

NESHBOY SOC PROFESSIONAL LIBRARY

MICROSOFT SENTINEL OPERATIONS

Workspace Architecture, Analytics Rules, Automation, and
Cost Governance — From Query to Platform

Build date: 2026-09-15

Table of Contents

CHAPTERS

Section A — Platform Foundations

- Part 1 — Why This Book: From Query to Platform
- Part 2 — Log Analytics Workspace Architecture and the Two Portals
- Part 3 — Data Ingestion: Connectors, Data Collection Rules, and Table Tiers

Section B — Detection Content Engineering

- Part 4 — Analytics Rules I: Scheduled and Near-Real-Time (NRT) Rules
- Part 5 — Analytics Rules II: Fusion, Anomaly, ML Behavior Analytics, and Custom Detections
- Part 6 — Entity Mapping and the Incident Model
- Part 7 — Detection Content Lifecycle: Content Hub, Solutions, and Detection-as-Code for Sentinel

Section C — Investigation, Hunting, and Reference Data

- Part 8 — Workbooks: Visualization, Reporting, and SOC Dashboards
- Part 9 — Hunting in Sentinel and the Defender Portal
- Part 10 — Watchlists and Reference Data

Section D — Behavioral Analytics and AI

- Part 11 — UEBA: User and Entity Behavior Analytics
- Part 12 — AI-Assisted Operations: Security Copilot and Agentic Platform Tooling

Section E — SOAR and Automation

- Part 13 — Automation Rules: Incident-Level Orchestration
- Part 14 — Playbooks: Logic Apps-Based Response and Remediation

Section F — Defender XDR Convergence

- Part 15 — Defender XDR Signal Convergence: Identity, Endpoint, Cloud Apps, and Email in One Incident
- Part 16 — Operating the Unified Security Operations Platform: Migration and Parity Gaps

Section G — Cost, Retention, and Governance

- Part 17 — Workspace Cost Models and Retention Tiers

Section H — Tuning and Troubleshooting

- Part 18 — Sentinel-Specific Tuning, Health Monitoring, and Troubleshooting

CHAPTERS

Section A — Platform Foundations

neshbooy

Part 1 — Why This Book: From Query to Platform

Why this part exists

The Detection Engineering Handbook (DEH) V2 ends a detection's story at the moment it becomes a correct, tested query. DEH Part 25 teaches the Kusto Query Language (KQL) dialect that Microsoft Sentinel and Microsoft Defender share — the pipe model, `where / join / summarize`, time-window functions, the join-strategy hints that keep a query fast — and DEH Part 23 frames the decision that produces that query in the first place: author it once in Sigma for portability across backends, or write it natively against one platform's table catalog and accept the maintenance cost of a single-platform rule. Both parts stop at a validated query. Neither part asks what happens to that query the moment someone has to run it forever, on someone else's infrastructure, under someone else's billing model, in front of an analyst who didn't write it.

That gap is this book. A KQL query that correctly matches DET-23-01 — DEH's own worked example, a process opening a suspicious handle to `lsass.exe` — is not yet a detection a SOC can operate. It has no workspace to run against, no rule type hosting its schedule, no entity mapping turning its output columns into an investigable incident, no automation deciding what happens the moment it fires, no assigned cost tier, and no settled answer to which of Microsoft's two portals the analyst who triages it will actually be looking at. *Microsoft Sentinel Operations* covers exactly that: the platform-operations layer sitting between a correct query and a durable, monitored, cost-accounted piece of production SOC infrastructure. This part states what that means precisely, names the one honesty constraint that governs every figure and claim in the other seventeen parts, and maps the book so a reader can find the part that answers their actual question instead of reading eighteen parts to find it.

1. Where the query stops and the platform starts

[CONCEPT] A validated query is a claim about telemetry: given this data, this logic correctly identifies this behavior. A platform is a claim about operations: given this logic, running continuously, at production volume, this organization can afford it, monitor it, respond to what it finds, and keep doing all three after the person who wrote it has moved to a different team. Those are different engineering problems, and treating them as one — writing a good query and assuming the platform question answers itself — is the specific failure this book exists to head off.

Microsoft Sentinel makes the distance between those two problems concrete in a way that's easy to underestimate from the query side alone. A query that becomes a **Scheduled rule** (the most configurable rule type — engineer-set query frequency and lookback window, full entity mapping and alert-grouping control) runs on a frequency and lookback window an engineer chose, against a workspace with its own retention and cost profile, producing alerts that a separate configuration layer (entity mapping) has to convert into something an investigation graph can reason about. The same query, packaged instead as a **Fusion** rule (Advanced multistage attack detection — single instance, not customizable, unavailable once Defender XDR incident integration is on), doesn't behave like a tunable detection at all — it's a single, non-deployable-by-you correlation engine you either have available or don't, depending on a platform configuration choice made somewhere else in the book (Part 16). The

query's correctness never changes across that difference. Everything about whether it's usable in production does.

Engineering Reality *A detection engineer who has only ever validated a query against a static dataset tends to treat "the query works" as the finish line. In Sentinel, the query is closer to the starting line: it still needs a workspace with retention that matches the investigation timeline it's meant to support, a rule type whose cadence and feature set actually fit the detection's latency requirement, entity mapping that survives the analytic's own edge cases (DEH Part 25 §4's shared-identifier problem shows up again here, not abstractly), an automation decision for what happens when it fires at 3 a.m. with no analyst watching, and a portal destination that, as of this writing, is actively changing under every workspace running the platform. None of that is optional infrastructure around the detection — it is the detection, in the only form that matters to the SOC actually running it.*

[SENTINEL ENGINEER] The entity-mapping point above deserves a concrete illustration, because it's the single most common place a query that behaved correctly in isolation starts behaving differently once it's wired into a rule. A query that returns a `DeviceName` column is not, by itself, a query that produces a usable Host entity in the resulting incident — entity mapping is a separate configuration step on the rule object, one that decides which output column maps to which entity type and which identifier field within it. Get that mapping wrong or leave it default, and two alerts that are actually about the same host can land in two different incidents, or two alerts about two different hosts that happen to share a mapped identifier can land in one — the same failure mode DEH Part 25 §4 documents for a `DeviceId` that isn't actually a unique key on a multi-user host, now expressed as a platform configuration choice rather than a query-logic one. Part 6 covers the full mechanics; the point here is narrower: this is exactly the kind of decision that doesn't exist yet at the moment a query is "just correct," and exists only once that query becomes a platform object.

2. The honesty constraint, stated once, in full

[CONCEPT] This book's author has no real Microsoft Sentinel tenant, no real Log Analytics workspace, and no Microsoft Defender portal license in their environment. Every claim in this book's other seventeen parts traces back to Microsoft's own public documentation, not to a screenshot taken from a live system or a result validated against production telemetry the author controls. That is a materially different evidentiary position than DEH's, where the parent handbook draws real captured evidence from a home-lab environment the author built and operates. No equivalent lab exists here, and this book does not pretend otherwise.

The practical consequence, enforced throughout by this book's `STYLE-GUIDE.md` §9, is that every figure in this book uses one of exactly two evidence-class tags: `OFFICIAL REFERENCE` (adapted from a cited Microsoft Learn page, with a full source citation and retrieval date) or `CONCEPTUAL` (an illustrative architecture or data-flow sketch that makes no claim of being captured from a real system). The two evidence classes DEH also defines — `CONTROLLED LAB EXAMPLE` and `REAL LAB EXAMPLE`, both meaning evidence captured from an environment the author actually built or observed — are retained in this

book's style guide only as a definition, never as a usable tag, because using either one here would assert something false about where the evidence came from. If a future edition of this book is written against a real tenant, that's a deliberate, recorded change to the book's evidence base, not a tag quietly picked because it seemed to fit a given figure (see [STYLE-GUIDE.md §9.4](#)).

This constraint shapes what this book can and can't do for a reader. It can tell you, with a citation, what Microsoft's documentation says a feature does, what a published limit or default is, and what a rule type's dependencies are. It cannot show you what your own workspace's specific data volume does to a scheduled rule's actual runtime, what your own tenant's Defender-portal onboarding does to an automation rule you already have deployed, or whether a documented default still matches what you see in your own environment on the day you read this. Every part in this book that makes a specific, checkable claim — a limit, a price, an availability rule, a retirement date — is written to be re-verified against the cited source, not trusted indefinitely; that's what this book's [PRODUCT VERSION NOTE](#) callout exists to force, and why it names a source and a re-check date every time it appears rather than asserting a fact as settled.

3. Six layers between a KQL query and an incident an analyst can act on

[CONCEPT] Stated as a single pipeline, the distance this book covers looks like Figure 1.1: a validated KQL query becomes a durable platform object by passing through a workspace, a rule type, an incident, an automation layer, a cost tier, and a portal — in roughly that order, with the content-packaging question (how the rule itself is versioned and distributed) wrapped around the whole thing rather than sitting at one point in the sequence.

flowchart TD

```
A["KQL query<br/>(DEH Part 25 syntax; DEH Part 23 Sigma-vs-native strategy)"] --> B
B["Log Analytics workspace<br/>(Part 2 – architecture and portals; Part 3 – ingestion and table tiers)"] --> C
C["Analytics rule<br/>(Part 4 – Scheduled/NRT; Part 5 – Fusion/Anomaly/UEBA/Custom Detections)"] --> D
D["Incident + entity mapping<br/>(Part 6 – SecurityIncident table, investigation graph)"] --> E
E["Automation rule / playbook<br/>(Part 13 – orchestration; Part 14 – Logic Apps response)"] --> F
F["Unified incident queue<br/>(Part 2 / Part 16 – Azure portal or Microsoft Defender portal)"]
C -->|packaged and versioned as| G["Content hub solution / CI-CD repository<br/>(Part 7 – rule lifecycle)"]
B -->|billed and retained under| H["Analytics tier or Data lake tier<br/>(Part 17 – cost and retention)"]
```

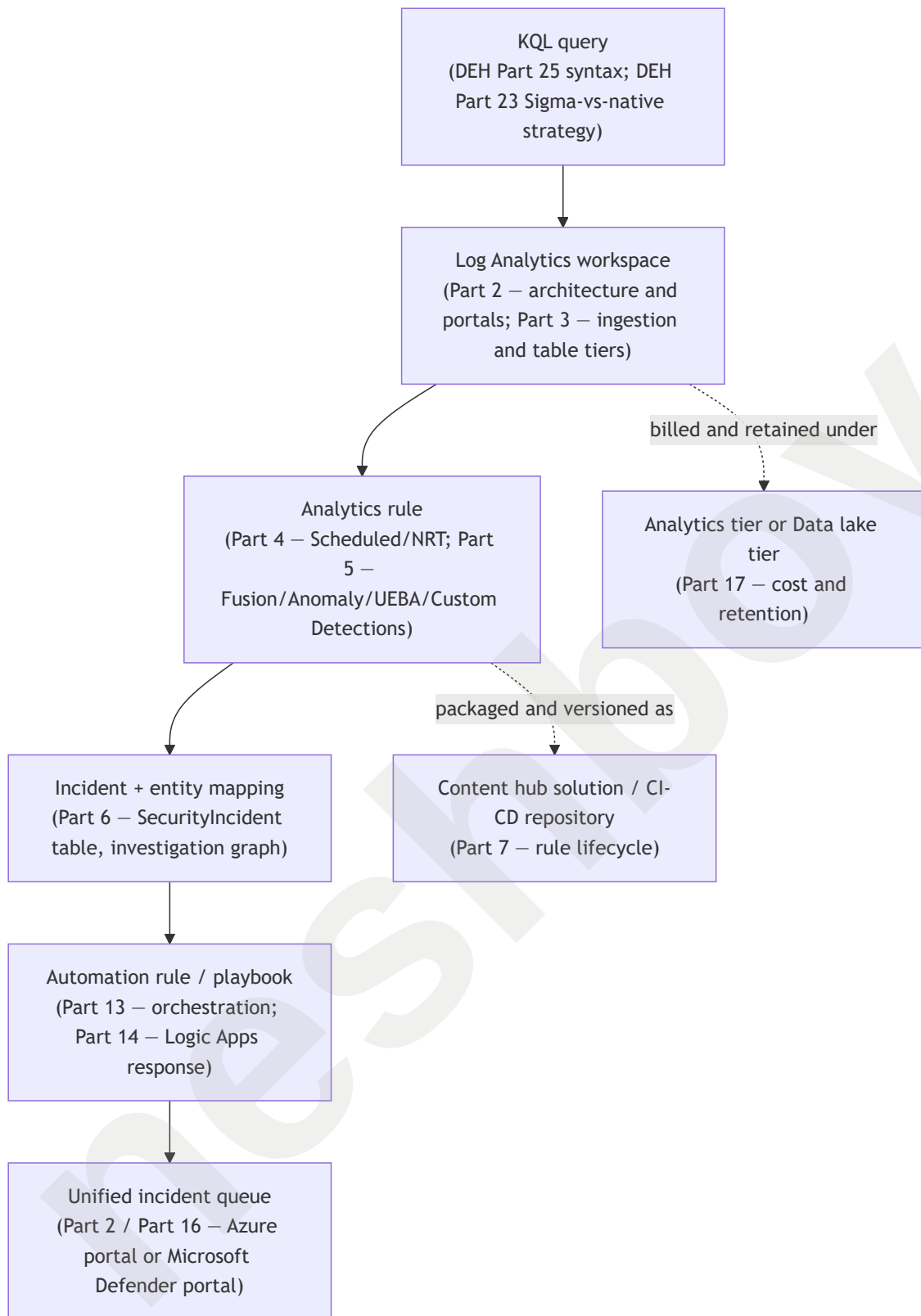


Figure 1.1 — From KQL query to SOC-visible incident: this book's six-layer path. *CONCEPTUAL.*

Illustrates the sequence of platform objects a validated query passes through before an analyst sees its output as a working incident, and which part of this book covers each layer. This is a structural sketch built from this book's own reading of Microsoft's Sentinel/Defender architecture documentation cited in

later parts' own reference lists — not a capture of any Sentinel UI screen, and not a reproduction of an official Microsoft architecture diagram.

[ENGINEERING] Two things about Figure 1.1 are worth naming up front, because they're easy to misread from the diagram's straight-line shape. First, the workspace layer isn't a passive container — which tier a table lands in (the analytics tier versus the data lake tier, Part 3 and Part 17) is itself a decision made before a rule ever runs, and it constrains what that rule can query cheaply. Second, the last box in the chain — the portal — isn't a fixed destination. It's the one layer in this diagram actively changing on a vendor timeline outside this book's or the reader's control, which §5 below covers as this book's first **PRODUCT VERSION NOTE**.

4. Platform layer, operations layer: the split this book reuses from DEH

[CONCEPT] DEH organizes its own content into a **telemetry layer** (what data exists and how it's structured) and an **analytic layer** (what a detection engineer builds on top of that data). This book reuses the same shape under different names because the underlying problem — don't conflate "what the substrate does" with "what you build on top of it" — is the same problem, applied to a vendor platform instead of a telemetry source:

- **Platform layer.** What Microsoft Sentinel and Microsoft Defender actually are, how data moves through them, and what a given feature does and doesn't do regardless of who's operating it — workspace architecture, the two-portal split, connector mechanics, table tiering, the mechanics of UEBA's baselines, what Security Copilot is actually wired into. Parts 2, 3, 6 (the mechanics half), 11, 12, 15, and 17 sit mostly here.
- **Operations layer.** What a SOC builds and runs on top of that platform — rule authoring, entity mapping choices, watchlist curation, automation-rule and playbook design, tuning, content lifecycle management. Parts 4, 5, 7, 8, 9, 10, 13, 14, and 18 sit mostly here.

The split isn't a hard wall — Part 6 explicitly straddles it, and Part 16 (the Defender-portal migration as a program-management problem) is operations layer content about a platform-layer fact — but it's the same discipline DEH's own architecture notes name as a defect to avoid when skipped: conflating "how ingestion billing works" with "how to write a good analytics rule" produces shallow coverage of both, because the two questions have different audiences, different failure modes, and different rates of change.

5. The two-portal fact, and the one date every later part has to account for

[ENGINEERING] Microsoft Sentinel is operated from one of two web experiences. **The Azure portal** is Sentinel's original home — a Sentinel-enabled Log Analytics workspace surfaced as an Azure resource inside the general Azure portal. **The Microsoft Defender portal** is the newer, converged experience: a Sentinel workspace onboarded there has its incidents, hunting surface, and (with caveats this book's Part 13 and Part 16 both treat as a first-class operational hazard, not a footnote) automation-rule behavior moved into the same queue and correlation engine Defender XDR already populates from Defender for Endpoint, Defender for Identity, Defender for Cloud Apps, and Defender for Office 365. Which portal a given workspace uses is not a cosmetic UI preference — it changes which menu paths

exist, which incident fields are populated, and in some documented cases which rule types are even available (Part 5's Fusion example above is one instance of this; Part 16 catalogs the rest).

PRODUCT VERSION NOTE (as of 2026-09-15) *Microsoft Sentinel in the Azure portal is scheduled for retirement on March 31, 2027; after that date, Sentinel is only available in the Microsoft Defender portal, and every workspace still in the Azure portal is redirected automatically. Microsoft's own "What's new in Microsoft Sentinel" page and its unified security operations platform overview documentation on Microsoft Learn carry this date as of this writing. If you're reading this after that date, or your organization has already migrated, treat every Azure-portal-specific menu path or procedure in Parts 2 and 16 as historical context for understanding a migration that has already happened to you, not as a live how-to — and re-check Microsoft Learn directly rather than trusting this book's date indefinitely.*

That date is why this book, starting in this part, never uses "Azure" or "Defender" alone to mean a portal — both words are also product names, and in a book about a platform actively moving between two experiences with those exact names, the ambiguity is precisely the kind of confusion that produces a wrong menu path in a reader's own tenant. Every later part expands both portal names in full on first mention and uses "the Defender portal" as the only acceptable short form after that.

What Would Change My Mind *This book treats "plan for the Defender portal as the only long-term option, on a timeline tied to the published retirement date rather than an indefinite one" as the correct default recommendation for any team still running Sentinel in the Azure portal today (Part 16 gives this its full program-management treatment). If Microsoft reversed the retirement decision, extended the date substantially, or shipped a durable Azure-portal-parity commitment beyond March 31, 2027, that recommendation would need to soften from "migrate on a schedule tied to this date" to "migrate on your own schedule" — a materially different message to give a SOC manager planning a multi-quarter project around a date this book currently treats as fixed.*

6. How the next seventeen parts are organized

[CONCEPT] The book runs in eight sections, eighteen parts total — deliberately fewer than the full range a book like this could run to, because the scope is one vendor's platform-operations layer, not a full detection-engineering curriculum DEH already owns by reference. The table below is the map; use it to jump directly to the part that answers the question you actually have instead of reading in sequence.

Section	Parts	Scope
A — Platform Foundations	1–3	This framing part; workspace architecture and the two-portal split; connectors, Data Collection Rules, and table tiers
B — Detection Content Engineering	4–7	Scheduled/NRT rules; Fusion/Anomaly/UEBA/Custom Detections; entity mapping and the incident model; content packaging and CI/CD
C — Investigation, Hunting, and Reference Data	8–10	Workbooks and dashboards; hunting queries, bookmarks, and notebooks; watchlists and reference data
D — Behavioral Analytics and AI	11–12	UEBA baselines and scores; Security Copilot and agentic platform tooling
E — SOAR and Automation	13–14	Automation rules as incident-level orchestration; Logic Apps–based playbooks
F — Defender XDR Convergence	15–16	Signal convergence into one incident queue; migration and portal parity gaps as an operations problem
G — Cost, Retention, and Governance	17	Analytics tier vs. data lake tier, commitment pricing, retention
H — Tuning and Troubleshooting	18	Rule health, auto-disable, correlation-engine surprises, canary-query discipline

Two things about that map are worth stating explicitly rather than leaving implicit. First, Sections F and the Part 2/16 portal split are the parts most likely to date fastest, because they describe an active platform migration with a published hard retirement date sitting inside this book's own planning horizon — every unit in those parts carries a mandatory **PRODUCT VERSION NOTE**, not an optional one. Second, Parts 11 and 12 look similar from a distance — both are "Microsoft's machine-learning layer on top of Sentinel data" — but they're split because UEBA is a mature, multi-year GA capability with a stable table schema, while Security Copilot and the platform's newer agent-facing tooling are the fastest-changing surface in the entire book; merging them would force one callout density onto content that mostly doesn't need it.

Three appendices sit behind the eighteen parts rather than inside them: **A1** collects the quick-reference decision tables (connector family, rule type, entity type) that Parts 3 through 6 each build up piece by piece; **A2** is the cost-and-retention decision matrix behind Part 17, kept separate specifically so it can be re-dated and re-verified on its own schedule as pricing changes without touching Part 17's narrative prose; **A3** is a single lookup table collecting every DEH cross-reference used anywhere in this book's Part Table, so a reader working from either book can navigate to the other without re-deriving the mapping from prose each time.

The table below states, topic by topic, which book owns which half of the query-to-platform pipeline this part introduced in §3 — a standing reference for whenever a later part in this book points back to DEH instead of re-explaining something DEH already teaches.

Topic	DEH's coverage	This book's coverage
KQL syntax — pipe model, <code>join</code> / <code>summarize</code> , time windows	DEH Part 25 — full syntax treatment against the Sentinel/Defender table catalog	Assumed known; cross-referenced on first use, never retaught
Sigma-vs-native authoring tradeoff, translation loss	DEH Part 23 — the general framing, using DET-23-01 as its canonical example	Referenced where a Sentinel-specific instance of the same tradeoff appears (Part 5, Part 7)
Analytics-rule YAML, CI/CD wrapper metadata, rule-repository structure	Explicitly excluded from DEH Part 25's own scope note	Part 7 — Detection Content Lifecycle owns this by name
Entity resolution and correlation theory	DEH <code>TERMINOLOGY.md</code> § Entity Key / Entity Resolution; DEH Part 30	Part 6 — Sentinel's concrete entity-mapping implementation and where it fails the same way DEH's own example fails
Baselining technique, in general	DEH Part 31	Part 11 — Sentinel's specific UEBA tables, peer-group scoring, and the two distinct anomaly scores
What a deployed rule costs, who sees it, and what portal it lives in	Out of scope for DEH by design	This book's core subject — Parts 2 through 18

7. What this book assumes you bring, and where to get it if you don't

[CONCEPT] This book assumes a working knowledge of KQL syntax and the Sigma-versus-native authoring tradeoff — not platform-specific fluency, just the query-language fundamentals DEH Part 25 and DEH Part 23 already teach. If a later part's KQL fragment (this book shows fragments, not full freestanding queries — see `STYLE-GUIDE.md` §3 for why a full detection query belongs in DEH, not here) doesn't parse cleanly on first read, that's the signal to go read DEH Part 25 first, not a gap this book will backfill; re-teaching KQL syntax here would duplicate content DEH already owns and this book explicitly declines to duplicate, per this book's own structural decision to cross-reference rather than re-teach.

What this book does not assume is access to a live Sentinel tenant. Every procedure, table reference, and rule-type description in the following seventeen parts is written to be useful to a reader with no workspace to test against — a SOC lead evaluating whether to adopt the platform, an engineer preparing for a migration that hasn't started yet, an analyst who uses a tenant someone else administers and needs to understand what they're looking at. That's a direct consequence of this part's §2: since the book itself has no tenant to validate against, it's written the way any reader without one would need it written — sourced, dated, and honest about which claims are a documented fact versus this book's own structural or editorial judgment.

8. How to read a part in this book

[CONCEPT] Every part carries the same front-matter contract: `title`, `part`, `author`, `reviewer`, `status`, `last_validated`, and `depends_on`. The last field is worth pausing on because it means something slightly different here than it does in a book with no companion volume — a `depends_on` entry can point at another part of this book by number, or at a DEH part directly using the form `DEH-23`, `DEH-25`, and so on, so a reader can tell without guessing which book a bare dependency number belongs to. `status` and

`last_validated` matter more in this book than they would in a book about stable architecture, precisely because of §5's retirement date and the broader pattern it exemplifies: a part can be well-written and still be stale, and the front matter is where a reader checks which is true before trusting a specific figure or limit.

Reading order matters less than the map in §6 suggests at first glance. A reader who already knows why the two-portal split exists can skip straight to the section covering their actual question; the callouts are built to make that safe. A `PRODUCT VERSION NOTE` always names its own source and re-check date inline, so a reader who jumps into Part 17 cold for a retention number doesn't need Part 1's framing to know whether to trust it — they need the callout itself, which is designed to stand alone. The table below is a quick reference for what each recurring callout signals, useful when a part's callout density (see `STYLE-GUIDE.md` §6.10) leaves a reader wondering why one box appeared and another didn't.

Callout	Signals
<code>PRODUCT VERSION NOTE</code>	A fast-moving platform fact — availability, price, retirement date — stated with a named source and a re-check date
<code>What Would Change My Mind</code>	A judgment call this book makes on top of a fact, plus the specific evidence that would reverse it
<code>Engineering Reality</code>	The operational cost hiding behind a feature that looks simple from the query side alone
<code>Blind Spot</code>	A documented gap in what a rule, entity map, or automation path actually covers
<code>False Positive Trap</code>	A platform- or logic-level noise source — in this book, as often a platform-behavior artifact as a legitimate user tripping a threshold
<code>Detection Autopsy</code> / <code>Detection Test</code>	Actual rule or automation logic — appears only in Parts 4, 5, 13, and 14, never elsewhere for the sake of quota
<code>SOC Management View</code>	A staffing, licensing, or risk-acceptance decision, not a technical one

The six content tags (`[CONCEPT]`, `[ANALYST]`, `[SENTINEL ENGINEER]`, `[THREAT HUNTER]`, `[ENGINEERING]`, `[SOC MANAGEMENT]`) work the same way across every part: they mark who the paragraph is actually for, so an analyst working the unified queue can skip a `[SENTINEL ENGINEER]` paragraph about watchlist curation without missing anything they needed, and vice versa. This part leans almost entirely on `[CONCEPT]`, with `[ENGINEERING]` and `[SENTINEL ENGINEER]` appearing only where a concrete platform or rule-configuration example was needed to ground the framing — later parts carry a denser, more evenly distributed mix across all six tags, per the "which column does this fall under" test `STYLE-GUIDE.md` §7 sets out.

Cross-references: DEH Part 23 (Query Language Strategy), DEH Part 25 (KQL — Sentinel/Defender).

Part 2 — Log Analytics Workspace Architecture and the Two Portals

Why this part exists

DEH Part 25 §1 already draws the distinction a working Sentinel operator needs first: KQL queries against Microsoft Sentinel run against a Log Analytics workspace's own table catalog, and the same query language against Microsoft Defender's Advanced Hunting surface runs against a different catalog entirely, even when the syntax on screen looks identical. This part exists to answer the question DEH Part 25 deliberately leaves alone: what is the thing a Sentinel query actually runs against, how many of them should an organization have, who is allowed to read what inside one, and — the fact that makes all three of those questions time-sensitive — which of two portals does an operator actually use to ask.

The workspace itself is the foundation everything later in this book sits on. Part 3 covers how data gets into a workspace (connectors, Data Collection Rules, table tiers); Part 6 covers what happens to that data once an analytics rule turns it into an incident; Part 17 covers what a workspace costs to run and how long it keeps what it's been fed. This part covers none of that. It covers the workspace as an Azure resource — where it lives, what governs who can query it, and how many an environment should actually deploy — and the operational fact sitting on top of every one of those decisions: Microsoft Sentinel is reachable from two different portals as of this writing, one of them on a published retirement date that falls inside this book's own planning horizon, and every screenshot, menu path, and RBAC assumption in the rest of this book has to be read against that fact.

This part does not cover: connector families, Data Collection Rules, or table-tier placement (Part 3); entity mapping or the incident model (Part 6); retention and cost mechanics for the Analytics tier and the Data lake tier (Part 17); or the operational mechanics of actually running a Defender-portal migration as a program (Part 16, which inherits this part's retirement-date fact and turns it into a project plan). What follows is the architecture and the access model underneath all of it, plus the two-portal fact every later part has to assume a stance on.

1. A Log Analytics workspace is an Azure Monitor resource, not a Sentinel-native one

[CONCEPT] A Log Analytics workspace is a data store — an Azure resource of type

`Microsoft.OperationalInsights/workspaces` — into which Azure Monitor collects log data from Azure and non-Azure resources alike (Microsoft Learn, "Log Analytics workspace overview,"

learn.microsoft.com/azure/azure-monitor/logs/log-analytics-workspace-overview, retrieved 2026-09-15).

Nothing about that definition mentions Microsoft Sentinel. A workspace exists, and is billed, and is queried, as an Azure Monitor object first; Microsoft Sentinel is a capability you enable *on top of* an existing (or newly created) workspace, not a separate resource with its own storage. Microsoft's own documentation is explicit about this: "Microsoft Sentinel documentation uses the term *Microsoft Sentinel workspace*. This workspace is the same Log Analytics workspace described in this article, but it's enabled for Microsoft Sentinel" (same source). Once Sentinel is enabled, every table in that workspace —

including tables Azure Monitor itself created for unrelated operational telemetry — is billed under Sentinel pricing, not Azure Monitor Logs pricing, whether or not the data in a given table has anything to do with security.

The practical consequence of "enabled on top of" rather than "a separate product": Sentinel's own configuration objects — analytics rules, automation rules, watchlists — are Azure resources scoped underneath the workspace resource, not independent top-level resources. An analytics rule is a `Microsoft.SecurityInsights/alertRules` resource; an automation rule is a `Microsoft.SecurityInsights/automationRules` resource; both live nested under the specific `Microsoft.OperationalInsights/workspaces` resource Sentinel was enabled on. Delete or move the workspace and every one of those child resources goes with it — there is no independent Sentinel resource to migrate separately. This is also why, once Sentinel is deployed on a workspace, that workspace cannot be moved to a different resource group or subscription afterward (Microsoft Learn, "Onboard to Microsoft Sentinel," learn.microsoft.com/azure/sentinel/quickstart-onboard, retrieved 2026-09-15) — the move restriction isn't an arbitrary Sentinel limitation, it falls directly out of the fact that Sentinel has no existence independent of the specific workspace resource it's attached to.

Engineering Reality *"You can run Microsoft Sentinel on more than one workspace, but data is isolated to a single workspace" (Microsoft Learn, "Onboard to Microsoft Sentinel," retrieved 2026-09-15) is easy to read past during a proof-of-concept and expensive to discover afterward. A team that enables Sentinel on a throwaway workspace to pilot one connector, likes what they see, and then wants to "just add the real data sources to the same workspace" has no problem — but a team that instead stands up a second workspace for production once the pilot looks good has just created two isolated data stores with no shared analytics-rule scope, no shared incident queue by default, and no path to consolidate the pilot's tuning work without re-deploying it. Decide the target workspace, and its resource group and subscription, before the pilot — not after — because the one thing you cannot do afterward is move the workspace to correct a placement decision made under pilot assumptions.*

[ENGINEERING] A workspace's tables are where this architecture becomes concrete. DEH Part 25 §1 already lists the table catalog split by surface (`SecurityEvent`, `Syslog`, and the rest for Sentinel; `DeviceEvents`, `DeviceProcessEvents`, and the rest for Defender's Advanced Hunting) — this part adds only the container-level fact DEH's table didn't need: every one of those Sentinel-side table names lives inside one specific workspace resource, and a query, an analytics rule, or a workbook that references `SecurityIncident` or `SentinelHealth` by bare name is implicitly scoped to whichever workspace it's running against. There is no global, cross-tenant `SecurityIncident` — there is a `SecurityIncident` table inside *this* workspace, and a different one, with different rows, inside any other workspace an organization runs. Section 2 below is about deciding how many "this workspace" instances an environment actually needs.

Retention inside that container is a two-part model: an interactive retention period, during which queries, workbooks, and alerts read the data directly, and a long-term retention period — up to twelve years per table — from which specific data has to be pulled back into interactive retention through a search job before it's queryable again (same "Log Analytics workspace overview" source). A workspace

with Sentinel enabled gets three months of free interactive retention by default rather than the 31-day default that applies to a workspace running only ordinary Azure Monitor workloads (Microsoft Learn, "Design a Log Analytics workspace architecture," learn.microsoft.com/azure/azure-monitor/logs/workspace-design, retrieved 2026-09-15) — one of several reasons, covered in full in Part 17, that combining or separating operational and security data in the same workspace is a cost decision as much as an architectural one.

2. Deciding how many workspaces to run

[ENGINEERING] Microsoft's own design guidance states the default plainly: "Your design should always start with a single workspace to reduce the complexity of managing multiple workspaces and in querying data from them. The amount of data in your workspace doesn't impose performance limitations" (Microsoft Learn, "Design a Log Analytics workspace architecture," retrieved 2026-09-15). A single workspace is not a starter-tier compromise a mature deployment outgrows on data-volume grounds alone — Microsoft explicitly disclaims a scale-driven reason to split. Every reason to run more than one workspace in the same environment is a *specific, named* requirement, not a general hedge against growth. The table below, adapted from the same source, supports one decision: whether a specific, named requirement is present in your environment, and if so, which side of the single-vs-multiple-workspace line it pushes you toward — not a general recommendation to split by default.

Requirement	Pushes toward —	Notes
No specific requirement below applies	Single workspace	Microsoft's stated default; reduces cross-workspace query complexity and RBAC surface.
Security team needs data ownership separate from IT/operations	Separate workspaces	Trade-off: separating from operational data usually raises the operational side's effective cost, since a workspace without Sentinel gets the shorter 31-day default retention.
Multiple Microsoft Entra ID tenants	One workspace per tenant (minimum)	Most connectors can only send data to a workspace in the same tenant as the resource.
Regulatory/data-residency requirement pinning data to a specific geography	Separate workspace per region with that requirement	Each workspace resides in exactly one Azure region; there is no cross-region replication built into the workspace itself.
Different cost ownership needing separate billing or charge-back	Separate workspaces per cost owner	Microsoft Cost Management + Billing reports by workspace; a single shared workspace can't cleanly split its bill without table- or resource-level query workarounds.
Different retention requirements for the <i>same table</i> across different resources	Separate workspaces	Retention is configurable per table within one workspace, but not per resource within the same table — two resources needing different retention for the same table name need two workspaces.
Ingestion volume near or above 100 GB/day, aiming for a Commitment tier discount	Consolidate into fewer workspaces (or a dedicated cluster)	Commitment tiers discount ingestion <i>per workspace</i> ; splitting volume across workspaces below the discount threshold gives up the discount unless a dedicated cluster pools it back together.
Legacy Log Analytics agent on Linux hosts	Constrains topology, doesn't drive it directly	The Linux Log Analytics agent can only connect to a single workspace at a time (unlike the Azure Monitor Agent); migrating off the legacy agent removes this constraint rather than resolving it by splitting workspaces.
Need for regional failover of monitoring data itself	Separate workspaces in different regions	Resilience requires deliberately duplicating ingestion into a second regional workspace; Microsoft is explicit that dependent resources like alerts and workbooks won't fail over automatically with the data.

[SOC MANAGEMENT] Cross-workspace querying exists — [union](#)/cross-workspace query expressions, plus Sentinel's own workspace-manager and multi-workspace hunting features — but it has a real ceiling: "Running a query over a large number of workspaces is slow and can't scale above 100 workspaces" (Microsoft Learn, "Design a Log Analytics workspace architecture," retrieved 2026-09-15). A Managed Security Service Provider (MSSP) or a large enterprise security operations center (SOC) planning a distributed architecture — one workspace per customer or business unit, accessed centrally through Azure Lighthouse delegation — is planning against that ceiling from day one, not discovering it after the fortieth customer onboards. The same source lays out three named patterns for this exact situation — distributed (one workspace per tenant, accessed via Lighthouse or Entra B2B guest access), centralized (one workspace in the service provider's own tenant, collecting from customer resources directly), and hybrid (per-tenant workspaces with a scripted or Logic Apps-based rollup into a central workspace for cross-customer reporting) — each with a stated trade-off between per-customer isolation and central query performance. None of those three patterns is specific to Sentinel; they're general multi-tenant Log Analytics topology, and Sentinel's own multi-workspace posture (covered in §5 below) is layered on top of whichever one an organization already runs.

3. Who can see what: workspace-level and resource-level access control

[ENGINEERING] Access to a workspace's data is governed by an *access control mode* set on the workspace itself, and the default setting is more permissive than it looks. Under the default mode, "Use resource or workspace permissions," a user who has ordinary read access to an Azure resource — a virtual machine, an App Service — automatically inherits read access to that resource's own monitoring data once it lands in the workspace, without ever being granted workspace access directly (Microsoft Learn, "Design a Log Analytics workspace architecture," retrieved 2026-09-15). Microsoft calls this *resource-context* access; it's a deliberate convenience so that a resource owner doesn't need a separate workspace-permission grant just to see their own resource's logs. The alternative mode, "Require workspace permissions," turns that inheritance off entirely and forces every reader to hold an explicit role assignment on the workspace resource itself. A separate, narrower control — *table-level RBAC* — lets an administrator grant or deny read access to specific named tables regardless of which access control mode the workspace runs under, which is the mechanism Sentinel documentation points to for scoping an internal audit team's or an MSSP customer's visibility to only the tables they should see (see Microsoft Learn, "Manage access to Microsoft Sentinel data by resource," referenced from the same design guidance).

Blind Spot *A workspace combining ordinary Azure Monitor operational data with Sentinel security data, left on the default resource-context access control mode, gives every resource owner in the subscription implicit read access to that resource's own security-relevant telemetry — not because anyone granted it, but because nobody changed the default. A VM owner with ordinary Contributor or Reader rights on their own virtual machine, who was never given a Sentinel role of any kind, can query that VM's `SecurityEvent` rows in the shared workspace the moment Sentinel starts collecting them, unless the workspace has been switched to "Require workspace permissions" or the security-relevant tables have been separately restricted with table-level RBAC. This is precisely the cost-versus-isolation trade-off the combined-workspace row in §2's table names in the abstract — here it's the concrete access-control failure mode that trade-off produces if nobody revisits the default after the decision to combine is made.*

[SOC MANAGEMENT] The choice to combine operational and security data in one workspace, from §2, and the choice of access control mode here, are not independent decisions — the second is the control that makes the first survivable from a security-standards standpoint. A security team asked to justify a combined workspace to an auditor or a compliance framework should be able to name which access control mode the workspace runs under and which tables carry table-level restrictions, not just point at the cost savings the combination produced. Reporting "we combined workspaces to hit a commitment tier" without reporting the access-control mode that decision now depends on is the same kind of incomplete cost story DEH Part 1 already warns against when a coverage number is quoted without the caveat that makes it honest.

4. Region and data residency

[ENGINEERING]

Each workspace resides in exactly one Azure region, chosen at creation and not changeable afterward (Microsoft Learn, "Design a Log Analytics workspace architecture," retrieved 2026-09-15). For most environments this is a one-time decision with no further consequence, but two situations turn it into a recurring one: a regulatory or contractual requirement that specific categories of data stay within a specific geography, and a multinational deployment sending logs from virtual machines across regional boundaries into a single workspace, which can incur bandwidth (egress) charges on the VM-to-workspace path — charges Microsoft describes as "usually minor relative to data ingestion costs for most customers" but still worth checking against the Azure pricing calculator before assuming they're negligible in a specific deployment (same source). Monitoring data sent via Azure diagnostic settings, rather than an in-VM agent, does not incur this egress charge regardless of region — the cost applies to the agent-forwarding path specifically, not to cross-region monitoring in general.

5. The two portals: how Sentinel ended up in both, and why that's temporary

[CONCEPT]

For most of Microsoft Sentinel's life as a product, there was one place to operate it: the Azure portal, alongside every other Azure resource. That changed with the introduction of a unified security operations experience inside the Microsoft Defender portal — the same portal Defender for Endpoint, Defender for Identity, Defender for Cloud Apps, and Defender for Office 365 customers already used — into which a Sentinel-enabled workspace can be *connected*, bringing Sentinel's incidents, hunting, and configuration surfaces into that portal alongside whatever Defender XDR signal the tenant already has. Connecting a workspace to the Defender portal does not require a Defender XDR license or an E5 subscription; Sentinel is generally available in the Defender portal on its own, for a tenant with no other Defender product at all (Microsoft Learn, "Microsoft Sentinel in the Microsoft Defender portal," learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal, retrieved 2026-09-15).

What changed the calculus from "an optional second way to see the same data" to "the destination" is the onboarding default and the published end date. As of this writing, in many cases, customers onboarding to Microsoft Sentinel after July 1, 2025 are automatically onboarded to the Defender portal rather than starting in the Azure portal at all (Microsoft Learn, "Connect Microsoft Sentinel to the Microsoft Defender portal," learn.microsoft.com/azure/sentinel/microsoft-sentinel-onboard, retrieved 2026-09-15) — specifically, a new customer with Owner or User Access Administrator permissions at the subscription level gets a workspace that's automatically onboarded, and those users work in the Defender portal exclusively from the start (Microsoft Learn, "Onboard to Microsoft Sentinel," learn.microsoft.com/azure/sentinel/quickstart-onboard, retrieved 2026-09-15). A workspace that isn't automatically onboarded can still be connected manually by an administrator with the right roles, and Microsoft's own onboarding guidance recommends doing so "for a unified experience in managing security operations (SecOps) across both Microsoft Sentinel and other Microsoft security services" (same source) — a recommendation this book treats as the reasonable default, not because the Azure portal is deficient today, but because of the date below.

PRODUCT VERSION NOTE (as of 2026-09-15) After March 31, 2027, Microsoft Sentinel will no longer be supported in the Azure portal and will be available only in the Microsoft Defender portal; every customer still running Sentinel in the Azure portal at that point is redirected automatically and uses Sentinel in the Defender portal from then on (Microsoft Learn, "Microsoft Sentinel in the Microsoft Defender portal,"

[Learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal](https://learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal), retrieved 2026-09-15).

This is the single most consequential date in this book, because it determines which portal's menu paths, navigation labels, and RBAC behavior every later part should be read against. If you're reading this after March 31, 2027, or your organization has already completed its own migration, treat any Azure-portal-specific procedure or screenshot anywhere in this book as historical context for a transition that has already happened to you, not as a live how-to — and check Microsoft's "Transition your Microsoft Sentinel environment to the Defender portal" guidance directly rather than this book's description of the pre-retirement state.

What Would Change My Mind This part treats "connect every Sentinel workspace to the Defender portal well ahead of March 31, 2027, and plan new deployments there from the start" as the correct default recommendation, matching Part 16's own program-management framing of the migration. If Microsoft reversed the retirement decision, published a durable Azure-portal-parity commitment extending meaningfully past that date, or the feature-comparison gaps named in §7 below turned out to block a specific, common workload from moving at all, the recommendation would need to soften — from "migrate proactively" to "migrate on a schedule driven by your own workload's parity gaps, not by the vendor's calendar" — a materially different message for a SOC manager currently building a multi-quarter migration plan around this date.

[ENGINEERING] Nothing about a workspace's own architecture — its resource ID, its region, its RBAC assignments, its tables — changes when it's connected to or disconnected from the Defender portal. Connecting is a relationship between the workspace and the Defender portal's tenant-level configuration, not a migration of the underlying data store; the same workspace resource is what both portals query. That single fact is what makes this part's placement in the book deliberate: everything in §§1–4 above is true regardless of which portal an operator happens to be looking through, and §6 below is about the specific, additional mechanics of establishing that relationship — not a second, parallel architecture.

6. Connecting a workspace to the Defender portal: roles, workspace topology, and two traps

[ENGINEERING] The Defender portal supports a single Microsoft Entra ID tenant per instance, connected to one *primary* workspace and any number of *secondary* workspaces; if a tenant has exactly one Sentinel-enabled workspace when it connects, that workspace becomes primary automatically (Microsoft Learn, "Connect Microsoft Sentinel to the Microsoft Defender portal,"

learn.microsoft.com/azure/sentinel/microsoft-sentinel-onboard, retrieved 2026-09-15). Only one workspace can be primary at a time, though the primary designation can be changed later — doing so automatically relinks the Defender XDR data connector from the old primary workspace to the new one (same source). The roles required to perform the connection itself are narrower than the roles required to use Sentinel day to day: onboarding needs either an unconditional Owner role assignment at the subscription scope, or User Access Administrator plus Microsoft Sentinel Contributor — and a tenant with more than one Sentinel-enabled workspace additionally requires at least a Microsoft Entra Security Administrator assignment to complete the connection (same source). Ordinary day-to-day use — reading incidents, querying tables, taking investigative action — continues to run on the same Sentinel Reader and Sentinel Contributor roles already in use before the connection, since "after you connect Microsoft Sentinel to the Defender portal, your existing Azure role-based access control (RBAC) permissions allow you to work with the Microsoft Sentinel features that you have access to," and any subsequent Azure RBAC change is reflected in the Defender portal automatically (same source) — this is the specific claim that makes §3's access-control-mode discussion above just as relevant after a workspace connects to the Defender portal as before.

One cross-tenant detail matters enough to name explicitly for any MSSP reading §2's distributed-architecture pattern: granular delegated admin privileges (GDAP) through Azure Lighthouse — the standard cross-tenant access mechanism named in §2 — is *not* supported for Sentinel data once it's viewed through the Defender portal. Microsoft's guidance for that scenario is to use Microsoft Entra B2B guest access instead (same source). An MSSP whose entire cross-tenant operating model was built on Lighthouse delegation into the Azure portal needs a genuinely different access mechanism for the Defender portal side of the same customer relationship, not a like-for-like swap — Part 16 covers what that means for a service-provider migration plan in full; this part exists so the underlying RBAC fact is on the record before that planning conversation starts.

Engineering Reality *Disconnecting a workspace from the Defender portal has a side effect worth checking for before doing it: "If your workspace has the Microsoft Defender XDR connector configured, offboarding the workspace from the Defender portal also disconnects the Microsoft Defender XDR connector" (Microsoft Learn, "Connect Microsoft Sentinel to the Microsoft Defender portal," retrieved 2026-09-15), which means Defender signal stops flowing into Sentinel's own incident data until someone reconnects that connector deliberately. A team troubleshooting an unrelated Defender portal issue by "just disconnecting Sentinel and reconnecting it" can silently create a gap in Defender-sourced incident correlation that has nothing to do with whatever they were actually debugging, and nothing on the Sentinel side announces that the connector went with it.*

7. What actually differs between the two portals, and what doesn't

[CONCEPT] Most of what a workspace is — everything in §§1–4 — is identical regardless of which portal displays it. What differs is what each portal builds on top of the same data, and how much of that difference matters depends heavily on whether the tenant also runs Defender XDR products. The table below summarizes the differences most relevant to workspace-level planning; Part 16 covers the full

navigation-path mapping and the migration-as-a-project version of this same comparison, and Part 5 covers the specific analytics-rule-type availability gap (Fusion and certain Microsoft-security incident-creation rules) this table only gestures at.

Capability area	Azure portal (retiring 2027-03-31)	Defender portal
Core SIEM functionality (ingestion, analytics rules, workbooks, hunting)	Full, unchanged	Full, same underlying workspace and rule engine
Incident queue	Sentinel's own queue, separate from any Defender XDR incident queue	One unified incident queue combining Sentinel and Defender XDR incidents, with automatic cross-domain correlation
RBAC model	Azure RBAC only	Same Azure RBAC continues to govern permissions (§6), surfaced through a unified Defender RBAC layer with row-level RBAC support
Cross-tenant / MSSP access	Azure Lighthouse (GDAP)	Native multi-tenant management (MTO); Lighthouse/GDAP not supported for Sentinel data — use Microsoft Entra B2B instead (§6)
AI-assisted investigation (Security Copilot)	Not available	Available natively — incident summarization, guided response, KQL generation, and (subscription-dependent) autonomous agents
Capabilities requiring a Defender XDR/E5 license even inside the Defender portal	Not applicable	Security Exposure Management, Defender-provided custom detection rules, and the Action center are limited or unavailable when a workspace is connected to the Defender portal without Defender XDR or an equivalent license (Microsoft Learn, "Microsoft Sentinel in the Microsoft Defender portal," retrieved 2026-09-15)
Ongoing feature investment	Maintenance and parity only, per Microsoft's own stated roadmap framing	Primary innovation surface — new Sentinel capabilities land here first
Support timeline	Supported until March 31, 2027	Long-term home for Sentinel

Table 2.1 — Azure portal vs. Defender portal, workspace-planning-relevant rows. *OFFICIAL REFERENCE.* The rows above are adapted from Microsoft Learn, "Microsoft Sentinel in the Microsoft Defender portal" (learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal), retrieved 2026-09-15, condensed to the subset relevant to workspace-level architecture decisions rather than reproduced in full; the source page's own tables cover hunting, automation, and cost-model differences in more detail than workspace planning needs at this stage.

[ANALYST] The unified incident queue row deserves one forward pointer rather than a full treatment here: an analyst working a Sentinel-sourced incident in the Defender portal, on a tenant that also runs Defender XDR products, sees that incident automatically enriched with related Defender signal and an "attack story" investigation graph spanning both sources — a materially different triage surface than the Azure portal's log-centric incident view, not just the same incident behind a different navigation menu. Part 15 covers what that convergence means for investigation practice in full; the point worth banking here is narrower: which portal an analyst is trained on is not a cosmetic preference, and a SOC that

trains analysts against Azure-portal screenshots today is training them against an interface with a fixed, published end date.

[SOC MANAGEMENT] None of the rows above are reasons to delay a Defender-portal connection — every one of them is either neutral (the underlying workspace and RBAC are unchanged) or a net addition (unified queue, native multi-tenant management, Security Copilot) for a tenant that already runs other Defender products. The one row that is a genuine planning constraint is the licensing-gated row: a Sentinel-only tenant with no Defender XDR license connecting to the Defender portal should expect Security Exposure Management, Defender-provided custom detection rules, and the Action center to stay unavailable regardless of the connection, and should not budget a migration plan around capabilities that require a license the organization hasn't bought. Reporting the migration to leadership as "moving portals" understates what's actually happening — it's a licensing and RBAC-surface review, timed against a vendor-controlled date, and the SOC management-facing version of the plan should say so explicitly rather than framing March 31, 2027 as a UI refresh.

8. Architecture at a glance

[CONCEPT] Figure 2.1 below ties §§1–7 together as one data-flow sketch: a single Log Analytics workspace resource, with Sentinel enabled on top of it, sitting underneath both portals at once for as long as both exist.

```
flowchart TB
    subgraph WS["Log Analytics workspace<br/>(Microsoft.OperationalInsights/workspaces)"]
        T["Tables: SecurityEvent, Syslog,<br/>SecurityIncident, SentinelHealth, custom tables"]
        R["Sentinel child resources:<br/>alertRules, automationRules, watchlists"]
    end
    C["Connectors / DCRs<br/>(Part 3)"] --> T
    T --- R
    WS -->|Azure RBAC on the workspace resource,<br/>unchanged by portal choice| AZ["Azure portal<br/>(retiring 2027-03-31)"]
    WS -->|same Azure RBAC,<br/>surfaced via unified Defender RBAC| DEF["Microsoft Defender<br/>portal<br/>(primary + secondary workspaces per tenant)"]
    DEF --- XDR["Defender XDR signal<br/>(Endpoint / Identity / Cloud Apps / Office 365)<br/>– if<br/>licensed and connected"]
    AZ --> ANALYST1["Analyst: Sentinel-only incident queue"]
    DEF --> ANALYST2["Analyst: unified incident queue<br/>(Part 15)"]
```

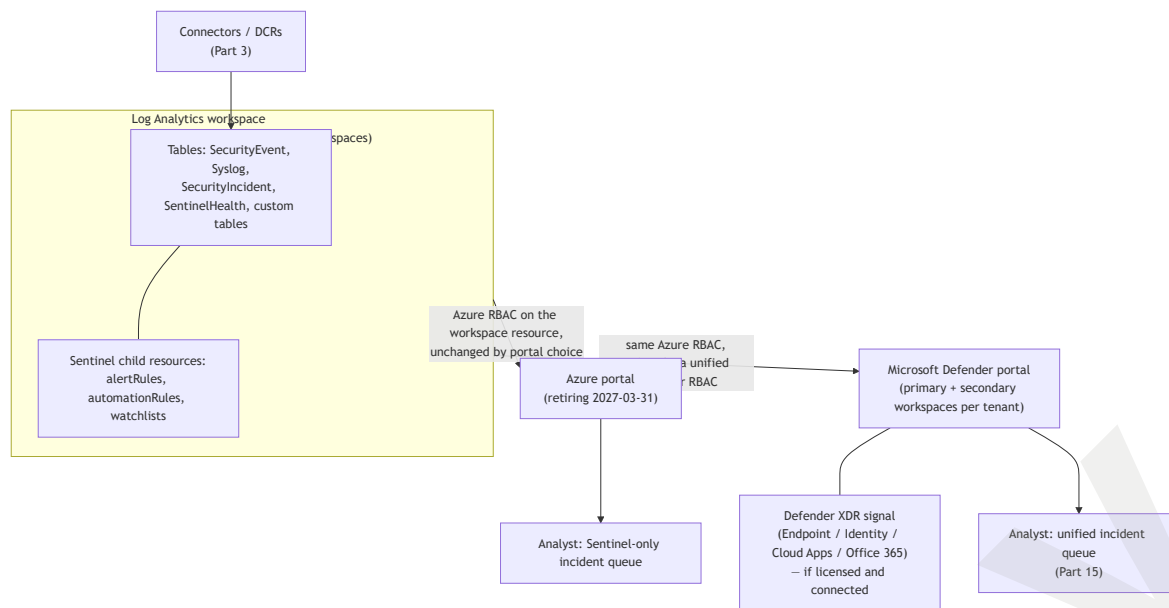


Figure 2.1 — One workspace, two portals, until 2027-03-31. *CONCEPTUAL*. Illustrates the architecture argued for in §§1–7: a single Log Analytics workspace resource holds the tables and the Sentinel child resources regardless of which portal is used to view them, and the same Azure RBAC assignments govern access in both. This is a structural sketch built from this part's own reading of the cited Microsoft Learn pages — not a capture of any Azure portal or Defender portal screen, and not a reproduction of an official Microsoft architecture diagram.

9. A decision checklist for a new or re-evaluated deployment

[SOC MANAGEMENT] Four questions, in order, cover the ground this part has argued for: First, does any requirement in §2's table apply — if not, one workspace is the correct starting point, and every additional workspace beyond that should be justified against a specific named row, not against a general instinct to isolate. Second, is the workspace's access control mode and table-level RBAC configuration actually documented and reviewed, especially if operational and security data share the workspace — §3's Blind Spot is a default, not an edge case, and it is trivially avoidable once someone checks it. Third, is the workspace already connected to the Defender portal, and if not, is there a specific, current parity gap (§7's licensing-gated row, or a rule-type gap Part 5 documents) actually blocking that connection today, or is the delay just inertia against a date that isn't moving. Fourth, for any organization running cross-tenant access via Azure Lighthouse, has the Entra B2B alternative required for Defender-portal Sentinel data (§6) actually been provisioned, or is that discovery still waiting for the day someone tries to use Lighthouse against a connected workspace and finds it doesn't reach.

None of these four questions require re-litigating this part's underlying architecture claims on a recurring basis — the workspace model in §§1–4 is stable, unlikely-to-change Azure Monitor plumbing. The fourth question, and the retirement date behind the third, are the parts of this checklist worth re-running on a schedule, not just once.

Cross-references: DEH Part 25 §1 (table catalog differs by surface — this part supplies the workspace container that catalog lives inside); Part 3 (connectors, Data Collection Rules, and table tiers — how data actually reaches the tables named in §1); Part 5 (analytics rule-type availability gaps between the two portals, referenced in §7); Part 6 (entity mapping and the incident model that runs on top of this workspace); Part 15 (Defender XDR signal convergence and the unified incident queue referenced in §7); Part 16 (the Defender-portal migration as a program-management problem, building directly on §5's retirement date and §6's RBAC mechanics); Part 17 (cost models and retention tiers, extending §1's retention note and §2's commitment-tier row in full).

neshboy

Part 3 — Data Ingestion: Connectors, Data Collection Rules, and Table Tiers

Why this part exists

[CONCEPT] DEH Part 25 starts from a working `SecurityEvent` or `SignInLogs` row already sitting in a table and teaches the query language that reads it. Part 2 of this book started one layer below that — the Log Analytics workspace the table lives in, and the two portals an analyst uses to see the result. This part sits between the two: it is about how a row gets into the table in the first place, what shape it's in when it arrives, and which of two billing-and-retention tiers it lands in — three decisions a Sentinel engineer makes once, per data source, that every later part in this book quietly assumes were made correctly. A rule that runs a perfect KQL query against a table nobody actually populates, or against a table that dropped the one column the query filters on during ingestion-time transformation, is not a detection gap you find by reading the rule. You find it here, in the connector and Data Collection Rule (DCR) configuration the rule never shows you.

Microsoft's own documentation lists a genuinely large number of ways data gets into a workspace — the current Microsoft Sentinel data connectors catalog documentation lists well over 100 connectors, spanning several distinct underlying mechanisms. This part groups those mechanisms into a small number of connector families, walks through the Data Collection Rule and Data Collection Endpoint objects that most of them now depend on, and treats "which tier does this table land in" as the ingestion-time decision it actually is rather than a retention-policy afterthought.

This part deliberately does not re-teach Advanced Security Information Model (ASIM) normalization or parser mechanics — DEH Part 6 (Normalisation) already covers how a raw event gets mapped into a common schema across ECS, OCSF, UDM, and ASIM, and DEH Part 5 (Parsers) already covers how parser drift silently breaks a detection. What follows assumes both and stays focused on the plumbing that exists *before* a parser or a normalization function ever runs: getting a raw record from a source into a workspace table at all.

1. The ingestion decision surface

[ENGINEERING] Onboarding any new data source to a Microsoft Sentinel workspace involves four separable decisions, and treating them as one decision ("turn on the connector") is where most avoidable ingestion problems start:

1. **Connector family** — which mechanism actually moves the data (a first-party Azure service integration, an agent-based pipeline, a codeless polling definition, a Logic App, or a direct API call).
2. **Transformation** — whether anything happens to the record between the source and the table it lands in (column filtering, redaction, reshaping), and where that transformation is defined.
3. **Destination table** — a built-in table with an existing schema Microsoft owns, or a custom table (suffixed `_CL`) this workspace owns and can alter.

4. **Tier** — whether that table's data is billed and retained under the Analytics tier or the Data lake tier (see §4).

The rest of this part works through these four decisions in order, because a connector family's technical limits (§2) constrain what transformation is even possible (§3), and the transformation and destination-table choice together constrain the tier decision (§4).

2. Connector families

[ENGINEERING] Microsoft Sentinel's connector catalog is large, but nearly every entry in it is an instance of one of five underlying mechanisms. Knowing which mechanism a given connector uses tells you what you can and can't configure — transformation support, latency, authentication model, and troubleshooting surface all follow from the family, not from the individual connector's marketing name in the Content Hub tile.

Azure-native (service-to-service) connectors

[ENGINEERING] The simplest family: an Azure service writes its own logs directly into a Log Analytics workspace, either because Sentinel's connector page enables a diagnostic setting on your behalf or because the service already supports Azure Monitor as a log destination. Azure Activity, Microsoft Entra ID sign-in and audit logs, Microsoft Defender for Cloud alerts, and most first-party Microsoft 365/Azure service connectors fall here. There is typically no agent, no DCR to author by hand for the basic case, and minimal ongoing maintenance — the tradeoff is that you generally can't reshape or filter the record before it lands; you get the service's own schema, in the service's own table, on the service's own delivery cadence.

Azure Monitor Agent, Data Collection Rules, and Data Collection Endpoints

[ENGINEERING] The dominant family for VM-resident telemetry — Windows Security Events, Syslog, Common Event Format (CEF) from a syslog-forwarding appliance, custom text-file logs, and Windows Firewall logs all move through the Azure Monitor Agent (AMA) today. AMA is a single agent, installed once per machine, whose behavior is entirely defined by the Data Collection Rules (DCRs) associated with it — a DCR is the Azure Resource Manager object ([Microsoft.Insights/dataCollectionRules](#)) that specifies what to collect, how to transform it, and which workspace and table to send it to. One machine can be associated with multiple DCRs; one DCR can be associated with many machines, which is how a fleet-wide "collect Sysmon Operational log" policy is expressed without touching each machine individually.

A Data Collection Endpoint (DCE, [Microsoft.Insights/dataCollectionEndpoints](#)) is a separate, optional-but-often-required resource that gives AMA (or a direct API caller — see below) a specific network ingestion point. A DCE is required when the agent needs to reach the ingestion service through Azure Private Link rather than the public endpoint, and it is unconditionally required for the direct Logs Ingestion API path described below, regardless of networking model.

PRODUCT VERSION NOTE (as of 2026-09-15) *The legacy Log Analytics agent (also called the MMA or OMS agent) and the older Azure diagnostics extension were Microsoft's stated retirement targets for VM-based log collection, with Microsoft's Azure Monitor Agent migration guidance naming August 31, 2024 as the retirement date for the legacy Log Analytics agent. If you are reading this after that date and still find a workspace or a solution's setup documentation referencing the legacy agent, treat that documentation as stale and check the current Azure Monitor Agent overview and migration pages on Microsoft Learn before building anything new against it — new deployments should be AMA/DCR-based by default.*

The Codeless Connector Platform (CCP)

[ENGINEERING] Not every source is an Azure service or a machine you can install an agent on. For a third-party SaaS API, a REST-based threat feed, or any source reachable only over HTTP on a polling schedule, Sentinel's Codeless Connector Platform (CCP — an actively developed connector-authoring surface, so treat any specific tooling detail below as worth re-checking against Microsoft Learn's "Create a codeless connector for Microsoft Sentinel" page before committing a design to it) lets an engineer define a connector declaratively — a JSON connector definition describing the authentication model (API key, OAuth2, Basic), the polling/pagination behavior against the source's REST endpoint, and the destination table — instead of writing and hosting custom collection code. CCP is the mechanism behind a large and growing share of newer Content Hub solution connectors, and it's also directly usable to build a connector for a source Microsoft hasn't shipped a template for.

Logic Apps-based connectors

[ENGINEERING] Before CCP existed, and still in use for a meaningful share of Content Hub solutions today, a data connector can simply be an Azure Logic Apps workflow: a playbook triggered on a recurring schedule that calls a third-party API, reshapes the response, and writes it into the workspace (historically via the legacy HTTP Data Collector API described below, and increasingly via the Logs Ingestion API). Because this is an ordinary Logic Apps resource, it inherits everything Part 14 covers about playbooks — managed-identity permissions, per-action billing, and Logic Apps' own reliability profile — as operational baggage a "connector" name doesn't suggest it has.

Direct ingestion: the Logs Ingestion API

[ENGINEERING] The most general mechanism, and the one every other custom-source path ultimately reduces to: the Logs Ingestion API is an Azure Monitor REST API that accepts a batch of JSON records and writes them to a specified table, using a DCR to define the target table's schema and any ingestion-time transformation, and a DCE to provide the network endpoint the caller sends to. Anything that can make an authenticated HTTPS call — a custom script, a SIEM-migration tool, a Logic App, an on-prem collector with no Sentinel-specific integration at all — can use this path, which is why it underlies both the Logic Apps family above and most custom, non-catalog integrations engineers build themselves.

PRODUCT VERSION NOTE (as of 2026-09-15) The Logs Ingestion API is the current, DCR-based replacement for the older Azure Monitor HTTP Data Collector API (the mechanism that historically wrote directly to a custom `_cl` table with no transformation step available), and Microsoft's own Azure Monitor documentation has flagged the Data Collector API as legacy, steering new work toward the Logs Ingestion API. This book could not confirm a specific, final retirement date for the Data Collector API against a source current as of this writing — check the Azure Monitor "Data Collector API" and "Logs Ingestion API" pages on Microsoft Learn directly before relying on either the older API's continued availability or any retirement timeline you find elsewhere. Either way, default new work to the Logs Ingestion API: it is the actively developed path regardless of exactly when the legacy one goes away.

Table 3.1 summarizes the five families side by side.

Table 3.1 — Connector family comparison. Supports an engineer deciding which mechanism fits a new source, based on what onboarding actually requires and what it allows afterward.

Connector family	Typical sources	Transformation available?	Auth/onboarding model	Ongoing maintenance surface
Azure-native (service-to-service)	Azure Activity, Entra ID sign-in/audit, Defender for Cloud	Rare — usually the service's own schema as-is	Portal toggle / diagnostic setting, Azure RBAC	Minimal — Microsoft owns the pipeline
Azure Monitor Agent (AMA) + DCR/DCE	Windows Security Events, Syslog, CEF, custom text logs	Yes — ingestion-time transformation in the DCR	Agent install (VM extension) + DCR association	Agent health per machine, DCR association drift
Codeless Connector Platform (CCP)	Third-party SaaS APIs, REST-based threat feeds	Yes, within the connector definition's mapping	API key / OAuth2 / Basic per connector definition	Connector-definition maintenance as the source's API evolves
Logic Apps-based	Legacy Content Hub solutions, custom scheduled pulls	Depends on the underlying write path (Data Collector vs. Logs Ingestion API)	Managed identity + API credentials in the Logic App	Full Logic Apps resource — see Part 14
Direct Logs Ingestion API	Custom scripts, migration tooling, anything with an HTTPS client	Yes — ingestion-time transformation in the DCR	Azure AD app registration/managed identity, DCE endpoint	Caller-side code and DCR/DCE lifecycle

3. Ingestion-time transformation: shaping data before it lands

[ENGINEERING] A DCR isn't only a routing instruction. It can carry a transformation — a KQL-like expression evaluated once per incoming record, before the record is written to the table — that filters columns, drops rows, computes a new column, or redacts a value. This is distinct from, and runs earlier than, ASIM normalization: a DCR transformation reshapes the raw record on its way into a table your workspace owns; ASIM (DEH Part 6) maps an already-landed record into a common schema so a query can treat many source tables uniformly. Confusing the two leads to trying to do full cross-table

enrichment in a DCR transform, which the model doesn't support — a transformation operates on one incoming record at a time, with no join against other tables.

The two transformation use cases that come up most often in practice:

- **Cost and noise control** — dropping columns a source always sends but no rule or hunt ever queries, before they're billed as ingested volume.
- **Compliance and data minimization** — redacting or hashing a column that carries sensitive data (a full email body, a raw credential-adjacent field) so the sensitive value never persists in the workspace at all, rather than persisting it and relying on RBAC/access control to protect it after the fact.

The following ARM template excerpt exports a Data Collection Rule

(`Microsoft.Insights/dataCollectionRules`) and illustrates only the `transformKql` property — the fragment that actually performs the shaping described above. The surrounding stream declarations, RBAC assignments, and DCR-to-DCE association a real deployment needs are omitted for brevity.

```
{
  "type": "Microsoft.Insights/dataCollectionRules",
  "apiVersion": "2023-03-11",
  "name": "dcr-custom-app-logs",
  "properties": {
    "dataFlows": [
      {
        "streams": ["Custom-AppEvents_CL"],
        "destinations": ["workspace-primary"],
        "transformKql": "source | project-away RawRequestBody | extend UserEmail =
tostring(hash_sha256(UserEmail))",
        "outputStream": "Custom-AppEvents_CL"
      }
    ]
  }
}
```

CONCEPTUAL SAMPLE – illustrative DCR transform, not a captured export. The `transformKql` line does two things in one pass: `project-away` drops a column the workspace never needs to store at all, and `extend` overwrites `UserEmail` with a one-way hash instead of the original address. Both edits are permanent at the point this transform runs — there is no "transform lightly now, decide later" option, which is exactly the Blind Spot named above stated as syntax rather than prose. The limitation worth stating plainly: `transformKql` supports a single incoming record's own columns and a fixed set of KQL operators suitable for row-level shaping (`project`, `extend`, `where`, `parse`, and similar) — it is not a general-purpose KQL surface, and DEH Part 25's join and cross-table `summarize` patterns have no equivalent here.

Engineering Reality *An ingestion-time transformation is evaluated per-record, with no state and no cross-record context — it cannot deduplicate against yesterday's data, cannot join against a watchlist, and cannot look at a second event to decide what to do with the first. If a transformation idea needs "compare this to something else," that's a job for a KQL query at read time (an analytics rule, DEH Part 25) or a summary/aggregation job, not for the DCR. What a transformation actually buys you is narrower and cheaper than that: shape and volume control on data you already know, in isolation, you don't need in full.*

Blind Spot *A column dropped at ingestion time is not recoverable later by any query, however clever — the data was never written to the table. This is a materially different failure mode from a detection gap in a rule's `where` clause, which a rule author can find and fix without touching historical data. If an investigation six months from now needs a field a transformation dropped on day one, there is no backfill; the record as stored simply doesn't have it. Treat every "drop this column, we'll never need it" decision in a DCR transform with the same scrutiny as a retention-period decision, because functionally it is one — for that column, retention is zero.*

4. Where a table lands: Analytics tier vs. Data lake tier

[ENGINEERING] Every table in a workspace is billed and retained under one of two tiers, and — this is the point this section exists to make — which tier a given table's data lands in is a decision made at ingestion configuration time, per source, not a retention setting applied uniformly after the fact. The **Analytics tier** is the interactive, full-featured tier: data here is immediately queryable with the full KQL surface, can drive analytics rules and workbooks, defaults to 90 days of retention and is extensible to two years, and is what every earlier example in this part has implicitly assumed. The **Data lake tier** is built for high-volume, lower-per-record-value secondary security data — cost-effective long-term retention, with ingestion, processing, storage, and query billed as separate components rather than folded into one per-GB analytics-tier rate.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's current documentation describes this two-tier model (Analytics tier and Data lake tier) as the present retention framework, per Microsoft Learn's "Log retention tiers in Microsoft Sentinel" page (learn.microsoft.com/azure/sentinel/log-plans), retrieved 2026-09-15 for this book. Training material and some older published guidance still circulate an earlier four-tier framing (Analytics / Basic / Auxiliary / Archive) that this two-tier model has superseded. If you encounter that older four-tier terminology in a vendor deck, a certification guide, or a colleague's notes, treat it as describing a prior version of this same retention concept, not a parallel option still on the menu — re-check the Log retention tiers page for the current mapping before designing around either the old or new terminology. Part 17 of this book covers the full cost-and-retention decision in depth; this part only establishes that the tier choice is made at the source, not discovered later.

[SOC MANAGEMENT]

The practical consequence for this part's scope: when you configure a DCR (or accept an Azure-native connector's default), you are choosing — sometimes explicitly, sometimes by accepting a default you didn't examine — which tier that table's incoming data is billed and retained under. A high-volume, rarely-queried source (bulk firewall traffic logs kept for compliance, not active hunting) is a candidate for the Data lake tier from day one; a source that feeds an analytics rule's scheduled query cannot go there, because Data lake tier data is not currently structured to drive analytics-rule alerting the way Analytics tier data is.

PRODUCT VERSION NOTE (as of 2026-09-15) The Data lake tier is a newer, still-evolving surface, and its relationship to analytics-rule alerting is exactly the kind of capability boundary Microsoft could extend later. As of this writing, Microsoft Learn's Sentinel data lake documentation describes data lake tier data as queried through search/KQL jobs and the data lake's own exploration surface, not as a source an analytics rule can alert on directly. Check the current "Microsoft Sentinel data lake" overview on Microsoft Learn before treating this as a permanent architectural limit rather than a snapshot of where the tier stands today.

Getting this backward — landing an actively-queried detection source in the wrong tier, or discovering months later that a compliance-retention source has been paying full Analytics tier rates the whole time — is exactly the kind of ingestion-time decision this part opened by naming as easy to treat as an afterthought and expensive to unwind later.

Table-management for existing tables (moving a table's plan, viewing tier assignment) also has portal-specific surface differences once a workspace is onboarded to the Microsoft Defender portal, covered in full in Part 16 — this part flags that dependency without duplicating it.

A second, easily missed consequence: tier is a property of the table's data as ingested, not a property you can trivially reassign after the fact for data already written. Moving a table's *plan* going forward is a supported operation, but re-tiering data that already landed under the old plan is a different and more constrained operation than adjusting a setting — so the ingestion-time decision this section describes

really is the point where the choice mostly locks in for that batch of data, not merely the point where a default first applies.

A note on retention vs. tier — two different knobs

[ENGINEERING] It's worth naming a distinction this part's scope requires but doesn't own: retention *period* (how many days a given table's data is kept before deletion or archival) and retention *tier* (Analytics vs. Data lake) are two different knobs, and a table's tier does not by itself tell you its retention period — each tier has its own retention defaults and extension mechanics, which Part 17 documents in full. This part's job is narrower: make sure the tier decision happens deliberately at ingestion configuration time, per source, rather than being discovered later as whatever the workspace default happened to be.

5. Putting it together: a data-flow view

[ENGINEERING] Figure 3.1 traces one record from source to query-ready table, showing where each of this part's four decisions (connector family, transformation, destination table, tier) actually happens in the pipeline.

flowchart LR

subgraph Sources

A1[Azure service
e.g. Entra ID sign-in log]
A2[VM: Syslog / CEF /
Windows Security Events]
A3[Third-party SaaS API]
A4[Custom script /
migration tool]

end

A1 -->|diagnostic setting| T1[Workspace table
service-owned schema]

A2 -->|Azure Monitor Agent| DCE1[Data Collection
Endpoint]

A3 -->|CCP poller| DCE1

A4 -->|Logs Ingestion API call| DCE1

DCE1 --> DCR[Data Collection Rule
schema + ingestion-time transform]

DCR --> T2[Workspace table
built-in or custom _CL]

T1 --> Tier{Tier assignment}

T2 --> Tier

Tier -->|Analytics tier| Q1[Analytics rules, hunting,
workbooks – full KQL]

Tier -->|Data lake tier| Q2[Long-term, lower-cost
secondary security data]

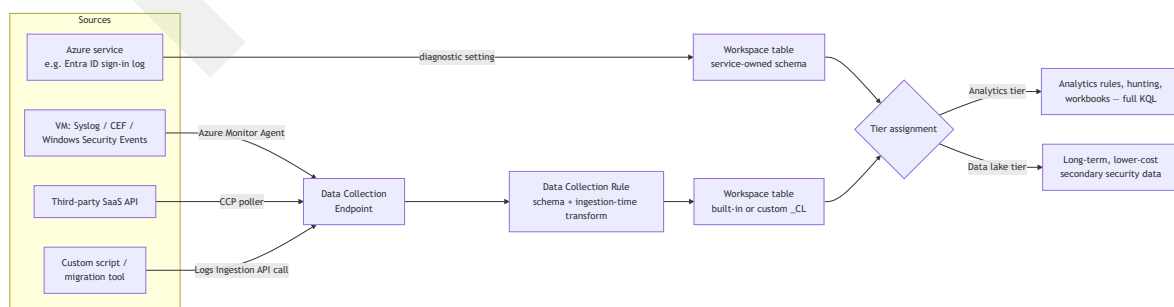


Figure 3.1 — Ingestion pipeline from source to tier. *CONCEPTUAL.* Illustrates, at a structural level, how the connector families in §2 converge on a Data Collection Rule and a destination table, and how that table's tier assignment (§4) determines what can be done with the data afterward. This is this book's own structural sketch built from the connector, DCR/DCE, and tier documentation cited throughout this part — not a reproduction of any official Microsoft architecture diagram, and not a capture of a live workspace, per this book's evidence-classification constraint.

6. Connector sprawl as a cost and ownership problem

[SOC MANAGEMENT] Every connector enabled is a source someone has to keep healthy — DEH's own SOC Management framing applies directly here: a connector nobody owns is a silent detection-coverage claim nobody is actually maintaining. Part 18 covers the health-monitoring mechanics (the `SentinelHealth` table, rule auto-disable) in depth; the point to make here, at the ingestion-design stage, is that connector count and table count are not free even before any rule runs against them. Each Azure-native connector adds an ingestion cost line; each AMA/DCR-based source adds an agent-health and DCR-drift surface to monitor; each CCP or Logic Apps connector adds a credential and an API-dependency to rotate and watch for breaking changes upstream. A SOC that enables a Content Hub solution's full connector bundle because it shipped as a bundle, without deciding whether every table it lands is going to be queried by anything, is choosing ingestion cost and maintenance surface it hasn't actually budgeted — a decision this part's four-question framing (§1) exists to force into the open before onboarding, not after the first monthly bill.

What Would Change My Mind This part treats "decide connector family, transformation, table, and tier deliberately, per source, before onboarding" as the correct discipline, on the assumption that ingestion cost and maintenance debt compound quietly if skipped. If Microsoft shipped a workspace-level default that reliably auto-assigned an appropriate tier and transformation per source type — something more than today's per-DCR manual choice — the specific four-step checklist in §1 would need to soften from "do this deliberately every time" to "verify the default, override only when it's wrong," a real reduction in the operational burden this part currently describes.

Cross-references

DEH Part 3 (Telemetry Engineering I: Host & Identity Sources), DEH Part 4 (Telemetry Engineering II: Network, Application & AI Sources), DEH Part 5 (Parsers), DEH Part 6 (Normalisation) — Part 17 (Workspace Cost Models and Retention Tiers) — Part 16 (Operating the Unified Security Operations Platform: Migration and Parity Gaps) — Part 18 (Sentinel-Specific Tuning, Health Monitoring, and Troubleshooting) — Part 14 (Playbooks: Logic Apps-Based Response and Remediation).

Section B — Detection Content Engineering

neshbooy

Part 4 — Analytics Rules I: Scheduled and Near-Real-Time (NRT) Rules

Why this part exists

DEH Part 25 already teaches the query language a Sentinel analytics rule runs — the KQL pipe model, `where`/`join`/`summarize`, time-window functions, and the join-strategy and performance concerns that apply once a query gets big — and DEH Part 23 already frames the decision that produces a query worth deploying in the first place: what an Analytic states versus what a Detection Rule literally evaluates. This part assumes both and starts exactly where DEH Part 25 §3 leaves off, at the one sentence its own Engineering Reality callout devotes to Sentinel: a scheduled analytics rule's "lookback" and "run frequency" are two independent settings, and mismatching them creates a real coverage gap. This part is that sentence's full platform-operations treatment — the rest of the rule wizard around that one setting, what each surrounding control actually does, and what an engineer trades away by getting one of them wrong.

This part does not re-teach KQL syntax (DEH Part 25) or the Sigma-versus-native-query tradeoff that precedes rule authoring (DEH Part 23). It also does not cover Fusion, ML Behavior Analytics, Anomaly rules, or Custom Detections — Part 5 owns every rule type that is single-instance, non-customizable, or ML-driven. This part covers the two rule types a Microsoft Sentinel `SENTINEL ENGINEER` builds and tunes by hand, from a query they already wrote: **Scheduled** rules, the default and most flexible type, and **Near-real-time (NRT) rules (fixed one-minute evaluation cadence, limited subset of scheduled-rule features)**, a constrained scheduled-rule subtype for the handful of detections where minutes matter more than customizability.

1. Rule types at a glance, and what this part actually covers

[CONCEPT] Microsoft Sentinel's rule-creation surface offers more than one rule type, and knowing the full set — even the ones this part defers — matters for orienting anything built later in this book. The table below is orientation only; every row past the first two gets its full treatment in Part 5.

Rule type	Distinguishing operational fact	Covered in
Scheduled	Fully customizable query, run frequency, and lookback; the default, most common rule type.	This part, §2–§3
NRT (Near-real-time)	Fixed one-minute run cadence, not configurable; a restricted subset of scheduled-rule query features and settings.	This part, §4
Fusion	Single built-in instance per workspace, ML-driven multistage-attack correlation across multiple low-fidelity signals; not customizable rule-by-rule.	Part 5
Microsoft security	Auto-creates incidents from alerts already raised by other Microsoft security products (for example Defender for Endpoint, Defender for Identity); not a query you write.	Part 5
ML Behavior Analytics	Single-instance, Microsoft-maintained ML models for anomalous SSH/RDP sign-in behavior; not customizable.	Part 5
Anomaly	ML-baselined, writes to the <code>Anomalies</code> table; does not raise its own alert without additional configuration.	Part 5
Custom Detections (Defender XDR)	Microsoft's KQL-based rule-authoring surface inside the Microsoft Defender portal, spanning Sentinel and Defender XDR tables together; a distinct authoring surface from the Sentinel analytics-rule wizard.	Part 5

[SENTINEL ENGINEER] Scheduled and NRT rules share almost everything about their internal anatomy — a KQL query, entity mapping, alert grouping, incident-creation behavior, and an automated-response attachment point. The difference that actually matters operationally is narrow: NRT trades away configurable frequency and lookback, and a portion of the KQL feature surface, for a fixed one-minute cadence no scheduled rule can match without running a query every sixty seconds indefinitely — which Microsoft Sentinel does not let a Scheduled rule do; five minutes is its fastest configurable frequency (see the PRODUCT VERSION NOTE in §2.2). Everything in §2–§3 below describes the Scheduled rule wizard in full; §4 then describes NRT purely as a delta against that same wizard, rather than repeating the shared ground twice.

2. Scheduled rules: anatomy of the rule wizard

2.1 General: naming, severity, and MITRE mapping

[SENTINEL ENGINEER] Both the Azure portal (in the Sentinel resource's own "Analytics" blade) and the Microsoft Defender portal (once a workspace is onboarded — see Part 2 for what that onboarding changes and the March 31, 2027 retirement date governing which portal is the long-term target) expose the same rule-creation wizard, built from a fixed sequence of steps: General, Set rule logic, Incident settings, Automated response, and Review and create. The Defender portal is the acceptable short form after this first full mention in this section.

The General step fixes the rule's identity independent of its query: a display name and description, a severity (`Informational`/`Low`/`Medium`/`High`), a status (enabled or disabled at creation), and one or more MITRE ATT&CK tactics and techniques the rule maps to. This mapping is not decorative — it is the field that later populates an incident's own tactic/technique display in the unified queue, and the field a coverage exercise (DEH Part 41) reads to answer "which techniques does this workspace actually have a rule for."

2.2 Set rule logic: query scheduling, threshold, and grouping

[SENTINEL ENGINEER] The Set rule logic step holds the query itself (a KQL query exactly like the ones DEH Part 25 teaches — this book supplies no new syntax here) plus four settings groups that decide how that query becomes an alert:

- **Query scheduling** — *run frequency* (how often the rule fires the query) and *lookback period*, usually labeled "lookup data from the last" (how far back each run's query window extends). These are independent controls, not one setting expressed two ways.
- **Alert threshold** — the condition applied to the query's result count before an alert fires, commonly expressed as an operator (greater than, fewer than, equal to, not equal to) and a number, defaulting to "greater than 0" — any non-empty result set alerts.
- **Event grouping** — whether every matching row becomes its own alert, or the whole result set for a given run becomes a single alert. This setting, not the query's own `summarize` logic, is what most directly determines whether a noisy query becomes a noisy analyst experience.
- **Suppression** — an optional "stop running the query for a period after this rule generates an alert" control, distinct from both alert grouping and the incident-level grouping in §2.4; it pauses the rule's own execution, not just the display of its results.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's documentation for creating a scheduled analytics rule states a run-frequency range from 5 minutes up to 14 days, with the lookback period independently configurable up to the same 14-day ceiling — see Microsoft Learn, "Create custom analytics rules to detect threats" (part of the Microsoft Sentinel documentation set under [Learn.microsoft.com/azure/sentinel/](https://learn.microsoft.com/azure/sentinel/)). Treat the exact numbers as the specific fact to re-check: Microsoft has adjusted rule-wizard bounds and defaults before, and a value this book states as a hard ceiling is exactly the kind of detail that page will show as current if this one has drifted.

Engineering Reality Setting the lookback period exactly equal to the run frequency looks efficient and is the most common mistake in this whole step: if a run fires every 15 minutes with a 15-minute lookback, any event whose ingestion into the workspace lags evaluation time by even a few minutes — which ingestion delay does routinely, not as an edge case — falls into the gap between when the previous run's window ended and when the event actually became queryable. The standard fix is a lookback period wider than the run frequency, commonly two to three times as wide, relying on the rule's own suppression or the incident-grouping logic in §2.4 to avoid re-alerting on an event a prior run already actioned. A rule authored once with matched values and never revisited degrades silently: the query still runs, still returns rows, and the gap only becomes visible during an incident retrospective that finds an event the rule should have caught but never did.

2.3 Entity mapping and custom details

[SENTINEL ENGINEER]

Entity mapping — assigning specific query-result columns to typed entities such as Account, Host, IP, or URL — is what lets a scheduled rule's alert participate in Sentinel's investigation graph and in the entity-based alert grouping described in §2.4. Part 6 of this book owns the full mechanics of how entity types, identifiers, and the incident model interact and where that mapping breaks down on a shared or ambiguous identifier; this part only needs the operational fact that entity mapping is configured per rule, in this same wizard step, and that the rule wizard documents a per-rule cap on how many entity types and identifiers a single rule can map — treat the exact cap as another value to check against the current rule-creation documentation rather than a number this book fixes here, since Microsoft has raised entity-mapping limits before.

A separate, simpler mechanism in the same step — **custom details** — surfaces specific query-result columns directly onto the alert and resulting incident as labeled key/value fields, without requiring an entity type at all. Custom details are the more direct choice when a query result carries context worth showing an analyst (a resource ID, a rule name, a scan target) that doesn't cleanly fit Sentinel's entity schema.

2.4 Incident settings: alert grouping and incident creation

[SENTINEL ENGINEER]

The Incident settings step controls two related but distinct decisions. First, whether this rule's alerts automatically create incidents in the unified queue at all — a toggle a rule feeding another automation path (for example, a playbook triggered directly off the alert, covered in Part 14) might reasonably leave off. Second, if incident creation is on, how alerts from *this rule* group into incidents: a matching method (commonly, all alerts from this rule into one incident; alerts matching on selected entities; or no grouping — an incident per alert) and a time window bounding how far apart two alerts can be and still group together. A "reopen matching closed incidents" toggle decides whether a new matching alert reopens an incident an analyst already closed, or starts a fresh one.

[ANALYST]

What these settings produce in the unified queue is the practical thing to hold onto: a rule grouped too loosely (all alerts, no entity match, a long time window) produces one incident that keeps absorbing unrelated matches for days, diluting its own investigation history; a rule grouped too tightly (per-alert, no grouping) produces an incident per match, burying a real pattern across many separate queue entries an analyst has to notice are related manually. Neither failure mode is visible from the rule's query alone — both are entirely a function of this step's settings, which is why a rule review that only reads the KQL misses half of what determines the analyst's actual experience of that rule.

False Positive Trap Grouping "all alerts triggered by this rule into a single incident," the most permissive matching method, is attractive because it minimizes queue clutter — but it also means a rule with a broad query (say, a login-anomaly rule scoped to an entire tenant rather than a specific entity) can fold two genuinely unrelated attacker sessions, days apart, into the same open incident, because "unrelated" isn't a concept this matching method evaluates. Grouping by matched entities is usually the safer default for anything but a genuinely single-purpose rule, precisely because it keeps the incident's own scope aligned with what the query result actually says happened, not with "which rule produced it."

2.5 Automated response

[SENTINEL ENGINEER] The final configuration step attaches automation rules (Part 13) that run when this rule's alert or incident fires — assigning an owner, applying a tag, or invoking a playbook (Part 14). This part treats that attachment point as the boundary of scheduled-rule anatomy: everything past it is orchestration logic that Part 13 owns in full, including the batching-window hazard that automation rule scope specifically names as a first-class operational risk once a workspace onboards to the Defender portal.

3. Worked example: from naive threshold to tuned scheduled rule

[SENTINEL ENGINEER] The example below tracks one rule through the exact failure the Engineering Reality box in §2.2 describes in the abstract, using Microsoft Entra ID (formerly Azure Active Directory) sign-in activity — ingested into the `SigninLogs` table by the Entra ID data connector — as the telemetry. It is shown as two ARM template excerpts, both exporting the `Microsoft.SecurityInsights/alertRules` resource type, `kind: "Scheduled"`. Per this book's own code-block rule (STYLE-GUIDE.md §3), the embedded KQL is a short fragment inside a larger rule definition, not a freestanding detection query — the fragment itself leans on DEH Part 25 for syntax, not the other way around.

The naive first draft below shows the failure mode directly.

```
// CONCEPTUAL SAMPLE – illustrative ARM excerpt of a Microsoft.SecurityInsights/alertRules
// resource, kind "Scheduled." Property and enum spellings shown here are illustrative of the
// documented schema shape, not exported from a live tenant – verify exact field and enum names
// against the current Microsoft.SecurityInsights/alertRules ARM template reference before
// treating this as a deployment template.
{
  "type": "Microsoft.SecurityInsights/alertRules",
  "kind": "Scheduled",
  "properties": {
    "displayName": "Any failed sign-in",
    "severity": "Medium",
    "enabled": true,
    "query": "SigninLogs | where ResultType != 0",
    "queryFrequency": "PT1H",
    "queryPeriod": "PT1H",
    "triggerOperator": "GreaterThan",
    "triggerThreshold": 0,
    "suppressionEnabled": false,
    "eventGroupingSettings": { "aggregationKind": "AlertPerResult" },
    "tactics": ["CredentialAccess"],
    "techniques": ["T1110"]
  }
}
```

MITRE: T1110 (Brute Force).

Detection Autopsy — "alert on any failed sign-in"

The rule: Alerts on any row where `ResultType != 0` (a failed sign-in), one alert per matching row (`AlertPerResult`), on an hourly run, with no query-side thresholding — the alert threshold is left at `GreaterThan 0`, so a single matching row alerts.

Why it shipped: It reads as the obvious first-draft brute-force detection — flag failed sign-ins, alert on anything that matches — and both the query and the wizard settings behind it are trivial to write and to defend in a design review.

How it failed: It is a mistyped-password counter running at organization scale, not a brute-force detection. A single traveling employee who fat-fingers a passphrase twice produces two alerts; an organization of any real size produces alerts by the dozens to hundreds per run, every run, none of them distinguishing a legitimate typo from an actual guessing campaign — the rule's own threshold and grouping are doing all of the damage; the query itself is syntactically fine.

The fix: Widen the lookback past the run frequency (closing the ingestion-lag gap the Engineering Reality box in §2.2 describes), move `aggregationKind` to `SingleAlert`, and push the actual thresholding into the query itself with `summarize` and a `FailCount >= 10` filter — shown in the tuned rule below.

Detection Test Setup: A Microsoft Sentinel workspace with the Entra ID sign-in-logs connector enabled and a test account available to sign in with a deliberately wrong password. **Action:** Attempt sign-in with the test account and an incorrect password three times within a few minutes, from the same client. **Expected result:** Against the naive rule above, three separate alerts, one per failed attempt, each carrying `Medium` severity and no volume context — indistinguishable in the queue from three unrelated single-attempt failures on three unrelated accounts. Against the tuned rule below, run the same test with at least ten attempts within one 15-minute window (the tuned rule's own threshold) and expect a single alert carrying a `FailCount` custom detail, not ten separate ones.

```
// CONCEPTUAL SAMPLE – same resource type and kind as above, tuned. Enum and property spellings
// remain illustrative; verify against the current ARM template reference before deployment.
{
  "type": "Microsoft.SecurityInsights/alertRules",
  "kind": "Scheduled",
  "properties": {
    "displayName": "Sign-in failure burst against a single account",
    "severity": "Medium",
    "enabled": true,
    "query": "SigninLogs\n| where ResultType != 0\n| summarize FailCount = count(), IPList =
make_set(IPAddress) by UserPrincipalName, bin(TimeGenerated, 15m)\n| where FailCount >= 10",
    "queryFrequency": "PT15M",
    "queryPeriod": "PT30M",
    "triggerOperator": "GreaterThan",
    "triggerThreshold": 0,
    "suppressionEnabled": true,
    "suppressionDuration": "PT1H",
    "eventGroupingSettings": { "aggregationKind": "SingleAlert" },
    "incidentConfiguration": {
      "createIncident": true,
      "groupingConfiguration": {
        "enabled": true,
        "reopenClosedIncident": false,
        "lookbackDuration": "PT5H",
        "matchingMethod": "Selected"
      }
    },
    "tactics": ["CredentialAccess"],
    "techniques": ["T1110.001"]
  }
}
```

MITRE: T1110.001 (Password Guessing).

Three changes carry the whole fix, and each maps to a setting named in §2.2–§2.4, not to any change in query sophistication beyond a `summarize`: the `queryPeriod` (30 minutes) is now wider than the `queryFrequency` (15 minutes), closing the exact ingestion-lag gap the Engineering Reality box in §2.2 warns about; `aggregationKind` moved from `AlertPerResult` to `SingleAlert`, so one run produces at most one alert per matching account/window pair, not one per row; and the query itself now does the thresholding work (`FailCount >= 10`) that the naive version left entirely to the rule's own `triggerThreshold: 0`.

Blind Spot The tuned rule's threshold (10 failures in 15 minutes) is tuned against a guessing campaign that announces itself through volume. A low-and-slow attacker making two or three attempts per account per hour across a large number of accounts never crosses this threshold for any single account, and the `by UserPrincipalName` grouping means the query never aggregates across accounts to notice that the same source IP is doing this to fifty of them — a T1110.003 (Password Spraying) shape this account-scoped rule is structurally blind to regardless of how the threshold number is tuned. Catching that shape needs a second rule grouped by `IPAddress` (or source ASN) instead of by account, not a lower threshold on this one.

4. Near-real-time (NRT) rules

4.1 What's fixed versus configurable

[SENTINEL ENGINEER] An NRT rule shares the same General, entity-mapping, custom-details, incident-settings, and automated-response surface described in §2 and §2.3–§2.5. What it removes is exactly the query-scheduling group in §2.2: there is no run-frequency or lookback field to set, because Microsoft Sentinel runs an NRT rule's query once every minute, fixed, with no configuration path to change that cadence. In ARM terms, an NRT rule's `alertRules` resource uses `kind: "NRT"` and simply has no `queryFrequency` or `queryPeriod` properties for that kind, rather than those properties existing with a locked value.

```
// CONCEPTUAL SAMPLE – illustrative ARM excerpt, kind "NRT." No queryFrequency/queryPeriod
// properties exist for this kind; cadence is fixed by the platform, not by a settable field.
// Verify current field presence against the Microsoft.SecurityInsights/alertRules ARM template
// reference before treating this as authoritative.
{
  "type": "Microsoft.SecurityInsights/alertRules",
  "kind": "NRT",
  "properties": {
    "displayName": "Impossible travel – same account, two distant sign-ins within minutes",
    "severity": "High",
    "enabled": true,
    "query": "SigninLogs | where ResultType == 0",
    "triggerOperator": "GreaterThan",
    "triggerThreshold": 0,
    "tactics": ["InitialAccess"],
    "techniques": ["T1078"]
  }
}
```

MITRE: T1078 (Valid Accounts).

Microsoft's own NRT documentation also states that NRT queries run against a restricted subset of KQL relative to what a scheduled rule's query can use — specific unsupported operators and constructs, and constraints on join complexity, that Microsoft's current NRT-limitations reference page lists explicitly. This book does not reproduce that list here, because it is exactly the kind of fast-moving enumeration a

printed or long-lived page shouldn't freeze in place; port a scheduled rule's query to NRT by checking the current list, not by assuming syntactic parity.

PRODUCT VERSION NOTE (as of 2026-09-15) *Microsoft documents both a maximum number of NRT rules permitted per workspace (separate from the overall cap on scheduled rules) and a specific list of KQL constructs NRT queries don't support — see Microsoft Learn, "Detect threats quickly with near-real-time (NRT) analytics rules in Microsoft Sentinel." This book intentionally does not restate either number or list here: the per-workspace NRT rule cap in particular has changed at least once as the feature matured from template-only toward general availability, and a specific figure quoted in a book chapter is far more likely to go stale than a live documentation page an engineer can check at deployment time. If you're building against a specific NRT rule count today, check that page directly rather than trusting a number from this or any other secondary source.*

Engineering Reality *NRT rules run once every minute, fixed — there is no lookback-vs-frequency tuning knob the way there is for a scheduled rule (§2.2, and DEH Part 25 §3 for why that mismatch matters at all). What this actually costs you: an NRT rule's query has to be cheap enough to finish comfortably inside a one-minute window on every run, forever, and a query that's "slow but fine" as an hourly scheduled rule can start missing its own cadence as data volume grows — silently degrading toward something closer to a scheduled rule's latency while still being billed and reasoned about as if it were near-real-time.*

4.2 Choosing scheduled versus NRT

[SENTINEL ENGINEER] The decision is narrower than it first appears, because NRT's only real advantage is cadence and its costs are all subtraction — less configurability, a smaller KQL surface, and (per the PRODUCT VERSION NOTE above) a workspace-level cap most scheduled rules never have to think about.

The table below supports one decision: whether a specific detection candidate justifies NRT's tradeoffs or is better served by a scheduled rule with the shortest sensible frequency instead.

Consideration	Scheduled	NRT
Run cadence	Configurable, 5 minutes to 14 days	Fixed at 1 minute, not configurable
Lookback period	Configurable independently of cadence	Not applicable — no lookback setting
KQL feature surface	Full surface DEH Part 25 teaches	Restricted subset; some operators and join shapes unsupported
Per-workspace rule count	Governed by the general scheduled-rule limit	Governed by a separate, smaller NRT-specific limit
Best fit	Most detections — anything not latency-critical to the minute	Detections where a 5–15 minute delay materially changes the outcome (active session hijack, mass credential exposure)
Query cost discipline	Matters, but a slow run mostly just runs long	Matters acutely — a query that can't finish in well under a minute degrades the rule's own reason for existing

5. Operating rule volume: tuning, health, and cost signal

[SOC MANAGEMENT] Every setting in §2–§4 is also a volume lever, and the aggregate effect across a workspace's full rule set is a SOC management problem, not just a per-rule tuning exercise. A handful of rules each grouped too loosely (§2.4) or thresholded too low (§2.2) compounds into an incident queue an analyst team can't keep current with — the platform-operations version of alert fatigue, produced entirely by rule configuration rather than by any actual increase in attacker activity. Reviewing a new or changed rule's scheduling, threshold, and grouping settings before it ships belongs in the same approval gate as reviewing its query logic, for the same reason DEH Part 25 §8's SOC Management View callout gives for query cost: the bill (here, analyst attention rather than ingestion dollars) arrives whether or not anyone reviewed the setting that caused it.

[ENGINEERING] A rule that fails repeatedly — most often from an expired or insufficiently scoped access token, or a permission change upstream of the query's source table — gets `(auto-disabled)` appended to its display name rather than continuing to fail silently forever. The `SentinelHealth` table is the queryable audit trail for that and other rule-health events; Part 18 owns the full troubleshooting treatment of both. The operational point worth carrying into this part specifically: a rule that stops firing because it's disabled looks, from inside the incident queue, identical to a rule that's still enabled and simply has nothing to report — a second, platform-level instance of the same "zero rows means nothing happened, or means something's broken" ambiguity DEH Part 25 §8 names for query schema drift.

SOC Management View A rule inventory review that only counts rules ("we have 140 analytics rules enabled") without also sampling their scheduling, threshold, and grouping settings is counting coverage that may not exist in practice — a rule enabled with a threshold and grouping combination that floods or a query that silently exceeds an NRT cadence isn't contributing detection value proportional to its presence on the list. Budget tuning review time proportional to rule count, not just deployment time; a rule that took an afternoon to author and never gets revisited is the most common source of both the false positive volume analysts complain about and the auto-disabled rules nobody notices went dark.

What Would Change My Mind This part treats "default to Scheduled; reach for NRT only when a specific, named latency requirement justifies its narrower feature set and its own per-workspace rule cap" as the correct default guidance. If Microsoft closed the KQL feature gap between Scheduled and NRT and substantially raised or removed the NRT-specific rule-count ceiling named in §4.1's PRODUCT VERSION NOTE, that guidance would need to soften toward "NRT is a reasonable default for any genuinely time-sensitive detection," since the remaining reason to prefer Scheduled — more configurability, not lower cost or lower reliability — would have mostly disappeared.

6. Data flow: from telemetry to incident

[CONCEPT] The diagram below places both rule types in the same data-flow position relative to ingestion and the incident queue — the difference between them is entirely in the cadence and feature-surface box, not in what happens before or after it.

flowchart LR

```

A[Data sources: connectors, DCRs] --> B[Log Analytics workspace tables, e.g. SigninLogs]
B --> C{Rule type}
C -->|Scheduled| D[Query runs on configured frequency, over configured lookback]
C -->|NRT| E[Query runs every 1 minute, fixed, restricted KQL subset]
D --> F[Threshold + event grouping]
E --> F
F --> G[Alert]
G --> H[Incident settings: alert grouping into an incident]
H --> I[Unified incident queue]
I --> J[Automation rule Part 13]
I --> K[Analyst triage Part 6]

```

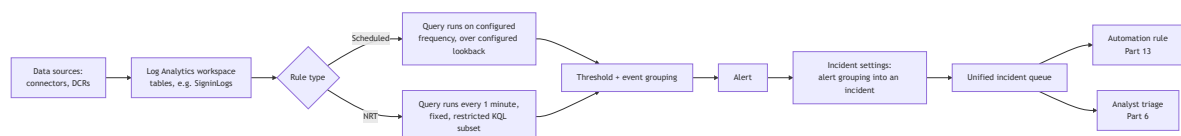


Figure 4.1 — Scheduled and NRT rule data flow into the unified incident queue. CONCEPTUAL. Illustrates where a Scheduled rule's configurable cadence and an NRT rule's fixed one-minute cadence

sit in an otherwise shared pipeline — the same ingestion layer, the same threshold and grouping mechanics, and the same downstream incident and automation surface described in §2–§5. This is a structural sketch built from this book's own reading of the rule-creation documentation cited throughout this part, not a capture of any Sentinel UI screen and not a reproduction of an official Microsoft architecture diagram. Rendering to a static image is pending a later production pass; the Mermaid source above is the editable version of record in the meantime.

Cross-references: DEH Part 25 §3 (lookback vs. frequency, the callout this part expands in full), DEH Part 25 §8 (query cost and schema-drift framing this part's §5 applies to rule volume instead of query volume), DEH Part 23 (Analytic vs. Detection Rule, and the canonical `lsass.exe` example this part's brute-force worked example parallels in structure); Part 2 (the Azure-portal-retirement date governing which portal hosts this wizard long-term); Part 5 (Fusion, Anomaly, ML Behavior Analytics, and Custom Detections — every rule type this part deferred); Part 6 (entity mapping and the incident model, full treatment); Part 13 (automation rules, the orchestration layer beyond §2.5's attachment point); Part 18 (rule health, auto-disable, and `SentinelHealth`).

Part 5 — Analytics Rules II: Fusion, Anomaly, ML Behavior Analytics, and Custom Detections

Why this part exists

[CONCEPT] Part 4 covered the two analytics rule types where the detection logic is a query you write and own: Scheduled rules (an analyst-authored KQL query running on a configurable cadence and lookback) and Near-real-time (NRT) rules (the same idea, fixed to a one-minute evaluation cadence with a narrower feature set — see Part 4). This part covers the rule types where that isn't true. Fusion and ML Behavior Analytics ship with the query hidden entirely — Microsoft's own documentation states plainly that Fusion's "logic is hidden and therefore not customizable." Anomaly rules sit one step less locked-down: the algorithm is fixed, but the thresholds it alerts on can be tuned without touching a query. And Custom Detections, the newest entrant, is a query you write again — but authored against a different, merged data surface that didn't exist when Parts 1 through 4 of this book's own outline was drafted.

[CONCEPT] DEH Part 23 frames a general problem: one analytic, expressed once, has to survive translation across however many query-language backends a detection engineering program actually runs. This part is that same problem's platform-operations mirror, and arguably a sharper version of it. DEH Part 23's translation loss happens once, at authoring time, and then holds still. The rule types in this part can have their *availability* — not just their translation fidelity — flipped off by a decision made in a completely different part of the organization: a Defender XDR licensing change, a portal-migration project with no connection to detection engineering, or simply the passage of a calendar date this book's Part 2 already named. A Fusion rule that was live and working yesterday can be gone today with no change to the rule itself, because the workspace it runs in was onboarded to the Microsoft Defender portal for an unrelated reason. That is this part's actual subject: which rule types exist, what's genuinely configurable about each one, and the specific, dated conditions under which four of them stop existing out from under you.

This part does not cover: KQL syntax itself (DEH Part 25); Scheduled and NRT rule anatomy, which Part 4 already owns; the `BehaviorAnalytics/IdentityInfo/UserPeerAnalytics` schema and peer-group baselining mechanics behind User and Entity Behavior Analytics (UEBA), which Part 11 owns in full — this part touches the `Anomalies` table only as far as Anomaly rules populate it, not as UEBA's own detection layer; or the CI/CD mechanics for deploying rules as code, which Part 7 owns and which §6 below points to for Custom Detections specifically.

1. The rule-type landscape: authored logic vs. fixed logic

[CONCEPT] Microsoft's own analytics-rule documentation splits its rule-type catalog into two groups, and this part follows that same split because it maps directly onto what an engineer can actually change. The first group — Scheduled rules, NRT rules, Anomaly rules, and Microsoft security rules (rules that automatically create Microsoft Sentinel incidents from alerts generated in other Microsoft security solutions, in real time) — are described as regular rule types you configure through the normal analytics-rule surface. The second group — Fusion (Advanced multistage attack detection — single

instance, hidden logic, unavailable once Defender XDR incident integration is on or the workspace is onboarded to the Defender portal), Machine learning (ML) behavior analytics (currently in Preview, also single-instance and non-customizable), and the Microsoft Threat Intelligence Analytics rule — are described as "specialized template types that can each create one instance of a rule, with limited configuration options" (Microsoft Learn, "Threat detection in Microsoft Sentinel," retrieved 2026-09-15). Anomaly rules are the interesting middle case: Microsoft groups them with the configurable rules on that page, but the underlying algorithm is exactly as fixed as Fusion's — what's configurable is a threshold, not the logic, and only after you duplicate the template (§4).

[SENTINEL ENGINEER] Table 5.1 supports one decision: given a detection need, which of these rule types can actually deliver it, and what does "configurable" mean for each one in practice.

Table 5.1 — Analytics rule types beyond Scheduled and NRT. Supports deciding which rule type fits a given detection need and what "configurable" actually buys you for each.

Rule type	Who owns the logic	Instance limit	Result lands in	Status (as of 2026-09-15)
Microsoft security	You (severity/text filter only, not detection logic)	One per Microsoft source service, more than one allowed if filters don't overlap	Incident directly, no separate alert	GA — unavailable once Defender XDR incident integration or Defender portal onboarding is on (§5)
Microsoft Threat Intelligence Analytics	Microsoft (fixed matching logic)	One	Alert, then incident	GA
Fusion (Advanced multistage attack detection)	Microsoft (fixed correlation; source-signal inclusion/exclusion configurable)	One	SecurityIncident directly, no SecurityAlert row	GA overall; several scenario-based detections in Preview (§2) — unavailable once Defender XDR incident integration or Defender portal onboarding is on
ML Behavior Analytics	Microsoft (fixed logic)	One per supported protocol	Alert, then incident	Preview
Anomaly	Microsoft (fixed baseline; thresholds tunable via a duplicated copy)	Many templates; each is single-instance until you duplicate it	Anomalies table only — no alert, no incident	GA
Custom Detection	You (advanced hunting KQL)	Tenant-level quota, not a single-instance cap	Alert, then incident; unified across Sentinel and Defender XDR data	GA as a Defender XDR feature; Sentinel-data scope and as-code deployment are newer (§6)

2. Fusion — advanced multistage attack detection

[CONCEPT] Fusion is a correlation engine, built into Microsoft Sentinel, that uses machine learning to detect multistage attacks by identifying combinations of anomalous behaviors and suspicious low-

fidelity signals across different stages of the kill chain and, by design, across different products (Microsoft Learn, "Advanced multistage attack detection in Microsoft Sentinel," retrieved 2026-09-15). It ships enabled by default as a single analytics rule literally named **Advanced multistage attack detection**, and because its correlation logic is hidden, there can only ever be one instance of it. A successful Fusion detection doesn't behave like a normal analytics-rule output: it's written directly to the `SecurityIncident` table as a Fusion incident, not to `SecurityAlert` as an alert that later gets grouped into an incident — Fusion skips the alert stage entirely. The stated design goal is low-volume, high-severity, high-fidelity incidents: each one is required to comprise at least two correlated alerts or activities, which is the mechanism behind "low-volume" — a single suspicious event, however severe, cannot become a Fusion incident on its own.

[SENTINEL ENGINEER] "Not customizable" doesn't mean "not configurable." You can enable or disable the rule, choose which source signals feed the Fusion ML model, and exclude specific detection patterns that don't apply to your environment — none of which touches the correlation logic itself (Microsoft Learn, "Advanced multistage attack detection in Microsoft Sentinel," retrieved 2026-09-15). Two named sub-capabilities extend Fusion beyond its default scenario library. **Fusion for emerging threats** (Preview) applies broader ML analysis across a wider signal set — out-of-the-box anomaly detections, alerts from a named list of Microsoft services (Microsoft Entra ID Protection, Defender for Cloud, Defender for IoT, Defender XDR, Defender for Cloud Apps, Defender for Endpoint, Defender for Identity, Defender for Office 365), and alerts from your own scheduled analytics rules — to surface threats with no pre-built scenario. **Fusion for ransomware** is narrower: it correlates alerts from Defender for Cloud, Defender for Endpoint, Defender for Identity, Defender for Cloud Apps, and scheduled analytics rules, specifically around the Execution and Defense Evasion stages, into an incident named **Multiple alerts possibly related to Ransomware activity detected**.

Blind Spot Both Fusion for emerging threats and Fusion for ransomware only consider your own scheduled analytics rules if those rules carry kill-chain (tactics) information and mapped entities (Microsoft Learn, "Advanced multistage attack detection in Microsoft Sentinel," retrieved 2026-09-15) — the same entity-mapping mechanics Part 6 covers in depth. A scheduled rule that fires constantly, with no tactic tagged and no entity mapped, is invisible to Fusion's cross-signal correlation even though it's generating alerts the entire time. This is a second, independent reason (beyond the False Positive Trap of an unfiltered rule) to treat entity mapping and tactic tagging as mandatory rule-authoring steps, not optional metadata — an unmapped rule doesn't just investigate worse on its own; it also can't contribute to the one correlation layer designed to catch what it misses.

Detection Test Setup: A workspace with Fusion enabled (its default state) and at least two of the data sources Fusion for ransomware consumes — for example, Microsoft Defender for Endpoint and Microsoft Defender for Cloud — actually connected and ingesting. **Action:** Trigger, or wait for, two or more distinct alerts from different connected sources on the same host within a short window, consistent with the Execution/Defense Evasion pattern Fusion for ransomware watches for — this is most reliably exercised with a scoped, approved simulation rather than live malware. **Expected result:** A single incident titled **Multiple alerts possibly related to Ransomware activity detected**, stored in `SecurityIncident`, distinct from — and not duplicating — the individual `SecurityAlert` rows each source alert produced on its own.

3. ML Behavior Analytics — anomalous SSH and RDP login detection

[CONCEPT] ML Behavior Analytics rules (currently in Preview) use Microsoft's own machine learning to flag anomalous SSH and RDP login behavior, scored against IP address, geolocation, and user history information (Microsoft Learn, "Threat detection in Microsoft Sentinel," retrieved 2026-09-15). Like Fusion, the logic isn't customizable, and each protocol gets its own single-instance rule — you enable it, you don't tune its query.

MITRE: T1021.001 (Remote Services: Remote Desktop Protocol), T1021.004 (Remote Services: SSH).

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's "Threat detection in Microsoft Sentinel" page states ML Behavior Analytics rules are "currently in Preview." Unlike Fusion and Microsoft security rules (§5), this same page's explicit unavailability callouts — the ones naming Defender XDR incident integration and Defender portal onboarding as disabling conditions — do not name ML Behavior Analytics. Treat that as "not documented as disabled," not as a confirmed guarantee: a Preview feature's availability list is exactly the kind of fact that can change between this chapter's last-validated date and whenever you're reading it. Re-check the same page before depending on ML Behavior Analytics rules surviving a Defender-portal migration unchanged.

[SENTINEL ENGINEER] The practical consequence of hidden, fixed logic is that there's no partial remedy for a bad fit. A scheduled rule with a threshold that's too sensitive for your environment gets a lookback or trigger-threshold adjustment; an ML Behavior Analytics rule that flags a distributed, frequently-traveling workforce's routine geolocation variance has exactly two states available to you — enabled, generating alerts you may not trust, or disabled, generating nothing.

Engineering Reality A fixed-logic rule's tuning surface is binary by design, not by an oversight Microsoft might eventually fix. The tradeoff you're accepting by enabling ML Behavior Analytics is: you get a detection you didn't have to build, tested by Microsoft against baseline attacker SSH/RDP behavior at a scale no single SOC's own dataset reaches — at the cost of not being able to adjust it when your own environment's normal doesn't match the assumption baked into the model. If that mismatch produces alert volume your team can't absorb, disabling the rule and building an equivalent, tunable Scheduled rule against `SignInLogs` / `DeviceLogonEvents` (Part 4) is a legitimate response, not a failure to use the built-in feature correctly.

4. Anomaly rules — tunable thresholds on a fixed ML baseline

[CONCEPT] Anomaly rules observe a specific type of behavior over an observation period to establish a baseline, then flag any occurrence that exceeds the boundaries that baseline sets (Microsoft Learn, "Threat detection in Microsoft Sentinel," retrieved 2026-09-15). Each out-of-the-box template ships with its own parameters and thresholds already tuned by Microsoft Sentinel's data science team — and, critically, anomaly rules do not generate their own alerts. A single anomaly is deliberately treated as weak evidence on its own; results are written to the `Anomalies` table, a dedicated tab on the Analytics page distinct from Active rules and Rule templates, for you to query, correlate, and act on deliberately rather than have surfaced as an incident automatically.

[SENTINEL ENGINEER] The out-of-the-box configuration of a template "can't be changed or fine-tuned" directly — the tuning path is to duplicate the template, then run the duplicate in **Flighting** mode alongside the unmodified original in **Production** mode, compare results, and promote the duplicate to Production once its performance satisfies you (Microsoft Learn, "Use customizable anomalies to detect threats in Microsoft Sentinel," retrieved 2026-09-15). This is the same underlying discipline as an A/B test: the original keeps running the whole time, so tuning the duplicate never leaves you with zero anomaly coverage for that behavior. A subset of anomaly detections comes specifically from the UEBA engine's peer-group and organizational baselining (Part 11 owns this scoring model in depth) — those rows land in the same `Anomalies` table alongside anomalies from Sentinel's other ML templates, distinguishable by template name, not by table.

```
// CONCEPTUAL SAMPLE – illustrative pivot on the Anomalies table, not validated against a
// live tenant. Verify the exact column set against your own workspace's Log Analytics
// schema browser before relying on field names here – this book has no live tenant to
// confirm them against (see Part 11 for the table's documented schema).
Anomalies
| where TimeGenerated > ago(7d)
| where AnomalyTemplateName has "Sign-in"
| project TimeGenerated, AnomalyTemplateName, Description, Score, CompromisedEntity
```

This query targets a workspace with Anomaly rule templates enabled and is meant as a threat-hunting pivot (Part 9), not a detection in itself; its main limitation is the one the section above already names — a row here is context for a hunt or an existing alert, not, by Microsoft's own design intent, a standalone verdict.

False Positive Trap Microsoft's own documentation is blunt about this: "Anomalies can be powerful tools, but they are notoriously noisy. They typically require a lot of tedious tuning for specific environments" (Microsoft Learn, "Use customizable anomalies to detect threats in Microsoft Sentinel," retrieved 2026-09-15). Treating a raw `Anomalies` row as an incident-worthy finding on its own reproduces exactly the noise problem the table's design already anticipates. The intended pattern is additive: use an anomaly to raise or lower confidence in a separate, already-triggered detection, or as a starting thread for a hunt — not as a rule that alerts by itself, which is a state Microsoft deliberately didn't build for the out-of-the-box templates.

Detection Autopsy — "we turned on the anomaly templates and got zero incidents" A team enables several out-of-the-box Anomaly rule templates expecting the usual outcome of turning on an analytics rule: alerts, then incidents, then something to triage. Nothing shows up in the incident queue. The postmortem conclusion isn't "the feature is broken" — it's that Anomaly rules were never going to produce an incident on their own; that's the one behavior explicitly and permanently designed out of this rule type (§4 above). The fix isn't debugging the rule; it's building the missing second half of the pipeline — a scheduled analytics rule or a hunting query that reads `Anomalies` and decides, deliberately, which combinations of anomaly plus something else should become an incident. Skipping that second half and expecting the anomaly template alone to carry the workload is the actual defect, not anything in Sentinel's configuration.

5. The availability trap: Defender XDR integration and Defender portal onboarding

[SOC MANAGEMENT] Two of the rule types this part covers can stop existing for reasons that have nothing to do with the rule. Microsoft's documentation states, for both Microsoft security rules and Fusion, the identical condition: each is "not available" if you have enabled Microsoft Defender XDR incident integration, or onboarded Microsoft Sentinel to the Microsoft Defender portal (in full: the Azure portal and Microsoft Defender portal are the two surfaces for operating Sentinel; see Part 2 for the full architecture and the retirement timeline). These are two independent triggers — either one alone is sufficient — and neither requires touching the rule itself. In both cases, the stated reason is the same: "Microsoft Defender XDR creates the incidents instead" (Microsoft Learn, "Threat detection in Microsoft Sentinel," retrieved 2026-09-15), via its own correlation engine, making Sentinel's Fusion and Microsoft-security incident-creation redundant with a system that now sits upstream of it.

Table 5.2 — Rule-type availability by portal. Supports deciding which rule types survive a Defender-portal migration unchanged and which need a replacement built before the switch is flipped.

Rule type	Azure portal (retiring 2027-03-31)	Defender portal
Microsoft security	Available; configurable per source service and severity filter	Not available — Defender XDR's correlation engine creates the incidents instead
Fusion (Advanced multistage attack detection)	Enabled by default	Not available — replaced by the Defender XDR correlation engine
ML Behavior Analytics	Preview, enabled	Not named in Microsoft's unavailability list as of 2026-09-15 (§3) — verify current status before relying on this
Anomaly	Available	Available — no correlation-engine overlap motivates disabling a rule type that never created incidents in either portal
Custom Detection	— (not a Sentinel Azure-portal rule type at all)	Available, including unified Sentinel and Defender XDR data (§6)

PRODUCT VERSION NOTE (as of 2026-09-15) Per Microsoft Learn's "Threat detection in Microsoft Sentinel" and "Advanced multistage attack detection in Microsoft Sentinel" pages, Microsoft security rules and the Fusion rule type are both unavailable once either Microsoft Defender XDR incident integration is enabled or the workspace is onboarded to the Defender portal — and "any such rules you had defined beforehand are automatically disabled." Both pages carry this fact as of this writing. If you're planning a Defender-portal transition (Part 16), re-check both pages for whether this list has grown or shrunk before you finalize a migration runbook.

Blind Spot "Automatically disabled" is silent by design — there's no incident, no alert, no health-monitoring event that specifically flags "your Microsoft security rule just stopped working because someone else enabled Defender XDR integration." If your team built Microsoft security rules with real severity filters — for example, a rule that only creates an incident from a high-severity Microsoft Defender for Identity alert, deliberately suppressing lower ones — that specific filtering intent doesn't carry over anywhere. Defender XDR's correlation engine has its own logic for which alerts become incidents, and it wasn't built to reproduce your rule's filter. A team that onboards to the Defender portal for an unrelated reason (unified advanced hunting, single-queue investigation) can lose that filtering behavior without anyone deciding to lose it.

What Would Change My Mind This book's default recommendation for a Sentinel-only shop planning a Defender-portal move is to treat the loss of Fusion and Microsoft-security auto-incident-creation as a migration task to complete before onboarding — rebuild the equivalent coverage as Scheduled rules or Custom Detections (§6), verify it fires the way the old rule did, and only then flip the switch — rather than a gap to discover afterward. If Microsoft reintroduced an equivalent, configurable incident-creation surface inside the Defender portal itself, instead of relying solely on the XDR correlation engine's own judgment, this recommendation would soften from "rebuild first" to "onboard, then evaluate parity" — a materially lighter migration-planning burden for the SOC manager owning Part 16's project.

6. Custom Detections — the unified rule-authoring surface

[CONCEPT] Custom detections are customizable detection rules, built from advanced hunting queries, that run at a configured interval and automatically trigger alerts and response actions (Microsoft Learn, "Overview of custom detections in Microsoft Defender XDR," retrieved 2026-09-15). They predate this book's subject matter — custom detection rules have existed in the Defender XDR advanced hunting experience for years — but what makes them belong in this part now is what changed once Microsoft Sentinel became reachable from the same Microsoft Defender portal.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's own documentation for Fusion, Anomaly rules, Microsoft security incident-creation rules, and the general "Threat detection in Microsoft Sentinel" overview all now carry an identical banner: "Custom detections is now the best way to create new rules across Microsoft Sentinel SIEM Microsoft Defender XDR. With custom detections, you can reduce ingestion costs, get unlimited real-time detections, and benefit from seamless integration with Defender XDR data, functions, and remediation actions with automatic entity mapping" (Microsoft Learn, retrieved 2026-09-15, appearing consistently across the pages cited elsewhere in this part). Strip the adjective and the falsifiable claim underneath is: for new detection content, Microsoft is now steering authors toward one rule-authoring surface that spans both platforms, rather than toward the older, platform-specific rule types this part otherwise documents. Nothing in any page cited in this part states that Scheduled or NRT rule creation (Part 4) is deprecated or scheduled for removal — the banner is a positioning statement about where to build new detections going forward, not a documented removal of the older surfaces. Re-check this banner's wording at the cited pages before treating it as a permanent policy rather than current guidance.

[SENTINEL ENGINEER] The mechanism behind that positioning is unified advanced hunting: once a Sentinel workspace is onboarded to the Microsoft Defender portal, the advanced hunting Schema tab shows that workspace's own tables — organized by solution, alongside the standard Defender XDR tables (`DeviceProcessEvents`, `IdentityLogonEvents`, and the rest of the catalog DEH Part 25 already documents) — and its saved functions and queries appear in the same interface, in folders marked **Sentinel** (Microsoft Learn, "Advanced hunting with Microsoft Sentinel data in Microsoft Defender,"

retrieved 2026-09-15). A custom detection rule authored from that unified query surface can therefore span both data worlds in one rule, with two documented limitations as of this writing: near-real-time detection frequency isn't available for a custom detection whose query includes Microsoft Sentinel data, and custom KQL functions created and saved in Microsoft Sentinel aren't supported inside a custom detection rule (same source). Both limitations disappear if the query stays entirely inside Defender XDR's own tables — the constraint is specifically about crossing the two data worlds inside one rule, not about custom detections generally.

PRODUCT VERSION NOTE (as of 2026-09-15) *The two cross-data-world limitations named above — no near-real-time frequency tier for a custom detection that references Sentinel data, and no support for Sentinel-saved custom KQL functions inside a custom detection rule — and the schema/folder integration mechanics they follow from are drawn from Microsoft's "Advanced hunting with Microsoft Sentinel data in Microsoft Defender" page as cited. This is the unified Sentinel/Defender-XDR advanced hunting surface Part 5's own scope note already flags as the fastest-changing ground in this book. Re-check that page directly before relying on either limitation still holding — a documented gap on an integration this new and this actively worked on is a likely candidate to shrink or disappear entirely on a future release, not a stable architectural boundary.*

Engineering Reality *Custom detection queries carry the same "keep it cheap" discipline DEH Part 25 §3 already establishes for a scheduled rule running on a tight cadence, plus one additional wrinkle this part's data adds: a query that blends Sentinel and Defender XDR tables doesn't just risk running slow — it forecloses the near-real-time frequency tier outright, by documented design, the moment it references a Sentinel table. A detection that genuinely needs sub-hour latency and also needs a Sentinel-only table has no custom-detection path to that combination as of this writing; it needs either a Sentinel-side NRT rule (Part 4) restricted to Sentinel data, or a redesign that keeps the time-sensitive half of the logic inside Defender XDR's own tables.*

[ENGINEERING] Custom detection rules can also be managed as code — currently in Preview — using the Microsoft Security Bicep extension, deployed either through Microsoft Sentinel's own Repositories feature for automatic sync or through the Bicep CLI for a custom pipeline (Microsoft Learn, "Overview of custom detections in Microsoft Defender XDR," retrieved 2026-09-15). Part 7 owns the CI/CD and rule-repository depth this implies; this part's job is only to flag that the path exists and that it's the same underlying deployment mechanism Part 7 already builds its detection-as-code case around, applied to a rule type this part covers.

7. Choosing a rule type

[SENTINEL ENGINEER] Collapsing §§1–6 into a single decision sequence: start from what the detection actually needs — a correlation across signals you already have, a fixed ML behavior you didn't have to

build, a tunable baseline you're willing to query rather than alert on directly, or a query you author yourself — and let the portal and latency constraints already covered above narrow the remaining choice.

flowchart TD

A["New detection need"] --> B["Correlating multiple existing
alerts/anomalies, not a single query?"]

B -->|Yes| C["Defender XDR incident integration on,
or Sentinel onboarded to Defender portal?"]

C -->|Yes| D["Rely on Defender XDR's own
correlation engine (Fusion unavailable)"]

C -->|No| E["Fusion – Advanced multistage
attack detection"]

B -->|No| F["Is the logic fixed ML,
not query-authored?"]

F -->|Fixed logic| G["Anomalous SSH/RDP
login specifically?"]

G -->|Yes| H["ML Behavior Analytics (Preview)"]

G -->|No, general baseline| I["Anomaly rule –
writes to Anomalies table only"]

F -->|I author the query| J["Sub-one-minute
evaluation cadence needed?"]

J -->|Yes| K["NRT rule (Part 4)"]

J -->|No| L["Need one authoring surface across
Sentinel and Defender XDR data?"]

L -->|Yes| M["Custom Detection"]

L -->|No / legacy workflow| N["Scheduled rule (Part 4)"]

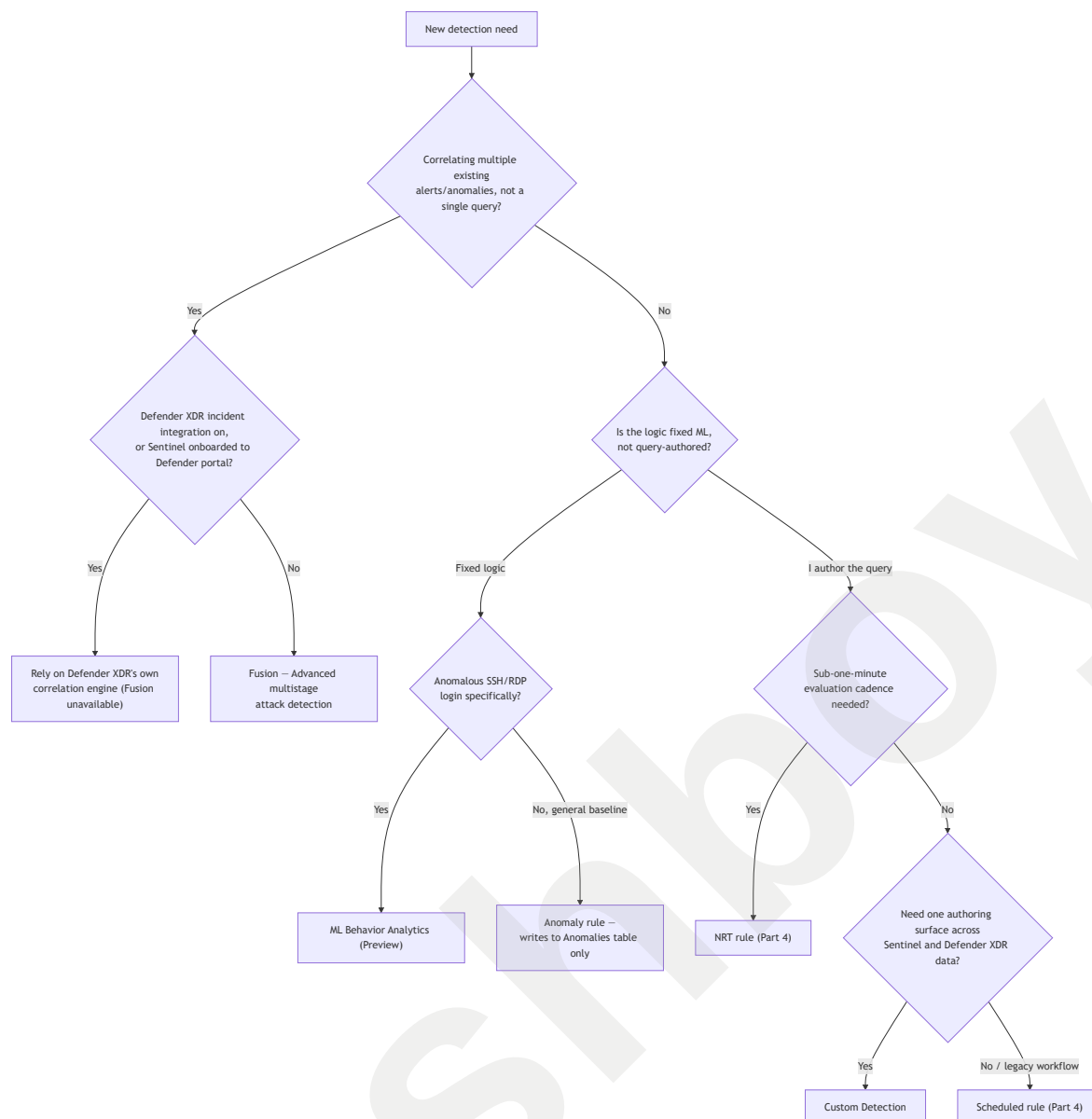


Figure 5.1 — Analytics rule type decision tree. CONCEPTUAL. Illustrates the decision sequence an engineer walks through when choosing among the rule types this part and Part 4 cover, based on latency need, customizability need, and Defender-portal onboarding status. This is a structural sketch built from this book's own reading of the official rule-type documentation cited throughout this part — not a capture of any Sentinel UI screen, and not a reproduction of an official Microsoft diagram.

[SOC MANAGEMENT] The one branch this diagram can't resolve on its own is the availability trap (§5): a decision made outside detection engineering entirely — onboarding to the Defender portal, turning on Defender XDR incident integration — can retroactively remove the Fusion and Microsoft-security branches from a workspace that was already relying on them. Walking this decision tree once, at authoring time, is necessary but not sufficient; Part 16's migration-planning discipline is what keeps the answer from silently going stale.

Cross-references: DEH Part 23 (one analytic, many backends — this part's availability trap is the platform-operations version of the same problem, where the "backend" is a portal-and-correlation-

engine choice rather than a query language); Part 2 (the two-portal architecture and the March 31, 2027 Azure-portal retirement date this part's availability facts depend on); Part 4 (Scheduled and NRT rules — the authored-query baseline every rule type in this part departs from in one direction or another); Part 6 (entity mapping and tactic tagging, required for a scheduled rule to be visible to Fusion for emerging threats and Fusion for ransomware); Part 7 (detection-as-code and the Repositories/Bicep deployment mechanics behind custom-detection-as-code); Part 9 (hunting mechanics behind the `Anomalies`-table pivot in §4); Part 11 (UEBA's `BehaviorAnalytics/IdentityInfo/UserPeerAnalytics` schema and scoring model behind the subset of `Anomalies` rows this part attributes to it); Part 16 (the Defender-portal migration as a program-management problem, not a toggle).

neshbooy

Part 6 — Entity Mapping and the Incident Model

Why this part exists

An analytics rule's KQL query, once it runs, produces rows and columns — `AccountName`, `SourceIP`, `Computer`, `SHA256`, whatever the query happens to `project`. None of those columns mean anything to Microsoft Sentinel's incident layer until something tells the platform which column represents which kind of real-world thing. That "something" is entity mapping: a per-rule configuration step, sitting between a query that matches and an alert that a human or an automation rule can act on, that turns an arbitrary result column into a typed entity — `Account`, `Host`, `IP`, and the rest of a defined schema Sentinel's incident-grouping, investigation-graph, and (where enabled) UEBA layers all read from.

This part covers the mechanics of that step and the incident model built on top of it: how a query column becomes a typed entity, how entities drive alert grouping into a single incident, what an analyst or threat hunter actually sees once entities are attached, and — the part of this that fails most often, and most quietly — what happens when the field chosen to represent an entity turns out not to uniquely identify it. This last problem is not new to this book. The *Detection Engineering Handbook* (DEH) names it as **Entity Resolution** in the abstract (TERMINOLOGY.md § Entity Key / Entity Resolution) and gives it a concrete, worked failure in DEH Part 25 §4, where a KQL join keyed on `DeviceId` alone silently misattributes an alert on a multi-user host. Entity mapping in Sentinel is that same problem, deployed as a standing platform feature rather than written by hand into one query — which makes it more convenient and, when curated carelessly, more consequential, because the same weak key now shapes every incident the rule ever produces, not just one query's output.

This part does not re-teach KQL join syntax (DEH Part 25) or correlation-window mechanics in the abstract (DEH Part 30) — it assumes both and covers the Sentinel-specific layer built on top of them: the entity schema itself, the rule-configuration surface that populates it, and the incident model that consumes it.

1. From query column to typed entity

[CONCEPT] A KQL query's result set is, to Kusto, just a table of columns with names and data types — a string here, a datetime there. Microsoft Sentinel's incident layer does not automatically infer that a column named `AttackerIP` holds an IP address entity any more than it infers that `AccountName` holds an `Account` entity; without an explicit mapping, both are just strings that happen to render in an alert's details pane. Entity mapping is the configuration step — performed once per analytics rule, on the rule's **Entity mapping** tab in the rule wizard — that names which output column corresponds to which entity type, and which of that entity type's identifier fields the column actually populates.

Once a column is mapped, every alert instance the rule produces carries structured entities alongside its raw column values, and those entities become the join key Sentinel's own platform logic uses for the rest of this part: alert grouping into incidents (§3), the investigation graph an analyst pivots through (§4), and — where the UEBA behaviors layer covered in Part 11 is enabled — behavioral baselining keyed on the same resolved entity across rules. Put in DEH's own vocabulary: entity mapping is the

practical, per-rule implementation of Entity Resolution (TERMINOLOGY.md § Entity Key / Entity Resolution) — the field or field combination asserting that two events, possibly from two different alerts or two different rules entirely, concern the same real-world `Account`, `Host`, or `IP`.

This configuration surface exists in **the Azure portal** and, once a workspace is onboarded, **the Microsoft Defender portal** — both terms given in full here on first mention, per this book's own notation convention, because "Azure" and "Defender" alone are also product names and the ambiguity matters more in this book than almost anywhere else. The Azure portal's rule wizard is the version described through this part; Part 16 covers what changes about rule configuration surfaces specifically once a workspace operates from the Defender portal instead.

[SENTINEL ENGINEER] Entity mapping applies to Scheduled and Near-real-time (NRT) rules (fixed one-minute evaluation cadence, limited subset of scheduled-rule features) — the rule types built directly on a KQL query, covered in Part 4 — the same way, through the same tab. Fusion (Advanced multistage attack detection — single instance, not customizable, unavailable once Defender XDR incident integration is on) and Anomaly rules, covered in Part 5, do not expose this tab at all: their entities are generated internally by Microsoft's own correlation or ML logic, not mapped by a rule author, which is one more reason Part 5 treats those rule types as a materially different authoring surface rather than a variant of the same one.

2. Sentinel's entity schema

[SENTINEL ENGINEER] Microsoft documents a defined set of entity types an analytics rule can map to, each with its own set of identifier fields — Microsoft Learn's "Map data fields to entities in Microsoft Sentinel" page and the companion entities reference page are the authoritative source for the full list; this part reproduces only the handful this book's examples return to most often.

Table 6.1 — Entity types this book's examples map to most often. The table below supports a Sentinel engineer's first decision when writing an entity mapping: which entity type a given query column actually represents, and which identifier field name Sentinel's own schema expects to receive it under. This is not the complete type or field list — `Azure resource`, `Cloud application`, `DNS`, `File`, `IoT device`, `Mailbox`, `Mail cluster`, `Mail message`, `Malware`, `Registry key`, `Registry value`, and `Security Group` all exist as mappable entity types beyond the six below — see the Microsoft Learn pages cited above for the rest.

Entity type	Typical source column	Representative identifier field(s)
<code>Account</code>	<code>AccountName</code> , <code>UserPrincipalName</code>	<code>Name</code> , <code>NTDomain</code> , <code>UPNSuffix</code> , <code>Sid</code> , <code>AadUserId</code>
<code>Host</code>	<code>Computer</code> , <code>DeviceName</code>	<code>HostName</code> , <code>NetBiosName</code> , <code>DnsDomain</code> , <code>AzureID</code>
<code>IP</code>	<code>SourceIP</code> , <code>RemoteIP</code> , <code>CallerIpAddress</code>	<code>Address</code>
<code>URL</code>	<code>RequestURL</code> , <code>Url</code>	<code>Url</code>
<code>FileHash</code>	<code>SHA256</code> , <code>MD5</code> , <code>InitiatingProcessSHA256</code>	<code>Algorithm</code> , <code>Value</code>
<code>Process</code>	<code>NewProcessName</code> , <code>InitiatingProcessFileName</code>	<code>ProcessId</code> , <code>CommandLine</code>

Microsoft's own documentation distinguishes identifier fields that are sufficient, on their own, to assert two rows concern the same entity (commonly called **strong identifiers** — an `IP` entity's `Address`, or an `Account` entity's `Sid`) from fields that only become reliable when combined with at least one other identifier for the same entity (an `Account` entity's bare `Name`, which needs `NTDomain` or `UPNSuffix` alongside it before Sentinel treats the match as trustworthy). Exactly which fields on which entity types currently carry which designation is a detail worth re-checking directly against the live rule wizard or the current Entities reference page before relying on it operationally — this book has no live tenant to confirm the current, complete strong/weak marking against (STYLE-GUIDE.md §9's honesty constraint), so treat the concept above as reliably real and the specific per-field marking as something to verify rather than take from this page alone.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's entity-mapping documentation ("Map data fields to entities in Microsoft Sentinel," part of the Microsoft Sentinel documentation on Microsoft Learn) currently documents a per-rule cap on entity mapping built through the Azure portal's rule wizard — up to 10 entity mappings per rule, each carrying up to three identifier fields (confirmed directly against learn.microsoft.com/azure/sentinel/map-data-fields-to-entities and the "Create scheduled analytics rules" page, current as of this check). This cap has moved before — this book's earlier drafts recorded a five-mapping limit — so treat "10 entities, three identifiers each" as current-as-checked rather than permanent, and re-confirm against the live rule wizard or the current Learn page if it's been a while since this chapter's `Last_validated` date. If the cap has since changed again, note that the rule's underlying ARM template already supports a JSON-based entity-mapping definition distinct from the wizard's own form fields; a rule that runs out of mapping slots in the portal UI may still be extendable by editing that definition directly rather than concluding the behavior can't be mapped at all.

3. Entity mapping and alert grouping into incidents

[SENTINEL ENGINEER] Mapped entities do more than decorate an alert's details pane — they are the join key Sentinel's own alert-grouping logic uses to decide whether a new alert extends an existing incident or opens a new one. Each analytics rule's **Incident settings** tab carries its own alert-grouping configuration: whether alerts this rule produces get grouped into incidents at all, and if so, whether grouping requires **all** mapped entities to match across two alerts or only a **selected subset**, within a bounded time window the rule itself defines. A rule with grouping turned off produces one incident per alert; a rule grouping on, say, a matching `Account` entity within a multi-hour window collapses every alert naming that same account inside the window into one incident instead of many.

[ANALYST] This matters directly to how an incident reads in the unified queue: two entirely different alert titles, from the same rule, sharing a grouped incident because they matched on entity, is normal and expected — it is the platform doing exactly what its grouping configuration told it to do, not a correlation-engine anomaly. What's worth checking, when an incident looks unexpectedly large or

unexpectedly narrow, is which entity type the rule actually grouped on and how wide its matching window is — both are visible on the rule's own configuration, not just inferable from the incident.

[SENTINEL ENGINEER]

This grouping mechanism is scoped to alerts from the *same* analytics rule. Combining alerts from *different* rules — or, once Defender XDR incident integration is on, from different Microsoft security products entirely — into one incident is a separate mechanism: Fusion's own multistage correlation (Part 5) on the Sentinel side, or Defender XDR's correlation engine once a workspace is onboarded to the Microsoft Defender portal (Part 15). Entity mapping still matters to both of those broader mechanisms — they still need a reliable entity to correlate on — but the per-rule Incident settings tab covered here is not the layer doing that wider correlation.

Engineering Reality Alert grouping into a single incident happens within a bounded time window the rule's own Incident settings define, not indefinitely. An intrusion that revisits the same entity after that window has already elapsed opens a second incident rather than extending the first, unless the rule's grouping configuration is also set to reopen a matching closed incident. This is the same class of silently-wrong-window problem DEH Part 30 §2 names for a correlation window measured against the wrong clock: the platform raises no error and drops no data — it simply produces two incidents where a human reviewing the underlying alerts side by side would see one continuous campaign.

4. The investigation graph and entity pages

[ANALYST]

Once an incident carries entities, Sentinel's incident page offers an **investigation graph** — opened from the incident's own investigate action — that renders the incident's alerts and their mapped entities as a connected graph, with expandable relationships an analyst can follow outward from any node: which other alerts, bookmarks, or related entities touch the same **Account**, **Host**, or **IP** this incident already named. This is the practical payoff of everything in §§1–3: an analyst working the incident is not reading a flat alert list and manually re-querying every entity by hand — the graph already reflects the entity resolution the rule author configured when they built the mapping.

Selecting an individual entity from that graph — or from any place in the portal where Sentinel renders an entity value as a link rather than plain text — opens an **entity page**: a dedicated view of that specific **Account**, **Host**, or **IP**'s own activity history, independent of any one incident. Where the UEBA behaviors layer covered in Part 11 is enabled, an entity page also surfaces behavioral insights for that entity; where it isn't, the page still shows the entity's alert and bookmark history drawn from ordinary entity-mapping data alone.

[ENGINEERING]

Two tables carry the underlying data this experience is built from, and they carry it differently, which matters if a query ever needs to reach entity data directly instead of through the graph UI: **SecurityAlert** carries each alert's mapped entities as a JSON-encoded column on the alert row itself, while **SecurityIncident** aggregates alerts by reference (an incident row names the alert IDs it contains) rather than duplicating each alert's entity payload onto the incident row directly. A KQL query reaching for "every entity on every alert in this incident" generally has to start from **SecurityIncident**,

resolve to the alert IDs it references, and pull entities from the corresponding `SecurityAlert` rows — not read them off the incident row in one step. Confirm current field names against the live table schema or Microsoft's data reference before building automation against this join directly; this book states the shape of the relationship with confidence and the exact column names with somewhat less.

PRODUCT VERSION NOTE (as of 2026-09-15) *The investigation graph and entity pages described above are Azure portal features as documented on Microsoft Learn's "Investigate incidents with Microsoft Sentinel" page. The Microsoft Defender portal presents its own incident-investigation surface once a workspace is onboarded, and this book has not verified feature-for-feature parity between the two — Part 16 owns that comparison in the depth it deserves. If you're reading this after that comparison exists, treat this part's description of the investigation graph as accurate to the Azure portal experience specifically, not as a claim about what an analyst sees after a Defender-portal migration.*

5. Where entity mapping breaks: identifiers that aren't actually keys

[SENTINEL ENGINEER] Every mapping in §2's table is a bet that the chosen identifier field actually, uniquely ties two events to the same real-world entity — and every entity-mapping failure this part cares about follows from that bet being wrong in a way that produces no error, no warning, and an incident that still looks entirely plausible.

Blind Spot *Entity mapping only asserts that two alerts share an entity if the mapped identifier field genuinely, uniquely identifies that entity — and the field most conveniently already present in a query's result set is often exactly the kind of field DEH Part 25 §4's `DeviceId` -on-a-multi-user-host example already names as unreliable for this purpose (see also `TERMINOLOGY.md` § Entity Key / Entity Resolution). Mapping a `Host` entity to a bare `Computer` or `DeviceName` value, with no `HostName` + `DnsDomain` combination or resource-scoped identifier behind it, means every alert that ever touches a shared jump box, a Citrix or VDI host, or a heavily multiplexed container node resolves to the same `Host` entity regardless of which of that host's several concurrent sessions or users actually produced each alert. The incident-grouping logic in §3 then has no way to distinguish "the same intrusion touched this host twice" from "two unrelated users happened to trigger unrelated alerts on the same shared machine" — both collapse into one incident, identically, once the entity key collapses them together. The same failure shape applies to `Account` entities mapped on a bare `Name` with no `NTDomain` / `UPNSuffix` alongside it: two different domains, or a cloud tenant and an on-prem forest, can genuinely have two different real people both named `jsmith`.*

[THREAT HUNTER] This is not purely a Sentinel-engineering problem to fix once and forget — it's also a standing reason a hunter should not trust an entity page's activity history as automatically complete or automatically precise, without checking which identifier field actually produced the resolution.

Hunter's Note *An entity page is a genuine hunting pivot, not just an incident-triage convenience — it surfaces every alert, bookmark, and (where Part 11's UEBA behaviors layer is enabled) behavioral insight tied to that resolved entity across every rule that ever mapped to it, not only the rule whose incident a hunter started from. A hunter chasing a single suspicious account or host across a suspected multi-stage intrusion generally gets further starting from that entity's own page than re-running the same filter by hand against each table Part 9 covers separately. The same §5 caveat applies while doing this: before treating a quiet entity page as evidence that "nothing else happened," confirm the entity was mapped on a genuinely unique identifier for the resource in question — a quiet page for a shared jump box's `Host` entity may just mean the actual attacker session never got a distinct identifier to resolve against in the first place.*

6. The operational cost: over-merged and under-merged incidents

[SENTINEL ENGINEER] Section 5's failure mode has two directions, and they feel opposite to an analyst even though both come from the same underlying cause — an identifier that doesn't actually uniquely key the entity it's mapped to.

False Positive Trap *Alert grouping by matching entities (§3) is a platform behavior, not a detection-logic decision, and it produces its own platform-shaped false-positive pattern: a shared service account, a NAT'd corporate egress IP, or a heavily used jump box, mapped as the matching entity across alerts with no real relationship to each other, pulls those alerts into one incident anyway, because the platform only evaluates "same entity, same time window" — it has no independent way to know the underlying activity isn't actually connected. The incident itself isn't wrong about what fired; its shape overstates how connected the underlying activity is, and an analyst opening it inherits a graph muddled with alerts that only ever shared a weak identifier, not a cause. Narrowing the rule's alert-grouping match from "any shared entity" to one specific, deliberately chosen entity type — the same allowlist-style discipline DEH's own False Positive Trap guidance already applies to detection logic — is the direct mitigation.*

The opposite direction produces no dramatic single incident to point at, which is exactly why it's easier to miss: a rule whose entity mapping is too narrow, or missing entirely, groups nothing, so an intrusion that trips the same rule five times becomes five separately opened, separately owned incidents with no shared context connecting them — each one individually unremarkable, collectively the same story DEH's own Correlation entry (TERMINOLOGY.md § Correlation) describes for events, now playing out one level up at incident scale.

SOC Management View Entity-mapping quality is easy to treat as a one-time rule-authoring checkbox, but its downstream cost shows up in analyst hours, not in the rule itself. An under-mapped rule produces five disconnected incidents for one intrusion, multiplying triage effort with no analyst realizing they're working the same event five times; an over-broadly-mapped rule produces one bloated incident that buries the one alert that mattered among unrelated activity riding along on a weak shared identifier. Neither failure shows up on a deployed-rule-count dashboard or a MITRE-coverage percentage — both describe whether a detection exists, not what shape its incidents take once it fires — so a SOC manager auditing platform quality has to ask about entity-mapping discipline explicitly, as its own review item, rather than trusting rule-count or coverage metrics to reflect it by proxy.

7. Data-flow summary

[CONCEPT] Figure 6.1 collects §§1–6 into one path: a query column becomes a typed entity through the rule's own entity-mapping configuration; that entity becomes the join key for alert-grouping into an incident, bounded by the rule's own grouping window; the resulting incident's entities populate the investigation graph an analyst pivots through; and the same resolved entity, independent of any one incident, is what a threat hunter reaches through an entity page.

flowchart TD

```
A[Analytics rule KQL query:<br>raw result columns] -->|Entity mapping tab, Part 4/5| B[Alert:<br>typed entities attached<br>Account, Host, IP, URL, FileHash, ...]
B --> C{Alert grouping settings:<br>matching entities within<br>the rule's own time window?}
C -->|match found, window open| D[Existing incident:<br>investigation graph extended]
C -->|no match, or window elapsed| E[New incident:<br>investigation graph seeded]
D --> F[Analyst: investigation graph]
E --> F
B -.same resolved entity,<br>any rule that ever mapped it.-> G[Entity page:<br>cross-rule alert<br>and<br>bookmark history]
G --> H[Threat hunter: pivot]
```

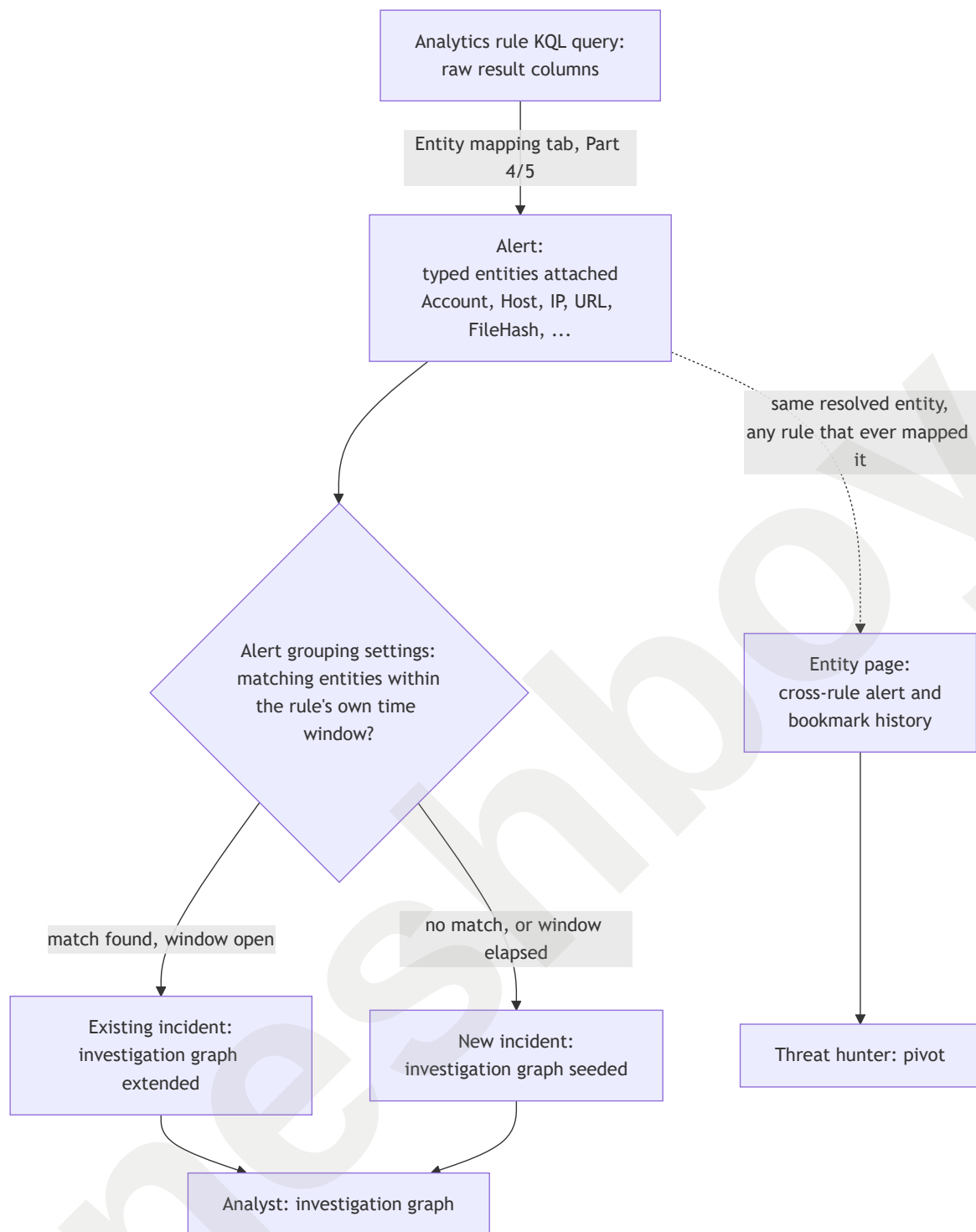


Figure 6.1 — From query column to incident entity graph. *CONCEPTUAL*. Illustrates the path an entity takes from a raw analytics-rule query column, through the entity-mapping configuration covered in §§1–2, into the alert-grouping and investigation-graph mechanics covered in §§3–4, and onward into the entity-page pivot a threat hunter uses independently of any single incident (§5–6's Hunter's Note). This is a structural sketch built from this book's own reading of the Microsoft Learn pages cited throughout this part — not a capture of any Sentinel portal screen, and not a reproduction of an official Microsoft diagram.

8. What this means for a rule built earlier in the book

[SENTINEL ENGINEER] Part 4 and Part 5's scheduled and NRT rule examples exist upstream of everything in this part: a query that returns a clean `AccountName`, `Computer`, or `SourceIP` column is doing the necessary but not sufficient work — the entity mapping tab is the step that actually connects that column to the incident model described here, and skipping it (or mapping a column to the wrong identifier field, per §5) is invisible at rule-authoring time and only surfaces later, as an incident that groups oddly or an entity page that returns less than expected. Reviewing a new rule's entity mapping belongs in the same authoring checklist as reviewing its query logic, not as an afterthought applied once an analyst first complains that two obviously related alerts landed in separate incidents.

Cross-references: DEH TERMINOLOGY.md § Entity Key / Entity Resolution; DEH Part 25 §4 (the `DeviceId`-on-a-multi-user-host join example this part's Blind Spot box mirrors at incident scale); DEH Part 30 (Correlation Engineering — entity keys and correlation windows, the event-level version of this part's incident-level problem); Part 4 (Analytics Rules I — the Scheduled/NRT rules entity mapping attaches to); Part 5 (Analytics Rules II — Fusion/Anomaly rules' internally-generated entities); Part 9 (Hunting in Sentinel and the Defender Portal); Part 11 (UEBA — the behaviors layer entity pages surface when enabled); Part 15–16 (Defender XDR Signal Convergence / portal migration — the broader, cross-product correlation and portal-parity questions this part explicitly defers).

Part 7 — Detection Content Lifecycle: Content Hub, Solutions, and Detection-as-Code for Sentinel

Why this part exists

DEH Part 25's own scope note is explicit about what it leaves out: "Sentinel analytics-rule YAML, CI/CD wrapper metadata, or rule-repository structure" — naming this part as where that detail belongs. DEH Part 22 already made the general case for Detection-as-Code: a Detection Rule is software with a lifecycle, and every failure mode application engineering solved decades ago (an untested change shipping broken, no record of who changed what, no rollback path) happens to a detection rule too, usually with less visibility, because a broken rule doesn't crash — it just goes quiet. This part does not re-argue that case. It answers the platform-specific question DEH Part 22 leaves open on purpose: what does author, reviewer, CI, CD, and rollback actually look like once the rule in question is a Microsoft Sentinel analytics rule, automation rule, workbook, or playbook, sitting in a Log Analytics workspace instead of an abstract repository?

Two distinct lifecycles converge in this part, and confusing them is the single most common source of the "why did my rule change" question a Sentinel engineer eventually asks. The first is **Content Hub**: Microsoft's and its partners' packaging system for out-of-the-box content, where installing a solution gives a workspace templates, not running detections, and updating that solution later touches the templates without touching anything an engineer already built from them. The second is **Repositories**: Sentinel's own native implementation of a source-controlled CI/CD pipeline, where a connected GitHub or Azure DevOps repository becomes the declared source of truth for custom content, and the platform's own sync mechanism — not a locked UI — is what keeps a portal-side edit from surviving past the next deployment. This part treats both as first-class, then gives a Sentinel engineer the specific ARM/Bicep resource types, portal terminology, and drift mechanics needed to reason about which lifecycle a given piece of content is actually living in.

Scope boundary: this part does not re-teach KQL (DEH Part 25) or the generic CI/CD case for detection-as-code (DEH Part 22) — it assumes both and supplies the Sentinel-specific mechanics on top. It does not cover the rule-type decision itself (Scheduled vs. NRT vs. Fusion vs. Custom Detection) — that's Part 4/5's territory; a rule template installed from Content Hub can be any of those types, and this part's packaging and versioning story applies uniformly across them.

1. Content hub and solutions: the packaging model

[CONCEPT] The Microsoft Sentinel **Content hub** is the in-portal catalog for out-of-the-box (OOTB) content, reachable from **Content management > Content hub** in both the Azure portal and the Microsoft Defender portal. Content arrives in one of two shapes: a **solution** — a packaged bundle of related content that supports one product, domain, or industry-vertical scenario — or a **standalone item** — an individual piece of content installed and kept current on its own, outside any bundle. The solutions experience itself is built on Azure Marketplace, the same mechanism Azure uses to distribute managed applications generally, which is why a solution's detail pane shows a publisher, a support

model, and a plan ID the way a marketplace listing does (Microsoft Learn, "Microsoft Sentinel Content and Solutions Overview," learn.microsoft.com/azure/sentinel/sentinel-solutions, retrieved 2026-09-15).

A solution can bundle any combination of eight content types Microsoft's own documentation enumerates: analytics rules, data connectors, hunting queries, parsers, playbooks (and their Logic Apps custom connectors), watchlists, workbooks, and summary rule templates (a newer content type for pre-built, cost-reducing aggregation rules — see Part 17 for what a summary rule is actually solving). The table below states, per content type, what installing a solution actually deposits into the workspace and whether a further step is needed before that content does anything.

Content type	What installation deposits	Needs a separate activation step?
Analytics rules	A read-only rule template in the analytics template gallery	Yes — Create rule
Data connectors	A connector configuration page	Yes — connect/configure credentials
Hunting queries	A read-only query template	Runnable as-is; Clone to customize/persist
Parsers	A workspace-scoped KQL function in Log Analytics	No — usable immediately as a function
Playbooks	A Logic Apps workflow template	Yes — Create playbook
Watchlists	A watchlist template/schema	Yes — populate or connect the data source
Workbooks	A gallery template	Yes — Save to create an editable instance
Summary rule templates	A summary rule template	Yes — deploy as an active summary rule

[SOC MANAGEMENT] Every piece of content also carries a **support model** — Microsoft-supported, partner-supported, or community-supported — visible on its detail pane, and this is a real operational fact, not a label: partner- and community-supported content is maintained by whoever authored it, and an issue with it goes to that party's support channel or the Microsoft Sentinel GitHub community's issue tracker, not to an Azure support ticket the way Microsoft-supported content does (same source). Most Content Hub solutions ultimately trace back to the public [Azure/Azure-Sentinel](#) GitHub repository, which is also where an engineer who wants to see a template's raw YAML/JSON before installing it, or convert one into a Repositories-managed file (§4), goes looking first.

Content sources: the label the portal actually uses

[SENTINEL ENGINEER] The portal itself tags every piece of content with one of four **content source** values, and this table is the one worth memorizing before touching anything in §2 or §4, because it's the fastest way to answer "who owns this object, and what will silently change it next."

Content source	How it got there	What updates it
Solution	Installed as part of a Content Hub solution	The solution's own Update action in Content hub — refreshes the template, not any content already created from it
Standalone	Installed individually from Content Hub, outside a solution	Kept current automatically by the platform
Custom	Authored or edited directly in the workspace (portal or API)	Nothing but a person, until it's exported into a repository
Repositories	Deployed by a connected GitHub/Azure DevOps sync	The next commit-triggered deployment from that repository

(Microsoft Learn, "Microsoft Sentinel Content and Solutions Overview," §"Content sources for Microsoft Sentinel content and solutions," same URL as above.)

2. From template to active object: instantiation and drift

[SENTINEL ENGINEER] Installing a solution is not the same act as turning on a detection, and the distinction matters more than it sounds like it should. An analytics rule template sits in the gallery, inert, until an engineer opens it and selects **Create rule**; a playbook template needs **Create playbook**; a workbook template needs **Save**. Each of those actions produces a genuinely independent object — a live **alertRules** resource, a **Logic Apps** workflow, a saved workbook instance — that the Content Hub tracks separately, under a **Created content** count on the template's own detail page, but no longer manages as the same thing as the template.

Blind Spot "Installed" and "protecting you" are two different claims, and Content Hub's own UI is honest about the gap once you know where to look — the **Created content** column on a template can legitimately read zero on a solution that's shown as fully installed for months. A SOC that treats "we deployed the Insider Risk solution" as equivalent to "we have working detections for insider risk" without checking that count is reporting a coverage number the platform never actually claimed. The template gallery is a shelf, not a running rule; every solution rollout should end with someone confirming which templates in it were actually turned into **Create rule** objects, not just that the solution's own status column says installed.

[SENTINEL ENGINEER] The reverse gap is the one that causes more quiet damage over time. Microsoft's own documentation states the update behavior for a solution plainly: "Any active or custom content created based on solutions or standalone content installed from content hub remains untouched" by a solution update (Microsoft Learn, "Discover and deploy Microsoft Sentinel out-of-the-box content from Content hub," learn.microsoft.com/azure/sentinel/sentinel-solutions-deploy, retrieved 2026-09-15). That sentence is the entire drift story in this part in one line: when Microsoft or a partner ships a better version of a template — a fixed field reference, a tightened threshold, an added entity mapping — a rule an engineer already created from an earlier version of that template does not move. It sits exactly where it was tuned, permanently, unless a person notices and manually reconciles it.

Engineering Reality The one durable trace of which template version a rule came from is its own `templateVersion` property (format `<major>.<minor>.<patch>`, alongside `alertRuleTemplateName` naming the template itself) — both real, documented fields on a `Scheduled` alert rule's ARM representation (Microsoft Learn, ARM template reference for `Microsoft.SecurityInsights/alertRules`, [Learn.microsoft.com/azure/templates/microsoft.securityinsights/alertrules](https://learn.microsoft.com/azure/templates/microsoft.securityinsights/alertrules), retrieved 2026-09-15). Nothing in the platform proactively compares that stamped version against the template's current version and pages anyone when they diverge — the comparison, if it happens at all, is a person opening the template gallery, noticing an `Update` indicator on the parent solution, and then separately checking whether any of their own created rules still reference the older version. A workspace with forty solution-derived rules and no scheduled review of this kind has, in practice if not in name, forty forked copies of upstream content that nothing keeps in sync.

3. Exporting and representing content as infrastructure

[ENGINEERING] Every content type a Content Hub solution or a hand-built rule produces is, underneath the portal, an ordinary Azure resource, and Sentinel supplies both a purpose-built export path and the generic Azure Resource Manager (ARM) tooling every Azure resource already has. Analytics rules have their own `Export` feature reachable from the analytics rule list (documented as "Import and export analytics rules in Microsoft Sentinel"), with an equivalent PowerShell path for scripted export; automation rules have the same pairing. Playbooks, being ordinary Logic Apps resources, export through a Logic Apps-specific ARM-template generator rather than a Sentinel-native button. Workbooks export through Azure Monitor Workbooks' own "automate" tooling, which produces the ARM template for a workbook instance or gallery template. The table below states the underlying resource type behind each content type — the piece of vocabulary that turns a portal screenshot into something version-controllable.

Content type	ARM/Bicep resource type	Reference
Analytics rules	<code>`Microsoft.SecurityInsights/alertRules`</code>	Sentinel ARM template reference, <code>alertRules</code>
Automation rules	<code>`Microsoft.SecurityInsights/automationRules`</code>	Sentinel ARM template reference, <code>automationRules</code>
Hunting queries, Parsers	<code>`Microsoft.OperationalInsights/workspaces/savedSearches`</code>	Log Analytics ARM template reference, <code>savedSearches</code> — both content types deploy through the same Saved Searches API
Playbooks	<code>`Microsoft.Logic/workflows`</code>	Azure Logic Apps ARM template reference
Workbooks	An Azure Monitor Workbooks resource (commonly <code>`Microsoft.Insights/workbooks`</code> / <code>`workbooktemplates`</code>)	Azure Monitor Workbooks "automate" documentation
Custom detection rules (Preview)	<code>`Microsoft.Security/detectionRules`</code>	Requires the Microsoft Security Bicep extension — see §4.1

Table built from the content-type-to-resource-type mapping Microsoft Learn's "Manage custom content with repository connections" page states directly (learn.microsoft.com/azure/sentinel/ci-cd-custom-content, retrieved 2026-09-15). The hunting-query/parser row is worth reading twice: neither content type has a Sentinel-specific resource type of its own — both ride on the same generic Log Analytics saved-search mechanism, which is also why selecting either one as a Repositories content type (§4) silently deploys both if a connected branch contains files of both kinds.

[SENTINEL ENGINEER] The `alertRules` resource's `kind` discriminator (`Scheduled`, `Fusion`, `MicrosoftSecurityIncidentCreation`, and — in earlier API versions — `NRT`, among others Part 4/5 covers by behavior rather than by schema) is exactly the kind of fast-moving, version-specific detail this book flags rather than freezes into a table meant to outlive its own review date.

PRODUCT VERSION NOTE (as of 2026-09-15) The `Microsoft.SecurityInsights/alertRules` resource's set of valid `kind` values, and the exact property shape under each `kind`, is defined per API version and has changed across the many versions Microsoft has shipped for this resource type (the ARM template reference page lists dozens, from `2019-01-01-preview` through `2025-09-01` as of this writing). A Bicep or ARM file written against one API version's schema is not guaranteed to deploy cleanly, or to expose the same `kind` values, against a different one. Before committing a rule definition to a repository (§4), pin an explicit `apiVersion` and check the live reference page at learn.microsoft.com/azure/templates/microsoft.securityinsights/alertrules for the `kind` values and property names that version actually supports — don't carry a `kind` value forward from an older sample without re-verifying it against the pinned version.

The following excerpt shows the `templateVersion` and `alertRuleTemplateName` fields from §2 in place on a `Scheduled` rule, targeting the `Microsoft.SecurityInsights/alertRules` resource type at API version `2025-09-01`:

```
// CONCEPTUAL SAMPLE – illustrative property usage, not validated against a live workspace.
// Confirm apiVersion-specific property availability against the live reference before deploying.
resource driftAwareRule 'Microsoft.SecurityInsights/alertRules@2025-09-01' = {
  name: guid('det-07-01-suspicious-oauth-grant')
  kind: 'Scheduled'
  properties: {
    displayName: 'Suspicious OAuth app consent grant'
    enabled: true
    severity: 'Medium'
    query: '<KQL query body – see DEH Part 25 for syntax; omitted here per STYLE-GUIDE.md §3>'
    queryFrequency: 'PT1H'
    queryPeriod: 'PT1H'
    triggerOperator: 'GreaterThan'
    triggerThreshold: 0
    suppressionEnabled: false
    suppressionDuration: 'PT5H'
    alertRuleTemplateName: '<template GUID this rule was created from>'
    templateVersion: '1.2.0'
  }
}
```

Its main limitation, beyond the omitted query body: this snippet declares `templateVersion` as a static string, but nothing about writing that value here makes it self-updating — it only ever reflects the version that was true at the moment this file was authored or exported, which is precisely why §2's Engineering Reality box calls the comparison a manual one.

4. Microsoft Sentinel Repositories: the platform's own CI/CD

[ENGINEERING] DEH Part 22 built the generic case for a git-based rule repository: the console becomes a deployment target, not an editing surface, and the file in version control becomes the actual source of truth. **Repositories** — reachable from **Content management > Repositories** in both portals — is Sentinel's own native implementation of exactly that model, for custom content rather than Content Hub solutions. A connection binds one GitHub or Azure DevOps branch, a chosen subset of content types, and a target workspace together; creating the connection generates a GitHub Actions workflow or Azure DevOps pipeline directly inside the connected repository, and every subsequent push to that branch runs it (Microsoft Learn, "Deploy Custom Content from your Repository,"

learn.microsoft.com/azure/sentinel/ci-cd, retrieved 2026-09-15).

[SENTINEL ENGINEER] The direction of authority is stated as plainly as Content Hub's update behavior was in §2: "Updates you make to the content in your Microsoft Sentinel repositories are synchronized to your Microsoft Sentinel workspace and overwrite any changes you make to that content through the Microsoft Sentinel portal. Your Microsoft Sentinel repositories become your *single source of truth*" (Microsoft Learn, "Manage custom content with repository connections,"

learn.microsoft.com/azure/sentinel/ci-cd-custom-content, retrieved 2026-09-15).

Engineering Reality *"Single source of truth" here is a behavioral guarantee enforced by the sync itself, not a locked field in the portal UI — an analyst can still open a repository-managed analytics rule in the Azure portal or the Defender portal and change its threshold, save it, and see the change take effect immediately. What changes nothing about that fact: the next time the connected branch's workflow runs, the deployed definition in the repository file overwrites that edit with no warning dialog and no diff shown to the person it affects. The platform's own guidance is to edit repository-managed content only in the repository, and if a portal edit is unavoidable, to export it back into the repo before the next sync — a discipline problem the tooling makes possible to violate, not one it prevents.*

Each connection is scoped to specific content types from the same list §1's solutions can bundle — analytics rules, automation rules, custom detection rules (Preview, see §4.1), hunting queries, parsers, playbooks, and workbooks — with one shared quirk: because hunting queries and parsers both ride the Log Analytics Saved Searches API (§3's table), selecting either one as a managed content type deploys both if the branch contains files of both kinds. A workspace supports at most five repository connections, and because every deployment is itself an Azure Resource Manager deployment, it counts against the hosting resource group's retained deployment-history limit of 800 — a high-volume content repository with frequent commits can exhaust that quota and surface a `DeploymentQuotaExceeded` error with no connection to anything obviously wrong in the content itself (same source, "Maximum connections and deployments"). Bicep is Microsoft's own recommended format over raw ARM JSON for this content, but Bicep support is gated on connection age: a connection created before a documented cutoff date has to be removed and recreated before Bicep files will deploy through it, and Bicep's own limitation — no `id` property — means an analytics rule exported from the portal (which includes `id`) needs that field stripped before it will decompile cleanly (same source, §"Deploy Bicep files").

PRODUCT VERSION NOTE (as of 2026-09-15) *Microsoft Learn's "Manage custom content with repository connections" page states the Bicep-deployment cutoff as a specific date (documented there as November 1, 2024 at last check) — any connection created before that date has to be removed and recreated before it will accept Bicep files. A fixed cutoff tied to a connection's creation timestamp is exactly the kind of narrow, easy-to-mis-copy fact that ages badly in a book with no live tenant to re-verify it against; re-check the live page's "Deploy Bicep files" section for the current cutoff (and whether it has been superseded or removed entirely) before treating a connection's Bicep eligibility as settled.*

[ENGINEERING] By default, a connection runs **smart deployments**: the generated pipeline tracks which files actually changed since the last run (via an audit file in a `.sentinel` folder in the repository) and redeploys only those, both for performance and specifically to avoid clobbering a rule's own runtime state — resetting a dynamic schedule field on every sync, for instance — on content nothing in the commit touched. Turning this off forces a full redeploy of every managed file on every trigger, which is available as an explicit customization for a team that wants it, not the default (Microsoft Learn, "Manage custom content with repository connections," §"Improve performance with smart deployments").

Custom detection rules as code (Preview)

[SENTINEL ENGINEER] Repositories' newest content type breaks from every other row in §3's resource-type table on purpose. Custom detection rules — Defender XDR's unified detection surface Part 5 covers in full — deploy through `Microsoft.Security/detectionRules`, a different resource provider entirely from `Microsoft.SecurityInsights`, and require the Microsoft Security Bicep extension — published at `br:mcr.microsoft.com/bicep/extensions/microsoftsecurity`, pinned to a specific version (`v1.0.1` as of this writing; see the paired PRODUCT VERSION NOTE below) — declared in a `bicepconfig.json` at the repository root. A minimal definition:

```
// CONCEPTUAL SAMPLE – illustrative structure per Microsoft Learn's Preview documentation;
// not validated against a live workspace. Preview limitations apply – see prose below.
extension MicrosoftSecurity

resource driftDetectionRule 'Microsoft.Security/detectionRules@2026-06-01-preview' = {
  id: 'det-07-02-custom-rule'
  displayName: 'Example custom detection rule'
  status: 'enabled'
  queryCondition: {
    queryText: '<Advanced Hunting KQL – see DEH Part 25 for syntax>'
  }
  schedule: {
    frequency: 'PT1H'
  }
}
```

Deploying this either through a Repositories sync or directly via `az deployment group create` requires a Microsoft 365 E5 license (or an equivalent license that includes Microsoft Defender XDR) and a Sentinel workspace already onboarded to the Microsoft Defender portal (Microsoft Learn, "Manage custom content with repository connections," §"Deploy custom detection rules as code (Preview)").

```
az deployment group create \
  --resource-group <RESOURCE_GROUP> \
  --template-file detectionRule.bicep \
  --name detection-rule-deployment
```

This command targets a resource group containing a Defender-portal-onboarded Sentinel workspace; its main limitation, beyond Preview status itself, is that Microsoft's documentation names two specific gaps as of this writing — custom query frequency for Sentinel-sourced data isn't supported, and custom details aren't supported — either of which can silently make a ported rule behave differently from the Sentinel-native analytics rule it was meant to replace.

PRODUCT VERSION NOTE (as of 2026-09-15) Custom detection rules support inside Repositories is Preview, governed by Azure's Supplemental Terms of Use for Previews, and depends on both an M365 E5-class license and Defender-portal onboarding — none of which is required for the `Microsoft.SecurityInsights`-based content types in §3's table. Separately, Microsoft has published that starting June 2026, older Repositories management API versions stop being supported; anyone managing connections through the API rather than the portal needs to be on API version `2025-09-01`, `2025-06-01`, or `2025-07-01-preview` by June 15, 2026 (existing connections themselves are stated as unaffected). Both facts are drawn from "Manage custom content with repository connections" ([Learn.microsoft.com/azure/sentinel/ci-cd-custom-content](https://learn.microsoft.com/azure/sentinel/ci-cd-custom-content)), retrieved 2026-09-15. The Microsoft Security Bicep extension's pinned version (`v1.0.1` above) is a third fast-moving detail from the same page and the same Preview surface — a Preview-stage extension gets version bumps on its own cadence independent of the API-version and GA-date facts above, so confirm the current extension version in `bicepconfig.json` examples on the live page rather than reusing the one printed here. If you're reading this after either date, re-check the Preview's GA status and the currently supported API version before relying on either claim.

What Would Change My Mind This part treats Content Hub instantiation (§2) as the right starting point for baseline coverage and a Repositories-managed Bicep pipeline (§4) as the right home for an organization's genuinely custom, high-value detections — two different lifecycles, deliberately kept separate. If custom detection rules as code exits Preview and becomes Microsoft's primary unified authoring path across both Sentinel and Defender XDR, that separation weakens: a team writing net-new detection logic would have less reason to start from a Sentinel-specific `AlertRules` template at all, and this part's guidance would need to shift toward "author as a Defender XDR custom detection rule from day one" rather than "start from a Content Hub template and repository-manage the ones worth keeping."

5. Choosing a content strategy for a SOC

[SOC MANAGEMENT] Nothing in §1 through §4 says a workspace has to pick exactly one lifecycle. In practice, the two coexist by design, and the decision that actually matters is which lifecycle a given piece of content should live in, not which one the workspace uses exclusively.

flowchart TD

```
A["Content hub solution<br/>Microsoft-, partner-, or<br/>community-supported"] -->|"Install / Update"| B["Workspace template:<br/>analytics rule, hunting query,<br/>workbook, or playbook template"]
B -->|"Create rule / Save workbook /<br/>Create playbook / Clone query"| C["Active content object<br/>alertRules, savedSearches,<br/>workbooks, Logic Apps workflow"]
C -->|"Export rule / Export template"| D["Bicep or ARM file<br/>committed to a repo"]
D -->|"git push"| E["Generated GitHub Actions workflow<br/>or Azure DevOps pipeline<br/>Repositories connection"]
E -->|"deploy: repo is the<br/>declared source of truth"| C
G["Analyst edits C<br/>directly in the portal"] -.->|"overwritten on the<br/>next repo-triggered sync"| C
H["Microsoft/partner updates<br/>the upstream template"] -.->|"Update button refreshes B only –<br/>C is left untouched"| B
```

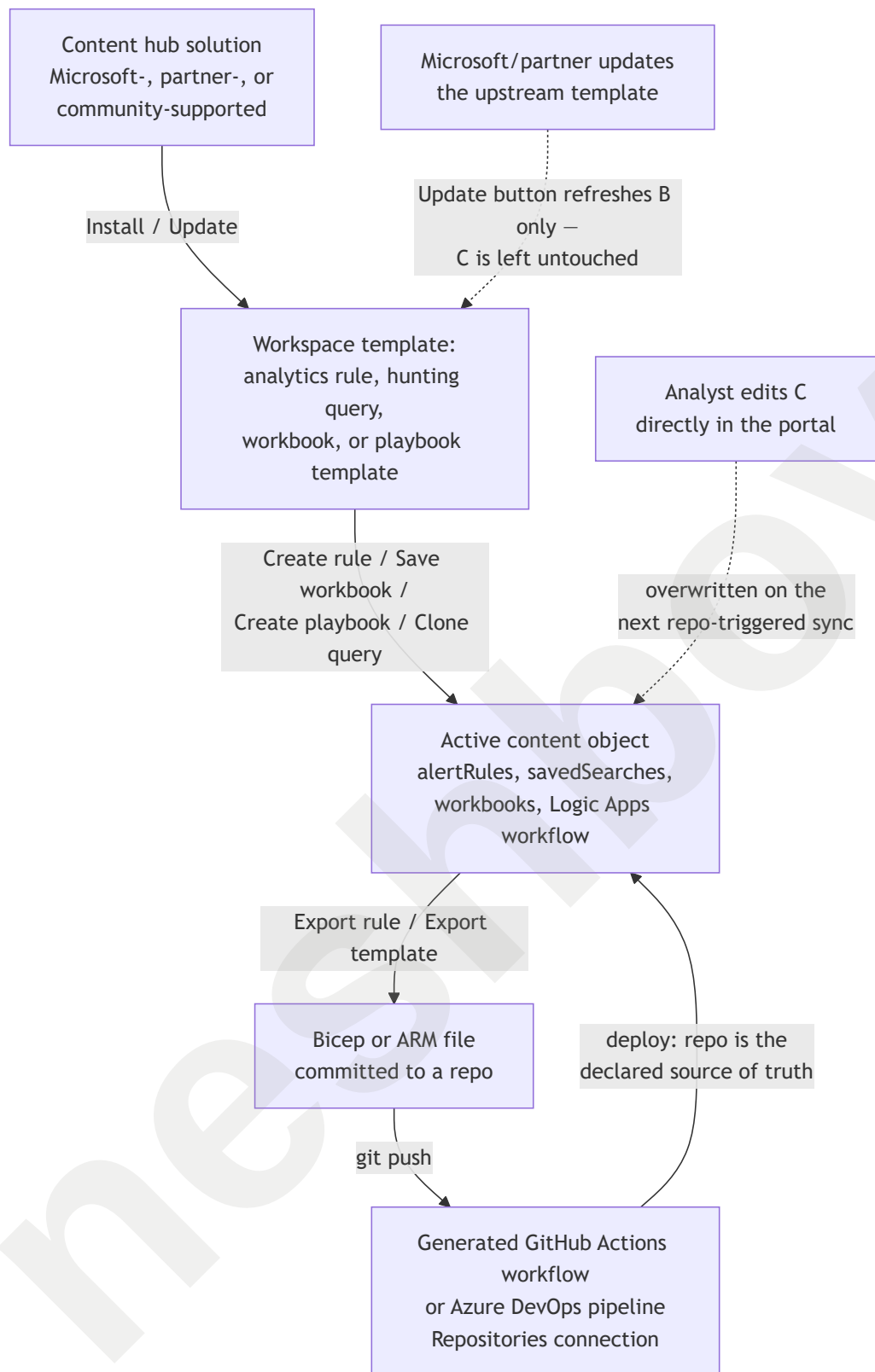


Figure 7.1 — Detection content lifecycle: Content Hub instantiation and Repositories-managed CI/CD, side by side. CONCEPTUAL. Illustrates two paths content takes into a running Sentinel workspace — installing and instantiating a Content Hub template, versus syncing Bicep/ARM files from a connected repository — and the two drift points (dashed edges) each path creates: a solution update that refreshes the template but never touches an already-created rule (§2), and a portal edit to a

repository-managed object that survives only until the next sync (§4). This is a structural sketch built from this book's reading of the Microsoft Learn pages cited in §1–§4, not a capture of any Sentinel UI screen.

[SOC MANAGEMENT] A small team with limited detection-engineering headcount gets real, ongoing value from leaning on Content Hub for baseline coverage: Microsoft- and partner-maintained templates absorb the maintenance burden of keeping pace with a connected product's own schema and API changes, and a solution's `Update` button is materially cheaper than rebuilding equivalent logic in house. The cost that comes with it is exactly §2's drift problem — every `Create rule` click produces one more workspace-local object that nothing but a person keeps in sync with its upstream template, and a growing count of solution-derived rules with no review cadence is the same Documentation/Testing Debt problem DEH Part 22 §9 names for any detection repository, just without the pipeline that would normally surface it. A more mature program's answer is to graduate its highest-value, most heavily customized logic — the rules an incident response has actually depended on, or the ones tuned enough times that "what does this actually do now" is no longer obvious from the template it started as — into a Repositories connection specifically so DEH Part 22's lint, test, and two-role review gates apply to it going forward.

Budgeting for either path has real prerequisites worth naming rather than discovering mid-rollout: installing or updating Content Hub content requires the **Microsoft Sentinel Contributor** role at the resource-group level, while creating a Repositories connection requires an **Owner** role (or an equivalent custom role with permission to create role assignments) on the same resource group, plus write access on the GitHub or Azure DevOps side of the connection (Microsoft Learn, "Deploy Custom Content from your Repository," learn.microsoft.com/azure/sentinel/ci-cd, and "Manage custom content with repository connections," learn.microsoft.com/azure/sentinel/ci-cd-custom-content, retrieved 2026-09-15 — check the current page for the exact named role on each side of the connection, since Azure RBAC role names and required GitHub/Azure DevOps access levels are exactly the kind of detail that gets renamed without a version-note-worthy announcement). The custom-detection-rules-as-code path in §4.1 adds a licensing prerequisite (M365 E5-class) on top of the portal-onboarding one. None of these is a large ask individually, but a rollout plan that assumes whoever clicks "Create rule" today can also "just connect a repository" tomorrow, without checking who actually holds Owner on the resource group, is a common way a detection-as-code initiative stalls in its first week for a reason that has nothing to do with the content itself.

Cross-references: DEH Part 22 (Detection as Code — the generic pipeline this part's Repositories mechanics implement), DEH Part 25 (KQL syntax and its own scope note deferring rule-repository structure to this part), Part 4/5 (analytics rule types and the Custom Detection Rules surface §4.1 previews), Part 6 (entity mapping — the `entityMappings` property this part's ARM examples reference), Part 16 (Defender-portal onboarding, a prerequisite for §4.1), Part 17 (summary rule templates, mentioned in §1's content-type table).

Section C — Investigation, Hunting, and Reference Data

neshboy

Part 8 — Workbooks: Visualization, Reporting, and SOC Dashboards

Why this part exists

DEH Part 25 already teaches the query surface a workbook's query steps are built from — the KQL pipe model, `where / join / summarize`, and the Sentinel/Defender table catalog those operators run against — and DEH Part 23 already frames the broader tradeoff between authoring a portable analytic and authoring a native, backend-specific query. This part assumes both and does not re-teach either one. What DEH Part 25 explicitly leaves alone is what happens to a KQL query once it stops being a `where / summarize` chain typed into a query box and becomes a saved, shared, parameterized report that a SOC runs every shift — the object that hosts it, where that object is stored, who is allowed to change it, what breaks when it's viewed from a different portal than the one it was built in, and where its usefulness as a metrics surface actually stops.

That object is a Microsoft Sentinel workbook. This part covers: what a workbook actually is underneath Sentinel's own gallery view (§1–2); where a workbook's template comes from and what Microsoft ships out of the box (§3); how a workbook differs from Sentinel's other built-in report surface, the Overview dashboard (§4); the specific, documented behavior gaps between the Azure portal and the Microsoft Defender portal that a workbook author has to plan around (§5); the operational mechanics of running a workbook in production — refresh behavior and access control (§6); and, following this book's own honesty constraint about evidence (see `STYLE-GUIDE.md` §9), the honest limits of treating a workbook's rendered number as a validated metric rather than whatever its underlying query happened to compute (§7).

1. A workbook is an Azure Monitor object wearing a Sentinel skin

[CONCEPT] Microsoft's own documentation states this directly: "Microsoft Sentinel workbooks are based on Azure Monitor workbooks, and add tables and charts with analytics for your logs and queries to the tools already available in Azure." (Microsoft Learn, "Visualize your Data by using Workbooks in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/monitor-your-data, retrieved 2026-09-15.) Sentinel does not have its own, separate dashboarding engine — every Sentinel workbook is an Azure Monitor Workbooks resource that Sentinel's own **Threat management > Workbooks** page filters, tags, and presents inside a Sentinel-specific gallery. The same underlying resource is equally reachable from the general Azure Monitor gallery (**Monitor > Workbooks** in the Azure portal, or the **Workbooks** tab on a Log Analytics workspace page) — Sentinel's gallery is a curated view onto it, not a different product.

That shared lineage matters operationally in one specific way: "each workbook is an Azure resource like any other, and you can assign it with Azure role-based access control (RBAC) to define and limit who can access" it (same source). A workbook saved from inside Sentinel is stored *within the Sentinel workspace's resource group* and tagged by the workspace it was created in — it does not live inside the Log Analytics workspace resource itself, and it does not store any data of its own. Saving a workbook "creates an Azure resource in the selected location based on the relevant template. Only the workbook's

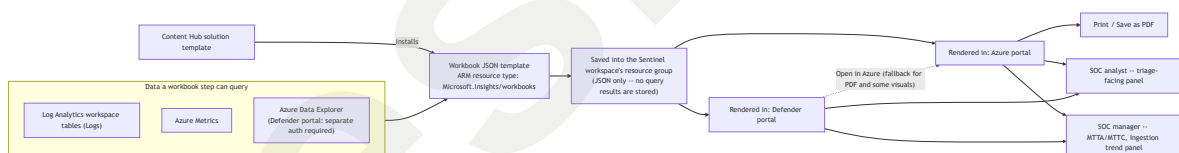
JSON file is saved in this location, and no data" (same source). A saved workbook is a template plus whatever parameter defaults it was last left with — every time someone opens it, every query step re-runs live against the workspace. There is no cached snapshot, no "as of" timestamp baked into the resource, and (see §6) no background refresh unless someone is actively viewing it with auto-refresh turned on.

Figure 8.1 — Where a Sentinel workbook's data comes from and where it's consumed.

CONCEPTUAL. Illustrates the relationship between a workbook's query sources, the underlying Azure Monitor resource, Content Hub packaging, and the two portals a rendered workbook can be viewed from — a structural sketch built from this part's own reading of the Microsoft Learn pages cited throughout §1, §3, and §5, not a capture of any Sentinel UI screen.

flowchart LR

```
subgraph SRC["Data a workbook step can query"]
    LA["Log Analytics workspace tables (Logs)"]
    MET["Azure Metrics"]
    ADX["Azure Data Explorer<br/>(Defender portal: separate auth required)"]
end
CH["Content Hub solution template"] -->|installs| WB
SRC --> WB["Workbook JSON template<br/>ARM resource type: Microsoft.Insights/workbooks"]
WB --> RG["Saved into the Sentinel workspace's resource group<br/>(JSON only -- no query results are stored)"]
RG --> AZP["Rendered in: Azure portal"]
RG --> DEF["Rendered in: Defender portal"]
AZP --> PDF["Print / Save as PDF"]
DEF -->| "Open in Azure (fallback for PDF and some visuals)" | AZP
AZP --> ANALYST["SOC analyst -- triage-facing panel"]
AZP --> MGR["SOC manager -- MTTA/MTTC, ingestion trend panel"]
DEF --> ANALYST
DEF --> MGR
```



2. Anatomy of a workbook: steps, parameters, and the resource underneath

[SENTINEL ENGINEER]

A workbook's editing surface is step-based: a text step for narrative context, a query step (data source set to **Logs**, resource type **Log Analytics**, one or more target workspaces) for a KQL result rendered as a grid, chart, or other visualization, a parameters step (a time-range picker, a dropdown driven by its own query, a free-text filter) that later steps can reference, and grouping constructs — groups and tabs — that let several steps sit inside one collapsible section or one page of a multi-page report. Microsoft's own authoring guidance for a custom query step is specific and worth repeating verbatim: "We recommend that your query uses an [ASIM] parser and not a built-in table. A query that uses an ASIM parser supports any current or future relevant data source rather than a single data source" (Microsoft Learn, "Visualize your Data by using Workbooks in Microsoft Sentinel," cited above). That recommendation is the workbook layer's version of the same normalization discipline DEH's own coverage of ASIM already argues for at the detection-rule layer — a workbook query built

against a single raw table (`SecurityEvent`, say) silently stops updating the day that source is replaced or supplemented by another one, while an ASIM-parser-backed query keeps rendering.

Saved, a workbook becomes an ARM resource of type `Microsoft.Insights/workbooks` — the same resource type any Azure Monitor Workbook uses, confirmed by the specific permission string (`microsoft.insights/workbooks/write`) Microsoft's RBAC documentation names as the write privilege a custom role needs to edit and save one (Microsoft Learn, "Azure Workbooks overview," learn.microsoft.com/azure/azure-monitor/visualize/workbooks-overview, retrieved 2026-09-15). The sketch below shows that resource's outer shape — not a working template, and not the place to look for KQL syntax, which DEH Part 25 already owns.

The following is a schematic excerpt of an ARM template exporting a `Microsoft.Insights/workbooks` resource; the `serializedData` value is itself a JSON string holding the step array described above, shown here only as a placeholder.

```
// CONCEPTUAL SAMPLE -- illustrative shape only, not a deployable template.
{
  "type": "Microsoft.Insights/workbooks",
  "apiVersion": "2022-04-01",
  "name": "[parameters('workbookId')]",
  "location": "[resourceGroup().location]",
  "kind": "shared",
  "properties": {
    "displayName": "SOC shift handover",
    "category": "sentinel",
    "sourceId": "[parameters('workspaceResourceId')]",
    "serializedData": "{ ...step array as an escaped JSON string... }"
  }
}
```

The limitation this excerpt doesn't show, and that DEH Part 25 §3/§8 covers in full: whatever KQL lives inside that `serializedData` string inherits every table-schema and query-performance concern an ordinary hunting or detection query does — a workbook step is not exempt from a slow join or a schema-drifted field just because it renders as a chart instead of a grid.

3. Where a workbook's template comes from

[SENTINEL ENGINEER] Most workbooks a SOC actually uses didn't start from a blank canvas. Three paths, all documented on the same Microsoft Learn page cited in §1:

- **Bundled with a Content Hub solution.** Installing a connector solution (the Microsoft Entra solution, a firewall vendor's solution, and so on) frequently installs one or more workbooks alongside it — this is Part 7's Content Hub packaging model applied specifically to workbooks rather than to rules or playbooks.
- **Installed standalone from the Content Hub**, without the rest of a solution's connector or analytics-rule content.
- **Built from scratch**, using **Threat management > Workbooks > Add workbook**, then **Edit** to add text, query, and parameter steps directly.

For a template installed either way, the **Templates** tab on the Workbooks page surfaces a **Required data types** field before you save it — a template built against Microsoft Entra sign-in logs renders nothing useful in a workspace that never onboarded that connector, and this field is the documented way to check that before wiring it in.

[ANALYST] The table below names a handful of workbooks and native views Microsoft's own documentation confirms by name, to ground "a workbook exists for this" in something more specific than a generic claim. It is not an exhaustive catalog — Content Hub's workbook gallery changes solution by solution — and Appendix A1's quick-reference table is the place for a fuller, independently re-verified list.

Name	Ships with / access path	What it shows
Microsoft Entra sign-ins	Microsoft Entra solution (Content Hub)	Sign-in trends over time; failed sign-ins broken out by application, device, and location
Microsoft Entra audit logs	Microsoft Entra solution (Content Hub)	Admin activity — user and group creation, modification, and other directory changes
SOC efficiency	Built in; linked directly from the Overview dashboard's Incidents section (see §4)	Mean time to acknowledge and mean time to close incidents, and related incident-handling metrics
Vendor firewall workbooks (for example, Palo Alto)	Installed with the vendor's own Content Hub solution	Firewall traffic correlated against threat events — illustrative of the general per-connector workbook pattern, not specific to one vendor
MITRE ATT&CK view	Native Sentinel page, linked from the Overview dashboard's Analytics section — not a Workbooks-gallery template	Which ATT&CK tactics and techniques currently have at least one active analytics rule mapped to them

Microsoft's own recommendation for how many workbooks a SOC should build is worth stating because it cuts against the instinct to build one "master" dashboard: "create different visualizations for each type of persona that uses workbooks, based on the persona's role and what they're looking for" — a network admin's firewall-focused workbook is a different artifact from a Tier 1 analyst's triage workbook, which is different again from the **SOC efficiency** workbook a manager checks weekly (same source as above). The same guidance extends the split by cadence, not just by role: build the workbook someone checks hourly (a live sign-in-anomaly view, say) separately from the one someone checks daily or weekly, rather than cramming both refresh expectations into one canvas whose auto-refresh interval (§6) can only be set once.

That last row is deliberately flagged as an exception: it looks like a workbook from a distance — a rendered, ATT&CK-shaped visualization inside the same product — but it's a purpose-built page Sentinel maintains itself, not a customizable template a SOC engineer edits the way they'd edit any row above it. §4 draws that distinction out further, and §7 comes back to exactly what this view's number does and doesn't prove.

4. Two different report surfaces: the Overview dashboard and a workbook

[ANALYST] Sentinel ships a second, non-workbook reporting surface: the **Overview** dashboard, a fixed layout of precalculated widgets Microsoft's own documentation walks through section by section (Microsoft Learn, "Use the Microsoft Sentinel Overview dashboard to view incidents, data, and

analytics," learn.microsoft.com/azure/sentinel/get-visibility, retrieved 2026-09-15). Unlike a workbook, its four sections — Incidents, Automation, Data, Analytics — are not editable steps; they're a fixed set of rollups Microsoft computes and refreshes on its own schedule, with the last refresh time shown per section and a manual **Refresh** available for the whole page. Where you find it differs by portal, following the same two-portal split Part 2 covers in full: "If your workspace is onboarded to the Microsoft Defender portal, select **General > Overview**. Otherwise, select **Overview** directly" (same source).

The Incidents section shows new/active/closed counts over the last 24 hours, severity totals, closing-classification counts, and — the figure most likely to end up on a manager's slide — mean time to acknowledge (MTTA) and mean time to close (MTTC), linked directly to the **SOC efficiency** workbook from §3. The Automation section states its own arithmetic openly: time saved by automation is $(avgWithout - avgWith) * resolvedByAutomation$, where `avgWithout` and `avgWith` are the average incident-resolution time without and with automation, and `resolvedByAutomation` is the count of incidents automation resolved. The Analytics section shows analytics-rule counts by status — enabled, disabled, and auto-disabled (the auto-disable mechanism itself, and why a rule ends up in that state, is Part 18's territory) — with a direct link labeled **MITRE view** into the ATT&CK page named in §3's table.

SOC Management View *The Automation section's time-saved formula is real arithmetic on real incident-resolution timestamps, not an invented number — but it is a modeled estimate (an average difference multiplied by a count), not a controlled before/after measurement of the same analyst doing the same incident with and without automation. Presenting `avgWithout - avgWith` to leadership as "hours of analyst time saved this quarter" is accurate to what the formula computes; presenting it as an audited, causally-isolated result is a claim the formula itself doesn't support. A SOC manager quoting this figure upward should say what it is: a documented estimate, not a controlled experiment.*

5. Portal parity: what changes once a workspace is onboarded to the Defender portal

[ENGINEERING] Workbooks are one of the areas where the Azure portal and the Microsoft Defender portal genuinely diverge in documented, specific ways — not just in menu layout. Four gaps Microsoft's own workbook documentation states directly:

Behavior	Azure portal (retiring 2027-03-31)	Defender portal
Print workbook / Save as PDF	Available, from the workbook's options menu	Not available — select Open in Azure to use it there instead
Full rendering of every visualization type	Supported	A subset of visualizations "can only be viewed in the Azure portal"; the workbook UI surfaces an Open in Azure link as the documented fallback
Overview dashboard navigation	Select Overview directly	Select General > Overview
Azure Data Explorer as a workbook data source	Configured in the Azure portal's own context	Must be separately configured and authenticated <i>from inside the Defender portal</i> before the step will run

(All four rows: Microsoft Learn, "Visualize your Data by using Workbooks in Microsoft Sentinel," cited in §1, retrieved 2026-09-15.)

PRODUCT VERSION NOTE (as of 2026-09-15) *Microsoft Sentinel in the Azure portal is scheduled for retirement on March 31, 2027; after that date Sentinel is reachable only through the Microsoft Defender portal, and workspaces still on the Azure portal are redirected automatically (Microsoft Learn states this identically on the workbooks page, the Overview-dashboard page, and the roles page cited throughout this part — all retrieved 2026-09-15). For workbooks specifically, that date is the deadline for the **Open in Azure** fallback in the table above: print/PDF export and any visualization type that only renders in the Azure portal lose their escape hatch entirely once that portal is gone, not just their preferred path. If you're reading this after that date, treat this table as a historical record of a gap that no longer has a workaround, not as a live how-to.*

What Would Change My Mind *This part treats "don't build a recurring PDF-export or Azure-portal-only-visualization workflow on top of Sentinel workbooks" as the right default for any team still relying on the Azure-portal fallback today. If Microsoft ships print/PDF parity or full visualization parity in the Defender portal before the March 31, 2027 retirement, that recommendation would soften from "migrate this workflow off workbooks now" to "the fallback simply moved portals, no separate action required" — a materially different message for a SOC manager planning around a fixed report a stakeholder already depends on.*

6. Operating a workbook in production

Auto-refresh — the fixed-interval, session-bound refresh model

[ENGINEERING] A workbook refreshes its query steps in one of two ways: a manual **Refresh** button, or **Auto refresh**, configurable to "supported auto refresh intervals rang[ing] from 5 minutes to 1 day" (Microsoft Learn, "Visualize your Data by using Workbooks in Microsoft Sentinel," cited above). Two

behaviors in that same documentation matter more than the interval range itself: auto-refresh is off by default, and it "is turned off again each time you close the [workbook] to optimize performance and prevent it from running in the background" — it has to be turned back on explicitly the next time the workbook is opened, and it pauses while the workbook is being edited, restarting only when the view switches back from edit mode (or on a manual refresh).

Engineering Reality *Treat a workbook the way a NOC wants to treat any wall-mounted dashboard — build it once, point a browser at it, and forget it — and the auto-refresh behavior above will quietly stop updating the display the first time someone closes that tab, restarts the browser, or the session times out. There's no server-side scheduled refresh independent of an open, in-view session: the workbook only recomputes while something is actively rendering it. A durable SOC video-wall workbook needs either a kept-alive, unattended browser session that nobody closes, or a different delivery mechanism entirely (a scheduled export, or a Power BI report built from the same underlying Log Analytics data) — not a workbook someone configured once and walked away from.*

Power BI — the scheduled, distributed alternative to a workbook

[SOC MANAGEMENT] The Engineering Reality box above names Power BI as the documented escape hatch from the "someone has to keep a browser tab open" problem, and it's worth stating precisely rather than leaving as a gesture. Log Analytics' own **Logs** query editor has an **Export** menu with two Power BI options: "Power BI (as an M query)," which exports a query to a Power Query (M-language) script for loading into Power BI Desktop, and "Power BI (new Dataset)," which creates a dataset directly in the Power BI service — from which reports can be shared, refreshed on a schedule, and (with a Power BI Pro or Premium license) refreshed incrementally rather than re-pulling the full result set every time (Microsoft Learn, "Log Analytics integration with Power BI," learn.microsoft.com/azure/azure-monitor/logs/log-powerbi, retrieved 2026-09-15). This is a genuinely different tool with a genuinely different job: a workbook is a live, interactive, ad hoc canvas an analyst edits and re-queries in the moment; a Power BI report built this way is a scheduled, distributable artifact for stakeholders who never touch the Sentinel portal at all. A SOC reporting to a monthly leadership meeting should be building that report in Power BI, not screenshotting a workbook — the export path above is the documented bridge between the two, not a workaround this book is inventing.

RBAC — who can see a workbook, and who can change it

[ENGINEERING] Workbook access rides on top of two independent role layers, both Azure RBAC. The first is Sentinel's own built-in role set, which grants workbook visibility as part of a broader Sentinel permission set:

Role	Workbook-relevant grant
Microsoft Sentinel Reader	View data, incidents, workbooks, recommendations, and other resources
Microsoft Sentinel Responder	All Reader permissions, plus incident management — no additional workbook-editing rights on its own
Microsoft Sentinel Contributor	All Responder permissions, plus install/update Content Hub solutions and create/edit resources
Workbook Contributor	Not a Sentinel role — an Azure Monitor role granting <code>microsoft.insights/workbooks/write</code> ; layered on top of whichever Sentinel role a user already holds

(Microsoft Learn, "Roles and permissions in the Microsoft Sentinel platform," learn.microsoft.com/azure/sentinel/roles, retrieved 2026-09-15.) The documented task-to-role mapping on that same page is worth stating precisely rather than paraphrasing: **creating or deleting a workbook requires Microsoft Sentinel Contributor (or a lesser Sentinel role) and Workbook Contributor** — the two roles are additive, not substitutes for each other. The same page notes the inverse case explicitly, with a footnote on its role-to-task table: a **Microsoft Sentinel Reader** can still edit analytics rules, workbooks, and similar resources if that same user is separately granted **Workbook Contributor** — the Reader role alone is read-only, but it isn't a hard ceiling on what a narrower, additional grant can unlock. Part 2 covers the broader resource-context-vs-workspace-context RBAC model these roles sit inside; this section states only the workbook-specific slice of it.

7. The honest limits of a workbook as a metrics surface

[SOC MANAGEMENT] Everything in §1–6 describes mechanics — how a workbook is stored, rendered, refreshed, and access-controlled. None of it says anything about whether the number a workbook renders is a *good* number, and that gap is where this part's scope stops and DEH's own coverage, quality, and debt model (DEH Parts 41–43) picks up. A workbook — or the MITRE ATT&CK view named in §3 — can render "38 of 200 ATT&CK techniques have an active analytics rule mapped to them" with complete technical accuracy: the count is real, the query genuinely counted what it says it counted. What that count cannot tell you, because it isn't designed to, is whether each of those 38 rules reliably fires when the technique it maps to actually happens (DEH Part 42's Detection Quality concern), whether any of them have quietly rotted since the last time someone validated them against current telemetry (DEH Part 43's Detection Debt concern), or whether "has a rule mapped" is even the right bar for calling a technique covered at all (DEH Part 41's Detection Coverage framing, which explicitly separates "a detection exists" from "the detection works"). A workbook renders whatever its query computes, and it inherits every scoping mistake in that query exactly the way any other KQL query does — DEH Part 25 §4's warning about a shared identifier that isn't actually a unique key applies just as much to a workbook's `summarize` step as it does to a detection rule's.

SOC Management View *A polished, auto-refreshing workbook is genuinely persuasive in a way a raw KQL result grid isn't — clean charts read as authoritative even when the underlying query hasn't been reviewed as carefully as the analytics rule it's summarizing. Before a coverage or quality metric goes into a leadership deck sourced from a Sentinel workbook, confirm it was built (or reviewed) against the same rigor DEH Parts 41–43 apply to a coverage claim made anywhere else — a dashboard rendering a number cleanly is not the same claim as a quality program having validated that number.*

Cross-references

DEH Part 25 (KQL syntax and the Sentinel/Defender table catalog underneath every workbook query step) · DEH Part 23 (Query Language Strategy — why a workbook's ASIM-parser recommendation in §2 is the same portability tradeoff at a different layer) · DEH Part 41 (Detection Coverage) · DEH Part 42 (Detection Quality) · DEH Part 43 (Detection Debt) — all three referenced in §7 · this book's Part 2 (Log Analytics Workspace Architecture and the Two Portals — the RBAC and portal-split model §5 and §6 apply specifically to workbooks) · this book's Part 3 (Data Ingestion — ASIM parsers referenced in §2) · this book's Part 7 (Detection Content Lifecycle — Content Hub packaging referenced in §3) · this book's Part 17 (Workspace Cost Models and Retention Tiers — what a workbook query against a Data lake tier table costs beyond the Analytics tier's included query capacity) · this book's Part 18 (Sentinel-Specific Tuning, Health Monitoring, and Troubleshooting — the auto-disabled rule count the Overview dashboard surfaces in §4).

Part 9 — Hunting in Sentinel and the Defender Portal

Why this part exists

DEH Part 34 already teaches why a SOC hunts at all (the gap between "what a detection rule catches" and "what a rule was never written to catch"), and DEH Part 35 already teaches the taxonomy of hunt types (structured, unstructured, situational) that gives a hunt its shape before a single query runs. DEH Part 36 closes the loop DEH's own way, teaching how a validated hunt finding becomes a candidate detection rule as a general discipline, independent of any one platform. This part assumes all three and does not re-teach any of them. What it supplies instead is the platform mechanics underneath that lifecycle, specifically as Microsoft Sentinel and the Microsoft Defender portal implement it: where a hunting query actually lives and runs, what a bookmark actually preserves, what the newer **Hunts (currently in Preview)** feature adds on top of a bare query-and-bookmark workflow, where notebook-based hunting fits, and the literal button a hunter clicks to turn a hunting query into a Part 4/5 analytics rule.

This part does not cover hypothesis formation, hunt-type selection, or the abstract hunt-to-detection maturity model — that is DEH's territory, cited above, not repeated here. It also does not re-teach KQL syntax; every query fragment shown is either a short illustrative extension DEH Part 25 doesn't already cover, or a direct quotation from Microsoft's own documentation, framed and cited as such.

1. Hunting queries vs. analytics rules: the platform's own line

[CONCEPT] A hunting query in Microsoft Sentinel is a saved KQL query that an analyst runs on demand — or, per §6 below, schedules as a one-time or recurring job — against workspace data, and that returns a results table for a human to review. Running a hunting query never creates an alert and never creates an incident on its own; nothing "fires" the way a Part 4 scheduled analytics rule fires. A hunting query only enters the incident queue if a person, or an automation acting on that person's behalf, deliberately promotes a specific finding — a bookmark, or the query itself — into something the platform tracks (§3, §7). This is the operational meaning of DEH's "hypothesis-driven, not detection-driven" framing: the platform enforces the distinction structurally, not just by convention. The table below supports one decision — whether a piece of KQL logic belongs on the hunting side of that line or the detection side of it — by mapping the same five properties on each side:

Property	Hunting query	Analytics rule (Part 4–5)
Trigger	Run on demand, or scheduled as a <code>KQL job</code> (§6)	Scheduled evaluation on a fixed cadence, or a Near-real-time (NRT) rule's fixed one-minute cadence
Output	Results table for review; no alert	Alert, then correlated into an incident
Entity/MITRE mapping	Optional, inherited by bookmarks that reference it	Mandatory design decision (Part 6)
Where it's authored	Hunting page <code>Queries</code> tab, or a Jupyter notebook (§5)	Analytics rule wizard (Part 4–5)
Typical source	Content hub solution, or hand-authored	Content hub solution, hand-authored, or promoted from a hunting query (§7)

In practice: a query an analyst wants to *look at periodically* stays a hunting query; a query whose match should *interrupt someone* belongs in Part 4 or 5 instead.

Hunting vs. Advanced Hunting — different pages, different capabilities

[THREAT HUNTER] The word "hunting" names two genuinely different surfaces in the Microsoft Defender portal, and the STYLE-GUIDE convention this book follows for portal terms exists precisely for cases like this one. **Advanced hunting** is Microsoft Defender XDR's own cross-product KQL query experience, unified across Defender for Endpoint, Defender for Identity, Defender for Office 365, Defender for Cloud Apps, and — once a workspace is connected — Microsoft Sentinel's own tables, functions, and saved queries, all queryable from one page (Microsoft Learn, "Advanced hunting with Microsoft Sentinel data in Microsoft Defender," retrieved 2026-09-15). **Hunting** — sometimes written as the Sentinel-specific hunting experience — is the older, narrower page reached via **Microsoft Sentinel > Threat management > Hunting** in the Defender portal, or **Threat management > Hunting** in the Azure portal, and it is the only one of the two that has a `Queries` tab tied to MITRE ATT&CK tactic/technique grouping, a `Bookmarks` tab, and (§4) the Hunts (Preview) feature. The two pages overlap in what data they can reach but not in what they let a hunter *do* with a result — most importantly, Advanced hunting does not support bookmarks at all (§3), and its custom-detection path has real gaps this chapter does not own (Part 5 covers custom detections directly). Get this distinction wrong and a procedure written for one page silently doesn't apply to the other.

2. The Hunting page: queries, results, and one retired feature

[THREAT HUNTER] The Hunting page's `Queries` tab lists every hunting query available to the workspace — built-in queries installed with a Content hub solution, plus any query an analyst wrote or cloned — grouped by MITRE ATT&CK tactic, with a `Techniques` column for the specific sub-technique each query targets. A hunter typically starts a session with **Run all queries** (or a selected subset) and then sorts the results by one of three columns: `Results` (raw match count for the selected time range), `Results delta`, or `Results delta percentage` — the latter two comparing the last 24 hours of matches against the previous 24-to-48-hour window to surface what changed recently (Microsoft Learn, "Hunting Capabilities in Microsoft Sentinel," retrieved 2026-09-15). A query with `N/A` in the `Results` column, rather than a `0`, specifically flags that the query's required data source isn't connected — Microsoft's

own filter separates that state from a genuine zero-match run, and hovering the info icon next to **N/A** names which connector to enable.

Blind Spot The **Results delta** sort is a day-over-day comparison by construction — the last 24 hours against the 24-to-48-hours-ago window — which makes it excellent at surfacing a sudden spike and structurally blind to anything that isn't one. A low-and-slow campaign that holds a steady low volume for weeks never produces a large delta in either direction, because each day looks almost exactly like the day before it. A hunter who prioritizes a session purely by sorting on delta is, by the sorting mechanism's own definition, deprioritizing exactly the patient, steady-state intrusion behavior that many real credential-abuse and persistence techniques are designed to look like — the opposite of noisy and spiky.

False Positive Trap A results column full of **0s** and **N/A**s invites the same misreading regardless of which value dominates: treating "we ran the sweep and it came back clean" as one state, when it's actually two very different ones collapsed together — a query that ran against real, connected data and matched nothing, and a query that never had real data to search because its data source was never onboarded. Microsoft's own **N/A** filter and per-query info icon exist specifically to separate these, but only for a hunter who opens that filter rather than skimming a mostly-blank **Results** column and concluding the environment is quiet.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft Sentinel's **livestream** feature — a continuously-running hunting query that generated a notification on new matches — is no longer available. Microsoft's own current "Hunting Capabilities in Microsoft Sentinel" page states this directly and points to three replacements depending on what livestream was actually being used for: **KQL jobs** (§6) for a persisted, queryable result set; an analytics rule, including a Near-real-time (NRT) rule (fixed one-minute evaluation cadence, a narrower feature set than a standard scheduled rule — see Part 4) where the goal was closer to alerting than hunting; or a playbook for the notification/messaging behavior livestream provided. If you're reading this after livestream has been gone long enough that no team in your organization remembers configuring one, treat this note as historical context for why an older runbook or training deck might still mention it — check the cited page directly if a procedure you're following references livestream at all, since the feature it describes no longer exists to configure.

3. Bookmarks: preserving a pivot mid-hunt

[THREAT HUNTER] A bookmark preserves three things at once from a moment inside a hunt: the specific row or rows of query results the hunter marked, the KQL query and time range that produced them, and whatever entity and MITRE ATT&CK mappings apply — inherited by default from the hunting

query that produced the results, but editable per bookmark (Microsoft Learn, "Hunt with bookmarks in Microsoft Sentinel," retrieved 2026-09-15). A bookmark needs at least one mapped entity before it can be opened in the investigation graph — the same entity-graph-and-timeline view Part 6 covers for incidents generally — so a hunter who wants to pivot visually from a bookmarked finding has to confirm the entity mapping carried over correctly, not just that the bookmark saved.

Every bookmark, across the whole workspace, is also a row in the `HuntingBookmark` table in the underlying Log Analytics workspace — queryable directly with KQL, including joins against other tables, exactly like the `HuntingBookmark`-table pattern DEH Part 25 §9's HUNT-25-01 example builds on for LSASS-access hunting. Deleting a bookmark from the UI doesn't remove its row; it sets that row's `SoftDelete` field to `true` on a new entry, so a query against the raw table has to account for soft-deleted bookmarks explicitly rather than assuming every visible row is still active.

Hunter's Note Querying `HuntingBookmark` directly is the fastest way to answer a question the `Bookmarks` tab's UI doesn't expose well: "which of my last 90 days of bookmarks touched this specific host or account." The UI caps the `Bookmarks` tab display at 1,000 entries and expects interactive filtering; a direct KQL query against `HuntingBookmark`, joined against the entity fields a hunt cares about, doesn't share that cap and can be saved as its own hunting query for reuse in a later session.

[THREAT HUNTER] Bookmark handling is one of the places where the two-portal split (Part 2, Part 16) is not yet symmetric, and it's worth naming precisely here because it sits inside this chapter's own workflow rather than being a distant migration concern:

Bookmark creation and incident-escalation are not available in both portals today, which matters directly for a hunting workflow this chapter otherwise treats as portal-agnostic:

Azure portal (retiring 2027-03-31)	Defender portal
Create a new bookmark from the Hunting page's <code>Queries</code> or <code>Bookmarks</code> tab	View bookmarks already created — no creation
Add one or more bookmarks to a new or existing incident directly	—
Delete or edit a bookmark's tags, notes, and mappings	—

PRODUCT VERSION NOTE (as of 2026-09-15) Per Microsoft Learn ("Hunt with bookmarks in Microsoft Sentinel," retrieved 2026-09-15): "You can only create bookmarks in the Azure portal... In the Microsoft Defender portal, you can view bookmarks that were already created, but you can't add new ones," and the same page's incident-escalation section is explicitly scoped "(Azure portal only)." Advanced hunting (§1) never supported bookmarks at all, in either portal — its documented alternative there is the **Link to incident** action against a query result. Given Part 2's March 31, 2027 retirement date for Sentinel in the Azure portal, this is not a cosmetic gap: as written today, the path to creating a new bookmark or escalating one to an incident by hand runs through the exact portal surface scheduled to stop working.

What Would Change My Mind This part currently tells a hunting team to expect a real workflow change, not just a menu relocation, once Azure-portal access to Sentinel actually ends — either Defender-portal bookmark creation ships before March 31, 2027, or teams need a documented interim substitute (the bookmarks page itself suggests incident tags, saved queries, or a custom hunting table as alternatives) built into their runbooks well ahead of the deadline. If Microsoft ships full bookmark-creation and incident-escalation parity in the Defender portal — plausible, given how actively the two-portal gap is being closed elsewhere (Part 16) — this stops being an operational risk worth flagging on its own and folds back into Part 16's general parity-tracking guidance instead of needing its own warning here.

4. Hunts (Preview): turning a pivot into a tracked investigation

[THREAT HUNTER] **Hunts (currently in Preview)** is a structured container built on top of the queries-and-bookmarks workflow above, not a replacement for it. Creating a hunt clones the hunting queries selected into it — the clones are independent of the workspace's shared query library, so editing a query inside one hunt doesn't touch any other hunt or the original — and gives that hunt its own name, description, owner, and a **Hypothesis** state the analyst updates as evidence accumulates (Microsoft Learn, "Conduct End-to-end Threat Hunting with Hunts," retrieved 2026-09-15). A hunt's detail page carries three tabs scoped to that specific investigation:

Tab	Contents	Typical action from this tab
Queries	Cloned hunting queries for this hunt only	Run, edit, clone, delete, or Create analytics rule (§7)
Bookmarks	Findings bookmarked specifically within this hunt	View source query, view raw bookmark logs, escalate to incident
Entities	Deduplicated entities pulled from every bookmark in the hunt	Open the UEBA entity page; add an IP to threat intelligence; run an entity-scoped playbook

A metrics bar at the top of the **Hunts (Preview)** tab tracks validated hypotheses, new incidents created, and new analytics rules created across the hunting program over time — a genuine, if still simple, way to report hunting output as more than "queries were run," which matters directly to the kind of

program-level accounting DEH's own coverage-and-debt framing (cross-referenced from Part 8) already argues a SOC needs.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft Learn's "Conduct End-to-end Threat Hunting with Hunts" page labels Hunts "(preview)" throughout, and lists its prerequisite as either the built-in Microsoft Sentinel Contributor role or a custom Azure RBAC role scoped to the `Microsoft.SecurityInsights/hunts` resource action — a narrower permission surface than the workspace-wide access a hunter might already have for the plain `Queries / Bookmarks` experience. A Preview label carries no committed GA date and no SLA; check that page before depending on Hunts for anything a SOC's incident-response runbook would call load-bearing, and confirm the RBAC assignment separately if a hunter can see the Hunting page generally but reports the `Hunts (Preview)` tab or its actions are unavailable.

5. Notebooks and MSTICPy: code-based hunting workflows

[THREAT HUNTER] Microsoft Sentinel's own `Notebooks` page integrates Jupyter Notebooks directly, giving a hunter a persistent, shareable environment that holds raw query results, the code run against them, and their visualizations together, rather than the query-at-a-time model the Hunting page's `Logs` search pane offers (Microsoft Learn, "Hunting Capabilities in Microsoft Sentinel," retrieved 2026-09-15). A notebook earns its overhead specifically where the Hunting page's model runs out of room: reusing a parameterized query across many dates without rebuilding it by hand, joining workspace data against something that genuinely isn't in the workspace at all — an HR system's VIP list, a geolocation lookup, a case-management API — or running a procedural, multi-step analysis (anomalous-session scoring, time-series decomposition) that KQL's own operators don't express directly.

MSTICPy, maintained by the Microsoft Threat Intelligence Center (MSTIC), is the Python library built specifically to make that workflow practical inside a Sentinel-connected notebook: querying several log sources at once, enriching results with threat intelligence and geolocation data, extracting indicators of activity from raw log text, and rendering interactive timelines and process-tree visualizations MSTIC's own investigators use day to day (same source). Notebooks support mixed-language cells within one notebook via Jupyter's "magics" — a PowerShell cell retrieving data, a Python cell processing it, a JavaScript cell rendering it — which is a genuine capability difference from the Hunting page's KQL-only query box, not a stylistic one.

[ENGINEERING] A notebook's authentication and connectivity are workspace plumbing distinct from the Hunting page's built-in session: MSTICPy needs its own configured connection to the target Log Analytics workspace (an Azure AD app registration or interactive sign-in, plus workspace ID) before its first query runs, and that configuration — not the KQL inside the notebook — is usually where a "the notebook doesn't return anything" support request actually traces back to, the notebook equivalent of the field-mapping-silently-returns-nothing failure mode DEH Part 25 §8 names for a live query.

6. Hunting at data-lake scale: KQL jobs and federated tables

[ENGINEERING]

An interactive hunting query against the analytics tier is bounded by the query timeout the portal enforces and by whatever retention that specific table carries. A hunt that genuinely needs to reach back months, or that needs to join workspace data against **federated tables** — external sources such as Microsoft Entra ID, Microsoft 365, or Azure Resource Graph data queryable alongside data lake tables without first ingesting them into the workspace — is the scenario KQL jobs exist for: a one-time or scheduled KQL query against the Microsoft Sentinel data lake and federated tables, run for investigation and analysis rather than for alerting (Microsoft Learn, "Create jobs in the Microsoft Sentinel data lake," retrieved 2026-09-15). A job can promote qualifying rows into the analytics tier — where the advanced hunting KQL editor and analytics-tier machine-learning tooling can reach them — or write back into the data lake tier to speed up a later investigation, and Microsoft ships built-in job templates explicitly categorized **Hunting**, **Anomaly detection**, and **Baseline**, which is a direct signal that KQL jobs are meant as a hunting-lifecycle feature, not merely a data-movement utility.

The following KQL fragment is adapted directly from Microsoft's own documentation for handling data lake ingestion latency inside a scheduled job, not an invented example:

```
let lookback = 15m;
let delay = 15m;
let endTime = now() - delay;
let startTime = endTime - lookback;
CommonSecurityLog
| where TimeGenerated between (startTime .. endTime)
```

This fragment targets a KQL job scheduled against the Microsoft Sentinel data lake; its stated purpose is to avoid querying rows that were ingested but not yet visible, since the data lake tier's cold storage carries a typical latency of up to 15 minutes before new rows are queryable. Its limitation, stated by the same source: this pattern only helps a job with a short lookback and frequent schedule — a job with a wide lookback window absorbs that same 15-minute latency as noise, and doesn't need the extra **delay** variable at all.

PRODUCT VERSION NOTE (as of 2026-09-15) *The Sentinel data lake tier and KQL jobs are recent additions to the platform, and several of the exact numbers governing them are the kind of figure this book's PRODUCT VERSION NOTE convention exists to flag rather than quote as permanent. As of Microsoft Learn, "Create jobs in the Microsoft Sentinel data lake": a tenant may run at most 5 KQL jobs concurrently, a single job's query is subject to a 1-hour execution timeout, at most 100 enabled jobs are permitted per tenant, and a job's queryable time range can span up to 12 years. **adx()**, **arg()**, **externaldata()**, and **ingestion_time()** are explicitly unsupported inside a data-lake job's KQL, and user-defined functions aren't supported either — a job that leans on any of those, copied from an ordinary hunting query, fails rather than silently degrading. Re-check that page's own limits table before sizing a hunting program's dependence on jobs; this is one of the fastest-moving numeric surfaces in the book, on a feature still young enough that these limits should be expected to move. Part 17 owns the analytics-tier-versus-data-lake-tier cost and retention tradeoff this feature sits on top of — this note covers the job mechanism's own limits, not the billing model underneath it.*

7. From hunt finding to detection candidate: the graduation path

[SENTINEL ENGINEER] DEH Part 36 already frames "hunt to detection" as a maturity model: a validated, recurring finding earns a permanent home as a detection rule instead of staying a manual, person-dependent sweep. The platform mechanic underneath that decision, in Sentinel, is a single context-menu action. From a hunting query's results — or, inside a specific hunt, from that hunt's own **Queries** tab — selecting **New alert rule > Create Microsoft Sentinel alert** (or, from within a hunt, **Create analytics rule** on a specific query) opens the analytics rule wizard (Part 4–5) with the query's name, description, and KQL body already populated. If the action was launched from inside a hunt, Microsoft Sentinel also records a link back to the new rule under that hunt's **Related analytics rules** list, so the hunt's own record shows which of its queries actually graduated (Microsoft Learn, "Conduct End-to-end Threat Hunting with Hunts," retrieved 2026-09-15).

What that wizard hands you is a starting point, not a finished rule. Entity mapping (Part 6) has to be added deliberately — a hunting query's own entity mappings, if it has any, don't automatically become the rule's alert-grouping keys. The rule's lookback and run frequency need the same deliberate tuning DEH Part 25 §3's lookback-versus-frequency Engineering Reality already covers, since a hunting query was never written with a fixed evaluation cadence in mind. And any allowlist logic the hunting query used informally to cut noise during manual review (Part 10 covers watchlists as the durable home for that allowlist) needs to be re-examined as production suppression logic, not carried over unexamined.

flowchart TB

```
A[Hypothesis: MITRE gap,\nnew campaign, or\nsuspicious behavior] --> B["Hunting page:\nQueries tab\n/\nHunts (Preview)"]
B --> C["Run queries against\nanalytics tier or,\nfor wide windows,\na KQL job / data lake"]
C --> D[Bookmark findings:\nentities + MITRE mapping,\nHuntingBookmark table]
D --> E[Hunt: Entities tab,\ncomments, hypothesis state]
E -->|validated, recurring| F["Create analytics rule\n(Part 4-5 wizard,\nent entity mapping/tuning\nstill required)"]
E -->|validated, one-off| G["Create incident\n(Part 6 entity/incident model)"]
E -->|indicator identified| H["Add to Threat Intelligence /\nwatchlist (Part 10)"]
E -->|not validated| I[Close hunt,\nrecord hypothesis outcome]
```

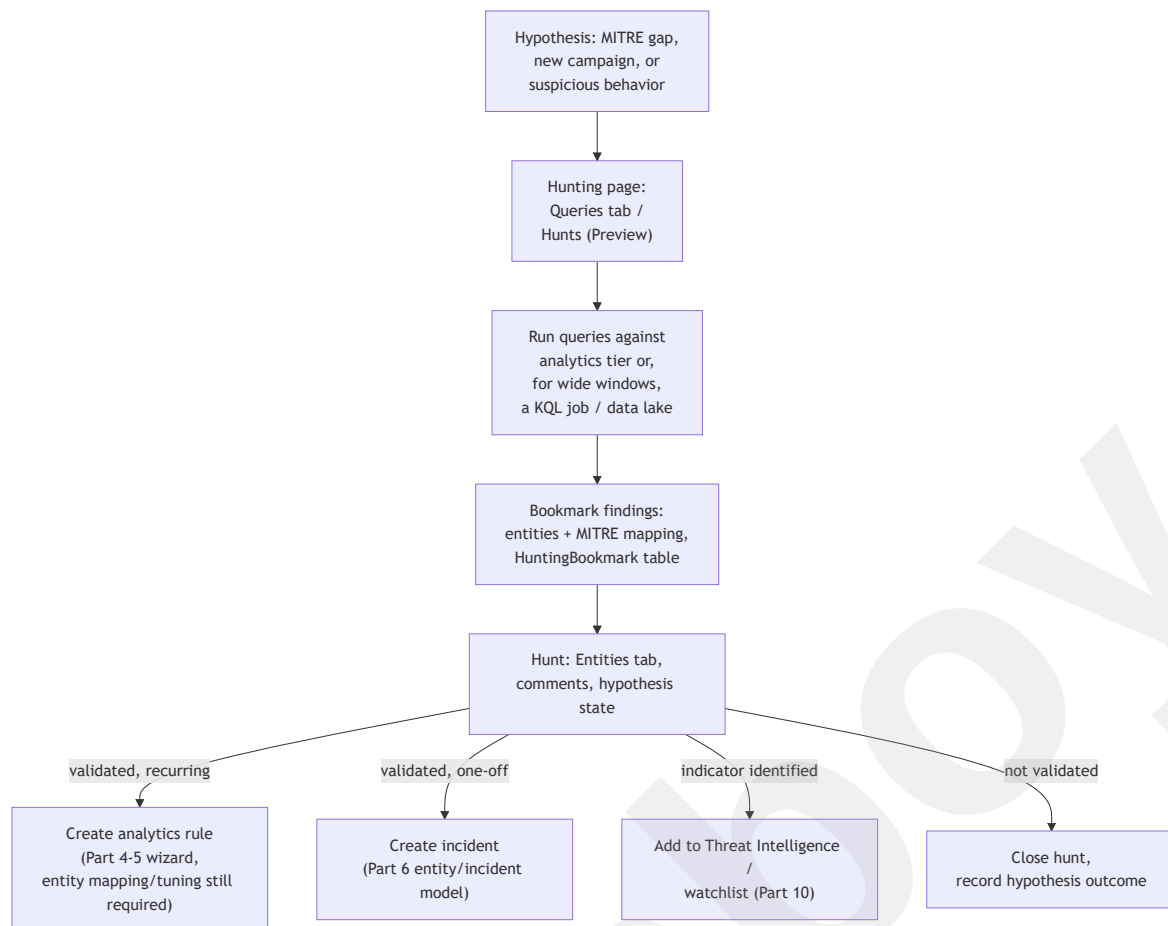


Figure 9.1 — Hunt lifecycle and its platform graduation paths. CONCEPTUAL. Illustrates how a single hypothesis moves through Sentinel's own hunting surfaces — queries, bookmarks, and a Hunt's tracked state — to one of four platform-recorded outcomes, one of which (a promoted analytics rule) is this chapter's link back to Part 4–5. This is a structural sketch of the book's own reading of the cited Microsoft Learn pages, not a capture of any live Hunting or Hunts page.

Detection Test Setup: A hunter arrives independently, via the low-and-slow-friendly hunting workflow above, at the same pattern DEH Part 25 §6 already finishes as DET-25-02 — repeated SSH `Failed password` entries from one source against one account — without having seen that worked example first, purely from sorting hunting-query results by `Results delta` and bookmarking a suspicious cluster. **Action:** Promote the bookmarked hunting query to an analytics rule using **Create analytics rule**, then deliberately add the entity mapping (source IP, target account) and a `bin()`-based grouping window the original hunting query never needed, before enabling the rule. **Expected result:** The promoted rule reproduces DET-25-02's alerting behavior — firing once a `SourceIP / Account` pair crosses the failure-count threshold inside the chosen time bucket — and, crucially, does not alert on the hunting query's own broader, unthresholded result set, confirming the graduation step actually added detection logic rather than just relocating the hunting query into the rule wizard unchanged.

MITRE: T1110.001 (Password Guessing) — the same mapping DEH Part 25 §6 assigns DET-25-02, carried forward here because this section reuses that analytic as its worked graduation example rather than defining a new one.

8. Where else a hunt's output lands

[THREAT HUNTER] An analytics rule is not the only destination a validated hunt finding has. From a hunt's **Entities** tab, an entity can be added directly to threat intelligence as an indicator; from the **Bookmarks** tab or the hunt's own action menu, one or more findings can be escalated straight to a new or existing incident without ever becoming a standing rule, which is the right choice for a genuinely one-off finding that doesn't warrant permanent detection logic. Both paths are intentionally brief here: Part 6 owns how an incident's entity graph actually works once a finding lands there, and Part 10 owns watchlists and threat-intelligence-as-reference-data as a durable platform object, including the allowlist-curation discipline DEH Part 23 §1 argues belongs "at the analytic level" rather than bolted on per rule. This part's job was narrower — showing that a hunt has more than one legitimate ending, and where the button for each one actually lives.

Cross-references: DEH Part 34 (Threat Hunting Fundamentals), DEH Part 35 (Hunt Types), DEH Part 36 (Hunt to Detection), DEH Part 25 §6, §9 (KQL hunting/detection worked examples this chapter reuses); this book's Part 4–5 (Analytics Rules), Part 6 (Entity Mapping and the Incident Model), Part 10 (Watchlists and Reference Data), Part 16 (portal parity), Part 17 (Workspace Cost Models and Retention Tiers).

Part 10 — Watchlists and Reference Data

Why this part exists

Every SOC accumulates a body of knowledge that isn't telemetry: the list of accounts that belong to executives and therefore deserve a lower alert threshold, not a higher one; the list of accounts that belong to people who no longer work here and should never authenticate again; the subnet ranges that belong to the vulnerability-scanning platform and will otherwise light up every anomaly rule in the workspace once a month, forever. DEH Part 23 §1 already made the architectural case for where this kind of exclusion belongs: "at the analytic level, not bolted on independently" — a maintained allowlist that a rule's own logic references, not an ad hoc `NOT` clause added under deadline pressure and never looked at again. This part is where that argument lands in a running Microsoft Sentinel workspace. A **watchlist** is Sentinel's own object for exactly this kind of reference data: an uploaded or query-driven table of name-value rows, cached in the workspace, joinable from a KQL query the same way any other table is.

This part does not re-teach the KQL required to actually join a watchlist into a query — DEH Part 25 already owns the pipe model, `join`, `where`, and `summarize` mechanics this part assumes throughout. What it covers instead is the platform-specific part DEH deliberately leaves to this book: how a watchlist gets created and kept current, the one function that reads it back inside a query, how it differs from Sentinel's separate threat-intelligence-indicator system, who is allowed to see or change one, and the operational discipline a watchlist needs to keep being an asset instead of becoming its own source of false positives and false negatives.

1. What a watchlist actually is

[CONCEPT] A watchlist is a named set of rows — a header row plus data rows, the same shape as a spreadsheet or a CSV export — that you load into a Microsoft Sentinel workspace and that Sentinel then caches for fast lookup from a KQL query. Microsoft's own reference page for the feature, "Use watchlists in Microsoft Sentinel" (learn.microsoft.com/azure/sentinel/watchlists), describes watchlists as a way to correlate data you provide with the events already flowing through your workspace — VIP user lists, terminated-employee lists, known-good service accounts, high-value asset inventories, or any other organization-specific reference table that isn't itself security telemetry but that a detection, a hunt, or a workbook needs to know about.

Two things a watchlist is not, stated up front because both are easy to assume incorrectly:

- **A watchlist is not a live view of its source.** Once loaded, it is a workspace copy, not a standing connection back to the CSV file, spreadsheet, or system that originally produced it. Updating the source does nothing to the workspace copy until the watchlist is explicitly refreshed — the Engineering Reality box in §3 below covers exactly how this bites a rule that assumes otherwise.
- **A watchlist is not the same object as a Sentinel threat intelligence indicator.** Both are reference data an analytic can join against, and it is genuinely easy to reach for the wrong one, so §6 below draws the distinction in full. The short version: a watchlist holds data your organization

produced about itself; a threat intelligence indicator holds data an external party produced about the outside world.

[SENTINEL ENGINEER] Every watchlist has, in addition to its data rows, a small set of metadata that governs how it's referenced: a display **name**, an **alias** (the short string a KQL query actually calls by name — see §3), a description, and a designated **search key** — one column chosen, at creation time, as the primary field a lookup or join is expected to match on. Choosing the search key matters the same way choosing a join key matters for any table: a watchlist of terminated employees keyed on a full display name will match inconsistently against a `SignInLogs` field populated with a user principal name, while one keyed on the UPN itself will not.

2. Building a watchlist: sources, aliases, and the search key

[SENTINEL ENGINEER] A watchlist can be populated from more than one source, and the choice of source is really a choice about who owns keeping the list current:

Table 10.1 — Watchlist creation and update methods. The table below supports choosing a source based on who maintains the underlying list and how often it changes.

Method	Driven from	Typical use	Update model
Local file upload (CSV) via the portal wizard	Whatever file an engineer has on hand	Small, infrequently-changing lists — a VIP roster, a short high-value-asset list	Manual: re-uploading replaces the watchlist's content
Azure Blob Storage source (SAS-secured container)	A blob another pipeline already maintains	Lists generated by a system of record — an HR export, a CMDB feed	Sentinel reads the blob on your own refresh cadence, not continuously
Watchlist and Watchlist Items REST API (<code>Microsoft.SecurityInsights/watchlists</code> resource type)	Any script or pipeline with the right workspace permissions	Programmatic, incremental updates — add or remove a single row without a full re-upload	Item-level create/update/delete calls
Built-in template gallery	A Microsoft-authored starting schema	Bootstrapping a common list shape (see §8) before populating your own values	Same upload paths above, seeded with a template's column layout

The REST API path matters more than a one-line table entry suggests: it's the difference between "someone remembers to re-upload the CSV" and "the HR offboarding workflow calls an API and the terminated-employee watchlist updates itself the same day." A watchlist maintained entirely by manual upload inherits every risk of any manually-maintained list — see the False Positive Trap in §7.

[SENTINEL ENGINEER] The built-in template gallery ships with a set of pre-shaped starting points — commonly cited examples include a VIP-users template and a terminated-employees template, each pre-defining the column layout a corresponding scenario expects. The exact set of templates on offer,

and the specific columns each one pre-populates, is exactly the kind of detail Microsoft updates independently of this book's release cycle.

PRODUCT VERSION NOTE (as of 2026-09-15) The built-in watchlist template gallery, the exact upload workflow (local-file vs. Azure Blob Storage source), and the numeric limits on watchlist size, row count, and refresh cadence are documented on Microsoft Learn's "Use watchlists in Microsoft Sentinel" page (learn.microsoft.com/azure/sentinel/watchlists) as of this writing. Both the template list and the specific limit figures have changed across past Sentinel releases, and this book has no live tenant to re-verify current numbers against. If a specific row-count or file-size figure matters to a design decision, check that page's current limits table yourself before building around a number quoted anywhere else — including, if it ever appears, a specific number in this book.

3. Reading a watchlist back: the lookup function

[SENTINEL ENGINEER] A watchlist is read back inside a KQL query through one function:

`_GetWatchlist("alias")`, which returns the watchlist's rows as an ordinary tabular result — the same shape a `where`, `join`, or `project` operator can act on exactly as if it were querying any other table. DEH Part 25 owns the full pipe-model, join-syntax, and time-window mechanics that operator chain relies on; the fragment below shows only the Sentinel-specific extension DEH Part 25 doesn't cover — the watchlist call itself — not a complete, freestanding detection.

```
// CONCEPTUAL SAMPLE – the watchlist-specific extension only, not a complete detection query.
// Full join/where/summarize syntax and filtering mechanics: DEH Part 25.
SigninLogs
| join kind=leftouter (_GetWatchlist("VIPUsers")) on UserPrincipalName
```

Note what this fragment is doing and not doing: it isn't suppressing anything, and it isn't itself a complete analytics rule body. The join alone hands the rest of the query — whatever filtering, classification, or automation-rule condition (Part 13) comes next, using the join/where mechanics DEH Part 25 owns — a per-row signal of whether the account matched the watchlist, which is the more common role a watchlist plays in an analytic than outright suppression. The allowlist-shaped use of the same mechanism (excluding rather than elevating) is covered in §7.

```
flowchart LR
    A["CSV upload\n(portal wizard)"] --> D
    B["Azure Blob Storage\n(SAS-secured container)"] --> D
    C["Watchlist / Watchlist Items API\nMicrosoft.SecurityInsights/watchlists"] --> D
    D["Watchlist store\n(cached name-value rows,\nalias + search key)"]
    D --> E["Analytics rule query\n_GetWatchlist(alias)"]
    D --> F["Hunting query\n(Part 9)"]
    D --> G["Workbook panel\n(Part 8)"]
    D --> H["Automation rule condition\n/ playbook input (Part 13, 14)"]
    D --> I["Entity investigation graph\nwatchlist-membership badge"]
```

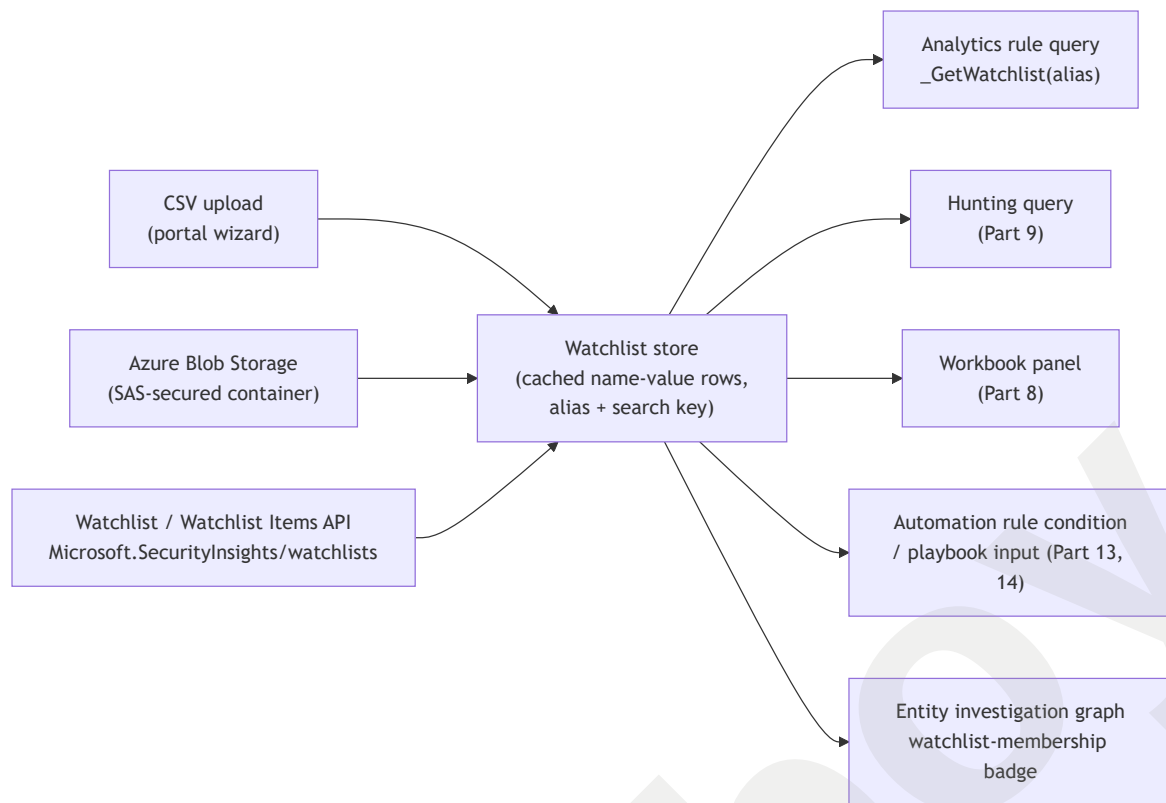


Figure 10.1 — Watchlist ingestion and lookup data flow. *CONCEPTUAL.* Illustrates how a watchlist's three creation paths (Table 10.1) feed one cached workspace object, and the five surfaces that object can then be read from. This is a structural sketch built from this book's own reading of the watchlist and platform-surface documentation cited throughout this part — not a capture of any Sentinel UI screen, and not a reproduction of an official Microsoft architecture diagram.

Engineering Reality A watchlist is not a live view of its source file or query — as §1 already stated, it's loaded into the workspace and cached for query performance, and an update to the underlying CSV or blob does not retroactively change the workspace copy until the watchlist itself is re-imported or refreshed through whichever path in Table 10.1 you're using. A scheduled rule calling `_GetWatchlist("TerminatedEmployees")` five minutes after an offboarding event is only as current as the watchlist's own last import — not as current as the query's own lookback window. Treat "the watchlist" and "the system of record it's supposed to mirror" as two objects with an update lag between them, the same discipline Part 18 asks you to apply to ingestion delay generally.

4. Watchlists across the two portals

[ENGINEERING] A watchlist is stored as one underlying object regardless of which portal created or last edited it — the same `Microsoft.SecurityInsights/watchlists` resource is what both the Azure portal and, once a workspace is onboarded, the Microsoft Defender portal surface. There is no separate Defender-

portal-native watchlist type and no conversion step: a watchlist built before onboarding continues to exist and continues to be readable through `_GetWatchlist()` exactly as before.

Table 10.2 — Watchlist behavior by portal. The table below supports predicting what changes, and what doesn't, when a workspace moves between the two operating surfaces (Part 2, Part 16).

Aspect	Azure portal (retiring 2027-03-31)	Defender portal
Where configured	Microsoft Sentinel > Configuration > Watchlists	Microsoft Sentinel navigation node, once the workspace is onboarded
Underlying resource	<code>Microsoft.SecurityInsights/watchlists</code>	Same resource — no separate object type
KQL lookup function	<code>_GetWatchlist("alias")</code>	Identical — the function is a Log Analytics/KQL capability, not a portal feature
Entity investigation view	Watchlist-membership indicator on the entity graph	Corresponding indicator on the unified incident's entity panel
Creation/edit permissions	Standard Sentinel RBAC (\$5)	Same RBAC model — onboarding does not introduce a separate permission set for watchlists specifically

PRODUCT VERSION NOTE (as of 2026-09-15) *Sentinel-in-the-Azure-portal is scheduled for retirement on March 31, 2027 (Part 2); after that date, watchlist configuration is only reachable from the Microsoft Defender portal. The exact menu placement inside the Defender portal's Microsoft Sentinel navigation node has already moved more than once during the unified-portal rollout, per Microsoft's own "Microsoft Sentinel in the Microsoft Defender portal" documentation. If a menu path in this section doesn't match what you see, use the portal's own search box rather than assuming this book is describing a hidden or removed feature — the underlying object and the `_GetWatchlist()` function are the stable parts of this table; the click path to reach the configuration screen is not.*

5. Who can see and change a watchlist

[SOC MANAGEMENT] Watchlist content is frequently more sensitive than the telemetry it's joined against. A terminated-employee list names people who no longer work at the organization, sometimes alongside a separation date or reason code someone thought was useful context; a VIP-user list is, by definition, a list of the people an attacker would most want to know are flagged as high-value targets. Both are worth handling with the same access discipline as the incident data they support, not as harmless configuration metadata.

Microsoft Sentinel's own role-based access control model — documented under "Roles and permissions in Microsoft Sentinel" — applies to watchlists the same way it applies to analytics rules and incidents: the built-in **Microsoft Sentinel Reader** role can view a watchlist's content, while creating, editing, or deleting one requires **Microsoft Sentinel Contributor** (or a broader role that includes those permissions, such as Owner on the resource). There is no watchlist-specific role that narrows access to

just this feature — a user with Contributor-level Sentinel access has that access across rules, automation, and watchlists alike, which is worth naming explicitly to whoever is deciding how broadly to grant Contributor in the first place, since it is not a "just for editing rules" grant.

Because watchlist rows can carry personal data about current or former employees, treat populating one the same way you'd treat any other system that stores that data: know who's allowed to export it, whether it needs to be included in a data-retention or right-to-erasure process, and whether HR or legal has a view on a security team independently maintaining a list of terminated staff. None of that is Sentinel-specific — it's the same governance question any internally-sourced PII table raises — but a watchlist is easy to overlook as "just configuration" precisely because it lives in a security tool's settings screen rather than a system an organization already has a data-handling policy for.

6. Watchlists vs. threat intelligence indicators

[CONCEPT] Sentinel has two distinct systems for reference data that an analytic can match against, and conflating them leads to reaching for the wrong tool. A watchlist holds data your organization produced about itself. A **threat intelligence indicator** — ingested via a TI feed, a TIP connector, or a TAXII feed and typically landing in the `ThreatIntelligenceIndicator` table — holds data an external party produced about the outside world: a hash, domain, IP, or URL that some other organization's collection process observed and judged malicious. DEH Part 32 covers how to consume that second category responsibly (confidence scoring, indicator age, infrastructure role, prevalence) in far more depth than this part needs to repeat; what matters here is knowing which of the two objects a given piece of reference data actually is before deciding how to store and join it.

Table 10.3 — Watchlists vs. threat intelligence indicators. The table below supports choosing the right object for a given piece of reference data before building a query against it.

Aspect	Watchlist	Threat intelligence indicator
What it holds	Organization-defined reference rows you author	Structured indicator objects (IP, domain, hash, URL) an external party produced
Typical source	CSV upload, blob storage, or API — data you or an internal system produced	A TI feed, TIP, or TAXII connector — data an external party produced
Primary access path	<code>_GetWatchlist("alias")</code>	<code>ThreatIntelligenceIndicator</code> table, plus dedicated TI-matching analytics rule templates
Confidence/age scoring	Not scored by the platform — every row is equally authoritative; scoring is your analytic's own job	Indicators carry source-supplied confidence and validity metadata (DEH Part 32)
Typical analytic role	The allowlist/context layer joined into a rule you wrote (§7)	The indicator-match signal a rule consumes and then has to weigh, not trust outright

A watchlist of "IP ranges belonging to our own vulnerability scanner" and a threat intelligence indicator for "an IP address a feed reported as a Cobalt Strike C2 node" are structurally similar — both are a list of IPs an analytic can join against — but they answer opposite questions, and building one where the other belongs produces predictable failures: a scanner-range watchlist accidentally uploaded as a TI feed would get scored for confidence and prevalence it was never designed to carry, and a genuine

threat feed treated as a flat watchlist loses every piece of source-confidence metadata DEH Part 32 §2 explains why you need.

7. The allowlist that actually lives here

[SENTINEL ENGINEER] DEH Part 23 §1 states the general principle this section operationalizes: an allowlist "belongs at the analytic level, not bolted on independently." A watchlist is the concrete Sentinel object that principle points to. Instead of a `SourceImage !in ("known-tool-a.exe", "known-tool-b.exe")` literal list buried inside a rule's own query text — invisible to anyone who isn't reading that specific rule, and duplicated by hand into every other rule that needs the same exclusion — a maintained watchlist referenced by `_GetWatchlist()` is one object, visible in its own configuration screen, reusable across every rule, hunting query, and workbook that needs the same exclusion, and RBAC-controlled independently of the rules that consume it (§5).

This is the same shape DEH Part 42's exception-ledger pattern already uses a Sentinel watchlist for — a named, dated, justified entry per exclusion, auditable for staleness — applied here specifically to the allowlist case: a watchlist of known-good service accounts, sanctioned scanner IP ranges, or approved administrative tools, joined into a rule's `where` clause as an exclusion rather than an elevation.

False Positive Trap *A watchlist-driven allowlist degrades the same way DEH Part 15's mail-relay watchlist example does: every account that changes role, every asset that gets decommissioned and reissued, and every scanner that gets a new IP range keeps generating false positives — or, in the terminated-employee case, false negatives, since a stale list is the one place "no alert" is the wrong outcome — until someone updates the list. Expect a watchlist-driven allowlist's real-world accuracy to track how well its owning team's offboarding, reissuance, or asset-inventory process feeds it, not how well the KQL join is written. The join is rarely where this actually fails.*

Blind Spot — the allowlist is only as strong as its match key *A VIP-user or terminated-employee watchlist keyed on a display name or a user principal name protects the analytic exactly as well as that string resists reuse. A terminated employee's UPN reassigned to a new hire before the corresponding watchlist entry is retired — or an on-premises AD account renamed post-departure while a synced cloud identity still carries the old UPN in cached telemetry — reproduces the same failure DEH Part 23's process-image allowlist Blind Spot names for a different kind of exclusion: the match is against an identity string, not against the state that identity is meant to represent, and closing the gap needs a compensating control (reconciling the watchlist against the identity source of record on a schedule) rather than a better string match.*

8. Analyst and hunter use of watchlists

[ANALYST]

From an incident-triage perspective, a watchlist most often shows up passively rather than as something an analyst goes and queries directly: when an entity (an account, a host, an IP) appears in the unified incident queue's investigation graph or entity panel, and that entity's identifying value also appears on a watchlist, the entity view surfaces that membership as context — "this account is on the VIP-users watchlist," for instance — without the analyst having to separately go check. That context should change triage behavior in both directions: a VIP-flagged account raises the priority of an otherwise-routine alert, while a scanner-range or known-good-service-account flag is a reason to close faster, not a reason to skip verification entirely — a watchlist entry is context for a human decision, not itself a verdict, the same caution DEH Part 32 §2 applies to a threat-intelligence match.

[THREAT HUNTER]

A watchlist is equally useful as an input to a hunt that hasn't produced an incident yet. Part 9 covers the mechanics of hunting queries and bookmarks in Sentinel and the Defender portal directly; a watchlist slots into that workflow as a join target the same way it does in an analytics rule — narrowing a broad hunting query to "activity involving an entity on this specific list" (a known-privileged-account watchlist, a high-value-asset inventory) without hardcoding the list of interest into the hunting query's own text every time it's run. DEH Part 34–36's hunt-to-detection lifecycle applies unchanged: a hunting query that repeatedly proves useful against a watchlist-scoped population is itself a candidate to graduate into a scheduled rule, at which point the same watchlist reference simply carries forward into the rule's own query.

Hunter's Note *A watchlist built for one purpose is frequently reusable for a hunt that has nothing to do with why the list was originally created. A high-value-asset inventory maintained for elevated-severity alerting is also exactly the population you want to scope a lateral-movement hunt to first, since that's where the impact of a successful hunt finding is highest — check whether a watchlist relevant to your hunt's hypothesis already exists before building a new reference list from scratch.*

9. What to verify before you trust a specific number here

[ENGINEERING]

This part has deliberately avoided stating specific numeric limits — maximum watchlist size, maximum row count, exact refresh cadence — because this book has no live Sentinel tenant to confirm current figures against, and those figures have changed across past Sentinel releases. Stating a specific number with confidence this book can't back up would be a worse outcome than not stating one at all.

What Would Change My Mind This part treats "check Microsoft Learn's current limits table before designing a control around a specific watchlist row count or file size" as the only responsible default, given the honesty constraint stated in Part 1 and [STYLE-GUIDE.md §9](#) — this book has no tenant to verify a current number against directly. If a future edition of this book is written against a real Sentinel tenant, the responsible default changes to citing a specific, lab-verified number directly in the prose. The caution here is a function of what this book's author can currently verify, not a claim that watchlist limits are inherently unknowable or unstable in practice.

What is stable enough to state without that hedge: a watchlist is cached, not live (§3's Engineering Reality); it is read back through `_GetWatchlist()` regardless of portal (§4); it inherits Sentinel's standard Contributor/Reader RBAC split rather than a feature-specific permission model (§5); and it is a categorically different object from a threat intelligence indicator (§6). Those are architectural facts, not release-cadence numbers, and they're the ones this part is willing to state plainly.

Cross-references: DEH Part 23 §1 (allowlist-at-the-analytic-level framing), DEH Part 25 (KQL join/where/summarize syntax), DEH Part 31 (Baselining), DEH Part 32 (Threat Intelligence in Detection), DEH Part 34–36 (Hunt lifecycle), DEH Part 42 (exception-ledger watchlist pattern); this book's Part 2 and Part 16 (the two-portal split and Azure-portal retirement date), Part 6 (entity mapping — what an entity panel's watchlist badge is built on), Part 9 (hunting mechanics), Part 13 (automation-rule conditions consuming a watchlist), Part 18 (ingestion/refresh delay, the same discipline applied to watchlist staleness).

Section D — Behavioral Analytics and AI

neshbooy

Part 11 — UEBA: User and Entity Behavior Analytics

Why this part exists

[CONCEPT] Every analytics rule discussed in Parts 4 and 5 answers a question about a single event or a single query result: did this pattern occur? UEBA (User and Entity Behavior Analytics) answers a different question that no single-event rule can: is this normal *for this entity*? A sign-in from a new country is not inherently suspicious — it's suspicious relative to where that specific user, or their peers, normally sign in from. That relative judgment requires a standing baseline, built over time, per entity, and Sentinel's UEBA feature is the platform component that builds and maintains it. DEH Part 31 already covers baselining as a general detection-engineering technique — why a static threshold rule and a baseline-relative rule solve different problems, and what a baseline actually needs to stay valid (enough history, enough peers, a refresh cadence shorter than attacker dwell time). This part doesn't re-derive that theory. It covers what Microsoft Sentinel specifically does with it: which data sources UEBA reads, which tables it writes, which two numeric scores it produces, and why those two scores answer different questions that this part explicitly refuses to collapse into one.

UEBA also sits downstream of a fact DEH Part 12–13 already establishes about identity as a detection surface: an authentication event (a token issuance, a sign-in) is thin evidence on its own, because a valid credential and a stolen credential produce the identical event. Behavioral context — where this account usually signs in from, what it usually accesses, who its peers are and what they do — is one of the few practical ways to add signal to an otherwise-clean authentication event. UEBA is Sentinel's implementation of that context layer, and it is a mature, GA capability: Microsoft ships it at no incremental license cost beyond the ingestion cost of the underlying tables it reads, and its core table schema has been stable for multiple years. That maturity is exactly why this part is split from Part 12 — UEBA is settled architecture with occasional additions (covered explicitly, and hedged, in §7 below); Security Copilot and agentic tooling are not, and get their own part precisely because a stable feature and a fast-moving one shouldn't share one callout density.

MITRE: T1078 (Valid Accounts) — the technique UEBA's baselining is most directly aimed at making harder to hide behind.

1. What UEBA is, and what it is not

[CONCEPT] UEBA is a workspace-level setting, not a rule. Enabling it (Microsoft Sentinel > Settings > Entity behavior analytics is the Azure portal path as of this writing — spelled out here on first mention, but a left-nav path is exactly the kind of detail that shifts between portal releases, so verify the current location against Microsoft Learn rather than trusting this sentence indefinitely; the equivalent surface exists in the Microsoft Defender portal under its own, separately-verified left-nav path, which may not match the Azure portal's) turns on a background process that reads identity and activity signal already flowing into the workspace and builds per-entity behavioral baselines from it. It does not, by itself, create incidents. What it produces is enrichment and scoring that other parts of the platform consume: entity pages display it, hunting queries can join against it, and a specific rule type — Anomaly rules (a rule type that is ML-baselined, doesn't alert on its own until a duplicated template's threshold is tuned,

and writes its output to the `Anomalies` table; full rule-type treatment in Part 5) — consumes it directly to decide whether to fire.

This distinction matters because it's easy to conflate "UEBA is on" with "UEBA will alert me." It won't, on its own. UEBA is the baseline; Anomaly rules (and, less directly, some Fusion and Custom Detection logic) are what turn a baseline deviation into something that shows up in the incident queue. A workspace can have UEBA enabled with zero Anomaly rules active, in which case UEBA is quietly building baselines and enriching entity pages with no effect on the incident queue at all — a common, and often unintentional, deployment state worth checking for during any Part 18-style health review.

Engineering Reality *UEBA needs history before its baselines mean anything. A newly enabled UEBA feature, or a newly onboarded entity with no prior activity in the workspace, doesn't have a peer-relative or self-relative baseline yet — it has to accumulate enough activity first. Microsoft's own UEBA documentation describes this as a learning period rather than an instant switch-on; treat the first stretch of any onboarding as "baselines are still forming," not "UEBA is broken," when investigation priority scores look thin or absent for entities that were only recently added to the data sources UEBA reads.*

2. Data sources: what UEBA actually reads

[ENGINEERING] UEBA doesn't ingest its own telemetry — it reads tables the workspace is already collecting through ordinary connectors, and it lets you toggle which of those sources it draws behavioral signal from independently of whether the underlying connector itself is on. The sources Microsoft documents as UEBA inputs center on identity and access activity: Azure Active Directory (Entra ID) sign-in and audit logs, Windows Security Events, Office 365 activity, and signal from Microsoft Defender for Identity where that product is connected. Each source has its own toggle in the UEBA settings surface, separate from the connector's own on/off state, so a workspace can be collecting Security Events for other purposes while explicitly excluding that source from UEBA's baselining — useful if a particular data source is noisy or not yet trustworthy enough to baseline against.

[SENTINEL ENGINEER] The scope description for this part also names multi-cloud connectors as a UEBA input, reflecting Microsoft's stated direction of extending entity-behavior baselining beyond the Entra ID-centric core. Treat the specific current list of supported multi-cloud sources as unsettled rather than memorize a list here — this is precisely the kind of enumerable fact (which connectors currently feed UEBA) that changes as Microsoft adds sources, and the authoritative, current answer is Microsoft Learn's UEBA data-sources reference, not this paragraph.

The table below is a decision aid for what to check before assuming UEBA is baselining an entity type — not an exhaustive schema reference.

Entity type	Primary baseline signal	Data sources typically required	Practical check
Account (user)	Peer-group deviation, sign-in geography/device/app novelty, activity volume	Azure AD sign-in and audit logs, Office 365 activity	Confirm the account's Azure AD object has group-membership data synced — a peer group can't be computed for an identity with no group attributes
Host	Sign-in target novelty, first-time account-to-host pairing	Security Events, Azure AD sign-in logs	Confirm the host emits Windows Security Events into the workspace, not just endpoint telemetry that bypasses that table
IP address	Geography and ASN novelty relative to the account's history	Azure AD sign-in logs, Defender for Identity	IP-based insight quality depends on sign-in logs carrying resolved geolocation — check for gaps behind corporate VPN egress, which flattens genuine geography signal

3. Peer-group analysis and blast radius

[CONCEPT] UEBA's baselining rests on two distinct methods, and conflating them is a common source of confusion when reading an entity page. Peer-group analysis compares an account's activity to a computed group of peers — accounts Microsoft's algorithm judges similar based on Azure AD attributes such as group membership, manager, and department — and flags activity that's unusual relative to that group even if it's routine for the individual account. Blast-radius evaluation is a different question entirely: not "is this normal," but "how much could go wrong if this specific account were compromised right now," based on what the account can reach — group memberships, role assignments, resource access. A help-desk account with broad reset-password rights can have a perfectly normal, unremarkable sign-in pattern and still carry a high blast radius, because the damage a compromise of that account could do is high regardless of how boring its day-to-day behavior looks. UEBA computes both and feeds both into the investigation priority score covered in §5.

False Positive Trap *Peer-group analysis needs an actual peer group to be meaningful, and small organizations, unique job functions, and newly created roles routinely don't have one. A sole database administrator, a newly hired specialist with no comparable Azure AD group siblings yet, or a service account with a one-of-a-kind naming pattern can register as behaviorally anomalous simply because Microsoft's peer-grouping algorithm can't find enough genuinely comparable accounts to build a stable baseline from. The fix isn't to disable UEBA for that account — it's to recognize that a thin-peer-group anomaly carries less evidentiary weight than the same anomaly flagged against an account with dozens of legitimate peers, and to weight investigation priority accordingly rather than treating every high score as equally strong evidence.*

[THREAT HUNTER] Peer-group membership itself is worth hunting on independently of any anomaly score. A hunt built on `UserPeerAnalytics` — asking "who does Sentinel currently consider this account's peers, and has that peer set changed recently" — can surface a role change, a group-membership drift, or a stale peer computation before it ever produces a scored anomaly. This is the kind of exploratory, hypothesis-driven pivot that belongs in a hunting notebook (Part 9) rather than a standing rule; the peer-set itself is reference data about the baseline, not an alert.

4. The table set

[SENTINEL ENGINEER] UEBA's output lives in four tables, and each answers a different question a rule author or hunter might ask.

BehaviorAnalytics is UEBA's primary output table — one row per scored activity, carrying the investigation priority score (§5), the peer-analysis and blast-radius insights that fed it, and identifiers for the entities involved. This is the table an Anomaly rule or a hunting query reads when the question is "show me scored behavioral activity for this entity."

IdentityInfo is broader than UEBA's own alerting purpose. It's a normalized, UEBA-populated table of identity attributes — UPN, group membership, manager, enabled/disabled state, and similar Azure AD context — that other parts of the workspace draw on for entity enrichment well beyond UEBA's own scoring. An analytics rule anywhere in the workspace that wants to know "is this account a member of a privileged group" without writing its own Azure AD lookup can query **IdentityInfo** instead, which is one reason enabling UEBA has enrichment value even for rules that have nothing to do with behavioral anomaly detection. This is the same identity-normalization role Part 6's entity-mapping mechanics and Part 10's watchlist-driven reference data both partly overlap with — **IdentityInfo** is UEBA's contribution to that shared reference-data layer, not a UEBA-exclusive silo.

UserPeerAnalytics records the peer-group computation itself — which accounts UEBA currently considers peers of a given account, and on what basis. This is the table the hunt in §3 reads.

Anomalies is written by Anomaly-type analytics rules, not by UEBA directly — it's downstream of UEBA's baselines but is a rule-type output table, one row per rule execution that judged an activity anomalous enough to record. This is the table that feeds the anomaly score covered next.

Blind Spot **BehaviorAnalytics** scores individual activities against a baseline; it does not, on its own, tell you whether that activity ever became an incident. A high-investigation-priority row in **BehaviorAnalytics** with no corresponding row in **SecurityIncident** means exactly what it looks like — Sentinel judged the activity behaviorally unusual and nothing in the rule layer acted on it. That's not necessarily a platform failure; it can just as easily mean no Anomaly rule's threshold was tuned to catch it, or the workspace has UEBA enabled with no Anomaly rules turned on at all (§1). Don't read the presence of scored anomalies in **BehaviorAnalytics** as evidence that the incident queue reflects them.

5. Two scores, two different questions

[ANALYST] This is the section Section D's own framing in **BOOK-INDEX.md** calls out by name, and it's worth stating as plainly as possible: Sentinel's UEBA and Anomaly-rule layer produces two numeric scores that look superficially similar — both are "how anomalous is this" numbers — and answer genuinely different questions. Reading one as if it were the other is a common analyst mistake.

The **investigation priority score** is a 0–10 value UEBA computes per scored activity in `BehaviorAnalytics`, combining peer-analysis deviation, blast-radius context, and (per Microsoft's documentation of the feature) prior investigation and incident history for the entity. It's evaluated close to the activity itself — near-real-time relative to the underlying sign-in or access event — and its purpose is triage prioritization: given a large number of scored activities on an entity page or in a hunting result set, which ones deserve an analyst's limited attention first. It answers "how much should I care about this specific entity right now."

The **anomaly score** is a 0–1 value written to the `Anomalies` table by an Anomaly-type analytics rule's underlying ML model, evaluated on the rule's own execution cadence rather than per-activity in real time — closer to a batch judgment than a near-real-time one. Its purpose is rule-triggering: an Anomaly rule template ships with a default sensitivity, and duplicating that template (never editing the built-in original directly — Part 5 covers why) and adjusting its threshold changes how high an anomaly score has to be before that specific rule instance creates an alert. It answers "should this specific rule fire on this specific model output."

The table below summarizes which of the two scores to consult for a given question, since the prose distinction above is easy to lose track of once both numbers are on screen at once.

Attribute	Investigation priority score	Anomaly score
Range	0–10	0–1
Computed by	UEBA baselining engine	Anomaly-type analytics rule's ML model
Written to	<code>BehaviorAnalytics</code>	<code>Anomalies</code>
Cadence	Near-real-time, per scored activity	Batch, per rule execution cycle
Answers	How much should an analyst care about this entity right now	Should this specific rule instance fire on this model output
Tunable by an engineer	No — it's a UEBA-computed enrichment value	Yes — via a duplicated rule template's threshold (Part 5)

Engineering Reality These two scores were built by different parts of the platform to serve different consumers — one for a human scanning an entity page, one for a rule deciding whether to create an alert — and Microsoft has never documented a fixed conversion between them. An investigation priority score of 9 does not imply any particular anomaly score, and a low anomaly score on one Anomaly rule doesn't mean the same activity has a low investigation priority. Treat a workbook or dashboard that tries to plot both on the same axis with the same skepticism you'd apply to averaging two unrelated units.

6. The UEBA behaviors layer — a newer, separately enabled capability

[CONCEPT] Beyond the mature `BehaviorAnalytics` / `IdentityInfo` / `UserPeerAnalytics` / `Anomalies` table set covered above, Microsoft has been extending UEBA with an additional, separately enabled behaviors layer referenced in this book's own part scope as writing to `SentinelBehaviorInfo` and

`SentinelBehaviorEntities`. Treat this section with more caution than any other in this part: it describes recently introduced, opt-in surface area, and this book has no lab tenant to validate its current behavior against (the honesty constraint stated in Part 1 applies with particular force here). What follows is a deliberately conservative description, not a full schema reference — verify every specific detail against Microsoft's current UEBA documentation before building anything against it.

PRODUCT VERSION NOTE (as of 2026-09-15) *Microsoft documents an additional entity-behavior surface beyond the classic UEBA table set, opt-in and separate from the core UEBA toggle covered in §2, with `SentinelBehaviorInfo` and `SentinelBehaviorEntities` as its table names. This book treats its scope, maturity, and exact enablement path as unsettled rather than describing specific fields or query patterns against it, because the feature has been under active development and this book has no tenant to confirm current behavior against. Check Microsoft Learn, "Identify advanced threats with User and Entity Behavior Analytics (UEBA)" (learn.microsoft.com/en-us/azure/sentinel/identify-threats-with-entity-behavior-analytics) — checked 2026-09-15 for this note — directly before relying on any specific claim about this layer — including whether the table names above are still current — and treat anything below this note as a description of that a newer layer exists, not how it currently works.*

What Would Change My Mind *This part's stance is "describe the newer behaviors layer's existence and purpose, but don't teach its mechanics without lab evidence." If a future edition's author gains access to a real Sentinel tenant with this layer enabled (the no-real-tenant honesty constraint lifting, per `STYLE-GUIDE.md` §9.4's provision for that day), the right move is to replace this hedge with a walkthrough validated against that real, captured evidence, documented per whatever evidence-class rules govern that future edition — not to keep hedging out of habit once real evidence exists.*

[SOC MANAGEMENT] The practical takeaway for a SOC deciding whether to enable this newer layer today: treat it as you would any preview or recently-GA'd capability elsewhere in this book — pilot it against a non-production workspace or a limited scope first, and don't build a standing detection or workbook dependency on its exact table schema until Microsoft's own documentation describes it as stable. The classic UEBA table set in §4 is the safe, multi-year-stable foundation to build production content against; this layer is not yet that, on the evidence available to this book.

7. Investigating with UEBA: the analyst's view

[ANALYST] UEBA's output surfaces in three places an analyst actually looks. First, entity pages (opened from an incident's investigation graph, or searched directly) show behavioral insights for Account, Host, and IP address entities — peer-group membership, blast-radius context, and any scored activity from `BehaviorAnalytics` relevant to that entity, alongside the ordinary activity timeline. Second, the built-in UEBA workbook (shipped with the Sentinel content solution set, not something you build from scratch) visualizes anomalous activity trends, geographic sign-in distribution, and investigation-priority-scored

entities over a selected time window — a starting triage view rather than a replacement for entity-level investigation. Third, any incident whose triggering alert came from an Anomaly rule carries the anomaly score and the specific model insight that fired it directly in the alert details, which is the fastest way to answer "why did this specific incident happen" without cross-referencing [Anomalies](#) manually.

[ANALYST] The practical triage discipline this part recommends: don't open an entity page and treat the investigation priority score alone as a verdict. Read the insights that produced it — which peer-group deviation, which blast-radius factor, whether the underlying activity has independent corroboration elsewhere in the incident (an IP reputation hit, a correlated alert from Defender for Identity, a watchlist match against a known-risky asset). A high score with thin supporting insight (§3's thin-peer-group trap) deserves a different level of confidence than a high score with three independent corroborating signals.

8. Hunting against the UEBA tables

[THREAT HUNTER] Syntax fundamentals — the pipe model, [join](#), [summarize](#), time-window functions — are DEH Part 25's scope, not this book's; what follows assumes that foundation and shows only the Sentinel-specific extension: joining a UEBA table into an identity-context lookup. The fragment below illustrates the shape of a peer-context hunt, not a complete, deployable query — see DEH Part 25 for the join-strategy and performance guidance that applies to any join like this at scale.

```
// CONCEPTUAL SAMPLE – illustrative peer-context hunt fragment, not a complete query
BehaviorAnalytics
| where ActivityInsights has "New Location"
| join kind=leftouter (
    UserPeerAnalytics
    | project UserName, PeerGroup
) on UserName
| where isnull(PeerGroup) or PeerGroup == ""
```

The specific pattern this fragment surfaces — a scored "new location" activity for an account with no computed peer group at all — is exactly the §3 thin-peer-group trap made queryable: it separates "genuinely novel behavior against a real peer baseline" from "novel-looking behavior against no baseline at all," which is the distinction an analyst needs before trusting the investigation priority score's face value.

flowchart LR

```
subgraph Sources["Data sources UEBA reads"]
  AAD["Azure AD / Entra ID<br/>sign-in and audit logs"]
  SEC["Security Events"]
  O365["Office 365 activity"]
  MDI["Defender for Identity signal"]
end

AAD --> ENGINE
SEC --> ENGINE
O365 --> ENGINE
MDI --> ENGINE

ENGINE["UEBA baselining engine<br/>peer-group analysis + blast-radius evaluation"]

ENGINE --> BA["BehaviorAnalytics"]
ENGINE --> II["IdentityInfo"]
ENGINE --> UPA["UserPeerAnalytics"]

BA --> IPS["Investigation priority score<br/>0-10, near-real-time"]
BA --> AR["Anomaly-type analytics rule<br/>(Part 5)"]
AR --> AN["Anomalies"]
AN --> ASC["Anomaly score<br/>0-1, batch ML"]
ASC --> INC["SecurityIncident,<br/>only if rule threshold crossed"]

IPS --> EP["Entity pages / investigation graph"]
ASC --> INC
II --> EM["Entity-mapping enrichment<br/>used workspace-wide (Part 6)"]
```

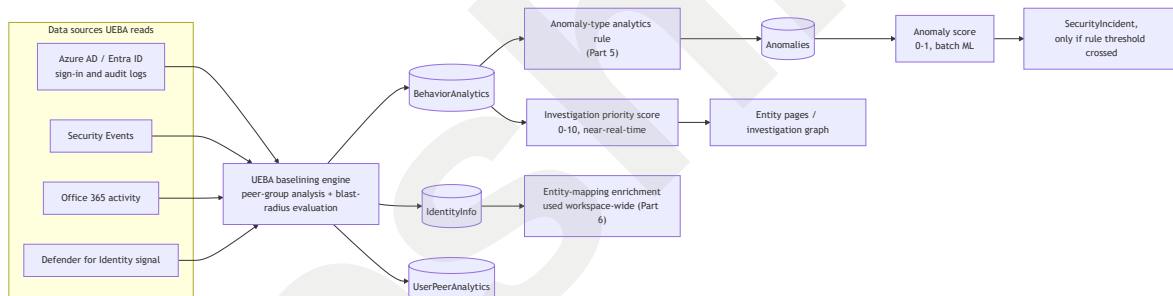


Figure 11.1 — UEBA data flow: sources, engine, tables, and the two scores. CONCEPTUAL.

Illustrates how UEBA's four inputs feed a single baselining engine that writes three tables, and how those tables diverge into two differently-scoped, differently-scaled scores — one consumed directly by analysts on entity pages, one consumed by an Anomaly-type analytics rule before it can ever reach the incident queue. This is a structural sketch built from this part's own reading of Microsoft's UEBA documentation, not a reproduction of an official Microsoft architecture diagram and not a capture from a running tenant.

9. Cost and program placement

[SOC MANAGEMENT] UEBA carries no separate license fee. What it costs is the ingestion cost of the underlying tables it reads (Azure AD sign-in and audit logs, Security Events, Office 365 activity, and whatever else is enabled as a source per \$2) — cost a workspace collecting those sources for other detection purposes is very likely paying already. That makes UEBA one of the more straightforwardly

justifiable additions in this book's whole platform for a SOC weighing where to spend tuning effort: turning it on adds identity-context enrichment and two new scoring signals without a separate billing line, and the actual cost center is the analyst time spent learning to read investigation priority and anomaly scores correctly (§5's conflation risk) rather than any additional spend.

SOC Management View *A program-management framing worth giving a director asking "why hasn't UEBA produced any incidents": the honest answer might be that it's working exactly as designed, with zero Anomaly rules currently tuned to act on its output (§1). That's a rule-layer gap, not a UEBA failure, and the fix is a Part 5 rule-tuning conversation — duplicating and thresholding an Anomaly rule template — not a request to re-evaluate whether UEBA itself is worth keeping enabled.*

10. Putting it together

[CONCEPT] UEBA's contribution to a Sentinel deployment is best understood as a context layer, not a detection layer in its own right. It doesn't replace the analytics rules covered in Parts 4 and 5; it makes the entities those rules fire on legible in a way a bare event never is on its own — this account's peers, this account's reach, whether this specific activity has ever happened before for this specific entity. The two scores it feeds (§5) exist because triage prioritization and rule-triggering are genuinely different jobs, and the newer behaviors layer (§6) is a reminder that even a mature, multi-year-stable feature like UEBA still has an actively developing edge — one this book describes honestly as unsettled rather than pretending to have validated it against evidence that doesn't exist in this author's environment.

Cross-references: DEH Part 31 (Baselining — the general technique UEBA's peer-group and blast-radius analysis operationalizes against identity data); DEH Part 12–13 (Identity Detection Engineering — the thin-evidence authentication-event problem UEBA's behavioral context exists to address); DEH Part 25 (KQL syntax fundamentals underlying the hunting fragment in §8); Part 5 (Anomaly-type analytics rules, the rule layer that consumes `BehaviorAnalytics` and writes `Anomalies`); Part 6 (entity mapping and the incident model, which `IdentityInfo` enrichment feeds); Part 10 (Watchlists and reference data, the adjacent reference-data layer UEBA's peer groups complement); Part 9 (Hunting in Sentinel and the Defender portal, for the notebook-based hunt workflow the §3 peer-set hunt belongs in).

Part 12 — AI-Assisted Operations: Security Copilot and Agentic Platform Tooling

Why this part exists

This part covers three things Microsoft ships on top of Sentinel and Defender data rather than inside a rule or a workspace setting: Microsoft Security Copilot itself (the standalone experience and its embedded panes), the newer Security Copilot agents that act on a trigger instead of a prompt, and Microsoft Sentinel's own agent-facing tool surface built on the Model Context Protocol (MCP). Every one of those three things shipped, changed shape, or moved out of prerelease status on a timeline measured in months, not years — this is, deliberately, the least stable ground in the book, and the part says so up front rather than dressing a snapshot up as settled architecture.

That instability changes how this part is written. Every specific claim about what a capability does, what it costs, or what release stage it's in carries a dated citation to a Microsoft Learn page, and most carry a **PRODUCT VERSION NOTE** rather than treating the fact as background knowledge. Where Microsoft's own documentation flags something as prerelease or preview, this part repeats that flag rather than smoothing it into confident prose.

This part does not re-teach KQL. When Security Copilot generates a Kusto Query Language (KQL) query from a natural-language prompt, the output is still KQL, and it still has to be read and validated against the same syntax fundamentals DEH Part 25 already teaches — a generated query that parses is not the same thing as a generated query that means what the analyst intended. This part also isn't the AI-security part of this library: DEH Part 20 (AI Systems Telemetry and Audit Trail Engineering) and DEH Part 21 (AI Security Detection Engineering) cover the inverse problem — treating an AI system as something to monitor and defend. This part covers the problem the other direction: using an AI system as a tool inside a SOC's own platform operations.

1. Security Copilot: what it is and where it lives

[CONCEPT] Microsoft Security Copilot is a generative-AI security-analysis product with a standalone web experience (at securitycopilot.microsoft.com) and a set of embedded panes inside other Microsoft security products. As of Microsoft's own current overview documentation, Security Copilot connects into Microsoft Defender XDR, Microsoft Sentinel, Microsoft Intune, and Microsoft Entra, along with third-party products including ServiceNow and Jamf (Microsoft Learn, "What is Microsoft Security Copilot?", learn.microsoft.com/en-us/copilot/security/microsoft-security-copilot, retrieved 2026-09-15). This part follows Part 2's framing of the platform's two-portal split and treats the Microsoft Defender portal as Copilot's durable home for anything Sentinel-relevant to an incident; whether a Copilot pane still ships inside the classic Azure portal's Sentinel blade by the time you're reading this is exactly the kind of fact Part 16's migration guidance already tells a reader to re-check rather than assume.

Microsoft's own use-case list for Security Copilot, stripped of its marketing framing, comes down to a short set of falsifiable jobs: turning an incident's alerts and entities into an actionable summary with step-by-step remediation suggestions, translating a plain-language question into a KQL query or

reverse-engineering a suspicious script, summarizing posture and policy information across a tenant, and — the newest addition — letting a developer build and add agents to the Copilot ecosystem (same source). The mechanism behind all of these is a fixed request/response cycle Microsoft calls grounding: a user's prompt is preprocessed through plugins that add organizational context, the modified prompt goes to the underlying language model, and the model's response is post-processed through plugins again before it's shown to the user (same source). Section 2 below is about what that plugin layer actually is and, more importantly, what it can't do.

2. Grounding, plugins, and the permission boundary

[CONCEPT] A plugin is the mechanism by which Security Copilot reaches into a specific Microsoft or third-party product's data — event logs, alerts, incidents, and policies — instead of answering purely from the underlying language model's general training (Microsoft Learn, "What is Microsoft Security Copilot?", retrieved 2026-09-15). Built-in plugins exist for the major Microsoft security products; Copilot's own usage-monitoring dashboard logs which plugin backed a given session, with `Microsoft Entra` and `Microsoft Defender XDR` given as the documented example plugin names (Microsoft Learn, "Manage security compute unit usage in Security Copilot," learn.microsoft.com/en-us/copilot/security/manage-usage, retrieved 2026-09-15). Building a custom plugin is a developer-facing task with its own manifest format that this book doesn't reproduce — the same "don't re-teach what another document already owns" discipline this book applies to KQL applies here too.

The operational fact worth internalizing is a permission boundary, not a feature list: Microsoft's own privacy and data-security documentation states plainly that "Security Copilot runs queries as the user, so it never has elevated privileges beyond what the user has" (Microsoft Learn, "Privacy and data security in Microsoft Security Copilot," learn.microsoft.com/en-us/copilot/security/privacy-data-security, retrieved 2026-09-15). Concretely, that means an incident summary Copilot writes is built from incident, alert, and entity data — including the `SecurityIncident` table's own columns where a Sentinel-connected workspace backs the incident — that the signed-in analyst could already see without Copilot's help. Copilot doesn't grant a junior analyst visibility into a workspace their role can't already query, and it doesn't quietly widen an investigation's scope beyond what the user's own Microsoft Entra role and Sentinel/Defender RBAC assignment already permit.

Engineering Reality *Because grounding runs as the signed-in user, the same prompt asked by two analysts with different RBAC scopes over the same incident can come back with two different Copilot summaries — one may see cross-workspace context the other's role doesn't expose, and there is no separate "Copilot-level" permission to reconcile the difference. A SOC that treats a Copilot summary as the record of an incident, rather than one analyst's own access reflected back through a summarizer, will eventually get surprised when a second analyst's summary doesn't match — not because Copilot is wrong, but because the two analysts were never looking at the same underlying dataset to begin with.*

3. What an analyst sees: capabilities inside the unified incident queue

[ANALYST]

Inside the unified incident queue this book describes in Part 15, an analyst working an incident that includes Sentinel-sourced alerts encounters Copilot as an embedded pane rather than a separate destination. The capabilities that pane exposes track directly to Microsoft's documented use cases: an auto-generated incident summary, guided step-by-step response suggestions, a natural-language-to-KQL translator for building an Advanced Hunting query, and script- or code-analysis for a submitted artifact (Microsoft Learn, "What is Microsoft Security Copilot?", retrieved 2026-09-15). Layered on top of individual prompts, promptbooks package a fixed multi-step prompt sequence — Microsoft's own SCU usage documentation names "Script Analysis" as one example, describing it as a promptbook that calls five underlying prompts in sequence (Microsoft Learn, "Manage security compute unit usage in Security Copilot," retrieved 2026-09-15).

Table 12.1 lays out what an analyst actually sees, what each capability is grounded on, and — the part that matters most for this book's own scope — what the analyst still owns after Copilot responds.

Capability	What it produces	Grounded on	Analyst still owns
Incident summarization	Narrative summary of an incident's alerts, entities, and timeline	Incident, alert, and entity data already visible to the signed-in user	Confirming the summary matches the entity graph, not just the prose
Guided response	Step-by-step remediation suggestions for the open incident	Plugin-mediated context from Defender XDR and connected products	Deciding whether to execute — the suggestion isn't the action
Natural-language KQL generation	A draft KQL query translated from a plain-language question	The workspace's own accessible table schema, surfaced through grounding	Validating syntax and logic against DEH Part 25 before running it
Script or code analysis	Deobfuscation and a behavioral explanation of a submitted script	The script content itself, plus threat-intelligence plugins	Independently confirming any verdict before acting on it
Promptbooks	A saved, reusable multi-step prompt sequence for a recurring task	Whatever plugins the promptbook's individual prompts call	Reviewing the steps at least once before trusting the output blindly

False Positive Trap *This one is a platform-behavior trap, not a query-tuning one. A Copilot-generated incident summary and an automation-rule comment added to the same incident (Part 13) can both restate the same underlying alert fields in slightly different words. A second-shift analyst who reads both and treats them as two independent pieces of corroborating evidence — rather than two paraphrases of one dataset — can walk away with false confidence in a finding that was never actually cross-checked against anything.*

Blind Spot Summarization is a smoothing function by design: it turns a mix of populated, empty, and unresolved fields into confident-sounding prose. An entity that failed to resolve, a field that came back null, or an alert that's still enriching in the background can all disappear into a fluent sentence that reads as more complete than the underlying data actually is. Reading only the summary, and never the entity graph or the raw alert it was built from, is how that gap stays invisible.

4. Licensing and Security Compute Units

[SOC MANAGEMENT] Security Compute Units (SCUs) are the billing and capacity unit for every Security Copilot workload — the standalone portal, embedded panes, Microsoft-built agents, partner-built agents, and any other Copilot feature all draw against the same SCU pool (Microsoft Learn, "Microsoft Security Copilot Security Compute Units and capacity," learn.microsoft.com/en-us/copilot/security/security-compute-units-capacity, retrieved 2026-09-15). Eligible Microsoft 365 E5 and E7 customers get a default capacity auto-provisioned; every other customer has to provision SCUs before using Security Copilot at all (same source).

Two capacity types apply once SCUs are provisioned. Provisioned capacity is a baseline allocation billed per hour: it refreshes on fixed clock-hour blocks (9:00–10:00, 10:00–11:00, not a rolling 60-minute window from whenever it was set up), and unused units in a given hour simply expire rather than rolling over (same source). Overage capacity absorbs demand above the provisioned baseline and is billed on consumption, to one decimal SCU rather than rounded to a whole unit (Microsoft Learn, "Manage security compute unit usage in Security Copilot," retrieved 2026-09-15). Whether a given capability's usage is billed at all depends on that capability's own release stage, not on the SCU model in general — Table 12.2 reproduces Microsoft's own breakdown.

Capability release stage	SCU consumption charge
Generally available (GA)	Yes
Public preview	Yes
Private preview	No

(Source: Microsoft Learn, "Manage security compute unit usage in Security Copilot," retrieved 2026-09-15.)

Engineering Reality The clock-hour refresh is a cost-planning trap disguised as a billing footnote. Microsoft's own documentation gives the example directly: provision three SCUs, and you get three SCUs every full clock hour — set up at 9:05, and you still only had until 10:00 before that hour's allocation expired unused, with no rollover into the next hour (Microsoft Learn, "Microsoft Security Copilot Security Compute Units and capacity," retrieved 2026-09-15). A SOC that sizes provisioned capacity by peak-hour demand and assumes idle hours "bank" unused units for a later burst will find the bill doesn't work that way — idle capacity is capacity paid for and thrown away, every single hour, by design.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's public pricing page lists an example rate of \$4 per provisioned SCU-hour and \$6 per overage SCU as of this writing, with an explicit disclaimer that these are estimates, not quotes, and that actual pricing varies by agreement, purchase date, currency, and applicable tax (microsoft.com/en-us/security/pricing/microsoft-security-copilot/, retrieved 2026-09-15; see also the Azure pricing calculator linked from Microsoft's own capacity documentation). If you're budgeting against this figure, run it through the calculator against your own agreement and region rather than citing this book's number — SCU pricing is exactly the kind of fact this callout exists to flag as provisional.

5. From assistant to agent

[CONCEPT] Everything in Sections 1–4 is Copilot responding to something a human typed. Security Copilot agents are a different shape: an agent runs on a trigger — a schedule, or a system event — rather than waiting for a prompt, and it keeps running with a defined scope of permissions rather than a one-off answer (Microsoft Learn, "Microsoft Security Copilot agents overview," learn.microsoft.com/en-us/copilot/security/agents-overview, retrieved 2026-09-15). Microsoft's own agents-overview page names threat-intelligence briefings and Conditional Access optimization as example resource-intensive tasks agents are meant to absorb (same source) — useful as illustrations of the shape of the work, not as a promise that either example still exists as a named, installable agent by the time you're reading this.

Security Copilot agents

An agent needs an identity to authenticate when it runs. Microsoft's documentation describes two models: a dedicated identity created through Microsoft Entra Agent ID — currently available only for Microsoft-built agents — or the agent simply running under the credentials of the user who set it up, inheriting that user's own access for as long as it's active (same source). The second model is the direct extension of Section 2's permission-inheritance principle into agentic territory: an agent running as a human's inherited identity is bounded by exactly that human's own RBAC scope, no wider. Agents draw against the same SCU pool as every other Copilot workload (same source), and — per Table 12.2 — whether a given agent's usage is even billed depends on whether that agent's underlying capability has left private preview.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's own agents-overview and custom-agent-development pages both open with the same disclaimer: "Some information in this article relates to a prereleased product which may be substantially modified before it's commercially released" (Microsoft Learn, "Microsoft Security Copilot agents overview" and "Security Copilot Agent Development Overview," [Learn.microsoft.com/en-us/copilot/security/developer/custom-agent-overview](https://learn.microsoft.com/en-us/copilot/security/developer/custom-agent-overview), both retrieved 2026-09-15). Treat every named agent, permission model, and identity mechanism in this section as subject to change without a deprecation notice resembling the kind Part 2 documents for the Azure-portal retirement — check the cited URLs directly before assuming any of this has stabilized.

What Would Change My Mind This book's default recommendation — chained directly from the PRODUCT VERSION NOTE above — is to require a human-in-the-loop approval step before any Security Copilot agent, Microsoft-built or custom, takes an action with an external effect (isolating a host, disabling an account, sending a notification outside the SOC). Microsoft's own framing already leans this way, describing agents as something that "keep you in control on the actions it takes" rather than something that acts unsupervised by default (Microsoft Learn, "Microsoft Security Copilot agents overview," retrieved 2026-09-15). If a durable, audited track record of unsupervised agent actions with a mature rollback mechanism becomes the documented norm rather than the exception, this recommendation should soften from "approve every external-effect action" to "approve the agent's scope once and audit its actions after the fact" — a materially different governance posture for a SOC manager to sign off on.

6. The Sentinel data lake's agent-facing surface

[ENGINEERING] Part 17 owns the cost and retention depth of the Sentinel data lake tier; this section covers only the piece relevant to AI tooling. The data lake tier mirrors data already in the analytics tier into an open Parquet format, supports retention of up to twelve years, and is queried through a KQL query editor (including scheduled or one-time jobs that promote data back into the analytics tier) and through Jupyter notebooks for Python-based analysis (Microsoft Learn, "Microsoft Sentinel data lake overview," learn.microsoft.com/en-us/azure/sentinel/datalake/sentinel-lake-overview, retrieved 2026-09-15). All lake activity — KQL queries, notebook runs, job management — is captured in an audit log enabled by default (same source).

The Sentinel MCP server

The Model Context Protocol (MCP) is an open, vendor-neutral protocol — not a Microsoft invention — that standardizes how an AI application connects to external tools and data through a client-server architecture: an MCP host (the AI application), an MCP client (the connection the host maintains), and an MCP server (the program that actually provides context and tools) (modelcontextprotocol.io; Microsoft Learn, "What is Microsoft Sentinel's support for MCP?", learn.microsoft.com/en-us/azure/sentinel/what-is-microsoft-sentinel-support-for-mcp).

[us/azure/sentinel/datalake/sentinel-mcp-overview](https://learn.microsoft.com/en-us/azure/sentinel/datalake/sentinel-mcp-overview), retrieved 2026-09-15). Microsoft's own documentation gives Visual Studio Code as the example MCP host, connecting as a client to the Sentinel MCP server (same source).

That Sentinel MCP server is documented as a unified, fully hosted interface requiring no infrastructure deployment on the customer's part, authenticating through Microsoft Entra, and organized into scenario-focused tool collections rather than one undifferentiated tool list (same source). As of the cited retrieval date, three collections are documented:

- **Data exploration** — natural-language querying over the data lake without requiring the caller to already know table schema or write well-formed KQL, plus an "entity analyzer" sub-tool for retrieving and reasoning over an entity (a URL, a user) across the organization's security data for enrichment during triage (same source).
- **Agent creation** — the same MCP-tool-driven agent-building path referenced in Section 7 below, letting a developer describe an agent's intent in natural language from inside an MCP-compatible IDE and have the tooling generate and iterate on the resulting agent (Microsoft Learn, "Security Copilot Model Context Protocol," learn.microsoft.com/en-us/copilot/security/developer/mcp-overview, retrieved 2026-09-15).
- **Triage** — tools that integrate an AI model with APIs supporting incident triage and threat hunting directly, aimed at reducing mean time to resolution (Microsoft Learn, "What is Microsoft Sentinel's support for MCP?", retrieved 2026-09-15).

This is, in practical terms, an MCP-mediated alternative front door onto the same territory DEH Part 34–36 and this book's own Part 9 already cover as native in-portal hunting — the data exploration and triage collections exist so an AI agent (or a developer working from an IDE) can do a version of the same enrichment and triage work a human hunter does natively in the Defender portal.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's own MCP documentation for Security Copilot notes that "Creating an agent using natural language (NL2Agent) is in Private Preview, but globally available" (Microsoft Learn, "Security Copilot Model Context Protocol," retrieved 2026-09-15) — read that literally: available in every supported region, but still a gated, actively-changing preview feature, not a stable capability every tenant can assume it has. Both the Sentinel MCP server and the NL2Agent capability it supports are, by Microsoft's own framing, the newest and least settled surface described anywhere in this book. If you're reading this more than a few months after 2026-09-15, treat every tool-collection name, and the client/server architecture description itself, as something to re-verify against the cited URLs rather than something this book has locked in for you.

flowchart LR

```
subgraph SENTINEL["Microsoft Sentinel"]
    AT[Analytics tier]
    DL["Data lake tier<br/>Parquet, up to 12 years"]
    AT <--> DL
end
```

```
subgraph XDR["Defender XDR signal"]
    EP[Endpoint]
    IDN[Identity]
    CA["Cloud Apps"]
    OFF["Office 365"]
end
```

```
DP["Unified incident queue<br/>Microsoft Defender portal"]
AT --> DP
EP --> DP
IDN --> DP
CA --> DP
OFF --> DP
```

```
subgraph COPILOT["Security Copilot"]
    OR["Orchestrator and grounding"]
    PL["Plugins: Sentinel, Defender XDR,<br/>Entra, Intune, third-party"]
    PB[Promptbooks]
    PL --> OR
    PB --> OR
end
```

```
DP -->|"incident context, user's own permissions"| OR
```

```
subgraph MCPSVR["Sentinel MCP server<br/>hosted, Entra ID auth"]
    DE["Data exploration and<br/>entity analyzer tools"]
    ACT["Agent creation tools"]
    TRI["Triage tool collection"]
end
DL --> DE
```

```
MCPC["MCP client<br/>e.g. VS Code plus GitHub Copilot"]
DE --> MCPC
ACT --> MCPC
TRI --> MCPC
```

```
AG["Security Copilot agent<br/>Entra Agent ID or inherited user identity"]
MCPC -->|builds and deploys| AG
OR --> AG
AG -->|human-in-the-loop checkpoint| DP
```

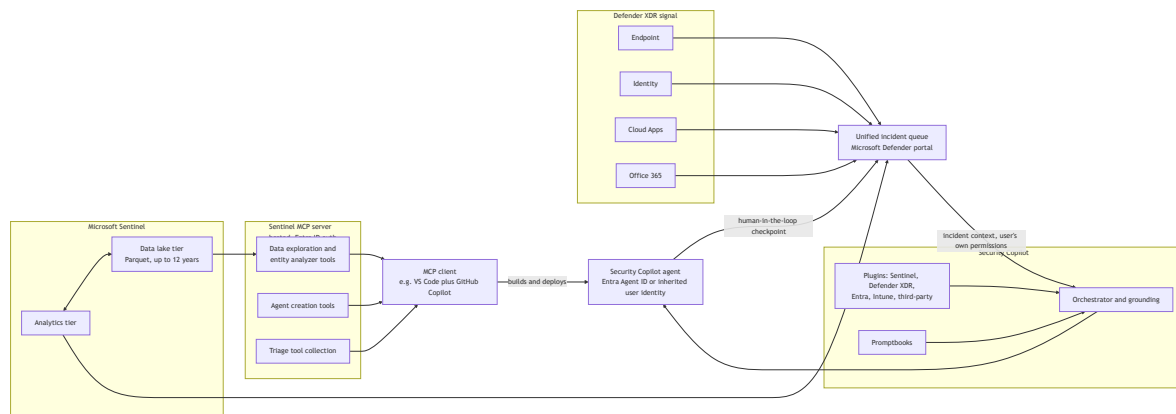


Figure 12.1 — How Security Copilot, Security Copilot agents, and the Sentinel MCP server sit on top of Sentinel/Defender data. *CONCEPTUAL.* Illustrates the relationship between the analytics and data-lake tiers and Defender XDR signal (Part 2, Part 15, Part 17), the unified incident queue an analyst already works in, Security Copilot's plugin-mediated grounding into that same data under the signed-in user's own permissions (Section 2), and the separate, developer-facing Sentinel MCP server surface used to build agents from an MCP-compatible IDE (Section 6). This is a structural sketch built from this book's own reading of the Microsoft Learn pages cited in this part's text — not a reproduction of an official Microsoft architecture diagram, and not a capture of any live tenant.

7. Building custom agents

[SENTINEL ENGINEER]

A custom Security Copilot agent is assembled from five documented components: tools (skills) the agent can call, triggers that initiate it, an orchestrator that decides how tasks execute, instructions the agent follows, and feedback stored to guide later runs (Microsoft Learn, "Security Copilot Agent Development Overview," retrieved 2026-09-15). Microsoft documents three ways to reach that structure — describing the agent in natural language (NL2Agent), building it interactively in an agent-builder interface, or authoring a YAML manifest directly in an IDE and uploading it — all three converging on the same YAML manifest format that actually gets deployed (same source). The MCP-based path described in Section 6 is a fourth entry point onto that same convergence point: an MCP-compatible IDE client calling the Sentinel MCP server's agent-creation tools to generate that YAML from a natural-language description (Microsoft Learn, "Security Copilot Model Context Protocol," retrieved 2026-09-15).

Publishing a custom agent is gated by Security Copilot's own role model, which Microsoft documents as distinct from Microsoft Entra and Azure RBAC roles: a Copilot contributor can build, test, and publish an agent at user scope, while a Copilot owner can additionally publish at workspace scope, making the agent visible to anyone in that Security Copilot workspace (Microsoft Learn, "Security Copilot Agent Development Overview," retrieved 2026-09-15).

The following sketch illustrates the shape of a manifest's contents, not a validated schema — build against Microsoft's own developer documentation, not this block, when authoring a real one.

```
# CONCEPTUAL SAMPLE – illustrative shape of a Security Copilot custom-agent manifest,
# not a validated or deployable manifest. Field and value names are simplified for
# teaching purposes; the real manifest schema is Microsoft's developer documentation
# to own, not this book's.
name: compromised-account-triage
description: Triage a suspected compromised-account incident and drafts a summary.
trigger:
  type: schedule
  interval: 15m
tools:
  - skill: get-incident-entities
  - skill: run-data-lake-query
  - skill: draft-incident-comment
orchestrator:
  type: sequential
instructions: >
  When a new high-severity identity incident appears, gather its entities,
  check sign-in risk history, and draft (but do not post) a triage summary
  for human review.
```

The limitation that block can't show: a real manifest's `skill` references bind to actual registered tools with their own permission and plugin dependencies, and the orchestrator field governs actual execution semantics (sequential versus something more branching) that this sketch only names, not implements.

8. Where AI assistance stops and platform operations resume

[ANALYST] The discipline this whole part comes down to is a boundary, not a checklist: Copilot and its agents produce drafts — a summary, a suggested response, a translated query, a generated manifest — and something else in the platform has to turn a draft into an effect. A Copilot-generated KQL query has to pass the same validation any hand-written query would, against the syntax fundamentals DEH Part 25 already teaches, before it's trusted inside a scheduled rule or a hunt. A guided-response suggestion isn't itself an action; Part 13's automation rules and Part 14's playbooks are what actually execute a remediation step, and an agent's "keep you in control on the actions it takes" framing (Microsoft Learn, "Microsoft Security Copilot agents overview," retrieved 2026-09-15) is only true in practice if a SOC's own runbooks make that human checkpoint explicit rather than assumed.

[SOC MANAGEMENT] That boundary is a governance decision, not a per-analyst judgment call. Deciding which agents get an Entra Agent ID with standing permissions versus which run only under an inherited user identity, deciding which capabilities are approved for use while still in preview (and therefore, per Table 12.2, potentially free to run but also potentially about to change shape), and deciding who reviews an agent's activity log for drift from its original instructions are all questions that belong at the same level as the Defender-portal migration decision this book's Part 16 already frames as a program-management problem, not a toggle a single engineer flips.

Cross-references: DEH Part 20 (AI Systems Telemetry and Audit Trail Engineering) and DEH Part 21 (AI Security Detection Engineering) — the inverse problem of securing an AI system rather than using one; DEH Part 25 (KQL syntax underlying any Copilot-generated query); this book's Part 2 and Part 16 (the Microsoft Defender portal as Copilot's durable home, and the Azure-portal retirement this part deliberately doesn't re-litigate); this book's Part 9 (native in-portal hunting, the workflow the MCP triage

and data-exploration collections mirror); this book's Part 13 and Part 14 (automation rules and playbooks — where a guided-response suggestion actually becomes an executed action); this book's Part 15 (the unified incident queue Copilot's embedded pane sits inside); this book's Part 17 (Sentinel data lake cost and retention depth this part deliberately left out).

neshbooy

Section E — SOAR and Automation

neshbooy

Part 13 — Automation Rules: Incident-Level Orchestration

Why this part exists

[CONCEPT] Parts 4 through 6 covered how an analytics rule — Scheduled, Near-real-time (NRT) (fixed one-minute evaluation cadence, limited subset of scheduled-rule features), Fusion (advanced multistage attack detection — single instance, not customizable, unavailable once Defender XDR incident integration is on), or Anomaly (ML-baselined, writes to the `Anomalies` table, doesn't alert on its own) — turns a KQL query into an alert, and how alerts group into an incident through entity mapping. None of that answers the next question a running SOC actually has to answer: once an incident exists, what happens to it in the seconds and minutes before a human looks at it?

Automation rules are Microsoft Sentinel's answer. An automation rule is a workspace-level, no-code orchestration object that watches for an alert or incident event and applies a defined set of actions — change status, change severity, assign an owner, add a tag, add a task, or hand off to a playbook — without requiring a Logic Apps workflow for the common cases. Before automation rules existed as a distinct object, this logic lived inside individual playbooks wired one-to-one to individual analytics rules; automation rules exist specifically to pull that logic out of Logic Apps and into a layer that can apply across many analytics rules at once, in a defined order, with conditions a SOC engineer can read without opening a workflow designer.

This part covers what an automation rule is made of, how triggers/conditions/actions/order interact, the specific batching behavior that can make an "incident updated" trigger miss an intermediate state, and the behavior changes this book's Part 16 treats as a full migration project once a workspace onboards to the Microsoft Defender portal. It does not cover playbook internals (Part 14), entity-mapping mechanics (Part 6), or KQL syntax (DEH Part 25) — an automation rule's conditions reference entities and fields that Part 6 already explains how they got there, and this part assumes that groundwork rather than re-deriving it. The governance angle here parallels DEH Part 22's separation-of-duties argument for detection rules, applied to the layer that can auto-close a real incident just as easily as a noisy one.

1. What an automation rule is, and where it sits in the pipeline

[CONCEPT] An automation rule is an Azure resource — `Microsoft.SecurityInsights/automationRules` — scoped to a Log Analytics workspace's Sentinel instance, not to a single analytics rule. That workspace scope is the point: one automation rule's conditions can select "any analytics rule," a named subset, or a specific alert provider, so a SOC that used to maintain a dozen playbook-per-rule bindings can instead maintain a handful of automation rules that each apply a policy across many rules at once.

Every automation rule is built from four parts:

- **Trigger** — the event that causes the rule to evaluate (§2).
- **Conditions** — the filter that decides whether this specific alert or incident matches the rule (§3).
- **Actions** — what happens if the conditions match, executed in the order specified inside the rule (§4).

- **Rule order and expiration** — where this rule sits relative to other automation rules in the same workspace, and an optional date after which the rule stops evaluating (§4).

Figure 13.1 places automation rules in the pipeline that starts with an analytics rule and ends with either a resolved incident or a handoff to a playbook.

flowchart LR

```
A["Data sources\n(connectors, DCRs)"] --> B["Analytics rule\nScheduled / NRT / Fusion / Anomaly"]
B --> C["Alert"]
C --> D["Alert grouping /\nincident correlation"]
D --> E["Incident\n(SecurityIncident)"]
C -. "When alert is created" .-> F["Automation rule\nevaluation, in order"]
E --> |"When incident is created /\nWhen incident is updated"| F
F --> |"Built-in actions"| G["Status, severity,\nowner, tags, tasks"]
F --> |"Run playbook action"| H["Playbook\n(Logic Apps workflow)"]
H --> I["External system:\nticket, containment,\nnotification"]
G --> E
```

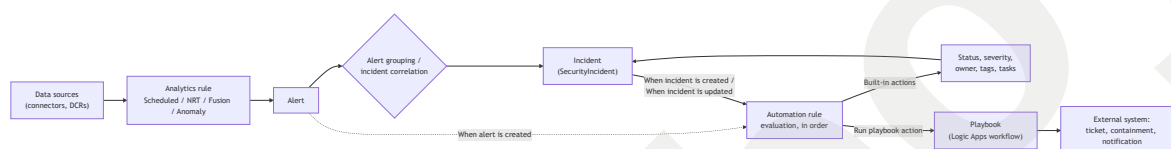


Figure 13.1 — Automation rules in the alert-to-incident pipeline. *CONCEPTUAL.* Illustrates where automation-rule evaluation sits relative to alert creation, incident correlation, and playbook handoff. This is a structural sketch built from this book's own reading of Microsoft's automation-rules and analytics-rule documentation (cited in §2 and §9 below) — not a capture of any Sentinel or Defender portal screen, and not a reproduction of an official Microsoft architecture diagram.

Microsoft's own documentation on this feature — "Automate incident handling in Microsoft Sentinel with automation rules" (learn.microsoft.com/azure/sentinel/automate-incident-handling-with-automation-rules) — is the load-bearing source for the trigger/condition/action model described through §4 of this part; re-check that page directly before relying on any specific field name below, since the automation-rule editor's exact condition list has grown since the feature's initial release.

2. Trigger types

[SENTINEL ENGINEER]

An automation rule fires on exactly one of three trigger types, chosen when the rule is created:

- **When alert is created** — fires on a new alert, before Sentinel's alert-grouping logic has decided which incident (new or existing) the alert joins. Conditions on this trigger evaluate the alert's own properties; there is no "current incident owner" or "current incident status" to condition on yet, because the incident this alert will belong to may not exist at the moment the rule runs.
- **When incident is created** — fires once, the first time an incident is created from one or more grouped alerts. This is the highest-frequency trigger in most workspaces and the one that carries the default triage action set (owner, severity, initial tags) most SOCs configure first.

- **When incident is updated** — fires on a subsequent change to an existing incident: a new alert grouped in, a manual status change, a severity change, a tag added. This trigger can be scoped to fire only on specific field changes rather than every update.

PRODUCT VERSION NOTE (as of 2026-09-15) All three trigger types described above are documented as generally available in Microsoft's current automation-rules reference. The alert-level trigger ("When alert is created") is the newer of the three relative to the original incident-only design and its exact action set (what you can do to an alert before it's grouped into an incident) has been the most actively extended part of the feature over time. If you're building against a workspace and the available conditions or actions for the alert trigger don't match what's described in §3–§4 below, treat the live editor as current and this part as the thing to re-check, not the other way around — this is exactly the kind of fast-moving, rule-availability detail this book flags rather than treats as settled.

[SOC MANAGEMENT] The practical reason to reach for the alert-level trigger instead of always using incident-created is timing: if you need a playbook to run against every single alert regardless of how alerts eventually get grouped — enriching an indicator, for example, before three related alerts merge into one incident — the alert trigger runs early enough to do that. If your action only makes sense once an incident exists (assigning an owner, setting an incident-level tag), incident-created is the correct trigger and using the alert trigger for it just adds an extra evaluation with no benefit.

3. Conditions

[SENTINEL ENGINEER] A condition narrows which alerts or incidents a triggered automation rule actually acts on. Most of the condition surface below belongs to the two incident triggers (When incident is created, When incident is updated); the alert-creation trigger's condition surface is much narrower — Microsoft's documentation describes the analytics-rule scope as the only condition currently configurable there, so an alert-triggered rule cannot condition on an entity value or an incident property the way an incident-triggered rule can:

- **Analytics rule scope** — restrict to a named analytics rule (or a small set), or leave unscoped to apply to every rule in the workspace. This is the condition that makes the "one rule instead of a dozen playbook bindings" consolidation in §5 possible, and the one condition available on every trigger type, including the alert trigger.
- **Alert provider / alert details** — the source product an alert came from (Microsoft Sentinel's own scheduled analytics vs. a connected Microsoft Defender product's native alert), or specific alert fields. Available on the incident triggers.
- **Entity conditions** — match on a mapped entity's value (an account name, a host, an IP), using the entity types Part 6 covers in depth. Available on the incident triggers.
- **Incident property conditions** — current severity, status, title, tags, owner, or custom details. Available on the incident triggers.
- **Updated-field conditions** (incident-updated trigger only) — restrict evaluation to updates that changed a specific property, rather than every update, including who or what made the change (a

user, an app, a playbook, another automation rule, or alert-grouping logic).

Microsoft's editor groups multiple conditions with AND/OR logic rather than a single flat AND-only list; treat the exact grouping UI as something to confirm in your own workspace, since this is one of the areas the automation-rules documentation has revised since the feature's original release.

4. Actions, order, and expiration

[SENTINEL ENGINEER] When an automation rule's conditions match, its actions run in the order listed inside that rule. Built-in actions include changing incident status, changing severity, assigning an owner, adding or removing tags, adding an incident task (a checklist-style item for whoever picks up the incident), and adding an incident comment. The action set available differs slightly by trigger type — an alert-level trigger can act on the alert itself but not on incident-level fields that don't exist yet at that point in the pipeline.

Two properties apply to the rule as a whole rather than to an individual action:

- **Order** — a number that determines the sequence automation rules in the same workspace evaluate against a matching event. A later-ordered rule can observe the effects of an earlier one (a severity change from rule 1 is visible to a condition in rule 2), which makes rule order a real design decision, not administrative housekeeping.
- **Expiration date** — an optional date after which the rule stops evaluating automatically. This is the built-in mechanism for a deliberately temporary rule (suppressing a known-noisy source during a planned maintenance window, for example) without leaving a stale rule active indefinitely.

Engineering Reality Rule order matters most exactly where it's easiest to overlook: two automation rules that each independently look correct in isolation can produce a different combined result depending on which runs first. A rule that reassigns owner based on current severity, placed before a rule that raises severity on the same incident, assigns based on the old severity every time — not a bug in either rule individually, but a sequencing defect visible only when you read both rules together in order. Review automation rules as an ordered list, not as a set of independent objects, every time you add or reorder one.

Run playbook — the action that hands off to Part 14

[SENTINEL ENGINEER] "Run playbook" is the action that connects this layer to Part 14: instead of (or alongside) a built-in action, an automation rule can invoke a Logic Apps playbook and pass it the triggering alert or incident. Everything about what the playbook does once invoked — its own triggers, its connectors, its managed-identity permissions — belongs to Part 14; this part's concern stops at the handoff. One detail that does belong here because it affects which rules can call which playbooks at all: an automation rule can invoke a playbook built on either Logic Apps hosting plan, Consumption or Standard — the hosting-plan choice does not by itself block the automation-rule handoff. The trigger

types must still match (an incident-trigger automation rule can only run an incident-trigger playbook, and likewise for the alert trigger); see Part 14 for the fuller cost/reliability comparison between the two hosting plans.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's "Automate threat response in Microsoft Sentinel with automation rules" page (learn.microsoft.com/azure/sentinel/automate-incident-handling-with-automation-rules) states that playbooks built on either version of Azure Logic Apps — Standard or Consumption — are available to run from automation rules. Older guidance and some training material describe Consumption as the only well-supported path for this specific integration; if you're working from one of those sources, treat the current Learn page as authoritative and re-check it directly, since hosting-plan parity for this integration is exactly the kind of platform detail Microsoft has actively extended over time.

The following illustrates the shape of an automation rule's ARM export — the resource type is `Microsoft.SecurityInsights/automationRules`, distinct from the `Microsoft.SecurityInsights/alertRules` type an analytics rule exports as.

```

{
  "type": "Microsoft.SecurityInsights/automationRules",
  "apiVersion": "2023-11-01",
  "properties": {
    "displayName": "Triage and escalate: high-severity identity incidents",
    "order": 10,
    "triggeringLogic": {
      "isEnabled": true,
      "triggersOn": "Incidents",
      "triggersWhen": "Created",
      "conditions": [
        {
          "conditionType": "Property",
          "conditionProperties": {
            "propertyName": "IncidentSeverity",
            "operator": "Equals",
            "propertyValues": ["High"]
          }
        }
      ]
    },
    "actions": [
      {
        "order": 1,
        "actionType": "AddIncidentTask",
        "actionConfiguration": {
          "title": "Confirm identity provider MFA logs for the affected account"
        }
      },
      {
        "order": 2,
        "actionType": "RunPlaybook",
        "actionConfiguration": {
          "logicAppResourceId":
"/subscriptions/<subId>/resourceGroups/<rg>/providers/Microsoft.Logic/workflows/<playbookName>"
        }
      }
    ]
  }
}

```

CONCEPTUAL SAMPLE — illustrative `automationRules` ARM export; field and action-type names shown at the confidence level of documentation review, not verified against a live tenant export.

The stated limitation this book's honesty constraint requires: treat the exact casing, nesting, and available `actionType` values as something to confirm against a real export or the current ARM template reference before using this as a deployment source, not as a validated schema.

5. The consolidation payoff — and how it breaks

[SENTINEL ENGINEER] The reason automation rules exist as their own object, rather than living inside every playbook, is consolidation: one automation rule, unscoped or scoped to a category of analytics rules, replaces what used to require a separate playbook trigger wired to each rule individually. That

consolidation is real and worth having — but the same breadth that makes one rule powerful also makes one bad condition powerful in the wrong direction.

Detection Autopsy — "close everything from the noisy scanner rule"

The rule: *An automation rule scoped to a single analytics rule (a detection that fires on traffic from a known vulnerability-scanner IP range), triggered on incident creation, with one action: set status to Closed, classification "benign positive."*

Why it shipped: *Scanner-triggered incidents were a known, high-volume nuisance, and auto-closing them by matching on the analytics-rule name looked like a clean way to keep the incident queue readable without editing the underlying query.*

How it failed: *The automation rule's only condition was the analytics-rule-name match — it never re-checked the source IP against the scanner's current address range. When the scanning vendor rotated its egress addresses (routine, and not announced to the SOC), the same analytics rule kept firing under the same name, now sometimes triggered by genuinely unrelated external activity that happened to match the same query logic. The automation rule closed every one of those incidents exactly as automatically as the real scanner traffic, with no analyst ever seeing the difference.*

The fix: *Move the IP-range check into the automation rule's own condition, ideally referencing a watchlist (Part 10) the scanner-operations team keeps current, rather than trusting an analytics-rule-name match alone to imply "this is definitely the scanner." A rule-name match that stops being true stays silent; a watchlist that stops matching produces an incident an analyst actually sees.*

6. The batching window

[SENTINEL ENGINEER] An automation rule triggered on "When incident is updated" evaluates the incident's state at the moment it runs — not a guaranteed record of every intervening change that happened to get it there.

Blind Spot Incidents can receive several updates in quick succession — most commonly, a short-frequency scheduled analytics rule re-running and grouping a new alert into an already-open incident every few minutes. Microsoft's own automation-rules documentation describes the incident-updated trigger's conditions as evaluating current incident state at the moment the rule runs, not a replay of every individual change — so a condition written to catch one specific transition (severity moving from Medium to High, say) can miss it if a later update moves severity again before the rule finishes evaluating. Once a workspace is onboarded to the Microsoft Defender portal, this stops being a subtle evaluation-timing detail and becomes an outright data-loss mechanic: Microsoft documents that if multiple changes land on the same incident within a 5–10 minute period, only a single update — carrying the most recent change — is sent to Microsoft Sentinel, and the intermediate changes are dropped, not merely superseded (see §9 for the fuller set of Defender-portal-onboarding behavior changes this belongs alongside). Either way, design conditions around the state that needs to be true right now (current severity, current status, current owner), not around the assumption that a specific change just happened a moment ago and is still the most recent thing that happened.

PRODUCT VERSION NOTE (as of 2026-09-15) The 5–10 minute batching figure above is documented specifically for workspaces onboarded to the Microsoft Defender portal, in Microsoft's "Automate threat response in Microsoft Sentinel with automation rules" page (learn.microsoft.com/azure/sentinel/automate-incident-handling-with-automation-rules), in its execution-order guidance. The general "evaluates current state, not a replay" behavior applies regardless of portal; the specific window and the outright loss of intermediate updates are described there as Defender-portal-onboarding-specific — re-check that page directly before assuming the same figure applies to a workspace still running Sentinel in the Azure portal.

7. A platform-behavior false positive

[SENTINEL ENGINEER] Part 6 discusses false positives as a detection-logic problem — legitimate activity that looks like an attack. This layer produces a different kind: a platform mechanic firing a response action for reasons that have nothing to do with the incident's actual severity.

False Positive Trap An automation rule conditioned on "status changed to Active" to trigger a paging playbook will re-fire every time a frequently re-triggering scheduled rule adds a new alert to an incident that had drifted back to "New," or that a different analyst manually reopened — not because a human judged the incident newly urgent, but because Sentinel's own alert-grouping logic touched the incident's status as a side effect of folding in one more alert. The fix mirrors Part 10's watchlist-allowlist pattern for detection logic: exclude the specific noisy analytics rule at the automation rule's condition level, or add a dedup/rate check before the paging action fires, rather than loosening the status condition until it stops paging on real re-opens too — that just moves the blind spot instead of closing it.

8. Verifying an automation rule actually ran

[SENTINEL ENGINEER] An automation rule that silently stops running — a deleted playbook it used to call, a permission change on its managed identity, a condition that no longer matches after an upstream field rename — fails the same way a broken analytics rule does: no error, just quiet absence.

Detection Test Setup: A workspace with at least one enabled analytics rule and one automation rule scoped to it, with a condition set to something directly observable (adding a specific tag is the simplest choice to verify). **Action:** Trigger the underlying analytics rule under controlled conditions so it produces a real alert and incident (see Part 4's rule-specific test guidance for how to generate one safely), or use a rule template's built-in sample-alert generation where the template offers it. **Expected result:** The resulting incident carries the tag or status change the automation rule specifies. If it doesn't, check the `SentinelHealth` table before assuming the condition logic is wrong — a silent automation-rule failure (a deleted playbook, a lapsed permission) is recorded there before it shows up anywhere incident-side.

```
// CONCEPTUAL SAMPLE – illustrative SentinelHealth query for automation-rule run status.  
// Exact column names and values should be confirmed against Microsoft's current  
// SentinelHealth schema reference; full schema walkthrough is Part 18's, not this part's.  
SentinelHealth  
| where SentinelResourceKind == "AutomationRule"  
| where Status == "Failure"  
| project TimeGenerated, SentinelResourceName, Status, Description
```

`SentinelHealth` is Sentinel's own health-monitoring table, covering rule, connector, and automation-rule/playbook run status without additional configuration; Part 18 owns the full schema and troubleshooting workflow. The fragment above illustrates only that automation-rule health is queryable the same way rule health is — it is not a substitute for Part 18's schema-accurate reference, and the exact field names above should be treated as illustrative pending that verification, consistent with this book's **CONCEPTUAL SAMPLE** convention for unverified KQL fragments.

9. Automation rules once Defender XDR is in the picture

[SENTINEL ENGINEER]

Onboarding a Sentinel workspace to the Microsoft Defender portal — and turning on Defender XDR incident integration — changes what an automation rule is actually reacting to, because the incident it's conditioned on may now be created by Defender XDR's own correlation engine instead of Sentinel's native logic. Four behavior changes are commonly described in this context, and this book treats them as hazards to check for rather than footnotes, because an automation rule's conditions are precisely the surface where a quiet platform change produces a quiet, hard-to-notice functional change:

- **Provider condition collapse.** A condition built on "alert provider" distinguishes Sentinel-native scheduled-analytics alerts from alerts natively generated by Microsoft Defender for Identity, Defender for Endpoint, Defender for Cloud Apps, and Defender for Office 365. Once Defender XDR's correlation engine is producing the incident, alerts from all of those sources can arrive inside a single Defender XDR incident, and a provider-scoped condition written for the pre-onboarding world may stop discriminating the way it used to.
- **Incident description field removal.** The `SecurityIncident` table no longer includes a `Description` field at all once a workspace is onboarded to the Defender portal — any automation-rule condition or ServiceNow-style integration keyed on it will fail every time, not just occasionally, per Microsoft's "Transition your Microsoft Sentinel environment to the Defender portal" page (learn.microsoft.com/azure/sentinel/move-to-defender).
- **Incident title instability.** Defender XDR's correlation engine can rename an incident's title as it merges in additional alerts over the incident's life. An automation rule condition matching on incident title text is checking a string that can change while the incident is still open.
- **Sharper incident-update batching.** §6's batching window stops being a general "current state, not a replay" caveat and becomes a documented 5–10 minute window in which only the most recent change reaches Sentinel at all — everything else in between is dropped, not just superseded.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft Sentinel in the Azure portal is scheduled for retirement on March 31, 2027; after that date Sentinel operates only in the Microsoft Defender portal, and every workspace still in the Azure portal is redirected automatically. Microsoft's Sentinel overview and "what's new" documentation carry this date as of this writing. The four behavior changes listed above stop being "the Defender-portal case" and become simply "how automation rules work" once that migration is universal — verify them against your own workspace's current behavior, not against this section alone, since portal-integration mechanics are exactly the detail most likely to have shifted between this chapter's `Last_validated` date and whenever you're reading it.

The author has not validated the four behaviors above against a live tenant — this book's honesty constraint (no Sentinel workspace in the author's environment) applies with particular force here, since portal-integration mechanics are easy to get subtly wrong from documentation review alone. Treat the list as a checklist to verify against your own workspace and against Microsoft's current onboarding and migration documentation before relying on any of it operationally.

The table below summarizes the same comparison for quick reference when deciding whether an existing condition needs rework before, or after, a Defender-portal migration.

Automation-rule surface	Azure portal (retiring 2027-03-31)	Defender portal
Alert provider condition	Distinguishes Sentinel-native and each connected Defender product as separate provider values	Reported to collapse toward a single Defender XDR provider value once XDR incident integration is on — reverify against your own workspace
Incident description (<code>SecurityIncident.Description</code>)	Populated by Sentinel's native incident-creation logic	Field removed entirely — any condition or ServiceNow-style integration keyed on it fails every time
Incident title stability	Set at creation by the analytics-rule template; stable unless manually edited	Can be rewritten by the Defender XDR correlation engine as additional alerts merge in
Incident-update batching	Evaluates current state at run time; no documented fixed batching window	Documented 5–10 minute window — only the most recent of several rapid changes reaches Sentinel, intermediate changes dropped
Automation-rule editor location	Sentinel resource blade, Automation tab	Microsoft Defender portal automation settings for the onboarded workspace

What Would Change My Mind This section treats the provider-condition collapse and incident-title instability as hazards to design around now, on the assumption Microsoft will not reintroduce a fine-grained, per-source provider condition once every incident is a Defender XDR incident. If the Defender portal's automation-rule editor shipped a documented, stable equivalent — a condition keyed on originating product or detection source rather than the collapsed provider field — the recommendation would soften from "don't build conditions that depend on the old provider distinction" to "use the new source-specific condition instead," and the table above would need a refresh, not just a footnote.

10. Governance: who can build and change automation rules

[SOC MANAGEMENT] An automation rule is response logic wearing an incident-management interface, and it carries a blast radius most SOC managers underweight relative to a detection rule: a misconfigured detection rule fails by staying silent, but a misconfigured automation rule can actively close or reclassify real incidents at scale before an analyst ever opens the queue.

SOC Management View *Treat a new or edited automation rule — especially one whose action set includes changing status to Closed or changing classification — with the same change-control discipline DEH Part 22 describes for a detection rule: a named author, a second reviewer who tries to break the condition logic before it ships, and a documented rollback (disabling the rule, or reverting its condition, is the rollback here — there is no git history for a change made directly in a portal editor). Scope Microsoft Sentinel's built-in RBAC roles narrowly: per Microsoft's Sentinel RBAC documentation, [Microsoft Sentinel Contributor](#) is documented to include "create/edit resources," which covers authoring or editing a workspace-wide automation rule from the main Automation page. [Microsoft Sentinel Responder](#) is documented only as "all Reader permissions, plus manage incidents" — that reference doesn't spell out whether "manage incidents" extends to authoring an unscoped, workspace-wide automation rule versus only the narrower per-incident suppression-rule shortcut on the Incidents page; confirm which of those your workspace's Responder role can actually reach before assuming it carries the same blast radius as Contributor. The separate [Microsoft Sentinel Automation Contributor](#) role is documented as letting Microsoft Sentinel itself add playbooks to automation rules — explicitly not intended for user accounts — and should be scoped to the resource group holding the playbooks rather than granted at the subscription level.*

Cross-references

DEH Part 22 (Detection as Code — separation-of-duties model applied to response automation in §10 above); DEH Part 25 (KQL syntax underlying the [SentinelHealth](#) fragment in §8); DEH Part 23 (Query Language Strategy — the backend-portability tradeoff §9's provider-collapse hazard is a platform-operations instance of); Part 4 and Part 5 (analytics rule types feeding the alert/incident pipeline in Figure 13.1); Part 6 (entity mapping and the incident model automation-rule conditions reference); Part 10 (watchlists — the allowlist pattern used to fix §5's and §7's failures); Part 14 (playbooks — what the "Run playbook" action hands off to); Part 16 (the Defender-portal migration as a program-management problem, of which §9's behavior changes are one instance); Part 18 ([SentinelHealth](#) full schema and health-monitoring workflow).

Part 14 — Playbooks: Logic Apps–Based Response and Remediation

Why this part exists

[CONCEPT] Part 13 covered automation rules — the ordered, incident-level layer that tags, assigns, and closes incidents, and that can invoke a playbook as one of its available actions. This part covers what actually happens once that invocation fires. A playbook is not a lightweight Sentinel setting the way an automation rule's "change severity" action is; it is a full Azure Logic Apps workflow, deployed as its own Azure resource, with its own trigger, its own connectors, its own execution history, its own throttling limits, and its own bill. Everything that makes Logic Apps a general-purpose integration platform — hundreds of connectors, conditional branching, human-approval steps, retry policies — is available to a Sentinel playbook, and everything that makes a general-purpose integration platform occasionally frustrating to operate (a connector authorized to a departed employee's account, a managed identity missing one specific API permission, a connector's own rate limit) is a Sentinel playbook's problem too. This part is about that second half: what a playbook actually is under the hood, the patterns that cover most real SOC response use cases, the specific permissions failure mode behind "the playbook ran but did nothing," and the cost and reliability profile that comes from being a real Azure resource rather than a Sentinel-native feature.

Two things this part does not do. It does not re-teach automation rules — see Part 13 for trigger types, ordering, and the Defender-portal behavior differences that apply there. And it does not walk through Logic Apps Workflow Definition Language syntax step by step; Microsoft's own Logic Apps documentation owns that in full, and this part cites it only where a Sentinel-specific detail sits on top of it.

1. What a playbook actually is

[CONCEPT] A playbook (an Azure Logic Apps workflow that Sentinel can trigger automatically from an automation rule or run manually against a selected alert, incident, or entity — not a Sentinel-native automation object the way an analytics rule or an automation rule is) is created and stored as an `Microsoft.Logic/workflows` resource in an Azure resource group, exactly like any Logic App built for a non-security integration scenario. Sentinel's Automation blade lists it under Active playbooks once it exists and carries the right trigger, but that listing is a view into the Logic App resource, not a separate copy of it. Opening a playbook from Sentinel's UI drops you into the same Logic Apps Designer (or code view) you'd use for any other Logic App, because it is the same product.

That has a direct, practical consequence covered by Microsoft's own Logic Apps documentation: a playbook is deployed on one of two hosting plans — **Consumption**, a fully serverless, per-execution-billed model with no dedicated compute to manage, or **Standard**, a dedicated-compute model (running on an App Service plan or in a container) that adds VNET integration, more predictable latency, and a flat compute cost regardless of how many times the workflow runs. Which plan a given playbook

uses is a decision made once, at creation time, in the Logic Apps resource creation blade — not a Sentinel-specific toggle — and it shapes every later question in this part about cost and reliability (§6).

Engineering Reality Sentinel does not give a playbook a dedicated monitoring surface, a dedicated SLA, or a dedicated support path distinct from Logic Apps' own. When a playbook misbehaves, the diagnostic trail — run history, per-action inputs/outputs, retry attempts, failure codes — lives in the Logic App resource's own **Overview** → **Runs history** blade in the Azure portal, not inside Sentinel's incident timeline. An analyst working an incident in Sentinel sees only that a "Run playbook" action was invoked and (depending on the automation rule's own logging) whether that invocation succeeded at the trigger level; whether every step inside the workflow actually completed is a separate question answered in a separate resource.

flowchart LR

```
AR["Analytics rule fires an alert"] --> INC["Incident created / updated<br/>(SecurityIncident)"]
INC --> AUTO["Automation rule<br/>(Part 13)"]
AUTO -->|Run playbook action| TRIG["Microsoft Sentinel<br/>incident trigger"]
TRIG --> LA["Playbook<br/>(Logic Apps workflow)"]
LA -->|enrichment| TI["Threat intel / WHOIS API"]
LA -->|containment| ENTRA["Microsoft Entra ID:<br/>disable account"]
LA -->|containment| MDE["Defender for Endpoint:<br/>isolate device"]
LA -->|ticketing| ITSM["ServiceNow / Jira"]
LA -->|notification| APPR["Teams / Email<br/>approval step"]
LA -->|write-back| INC
```

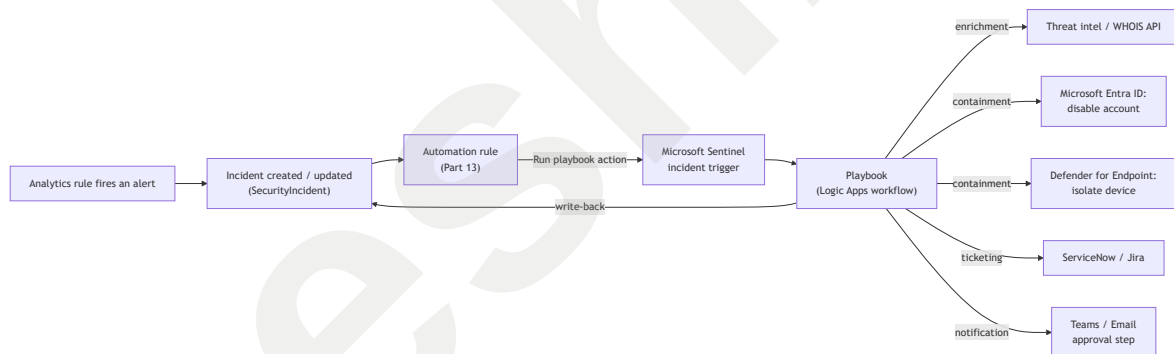


Figure 14.1 — Playbook invocation and fan-out. CONCEPTUAL. Illustrates the path from a fired analytics rule through an automation rule's "Run playbook" action into a Logic Apps workflow, and the workflow's typical fan-out to enrichment, containment, ticketing, and notification systems before writing a result back to the incident. This is a structural sketch of the mechanics described in §§1–4, not a capture of any deployed workflow's actual run graph.

2. Triggers: how a playbook starts

[SENTINEL ENGINEER]

A Logic App built as a Sentinel playbook is distinguished from any other Logic App by which trigger sits at the top of its workflow. Microsoft documents three Sentinel-specific trigger types, and which one a given playbook uses determines what can invoke it and what data it receives.

Microsoft Sentinel incident trigger

The current, recommended entry point for a playbook meant to be invoked from an automation rule. A workflow built on this trigger ("When Microsoft Sentinel incident creation rule is triggered" in the Logic Apps designer's trigger picker) receives the full incident object — its alerts, entities, tags, severity, and status — as its trigger payload, which is what lets a single playbook branch its logic on entity type or alert count rather than needing a separate workflow per analytics rule.

Microsoft Sentinel alert trigger

An older trigger type, scoped to a single alert rather than the incident it belongs to. It predates automation rules' incident-level "Run playbook" action and is the trigger you'll still see on playbooks written before that feature existed. Microsoft's guidance is to build new playbooks on the incident trigger; an alert-triggered playbook can still be invoked manually against a specific alert but cannot be the target of an automation rule's incident-level action.

Microsoft Sentinel entity trigger

Used for the "Run playbook" action available directly on an entity inside the investigation graph or entity page — an on-demand enrichment or lookup against one account, host, IP, or file hash, run by an analyst mid-investigation rather than fired automatically.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's "Automate threat response with playbooks in Microsoft Sentinel" documentation is the authoritative source for which trigger types exist and which one automation rules can target — this is exactly the kind of platform-mechanics detail that has already changed once (the incident trigger was added after the alert trigger, specifically to support automation rules) and could change again. If you're building or auditing playbooks after this date, re-check that page for the current trigger list before assuming the two-trigger-plus-entity-trigger picture above is still complete.

[SENTINEL ENGINEER] A playbook can also be run manually from an incident or alert in the Azure portal by selecting "Run playbook" and choosing from the playbooks whose trigger type is compatible with what's selected. Manual run behavior for playbooks differs in some documented respects once a workspace is onboarded to the Microsoft Defender portal — Part 16 owns that migration-parity detail in full; this part flags only that the difference exists so a reader doesn't assume portal parity by default.

3. Invocation: automation rules versus a standing analytics rule action

[SENTINEL ENGINEER] Two things can start a playbook running against real incident data: an automation rule's "Run playbook" action (Part 13's ordered, incident-level orchestration layer — the action sits alongside tag/assign/close/status-change actions in the same rule, and the rule's own ordering determines whether the playbook runs before or after another action touches the same incident), or a manual "Run playbook" invocation from the portal. A playbook is never invoked directly

by an analytics rule; the analytics rule creates the alert and incident, and everything downstream of that is automation-rule or manual-invocation territory.

Getting an automation rule to successfully invoke a playbook requires a permission grant that is easy to miss because it's separate from anything on the playbook itself: Sentinel's own service principal needs a role — documented as the **Microsoft Sentinel Automation Contributor** role — assigned on the resource group containing the target playbook, so that Sentinel's automation-rule engine is authorized to start someone else's Logic App on the incident's behalf. In practice, the Azure portal's automation-rule editor prompts for this grant the first time you add a "Run playbook" action pointing at a playbook in a resource group that hasn't been granted it yet, and accepting the prompt performs the role assignment for you. It is a separate concern from the managed-identity permissions covered in §5, which govern what the playbook can do once it's running, not whether it's allowed to start.

PRODUCT VERSION NOTE (as of 2026-09-15) *The exact built-in role name and the exact wording of the in-portal permission prompt are the kind of platform-mechanics detail Microsoft has renamed before during the Azure Sentinel-to-Microsoft Sentinel rebrand and could rename again. Verify the current role name against Microsoft's "Automate incident handling in Microsoft Sentinel with automation rules" documentation (the prerequisites section covering playbook permissions) before writing this into a runbook as a literal role name to search for in Azure RBAC.*

4. Common playbook patterns

[SENTINEL ENGINEER] Most production playbooks fall into a small number of shapes, distinguished by what kind of connector call dominates the workflow and how reversible the resulting action is. The table below supports a design decision every new playbook forces: how much of this workflow can run fully automated, and where a human approval step belongs.

Pattern	Typical trigger context	Representative connector / action	Operational note
Enrichment lookup	Alert or incident created	Threat-intelligence, WHOIS, or internal-API connector; generic HTTP action	Read-only against external state; the safest pattern to run fully automated
Ticket creation and sync	Incident created or updated	ServiceNow / Jira connector Create record action	Two-way status sync back into the incident usually needs a second, inbound-triggered playbook or a polling step, not one workflow
Notification and approval	High-severity incident created	Microsoft Teams / Outlook connector, adaptive card with a wait-for-response step	Human-in-the-loop by design; adds the latency discussed in §7
User containment	High-confidence identity alert, usually automation-rule-driven	Microsoft Entra ID connector — disable account / revoke sign-in sessions action	Needs a Microsoft Graph API permission grant on the playbook's own managed identity, separate from any Sentinel role (§5)
Host containment	Endpoint alert, usually automation-rule-driven	Microsoft Defender for Endpoint connector — isolate device action	Needs a Defender for Endpoint API permission scope on the managed identity, entirely separate from both Sentinel RBAC and the Entra ID permission above
Network containment	Automation rule or manual, often paired with an approval step	Network Security Group / firewall connector — update or block-rule action	Often the least reversible action in this table; §7 covers why that argues for keeping a human in this specific loop

[SENTINEL ENGINEER] An enrichment playbook occasionally needs to run a query against the workspace itself rather than call an external API — pulling recent sign-in activity for an account before deciding whether to contain it, for example. The connector for this is Azure Monitor Logs, and its "Run query and list results" action takes a literal KQL query as one of its inputs:

```
// CONCEPTUAL SAMPLE – illustrative enrichment fragment inside a playbook's
// Azure Monitor Logs connector action, not a freestanding detection query.
// Syntax fundamentals (where/join/summarize, time-window functions) are
// DEH Part 25's and DEH Part 23's territory, not re-taught here.
SigninLogs
| where TimeGenerated > ago(1h)
| where UserPrincipalName == "{account from trigger payload}"
| project TimeGenerated, IPAddress, AppDisplayName, ResultType
```

The query body itself is ordinary KQL — for the pipe model, join strategy, and time-window function choices behind a query like this, see DEH Part 25 (KQL for Sentinel/Defender) and DEH Part 23 (Query Language Strategy) rather than expecting this book to re-derive syntax it already covers. What's Sentinel-specific here is only the connector action wrapping it and the fact that its output becomes an input to a later step in the same workflow (a condition, a notification, a containment action) — that wiring is Logic Apps mechanics, not KQL.

5. Managed identities and the permissions problem behind "the playbook ran but did nothing"

Managed identity versus a delegated user connection

[SENTINEL ENGINEER] Every connector a playbook uses — including the Microsoft Sentinel connector supplying its own trigger — needs an authenticated connection to whatever it's calling. Historically, the default way to authorize the Microsoft Sentinel connector was a delegated connection tied to whichever analyst's account happened to be signed in when the connection was created. Microsoft's current guidance favors authorizing the connector with the Logic App's own **system-assigned managed identity** instead, specifically because a managed identity's authorization doesn't depend on any one human's account staying enabled, licensed, and password-current. The tradeoff a managed identity introduces in exchange is that it needs its own, explicit role or API-permission grants for everything it's going to do — it doesn't inherit whatever access the analyst who built the workflow happened to have.

Detection Autopsy — the naive "add the analytics rule, add the containment step, ship it" playbook

The rule: *On a high-confidence identity alert, an automation rule runs a playbook that calls the Defender for Endpoint connector's isolate-device action against the alert's host entity, then posts a Teams notification confirming isolation.*

Why it shipped: *The workflow's own run history in the Logic Apps designer showed a green checkmark on every test run during build — the trigger fired, the Teams notification posted, and nothing in the designer's default view flagged a problem. It looked done.*

How it failed: *The playbook's managed identity had been granted the Sentinel-side role needed to read the incident and post a comment back to it, but no one had separately granted it the Defender for Endpoint API permission scope the isolate-device action actually requires. The isolate-device action returned a permission-denied response, the workflow's default error handling caught that failure and continued to the next step anyway, and the Teams notification — which only confirmed that the workflow completed, not that isolation succeeded — posted exactly as designed. The host stayed fully connected through an incident everyone believed had been contained.*

The fix: *Grant the specific Defender for Endpoint API permission scope to the managed identity directly (a Microsoft Entra ID application-permission grant, not a Sentinel role, and not something the Sentinel-side "grant permissions" prompt in §3 covers), and change the Teams step to fire only after checking the isolate-device action's actual response code — a condition step, not a fixed sequence — so a failed containment action produces a failure notification instead of a success one.*

[SENTINEL ENGINEER] The general shape of that failure — a workflow that completes without error while one specific action inside it silently didn't do what it looked like it did — is the single most common reason a playbook "ran but did nothing" in practice. It is almost never a Sentinel-side problem once the automation rule has confirmed the playbook was invoked at all; it is a downstream connector's own authorization boundary, checked (or not checked) once per action, independently of every other action in the same workflow.

Engineering Reality Individual connector actions (Defender for Endpoint's isolate-device action, Microsoft Entra ID's disable-account action, a third-party firewall connector's block-rule action) are each maintained and versioned by their own connector team, on their own release cadence, independent of Sentinel's. An action's required permission scope, its exact input schema, or its very existence in the connector's action list can change without a Sentinel-specific deprecation notice, because from the connector's point of view it isn't a Sentinel feature — it's one action among the connector's full catalog. Treat every containment action in a production playbook as something to re-validate against the connector's current documentation on a schedule, not as a fact fixed at build time.

6. Cost and reliability: inheriting a full Azure resource's profile

[ENGINEERING] A playbook on the Consumption plan is billed per action execution — every step in the workflow that actually runs on a given trigger, not just the trigger firing itself, counts toward the bill. A short enrichment workflow with three or four steps is inexpensive at almost any incident volume; a longer containment-and-notification workflow with conditional branches, a wait-for-approval step, and several connector calls costs more per run, and that cost scales linearly with how many incidents actually invoke it. A playbook on the Standard plan instead pays for its underlying compute (an App Service plan or container) at a flat rate regardless of how many times it runs that month, which changes the economics in the opposite direction — cheap at very high volume, and a fixed cost you pay even on a quiet month.

SOC Management View The Consumption-versus-Standard choice is a capacity-planning question before it's a technical one: a small number of high-volume, latency-sensitive containment playbooks running against a busy Defender for Endpoint or Microsoft Entra ID tenant are the case for Standard's flat, predictable compute cost and VNET-integration option; a larger number of low-volume, occasionally-run enrichment or ticketing playbooks are the case for Consumption's pay-only-for-what-runs model. Mixing both plans across a playbook library is normal and often the cheaper overall outcome — it isn't an either/or decision made once for the whole automation program.

PRODUCT VERSION NOTE (as of 2026-09-15) Per-action Consumption pricing and Standard plan compute pricing are both published on Azure's own Logic Apps pricing page and change on Azure's general pricing-update cadence, independent of anything Sentinel-specific. This book does not quote a per-action dollar figure or a free-tier action count here because either would likely be stale by the time this is read — check the current Azure Logic Apps pricing page directly before budgeting a playbook library, and re-check it again before assuming a cost model built a year earlier still holds.

[ENGINEERING]

Reliability follows the same inherited-resource logic as cost. A Logic Apps workflow has a default retry policy on most connector actions — a fixed number of retries on a fixed or exponential interval before an action is marked failed — and a connector-specific throttling limit that governs how many calls per minute or per hour that connector's API will accept before returning a throttling error rather than executing the call. Neither of these is a Sentinel setting; both live on the action and the connector respectively, and both apply identically whether the workflow is a Sentinel playbook or an unrelated business-process Logic App. A containment playbook invoked simultaneously across a batch of incidents during a genuinely bad day can hit a connector's own rate limit purely from the volume of near-simultaneous calls, producing queued or retried — not failed outright, but delayed — actions at exactly the moment speed matters most.

Diagnosing any of this happens in the Logic App resource's own run history, per §1's Engineering Reality box — not in Sentinel's incident timeline. Part 18 covers the `SentinelHealth` table as a queryable audit surface for rule and connector health inside the workspace itself; automation-rule and playbook invocation events are among the record types that surface there, which makes `SentinelHealth` a useful first stop for "did this even get invoked," but it does not substitute for the Logic App's own run history when the question is "why did this specific action inside the workflow fail."

7. What a containment playbook is actually racing against

[SENTINEL ENGINEER]

A detection-to-containment playbook exists to shorten the window between an analytics rule firing and an attacker losing the access that triggered it. DEH Part 44 (Adversary Behaviour for Defenders) walks the ATT&CK tactic sequence a real intrusion runs through, including how quickly an adversary already inside a network moves from one compromised identity to the next — T1078 (Valid Accounts) reused across hosts, T1021 (Remote Services) for the lateral hop itself. Every wait-for-approval step, every notification-before-action design choice, and every connector retry interval a playbook design adds is time added to that same window, on the defender's side, while the attacker's side of the clock keeps running regardless of how the playbook is built.

That isn't an argument for removing every human checkpoint from every containment playbook — §4's table already flagged network containment as often the least reversible action in a typical playbook library, and an automated firewall rule change that turns out to be wrong (a false positive disabling a production service, not just a user account) has its own cost. It's an argument for making the automation-versus-approval choice deliberately, action by action, rather than defaulting either to full automation everywhere or an approval step on everything because that feels safer.

What Would Change My Mind This part treats "keep a human approval step on the least-reversible containment actions (network and production-account changes) and fully automate the cheaply-reversible ones (session revocation, device isolation)" as a reasonable default split. If an organization's own incident data showed that its actual mean time from alert to human approval consistently exceeded the attacker's own observed dwell time before lateral movement on the technique the playbook targets, the right conclusion would flip specifically for that technique — the approval step would be costing more in exposure window than it's saving in avoided false-positive containment, and the case for full automation on that one playbook would get stronger, not the reverse.

False Positive Trap An automation rule configured to run a playbook on incident update, not just incident create, re-invokes that playbook every time a matching field changes — including an analyst adding an investigation comment, changing severity during triage, or a correlation engine merging in a new alert. A containment playbook wired to that trigger condition can re-isolate an already-isolated host or re-disable an already-disabled account on every one of those updates, which is harmless against the host itself but produces a burst of duplicate helpdesk tickets and Teams notifications that reads as "the playbook is broken" when the actual problem is the trigger condition. Scope the automation rule's trigger to incident creation (or a specific, narrow update condition) for any playbook whose action isn't idempotent, and note the idempotence question explicitly in the playbook's own documentation rather than assuming the next person configuring the automation rule will re-derive it.

Cross-references

Cross-references: DEH Part 44 (Adversary Behaviour for Defenders — the tactic sequence and dwell-time behavior a containment playbook is racing against, §7); DEH Part 25 (KQL for Sentinel/Defender — syntax for any query a playbook's Azure Monitor Logs action runs, §4); DEH Part 23 (Query Language Strategy — the general translation-cost framing behind that same query); Part 13 (Automation Rules: Incident-Level Orchestration — the layer that invokes a playbook and the permission grant covered in §3); Part 16 (Operating the Unified Security Operations Platform — the Defender-portal manual-run difference flagged in §2); Part 18 (Sentinel-Specific Tuning, Health Monitoring, and Troubleshooting — the `SentinelHealth` table referenced in §6).

Section F — Defender XDR Convergence

neshbooy

Part 15 — Defender XDR Signal Convergence: Identity, Endpoint, Cloud Apps, and Email in One Incident

Why this part exists

Parts 4 through 6 treat a Sentinel analytics rule's alert and the incident it produces as if they existed in a room by themselves — one query, one `SecurityIncident` row, one entity graph built from that rule's own entity mapping. That framing is correct as far as it goes, and it is also incomplete for almost any organization actually running Microsoft Sentinel today, because a Sentinel workspace rarely operates alone. Most production deployments sit next to at least one, and usually several, of Microsoft's other Defender workloads — Microsoft Defender for Identity, Microsoft Defender for Endpoint, Microsoft Defender for Cloud Apps, and Microsoft Defender for Office 365 — each generating its own alerts from its own signal, on its own detection logic, that this book's earlier parts never touch. Microsoft Defender XDR (the platform-wide correlation and incident layer that used to be branded Microsoft 365 Defender) is where those alerts, plus a Sentinel workspace's own analytics-rule alerts once that workspace is onboarded to the Microsoft Defender portal, stop being four or five separate stories and become one incident.

That convergence is a genuine operational payoff — an account takeover that started with a phishing email, moved through a cloud-app session, and ended with a device compromise shows up as one connected narrative instead of four unrelated alerts an analyst has to notice were related on their own. It is also a real investigative tradeoff, because the per-product incident boundary each workload used to draw around "its" alerts mostly disappears once everything lands in one queue, and knowing where a specific piece of evidence actually originated takes deliberate digging that a single-product console never required. This part covers both halves honestly: what actually correlates, what automatic attack disruption does with that correlation, and what an analyst gives up — and has to compensate for — once four products' alerts stop having their own separate front doors.

MITRE: T1078 (Valid Accounts) is the technique most of this part's worked scenarios return to, because a converged incident's most common shape is a single valid-looking credential moving across identity, endpoint, cloud-app, and email surfaces in turn — exactly the kind of multi-surface movement a single-product alert queue is structurally poor at telling one connected story about.

1. Microsoft Defender XDR: what it correlates, and what feeds it

[CONCEPT] Microsoft Defender XDR is Microsoft's cross-product detection, correlation, and incident layer, operated from the Microsoft Defender portal (the same portal Part 2 and Part 16 already establish as Sentinel's newer, converged operating surface — expanded here in full on first mention per this book's notation convention, "the Defender portal" as the acceptable short form afterward). It does not generate its own primary telemetry the way an analytics rule or an EDR sensor does. It ingests alerts already produced by Microsoft's individual Defender workloads and a connected Sentinel workspace's own analytics rules, and it runs a correlation layer on top of that combined alert stream to decide which alerts belong to the same underlying incident.

This part's scope — per this book's own Part Table — is the four workloads named in the part's title: Defender for Identity, Defender for Endpoint, Defender for Cloud Apps, and Defender for Office 365. Microsoft Defender XDR also correlates signal from Microsoft Entra ID Protection and Microsoft Defender for Cloud in a full deployment; this part doesn't cover those two in depth, both because the part's own scope is bounded and because Entra ID Protection's risk-signal role overlaps closely with the identity material Part 6 (entity mapping) and DEH Part 46 (Identity Compromise Model) already cover from adjacent angles.

Engineering Reality *Correlation only spans what's actually licensed and connected. A tenant with Defender for Endpoint and Defender for Office 365 active but no Defender for Identity license simply has no identity-workload alerts to correlate — Defender XDR doesn't synthesize a workload's signal from nothing, and an incident that "should" include an identity alert on the strength of the attack story alone won't, if that workload was never in the picture to begin with. Before treating a converged incident's silence on one signal type as reassuring, confirm the relevant workload is actually licensed, connected, and healthy — not just absent from this particular incident.*

2. The four signal sources this part covers

[CONCEPT] Each of the four workloads below produces its own alerts from its own detection logic running against its own telemetry. What Defender XDR adds is not more detection logic of its own — it's the correlation step described in §3. The table after the four short descriptions is a decision aid for knowing which workload's evidence to go look at first, not a substitute for each product's own documentation.

Defender for Identity — on-premises Active Directory and Entra ID signal

Defender for Identity monitors identity infrastructure through sensors deployed on domain controllers and, for federated environments, Active Directory Federation Services (AD FS) servers, correlating that on-premises signal with Entra ID sign-in and directory activity for hybrid identity environments. Its detections center on the kind of identity-attack techniques a network-layer or endpoint sensor structurally can't see well — reconnaissance against Active Directory (enumeration, directory-service queries at unusual volume), lateral-movement patterns that abuse legitimate authentication protocols, and privilege-escalation paths through group membership and delegation.

Defender for Endpoint — device signal

Defender for Endpoint is Microsoft's EDR product: an agent-based sensor running on the device itself, producing process, file, network-connection, and behavioral telemetry, and the workload most likely to be already familiar to a reader coming from a general SOC background. Its alerts anchor to the Device entity, and its device timeline is the deepest per-asset investigation surface among the four workloads this part covers.

Defender for Cloud Apps — SaaS and cloud-app signal

Defender for Cloud Apps (a Cloud Access Security Broker, or CASB, in Microsoft's own product category language) sees activity inside sanctioned SaaS applications through API connectors, session-level control through Conditional Access App Control, and shadow-IT discovery through firewall and proxy log ingestion. Its detections center on anomalous in-app activity — impossible-travel sign-ins, mass-download or mass-share events, risky OAuth app grants — that neither an endpoint agent nor an identity sensor observes directly, because the activity in question happens entirely inside the SaaS application's own session.

Defender for Office 365 — email and collaboration signal

Defender for Office 365 inspects email and collaboration content directly — messages, attachments, and embedded URLs, evaluated both at time of delivery (Safe Links and Safe Attachments detonation) and after delivery, where a message judged benign at delivery time can still be pulled back from a mailbox (zero-hour auto purge) if later intelligence reclassifies it. Threat Explorer remains the deep, email-specific investigation surface for this workload even after an alert has already surfaced in a converged incident.

Workload	Primary signal	Typical entity anchor	Portal note
Defender for Identity	Domain-controller and AD FS sensor telemetry, correlated with Entra ID sign-in/directory activity	Account, Device (domain controller)	Alerts and investigation surface in the Defender portal; sensor health and directory service account configuration use their own settings pages
Defender for Endpoint	Process, file, network, and behavioral sensor data from onboarded devices	Device	Alerts and the Device timeline live in the Defender portal
Defender for Cloud Apps	API-connected SaaS activity, session-control signal, discovery-log-derived shadow-IT signal	Account, Cloud app, IP address	Alerts flow into the Defender portal; app-connector and policy configuration largely stays in Defender for Cloud Apps's own administrative surface
Defender for Office 365	Email and collaboration content — messages, attachments, URLs, post-delivery re-classification	Mailbox/Account, URL, File	Alerts flow into the Defender portal; Threat Explorer remains the deep email-specific investigation tool

3. How alerts become one incident: the correlation engine

[ENGINEERING] Defender XDR's correlation logic groups alerts into a single incident based on shared entities (the same account, device, mailbox, IP address, or cloud app appearing across two or more alerts) and known attack-chain relationships between the techniques those alerts represent, rather than on time proximity alone. Two alerts about the same user account — one an anomalous sign-in flagged by Defender for Identity, one a mass-download flagged by Defender for Cloud Apps twenty minutes later — correlate into one incident because they share the Account entity and fit a plausible sequential attack pattern, not merely because they happened close together in time.

The unified evidence backing this correlation is queryable directly: Microsoft's advanced hunting schema in the Defender portal includes `AlertInfo` and `AlertEvidence` as cross-product tables —

`AlertInfo` carrying one row per alert regardless of which workload generated it, `AlertEvidence` carrying the entities and artifacts tied to each alert — so a hunt or a workbook built against these two tables sees every workload's alerts through one consistent schema rather than four product-specific alert formats. Each workload also contributes its own detailed event tables to the same advanced hunting surface — `IdentityLogonEvents` and `IdentityDirectoryEvents` for Defender for Identity, `DeviceEvents`/`DeviceProcessEvents`/`DeviceNetworkEvents` for Defender for Endpoint, `CloudAppEvents` for Defender for Cloud Apps, and `EmailEvents`/`EmailAttachmentInfo`/`EmailUrlInfo` for Defender for Office 365 — which is what makes a cross-product hunt (§8) possible without four separate query languages.

The incident itself carries a generated name and a visual attack-chain representation intended to summarize the correlated story rather than list alerts in arrival order. Exactly how that graph view is labeled and laid out in the current portal has shifted across recent UI updates; treat the existence of an incident-level attack-chain visualization as stable and its exact current tab name and layout as something to confirm against the live portal rather than something this part pins down permanently.

flowchart TD

```
subgraph Signals["Defender XDR signal sources (this part's scope)"]
    MDI["Defender for Identity<br/>on-prem AD + Entra ID signal"]
    MDE["Defender for Endpoint<br/>device signal"]
    MDA["Defender for Cloud Apps<br/>SaaS / cloud-app signal"]
    MDO["Defender for Office 365<br/>email + collaboration signal"]
end
```

```
SENT["Sentinel analytics-rule alerts<br/>(Parts 4-5) – only merges here if the<br/>workspace is onboarded to the Defender<br/>portal (Part 2, Part 16)"]
```

```
MDI --> CORR
```

```
MDE --> CORR
```

```
MDA --> CORR
```

```
MDO --> CORR
```

```
SENT -.->|onboarding-dependent merge| CORR
```

```
CORR["Correlation engine<br/>shared entities + attack-chain relationships"]
```

```
CORR --> INC["Unified incident<br/>one queue, one entity graph"]
```

```
INC --> AAD["Automatic attack disruption<br/>scenario-scoped, high-confidence only (Section 5)"]
```

```
AAD --> ACT1["Device contained<br/>Defender for Endpoint"]
```

```
AAD --> ACT2["Account disabled / session revoked<br/>Entra ID / Defender for Identity"]
```

```
AAD --> ACT3["User restricted from sending mail<br/>Defender for Office 365"]
```

```
INC --> ANALYST["Analyst investigation<br/>incident graph, evidence, MITRE tags (Section 7)"]
```

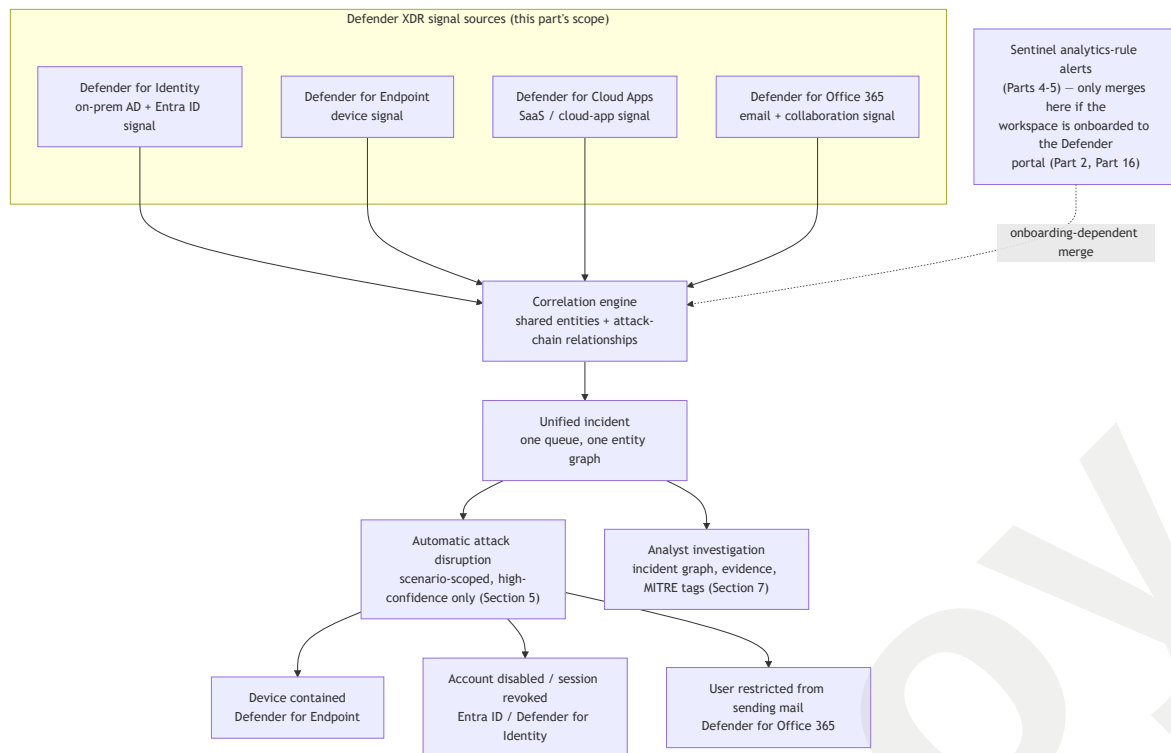


Figure 15.1 — From four workloads' alerts to one Defender XDR incident. *CONCEPTUAL.* Illustrates how alerts from the four workloads this part covers, plus Sentinel's own analytics-rule alerts when the workspace is onboarded to the Defender portal, feed a shared correlation engine that groups them into one incident by entity and attack-chain relationship, and how that incident in turn feeds both an automated-disruption path and the ordinary analyst-investigation path. This is a structural sketch built from this part's own reading of Microsoft's Defender XDR incident and correlation documentation — not a capture of any Defender portal screen, and not a reproduction of an official Microsoft architecture diagram.

4. Sentinel's own alerts joining the same queue

[ENGINEERING] Everything in §3 describes correlation among the four Defender workloads on their own. A Sentinel workspace's analytics-rule alerts join that same correlation engine and the same unified incident queue only once the workspace is onboarded to the unified security operations platform in the Defender portal — the specific mechanics and the retirement timeline driving that onboarding decision belong to Part 2 and get their full program-management treatment in Part 16, and this part assumes that context rather than re-deriving it. Before onboarding, a Sentinel workspace's incidents and a Defender XDR tenant's incidents are two separate queues that an analyst has to check independently; after onboarding, they're the same queue, and a Sentinel analytics rule's alert can correlate into an incident alongside a Defender for Identity or Defender for Cloud Apps alert on exactly the same shared-entity basis described in §3.

This matters for this part specifically because it changes what "a converged incident" can contain. A workspace not yet onboarded to the Defender portal can still see Defender XDR incidents that correlate across the four workloads this part covers — that correlation doesn't depend on Sentinel at all — but it won't see a Sentinel analytics-rule alert folded into one of those incidents until onboarding happens.

Part 13's own documented hazard list (incident-provider condition collapsing to one value, `SecurityIncident.Description` behaving differently, incident-title instability from the correlation engine) is the automation-facing version of this same onboarding-dependent change; this part's version of the same fact is investigative rather than automation-facing — an analyst reading a converged incident needs to know whether the workspace they're looking at even can show a Sentinel alert in that incident before concluding one's absence means anything.

5. Automatic attack disruption — scenario-scoped automated containment

[SENTINEL ENGINEER] Automatic attack disruption is Defender XDR's capability for taking a containment action automatically, mid-attack, based on a high-confidence correlated signal rather than waiting for an analyst to act on an incident. It is the clearest payoff of the convergence this part describes: because the correlation engine already has the entity graph connecting an identity signal to a device to a mailbox, a disruption action can reach across workloads — isolating a device through Defender for Endpoint, disabling or suspending an account through Entra ID and Defender for Identity, or restricting a user's ability to send mail through Defender for Office 365 — in response to a single correlated incident, without requiring an analyst to separately open three product consoles and take three separate actions.

Microsoft documents automatic attack disruption as scoped to specific attack scenarios where the platform's confidence in the correlated signal is high enough to act without human review first, rather than as a general-purpose "contain anything Defender XDR sees" switch. As of this writing, the scenarios Microsoft's own attack-disruption documentation names most consistently are human-operated ransomware, business email compromise (BEC), and adversary-in-the-middle (AiTM) phishing — T1557 (Adversary-in-the-Middle) — each chosen because the attack pattern is well-characterized enough for high-confidence automated action, and because the cost of acting a few minutes late (ransomware detonation, a BEC wire-transfer instruction, a stolen session token being used) is severe enough to justify acting before an analyst has looked at the incident at all.

Attack scenario (as Microsoft documents it)	Example automated containment action	Workload enacting the action
Human-operated ransomware	Device isolated from the network	Defender for Endpoint
Business email compromise (BEC)	Compromised user's ability to send mail restricted	Defender for Office 365
Adversary-in-the-middle (AiTM) phishing	Compromised account disabled / session revoked	Entra ID / Defender for Identity

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft's automatic attack disruption capability is documented on Microsoft Learn's "Automatic attack disruption in Microsoft Defender XDR" page as scoped to a specific, named set of attack scenarios rather than general-purpose automated response, and that scenario list has grown since the capability's initial release. This part names ransomware, BEC, and AiTM phishing as the scenarios most consistently documented as of this writing; treat the exact current scenario list, the licensing prerequisites for each scenario, and the specific containment actions available per workload as subject to change, and re-check the current Learn page rather than treating this table as a permanent inventory. If the current list has grown or the licensing prerequisites have changed since this date, this table's scope — not its underlying mechanism — is what's out of date.

Blind Spot Automatic attack disruption acting on an incident is not the same thing as an incident being handled. The capability is scenario-scoped by design (the table above), which means the majority of incidents in a typical converged queue — anything that isn't a high-confidence ransomware, BEC, or AiTM match — receive no automated action at all and still need an analyst to triage and respond manually. Reading a quiet incident queue as "disruption is handling things" without checking whether disruption actually fired on anything is a program-level version of the UEBA blind spot Part 11 §1 already names for a different feature: a capability being enabled and available is not evidence that it acted on the incident in front of you.

[ANALYST] An automated containment action is reversible and visible in the incident record — Defender XDR logs the specific action taken, when, and on what entity, and provides a way to undo an action an analyst determines was a false-positive-driven overreach (a legitimate but unusual admin sign-in pattern misclassified as AiTM, for instance). Treat a disruption action the same way you'd treat any other automated response in this book — Part 13 and Part 14's own automation-rule and playbook guidance applies here just as much as it does to a Sentinel-native automation path — verify the action fired for the reason the incident record claims, not just that something happened.

6. The investigative tradeoff: what a unified incident costs you

[ANALYST] The payoff described in §5 has a corresponding cost, and this part's own title exists specifically to name it rather than let it pass unstated: once alerts from four separate workloads stop having their own separate incident boundary, an analyst loses the built-in cue that used to say "this alert belongs to the email team's investigation, that one belongs to the endpoint team's." A converged incident's entity graph and attack-chain narrative are genuinely useful for seeing the whole story, but they don't automatically tell you which workload's detection logic actually judged which piece of evidence suspicious, or which product's own console has the deepest tooling for investigating that specific piece of evidence further.

In practice, that means an analyst working a converged incident still needs to open the alert-level detail — not just the incident-level summary — to find out which workload generated a given alert, and often still needs to pivot into that workload's own deeper investigation surface (Threat Explorer for an Office 365 email judgment, the Device timeline for an Endpoint process chain, Defender for Cloud Apps's own activity log for a cloud-app session) to get the level of product-specific context the unified incident view deliberately doesn't try to replicate. Some workloads also retain a genuinely separate administrative surface outside the Defender portal for configuration — Defender for Cloud Apps's own policy and app-connector administration is the clearest example among the four workloads this part covers — which means "everything about this workload is now in the Defender portal" is true for investigation and mostly untrue for configuration, a distinction worth confirming directly rather than assuming from the incident queue's own unified appearance.

Blind Spot *A converged incident's entity graph shows what correlated, not everything that happened. An alert that never fired — because a detection's threshold wasn't met, because a workload wasn't licensed (§1's Engineering Reality), or because the relevant telemetry simply wasn't collected — leaves no gap in the graph for an analyst to notice, because there's nothing there to notice. The unified incident view is strong evidence of what Defender XDR judged connected; it is not evidence that nothing else relevant happened across the four workloads, and DEH Part 44's own framing of adversary behavior spanning multiple kill-chain stages is worth re-reading here as a reminder that a real attack chain is frequently wider than what any one incident's correlated alert set captures.*

What Would Change My Mind *This part treats the convergence described above as a net positive for investigative speed and automated-response reach, worth the per-product-boundary cost given deliberate compensating habits (opening alert-level detail, pivoting into a workload's own console when its evidence needs deeper scrutiny). If Microsoft's own incident view stopped surfacing which workload generated a given alert clearly enough for an analyst to find it without cross-referencing multiple screens, or if correlation false-positives (two alerts sharing an entity by coincidence rather than by genuine attack relationship — a shared service account touching unrelated systems, for instance) became common enough to regularly produce misleadingly broad incidents, that tradeoff would need re-evaluating: a queue that's faster to open but slower to actually resolve correctly is a worse trade than four separate, narrower queues.*

7. Reading a converged incident as an analyst

[ANALYST] A practical triage sequence for a converged incident, building on the alert-level-detail habit §6 already names: start with the incident's overall severity and the entity list (which accounts, devices, mailboxes, and cloud apps are actually involved), then work through the individual alerts in roughly chronological order rather than reading the attack-chain summary as a substitute for that walk-through. For each alert, confirm which workload generated it and what that workload's own detection

logic actually claims — a Defender for Identity alert flagging a lateral-movement pattern and a Defender for Cloud Apps alert flagging a mass-download are both "suspicious," but they're suspicious for entirely different reasons that a shared incident summary can flatten into sounding more similar than they are.

MITRE ATT&CK tags attached at the alert level are the fastest way to see which stage of an attack chain each correlated alert represents without re-deriving it from the raw evidence — a converged incident spanning initial access through impact typically carries a spread of technique tags across its constituent alerts rather than one, and that spread is itself useful triage signal: an incident with only early-stage tags (initial access, reconnaissance) is a different priority than one whose tags already include impact-stage techniques like T1486 (Data Encrypted for Impact).

[ANALYST] Watchlist context (Part 10) and UEBA entity enrichment (Part 11) both remain relevant inside a converged incident exactly as they would inside a Sentinel-only one — a device or account flagged high-value on a watchlist, or carrying a high investigation priority score from `BehaviorAnalytics`, is worth weighting more heavily regardless of which workload's alert brought that entity into the incident in the first place. Neither piece of context is workload-specific; both apply to any entity the incident's graph surfaces.

8. Hunting across the converged signal set

[THREAT HUNTER] The join-strategy and syntax fundamentals for a cross-table hunt — `join`, `summarize`, time-window functions — are DEH Part 25's scope, not this book's; what follows illustrates only the Sentinel/Defender-specific extension DEH Part 25 doesn't cover, joining the cross-product `AlertEvidence` table against a workload-specific event table. The fragment below is illustrative of the join shape, not a complete, deployable hunt — see DEH Part 25 for join-strategy and performance guidance that applies to any join like this at production data volume.

```
// CONCEPTUAL SAMPLE – illustrative cross-workload evidence join fragment, not a complete query
AlertEvidence
| where EntityType == "MailboxConfiguration" or EntityType == "Mailbox"
| join kind=inner (
    EmailEvents
    | where ThreatTypes has "Phish"
) on $left.RecipientObjectId == $right.RecipientObjectId
| project AlertId, Title, Timestamp, SenderFromAddress, ThreatTypes
```

A hunt built on this shape answers a specific question a converged incident's own summary doesn't directly surface: given an alert already correlated into an incident on the strength of a mailbox entity, what other phishing-flagged mail activity touched that same mailbox that didn't itself rise to alert severity. That's the same "look beyond what already correlated" discipline §6's Blind Spot names, made queryable rather than left as a general caution — a hunt against `AlertEvidence` alongside a workload-specific table is one concrete way to check for exactly the kind of relevant-but-uncorrelated activity a converged incident's own graph has no obligation to show.

9. Program and licensing considerations

[SOC MANAGEMENT]

Convergence is a licensing story as much as a platform-mechanics one. Each of the four workloads this part covers is its own licensed product (typically bundled within Microsoft 365 E5 or available as a standalone add-on — Defender for Endpoint Plan 2, Defender for Identity, Defender for Cloud Apps, Defender for Office 365 Plan 2, in Microsoft's current naming), and a converged incident can only correlate across the workloads a given tenant has actually licensed and connected. A SOC evaluating why an incident "should have" included a workload's evidence and didn't needs to check the licensing and connection status of that workload before treating the gap as a correlation-engine failure — the same Engineering Reality point from §1, restated here as a program-management checklist item rather than a mechanics fact.

Automatic attack disruption (§5) carries its own licensing dependencies per scenario, tied to which workloads a given disruption action needs to be able to reach — a disruption action that restricts mail sending needs Defender for Office 365 licensed and connected; one that isolates a device needs Defender for Endpoint. A partial-workload deployment gets partial disruption coverage, scoped to whichever actions the licensed and connected workloads can actually perform, which is a genuinely different, more limited capability than a full four-workload deployment and worth stating plainly to a director who read about attack disruption as a single feature rather than a capability that scales with licensing breadth.

PRODUCT VERSION NOTE (as of 2026-09-15) *The plan/SKU names used above — Defender for Endpoint Plan 2, Defender for Office 365 Plan 2, and Microsoft 365 E5 bundling — reflect Microsoft's licensing documentation as of this writing. Microsoft has renamed and re-bundled Defender licensing tiers before, and Defender for Cloud Apps and Defender for Identity have each been sold under prior product names. Before using this section to answer a director's or procurement team's question, re-check the current licensing page rather than treating these SKU names as permanent — the underlying point (correlation and disruption scope are bounded by what's licensed and connected) is stable even when the specific plan names aren't.*

SOC Management View *The staffing implication of this part's whole subject is real and worth naming directly: an analyst trained only on Sentinel-native incidents, or only on one Defender workload's own console, is under-prepared for a converged incident that spans four workloads' worth of detection logic and evidence formats. The training investment this part implies isn't "learn a fifth product" — it's learning to read one incident's evidence as coming from four different detection engines with four different confidence models, and knowing which workload's own deeper console to pivot into when the unified view's summary isn't enough (§6). Budgeting analyst ramp-up time for that skill, not just for the tools themselves, is the practical program cost of the convergence this part describes.*

10. Putting it together

[CONCEPT] Defender XDR's convergence of identity, endpoint, cloud-app, and email signal — and, once a workspace is onboarded to the Defender portal, Sentinel's own analytics-rule alerts alongside them — turns what used to be four or five separately-triaged alert streams into one incident with one entity graph and, for a scoped set of high-confidence attack scenarios, one automated containment response reaching across all of them. That's a genuine capability gain over the pre-convergence world, and the automatic attack disruption scenarios in §5 are the sharpest illustration of what shared correlation actually buys a SOC that a single-product alert queue structurally couldn't.

It is not, however, a reason to stop asking which workload generated a specific piece of evidence, or to assume a quiet incident means disruption handled it, or to assume a converged incident's graph is a complete account of everything relevant that happened across four workloads' worth of telemetry. This part's honest position is the same one DEH Part 44 takes about adversary behavior generally: an attack chain spanning multiple domains is genuinely easier to see as one story once the platform correlates it for you, and that ease is exactly why it still needs the same skepticism, evidence-tracing discipline, and awareness of what the correlation engine didn't include that a single-alert investigation always required — just applied at incident scope instead of alert scope.

Cross-references: DEH Part 44 (Adversary Behaviour for Defenders — the multi-domain attack-chain framing this part's converged incidents make visible as one story); DEH Part 46 (Identity Compromise Model — the identity-specific compromise reasoning behind Defender for Identity's and Entra ID's contribution to a converged incident); DEH Part 18 (Cloud Identity & SaaS Detection Engineering — the detection logic underneath Defender for Cloud Apps's contribution to this part's signal set); DEH Part 25 (KQL syntax fundamentals underlying the §8 hunting fragment); Part 2 (Log Analytics Workspace Architecture and the Two Portals — the Defender-portal onboarding step this part's §4 depends on); Part 6 (Entity Mapping and the Incident Model — the entity-key mechanics the correlation engine in §3 depends on at Sentinel's own layer); Part 10 (Watchlists and Reference Data); Part 11 (UEBA — the entity-enrichment context §7 draws on); Part 13 (Automation Rules — the onboarding-dependent behavior changes this part's §4 parallels from an automation angle); Part 16 (Operating the Unified Security Operations Platform — the full migration and parity-gap treatment this part's portal dependency assumes).

Part 16 — Operating the Unified Security Operations Platform: Migration and Parity Gaps

Why this part exists

[CONCEPT] Part 2 established the fact this whole book now operates against: the Azure portal (never bare "Azure" when the portal is meant, per this book's own notation convention) and the Microsoft Defender portal are, as of this writing, the two surfaces for operating Microsoft Sentinel, and the Azure portal carries a published retirement date — March 31, 2027 — that sits inside this book's own planning horizon. Part 5 turned that fact into a specific detection-content hazard: Fusion (Advanced multistage attack detection — single instance, hidden correlation logic, unavailable once Defender XDR incident integration is on or the workspace is onboarded to the Defender portal) and Microsoft security rules (rules that automatically create Microsoft Sentinel incidents from alerts generated in other Microsoft security solutions, in real time) both stop existing the moment either condition is true, silently, whether or not anyone planned for it. Part 13 turned the same fact into an automation-rule hazard list — provider-condition collapse, `SecurityIncident.Description` disappearing, incident-title instability, a documented 5–10 minute batching window that drops intermediate incident updates outright. Part 15 turned it into an investigative one — a unified incident queue that correlates more but tells an analyst less about which product generated which piece of evidence.

[CONCEPT] None of those four parts is wrong to stop where it stops. Each one owns a specific layer — workspace architecture, analytics-rule availability, automation-rule behavior, cross-product investigation — and each one correctly treats the Defender-portal transition as a fact that layer has to account for, not a project that layer has to run. This part is where the running happens. Three questions carry the rest of this chapter: what changes operationally, on Defender-portal onboarding, that the four parts above don't already own (administration surfaces that relocate, or conspicuously don't; the health-monitoring surface; playbook manual-run mechanics; multi-workspace and MSSP topology); how Part 5's rule-type gap turns from a feature-availability fact into a rebuild sequence with a deadline; and what a migration actually looks like as a staffed, phased project rather than a single onboarding wizard. DEH Part 23 frames the general version of this problem — one analytic, expressed once, surviving translation across however many backends a detection program runs. This part is that problem's sharpest platform-operations instance in the whole book: the "backend" here isn't a query-language target, it's an entire operating surface with its own retirement date, and DEH's own bridging part deliberately left the operational mechanics of that specific migration to this book — this part is where they land.

[CONCEPT] This part does not cover: the workspace architecture or the RBAC roles required to perform the connection itself (Part 2, in full, including §6's roles table); which rule types are or aren't available and why (Part 5); automation-rule trigger, condition, and batching mechanics (Part 13, in full — this part references its conclusions rather than re-deriving them); or the unified incident queue's investigative tradeoff and the analyst-training implications of convergence (Part 15, in full). What follows assumes all four and adds the layer none of them owns: the practical, day-to-day operating consequences of the transition, and the project-management discipline that keeps those consequences from arriving as surprises.

1. What still requires the Azure portal, even after full onboarding

[ENGINEERING] Microsoft's own framing of the transition is additive, and Part 2 §5 already established why that framing is accurate at the architecture level: connecting a workspace to the Defender portal changes nothing about the workspace's resource ID, region, RBAC assignments, or tables. "Additive" is not the same claim as "complete parity," though, and a team that assumes the Defender portal fully replaces the Azure portal for every operational task will find real gaps — most of them documented not as a single named list but scattered across several feature-specific pages, which is exactly why this section exists as one place to check before a team discovers a gap operationally instead of on paper.

[ENGINEERING] Four gaps are worth naming specifically, in ascending order of how much they cost to discover late:

- **Workspace Manager.** Microsoft's own transition guidance states plainly that "Microsoft Sentinel's Workspace Manager isn't available in the Defender portal" (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," learn.microsoft.com/azure/sentinel/move-to-defender, retrieved 2026-09-15), and names two alternatives for the job Workspace Manager did — deploying content as code from a repository (Part 7 owns this mechanism in full) or the Defender multitenant portal for cross-tenant content distribution. Neither is a like-for-like swap for a deployment that leaned on Workspace Manager specifically to push analytics rules, watchlists, or other content across a fleet of workspaces from one console; a team in that position has a content-distribution redesign to do, not a settings toggle to flip.
- **Workbook rendering.** Printing a workbook or saving it as a PDF is documented as an Azure-portal-only capability, and some workbook visualizations render only in the Azure portal at all — the Defender portal's own workbook view offers an "Open in Azure" link rather than full rendering parity (Microsoft Learn, "Visualize and monitor your data by using workbooks in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/monitor-your-data, retrieved 2026-09-15). A weekly leadership report built around a printed or PDF-exported workbook (Part 8) keeps working exactly as long as someone remembers to open that workbook in the Azure portal specifically to produce the export — the Defender portal's own toolbar simply doesn't offer the button.
- **IdentityInfo table-level RBAC.** This is the sharpest gap in this section, because it is a security-control loss rather than a UI inconvenience. Once a workspace transitions, "the `IdentityInfo` table becomes a native Defender table that doesn't support table-level role-based access control (RBAC)," and an organization that used table-level RBAC to restrict which analysts could query this table in the Azure portal finds that "this access control will no longer be available" after the transition (same "Transition..." source). A UEBA identity table (Part 11) an organization deliberately scoped down to a subset of analysts becomes queryable by anyone with ordinary hunting access the moment onboarding completes, with no separate warning at the moment access actually widens.
- **Similar incidents (Preview).** Sentinel's own similar-incidents feature for case investigations is documented as unsupported in the Defender portal, and the tab simply isn't present on an incident's details page once a workspace is onboarded (same source). A case-investigation habit

built around that tab loses it at the same moment as everything else in this list — quietly, alongside the change, not as a separately flagged removal.

Blind Spot Of the four gaps above, only the `IdentityInfo` RBAC loss is a genuine security-control regression rather than a workflow inconvenience, and it is the one most likely to go unnoticed, because nothing about the incident queue, the analytics rules, or the automation layer changes to announce it. A security team that built a defensible access-control story — "identity behavior data is restricted to the identity-investigations team, enforced with table-level RBAC" — around the Azure portal's own access model has that specific enforcement mechanism silently removed on the same day the workspace connects to the Defender portal, unless the underlying access requirement gets re-solved some other way (a narrower `Microsoft Sentinel Reader` / `Microsoft Sentinel Contributor` role assignment scoped more tightly than before, or a process control substituting for the removed platform control) before that day arrives, not after an auditor asks who can see the table now.

[ENGINEERING] Table 16.1 collects the four gaps above alongside one already covered in depth elsewhere in this book, to support one decision: which Azure-portal-only habits a team needs a documented replacement for before treating onboarding as complete, rather than discovering the gap the first time someone reaches for the missing capability.

Table 16.1 — Capabilities that don't carry over cleanly to the Defender portal. Supports deciding which Azure-portal-only habits need a documented replacement before onboarding, not after.

Capability	Azure portal (retiring 2027-03-31)	Defender portal
Cross-workspace content distribution	Workspace Manager	Not available — use content-as-code from a repository (Part 7) or the Defender multitenant portal
Workbook PDF export / print	Available from the workbook toolbar	Not available — open the workbook in the Azure portal to export or print
Full workbook visualization rendering	All visualization types render	Some visualization types render only after selecting "Open in Azure"
<code>IdentityInfo</code> table-level RBAC	Supported, per-table access restriction	Not supported — the table becomes a native Defender table once onboarded
Similar incidents (Preview)	Available as an incident-details tab	Not supported — tab isn't present
Fusion / Microsoft security rule incident creation	Available (Part 5)	Unavailable — Defender XDR's correlation engine replaces both (§5 below)

2. The health-monitoring surface after onboarding

[ENGINEERING] Sentinel's built-in health and audit monitoring writes to two tables in the workspace — `SentinelHealth`, which records automation-rule run results and on-demand playbook triggers (including who triggered them and whether they succeeded), and `SentinelAudit`, which records user actions taken

within the service (Microsoft Learn, "Auditing and health monitoring in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/health-audit, retrieved 2026-09-15). Microsoft's own guidance recommends querying both through pre-built functions — `_SentinelHealth()` and `_SentinelAudit()` — rather than the raw tables directly, specifically so a query survives a future schema change without rewriting; that's the same schema-drift discipline DEH Part 25 §8 already argues for against the Advanced Hunting catalog, applied here to Sentinel's own health telemetry instead of a third-party log source.

[ENGINEERING] One cost distinction is easy to miss: `SentinelHealth` is not billable — Microsoft states plainly that it "isn't billable and incurs no charges for ingesting health data" — while `SentinelAudit` is a billable table, with cost driven by activity volume on the rules and resources it's tracking (same source). Turning on verbose audit logging as a blanket "more visibility is always better" default is therefore a real, ongoing cost decision, not a free security-hygiene improvement, and Part 17's cost-model framework is the place to actually size that decision before switching it on tenant-wide.

Engineering Reality *Health data costs nothing to collect; audit data does. A team migrating its monitoring habits into the Defender portal often reaches for "turn everything on" as the safe default during the transition, on the reasonable assumption that more logging can only help a migration go smoothly. For `SentinelAudit` specifically, that default has a real, metered cost attached to it that scales with how much configuration activity the migration itself generates — ironically, the busiest period for enabling verbose audit logging (the migration project itself, full of configuration changes) is also the period that logging is most expensive to run.*

[ENGINEERING] Where you go to query this data changes with the portal. In the Azure portal, the documented path is Microsoft Sentinel's own **Logs** page, plus the auditing-and-health-monitoring workbooks Microsoft ships for data-connector, automation, and analytics-rule health specifically (same "Auditing and health monitoring" source). Once a workspace is onboarded to the Defender portal, the equivalent general-purpose query surface for this same data is the unified **Advanced hunting** page rather than a separate Logs blade — both `SentinelHealth` and `SentinelAudit` are ordinary workspace tables, and Microsoft's own onboarding documentation is explicit that a connected workspace's existing tables, saved queries, and functions all surface inside that same unified hunting schema (Microsoft Learn, "Microsoft Sentinel in the Microsoft Defender portal,"

learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal, retrieved 2026-09-15). The health-monitoring workbooks themselves are Azure resources under §1's general workbook-rendering caveat — most of what they show renders fine in the Defender portal's own Workbooks page, but confirm any specific visualization actually renders there before relying on it mid-incident rather than discovering the "Open in Azure" fallback during a live health investigation.

This section names where health and audit data lives and where the general query surface for it moves once a workspace is onboarded; it does not walk the `SentinelHealth` / `SentinelAudit` schema itself, the specific health-monitoring workbooks Microsoft ships per feature area, or the troubleshooting workflow built on top of either table — Part 18 owns that full depth, including the canary-query discipline this book uses as its direct answer to DEH Part 25 §8's schema-drift warning.

PRODUCT VERSION NOTE (as of 2026-09-15) *Where a workspace's health/audit query surface actually lives is itself a migration-dated fact, not settled architecture: Microsoft Learn's "Auditing and health monitoring in Microsoft Sentinel"*

([Learn.microsoft.com/azure/sentinel/health-audit](https://learn.microsoft.com/azure/sentinel/health-audit)) names the Azure portal's Logs page and its health-monitoring workbooks as the query surface, and Microsoft Learn's "Microsoft Sentinel in the Microsoft Defender portal" ([Learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal](https://learn.microsoft.com/azure/sentinel/microsoft-sentinel-defender-portal)) names Advanced hunting as the equivalent surface once onboarded, both as of the retrieval dates cited earlier in this section. Re-check both pages and Part 18 directly before building a health-monitoring runbook on this section alone, since a query surface moving is exactly the kind of platform-navigation fact this part's broader migration subject matter puts on the active-change list.

3. Playbook manual-run restrictions

[SENTINEL ENGINEER] Part 14 owns playbook mechanics in full; this section names one specific, documented behavior change that belongs to the migration rather than to playbook design itself. Microsoft's own automation documentation lists, under "Running playbooks manually on demand," two procedures that are "not currently supported in the Defender portal": running a playbook manually on an alert, and running a playbook manually on an entity (Microsoft Learn, "Automation in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/automation/automation, retrieved 2026-09-15). Running a playbook manually against an incident as a whole is supported — but with its own sync-dependent wrinkle: if an analyst tries to run a playbook on an incident from the Defender portal before that incident has finished synchronizing to the underlying Sentinel workspace, the action fails with an error directing the analyst to wait and refresh, and the fix is exactly that — wait for sync, then retry (same source). This book paraphrases that error rather than reproducing it verbatim, since the book's no-tenant honesty constraint ([BOOK-INDEX.md](#), "What this book is") means the exact on-screen wording hasn't been captured firsthand; check the cited Microsoft Learn page directly if a runbook needs the literal string.

Engineering Reality A SOC that built its triage runbook around an analyst manually firing a specific enrichment playbook against one alert, or against one entity mid-investigation — "pull threat-intel context on this IP right now, without waiting for an automation rule's conditions to happen to match" — loses that specific manual action once operating from the Defender portal, even though the exact same playbook keeps running perfectly well when invoked automatically by an automation rule (Part 13's "Run playbook" action) or manually against the incident as a whole. The fix is not "the feature is broken, escalate a support ticket" — it's rebuilding the manual, per-alert or per-entity trigger as something the platform still supports: either an automation rule scoped narrowly enough that an analyst can trigger the same effect indirectly (adding a specific tag or comment that a condition watches for), or, for as long as the Azure portal exists, performing that one action there specifically. The second option is a stopgap with a countdown attached to it (Part 2's retirement date), not a durable answer.

[SENTINEL ENGINEER] The incident-level sync dependency is worth naming as its own operational fact, separate from the manual-run restriction: an incident that exists in the Defender portal is not instantly the same object the Sentinel-side automation and playbook layer can act on — there's a real, if usually brief, propagation delay, and the "up to 10 minutes" figure Part 13 §9 already documents for automation-rule execution latency is the general version of the same underlying mechanic this section's playbook-sync message is a symptom of. Treat both as evidence of one fact, not two unrelated quirks: the Defender portal's incident and the Sentinel workspace's automation/playbook layer are two systems kept in sync, not one system with two skins, and every "it didn't run yet" surprise in this part traces back to that same seam.

4. Multi-workspace and MSSP topology changes

[ENGINEERING] Part 2 §2 already laid out the distributed, centralized, and hybrid patterns a Managed Security Service Provider (MSSP) or large multi-business-unit enterprise chooses among for Log Analytics workspace topology, and Part 2 §6 already established the Defender portal's own primary-plus-secondary-workspaces model on top of that topology. This section adds the specific operational consequence a distributed deployment runs into the moment more than one of its workspaces connects to the same Defender portal tenant.

[ENGINEERING] To prevent duplicate tenant-based alerts landing in more than one workspace at once, Microsoft's transition guidance states that standalone data connectors for Microsoft Defender for Office 365, Microsoft Entra ID Protection, Microsoft Defender for Cloud Apps, Microsoft Defender for Endpoint, and Microsoft Defender for Identity "are automatically disconnected in secondary workspaces during onboarding," with the result that "tenant-based alerts from these Microsoft security products are available only in the primary workspace" (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," retrieved 2026-09-15). For a single-tenant deployment running one primary workspace, this is mostly invisible — there's only ever one workspace for that alert stream to live in anyway. For an MSSP running Part 2 §2's distributed pattern — one workspace per customer tenant, several of which might get connected under the same Defender multitenant setup, or a single

tenant running more than one Sentinel-enabled workspace for internal reasons — this is a real, silent narrowing: a secondary workspace that used to independently ingest its own copy of, say, Defender for Endpoint alerts stops doing so the instant it becomes secondary, and those alerts now live only in whichever workspace holds the primary designation.

Blind Spot *"Which workspace is primary" already mattered, per Part 2 §6, for which workspace's Defender XDR connector stays live after a primary-designation change. This section adds that it now also determines which workspace actually receives a tenant's own Microsoft-product-native alerts at all, for every workspace in that tenant that isn't primary. A team that changes the primary designation for an unrelated operational reason — rebalancing ingestion cost, consolidating after an acquisition — can inadvertently relocate where a whole category of alerts lands, without touching a single data connector setting directly. Anyone with authority to change a tenant's primary workspace should treat that action with the same change-control weight Part 13 §10 already argues for around a high-blast-radius automation rule, because the two failure modes rhyme: an action that looks like relabeling something actually redirects real signal.*

[SOC MANAGEMENT]

Cross-tenant access for an MSSP is the other topology fact worth updating from Part 2 §6's own statement of it. Part 2 documents, correctly as of its own last-validated date, that granular delegated admin privileges (GDAP) through Azure Lighthouse is not supported for Sentinel data viewed through the Defender portal, and that Microsoft Entra B2B guest access is the documented alternative. Microsoft's own transition guidance, read alongside that same point, adds one more precision worth flagging here rather than silently correcting: it describes "Granular Delegated Admin Privileges (GDAP) for Microsoft Sentinel" as currently "in preview" for the Defender portal (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," retrieved 2026-09-15) — meaning the gap Part 2 names as a settled "use B2B instead" fact may already be narrowing on Microsoft's own roadmap, in preview form, at the time of this writing.

PRODUCT VERSION NOTE (as of 2026-09-15) *GDAP support for Microsoft Sentinel data in the Defender portal is documented, per Microsoft Learn's "Transition your Microsoft Sentinel environment to the Defender portal" ([Learn.microsoft.com/azure/sentinel/move-to-defender](https://learn.microsoft.com/azure/sentinel/move-to-defender)), as being in preview as of this writing, which sits alongside — and may eventually revise — Part 2 §6's "use Microsoft Entra B2B instead of Lighthouse/GDAP" guidance. An MSSP planning a cross-tenant access model around this book's Part 2 recommendation should re-check the current GDAP-for-Sentinel documentation directly before finalizing that model, since a preview feature narrowing a documented gap is exactly the kind of fact this book's honesty constraint requires flagging rather than either asserting as resolved or ignoring as unchanged.*

5. The Fusion and Microsoft security rule gap, revisited as a migration task

[SOC MANAGEMENT] Part 5 §5 already established the underlying fact in full: Fusion and Microsoft security rules are both unavailable once a workspace onboards to the Defender portal, because Defender XDR's own correlation engine creates the incidents instead. This part's job is not to re-argue that fact — it's to name what actually happens operationally at the moment onboarding completes, and to turn "unavailable" into a sequencing decision a migration project can plan around instead of discover.

[SOC MANAGEMENT] Microsoft's transition guidance states the mechanics plainly: "Any active Microsoft security incident creation rules are deactivated to avoid creating duplicate incidents," and separately, "The Fusion analytics rule, which in the Azure portal, creates incidents based on alert correlations made by the Fusion correlation engine, is disabled when you onboard Microsoft Sentinel to the Defender portal. You don't lose alert correlation functionality because the Defender portal uses Microsoft Defender XDR's incident-creation and correlation functionalities to replace those of the Fusion engine" (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," retrieved 2026-09-15). Both actions are automatic, immediate, and tied to the onboarding event itself — nobody schedules the deactivation separately, and nobody gets a grace period to verify replacement coverage first unless the migration plan deliberately builds that verification in before flipping the connection on.

[SOC MANAGEMENT] That's the sequencing point this section exists to make: the question that actually matters isn't whether Fusion and Microsoft security rules go away on onboarding — Part 5 already settled that they do — it's whether the replacement coverage (Defender XDR's own correlation engine picking up Fusion's job; rebuilt Scheduled rules or Custom Detections, per Part 5 §6, picking up whatever a Microsoft-security rule's severity filtering used to do) is verified as equivalent *before* that automatic deactivation happens, or discovered to be missing *after*. Part 5's own What Would Change My Mind box already recommends "rebuild first, verify it fires the way the old rule did, and only then flip the switch" as this book's default position; this part adds the specific tooling pointer for the alert-grouping and incident-creation half of that rebuild — Microsoft publishes dedicated guidance titled "Migrate Microsoft Sentinel incident creation rules and alert grouping settings to Defender XDR" (learn.microsoft.com/azure/sentinel/migrate-sentinel-incident-creation-rules-alert-grouping) specifically for this transition. This book has not validated that migration tool's own workflow against a live tenant, consistent with its honesty constraint, so this section names the page rather than walking its steps — a migration project's engineering-phase checklist (§6 below) should include working through that page directly, not substituting this paragraph for it.

6. Planning the migration as a project

[SOC MANAGEMENT] Microsoft's own transition documentation is organized around three named audiences — security architects, security engineers, and security analysts — each with a distinct phase of work (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," retrieved 2026-09-15). That structure is worth adopting directly, not reinventing, because it already reflects the actual shape of the work: a plan-and-prerequisites phase that has to finish before onboarding, a configure-and-review phase that rebuilds and re-verifies content immediately around the onboarding event, and a run-operations phase that continues indefinitely afterward as the new steady state.

- **Plan and prerequisites (security architects).** Confirm the roles required to perform the connection (Part 2 §6); decide primary-versus-secondary workspace topology and account for §4 above's secondary-workspace connector-disconnection consequence before, not after, designating a primary; check for customer-managed key (CMK) encryption in use on the workspace, because alerts and incidents lose CMK encryption on onboarding even though log data and Sentinel's own rule and automation-rule content stay CMK-encrypted (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," retrieved 2026-09-15) — a fact a regulated-industry SOC manager needs on record before, not after, a compliance review asks about it; and confirm which data-residency and privacy policy governs the data going forward, since Defender XDR's own data-handling policies apply to data viewed through the Defender portal in place of Sentinel's own data-residency policies, per the same source.
- **Configure and review (security engineers).** Rebuild Fusion and Microsoft-security-rule coverage before cutover (§5 above); update every automation-rule condition affected by Part 13 §9's hazard list (provider-condition collapse, `SecurityIncident.Description` disappearing, incident-title instability, the 5–10 minute batching window); identify any analyst workflow depending on §3 above's manual per-alert or per-entity playbook run and rebuild it as a supported pattern; confirm a Workspace-Manager-dependent content-distribution habit (§1 above) has a working as-code replacement; and review the alert-schema differences that follow from Microsoft-product alerts now routing through the Defender XDR connector rather than each product's own standalone connector — Microsoft documents this schema difference on a dedicated page ("Alert schema differences: Standalone vs. Microsoft Defender XDR connector"), which this section names rather than reproduces, since a field-by-field schema comparison belongs to that page's own currency, not this book's.
- **Run operations (security analysts).** Retrain analysts on the unified triage process Part 15 §9 already budgets staffing time for; retire any runbook step written against a Sentinel-only incident queue; confirm health-monitoring queries and workbooks still resolve the way §2 above describes; and treat the first several weeks of live operation as a verification window, not a victory lap — several of this part's hazards (the secondary-workspace connector disconnection, the batching window, the manual-playbook-run gap) are the kind of thing that surfaces only the first time someone actually needs the missing capability, not at cutover itself.

flowchart TD

```
A["Plan & prerequisites\n(security architects)"] --> B["Roles confirmed?\nPrimary/secondary topology decided?\nCMK / data-residency reviewed?"]
B -->|No| A
B -->|Yes| C["Configure & review\n(security engineers)"]
C --> D["Fusion / Microsoft-security-rule\nreplacement coverage verified?\n(Part 5 §6, this part §5)"]
D -->|No| C
D -->|Yes| E["Automation-rule conditions\nupdated for Part 13 §9 hazards?"]
E -->|No| C
E -->|Yes| F["Workspace Manager and manual\nplaybook-run dependencies\nreplaced? (§1, §3)"]
F -->|No| C
F -->|Yes| G["Connect workspace to\nthe Defender portal"]
G --> H["Run operations\n(security analysts)"]
H --> I["Analyst retraining\n(Part 15 §9)"]
H --> J["Post-cutover verification window\n(secondary-workspace connectors,\nbatching window, health queries)"]
I --> K["Steady-state operation\nin the Defender portal"]
J --> K
```

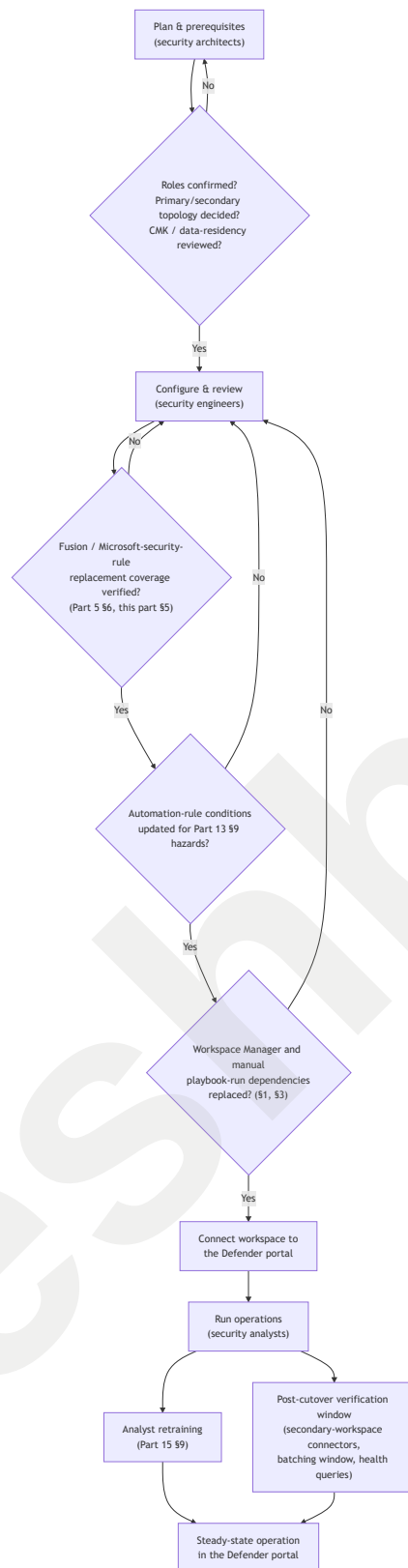


Figure 16.1 — A Defender-portal migration as a gated, three-phase project. *CONCEPTUAL.*

Illustrates the plan/configure/run-operations structure Microsoft's own transition documentation organizes the migration around, mapped onto this book's own parts as gating checks rather than a single onboarding step. This is a structural sketch built from this part's own reading of the cited Microsoft Learn transition guidance and this book's own Parts 2, 5, 13, and 15 — not a capture of any

Azure portal or Defender portal screen, and not a reproduction of an official Microsoft architecture diagram.

SOC Management View *The connection step itself — the wizard Part 2 §6 documents, roles and all — takes minutes. Everything named in this part's §§1–5 is the actual project: named owners for each gating check in Figure 16.1, a rebuild-and-verify pass on detection coverage before cutover rather than after, and a post-cutover verification window long enough to actually exercise the hazards that don't surface until someone needs the missing capability. A SOC manager who reports this migration to leadership as "we connected the workspace to the new portal" is reporting the five-minute step and omitting the project around it — the same underreporting Part 2 §7 already warns against when it calls the migration "a licensing and RBAC-surface review, timed against a vendor-controlled date," not a UI refresh. Report the phases, the gating checks, and the verification window as the actual scope, because that's what a leadership sponsor is actually approving staff time for.*

PRODUCT VERSION NOTE (as of 2026-09-15) *Microsoft states directly that "transitioning to the Defender portal, even for non-E5 customers, has no extra cost for the customer. The customer continues to be billed as usual for their consumption on Sentinel only" (Microsoft Learn, "Transition your Microsoft Sentinel environment to the Defender portal," retrieved 2026-09-15). This is worth having on record verbatim, because it's the single fastest way to answer the leadership question this migration reliably generates — "does moving portals cost us anything beyond what we already pay" — without leaving room for the licensing-gated capabilities named in Part 2 §7 (Security Exposure Management, Defender-provided custom detection rules, the Action center) to be mistaken for a cost of the portal move itself, when they're actually a cost of a Defender XDR license this book treats as a separate purchasing decision.*

7. Post-migration verification checklist

[SOC MANAGEMENT] Table 16.2 collects this part's own gaps into one verification pass, to support a single decision: is a specific workspace's migration actually complete, or does it only look complete because the connection succeeded and nobody has yet reached for one of the capabilities this part names.

Table 16.2 — Post-migration verification checklist. Supports confirming a workspace's Defender-portal transition is operationally complete, not just technically connected.

Gap area	What to verify	Owning part for full mechanics
Fusion / Microsoft security rule coverage	Replacement correlation or rebuilt rule confirmed to fire equivalently, before cutover	Part 5 §5–§6, this part §5
Automation-rule conditions	No condition depends on the removed <code>SecurityIncident.Description</code> field, a stable-provider assumption, or a stable incident title	Part 13 §9
Manual playbook runs	Any analyst habit of running a playbook on a specific alert or entity rebuilt as a supported pattern	Part 14, this part §3
Workspace Manager dependents	Cross-workspace content distribution has a working as-code or multitenant-portal replacement	Part 7, this part §1
<code>IdentityInfo</code> table RBAC	Access-restriction requirement re-solved by role scoping or process control, if table-level RBAC was in use	Part 11, this part §1
Secondary-workspace product connectors	Confirmed disconnection of standalone Microsoft-product connectors on every secondary workspace, and that primary-workspace ingestion actually captures what those connectors used to carry	Part 2 §2, §6, this part §4
Health-monitoring queries and workbooks	<code>SentinelHealth</code> / <code>SentinelAudit</code> queries and health workbooks resolve via Advanced hunting and the Defender portal's Workbooks page	Part 18, this part §2
Analyst training	Unified triage process training completed, not just scheduled	Part 15 §9

8. Putting it together

[CONCEPT] Nothing in this part is new architecture — every fact above traces back to Part 2's calendar, Part 5's rule-type availability trap, Part 13's automation-rule hazard list, or Part 15's investigative tradeoff. What this part adds is the operational discipline that keeps those facts from arriving as surprises: a named list of Azure-portal-only capabilities that need a documented replacement (§1), the health-monitoring surface's actual new location (§2), the specific manual-response action that has to be rebuilt rather than assumed to carry over (§3), the multi-workspace consequence a distributed or MSSP deployment specifically has to plan for (§4), a sequencing answer for Part 5's rule-type gap instead of just a restatement of the fact (§5), and a three-phase project structure with gating checks, mapped onto owners, instead of a single onboarding step (§6–§7).

What Would Change My Mind This part treats every gap named above as temporary migration friction with a deadline attached to it — worth a rebuild task and a verification check now, because the Azure portal side of each comparison stops being an option at all on March 31, 2027 — rather than as a durable trade-off a team could reasonably choose to live with indefinitely. If Microsoft reversed the retirement decision, published a durable Azure-portal-parity commitment extending meaningfully past that date, or if enough of this part's gaps turned out to be permanent rather than closing over time, that framing would need to change from "rebuild this before the deadline" to a genuine cost-benefit comparison between two co-existing operating models — a materially different message for a SOC manager currently staffing a migration project against a fixed date rather than an open-ended choice.

Cross-references: DEH Part 23 (one analytic, many backends — the general problem this part's operational migration work is the sharpest platform-specific instance of, and the detail DEH's own bridging part deliberately deferred here); Part 2 (workspace architecture, the two-portal split, the retirement date, and the connection roles this part assumes throughout); Part 5 (the Fusion and Microsoft-security-rule availability trap this part's §5 turns into a rebuild sequence); Part 7 (detection-as-code and repository deployment — the Workspace Manager replacement named in §1); Part 8 (workbooks — the PDF/print and rendering gaps named in §1); Part 11 (UEBA and the `IdentityInfo` table — the RBAC loss named in §1); Part 13 (automation-rule trigger, condition, and batching mechanics this part's §6 configure-and-review phase updates against); Part 14 (playbook mechanics underneath the manual-run restriction in §3); Part 15 (the unified incident queue's investigative tradeoff and the analyst-training cost this part's §6 run-operations phase budgets for); Part 17 (cost models — the billing distinction behind `SentinelAudit`'s cost in §2); Part 18 (`SentinelHealth`/`SentinelAudit` full schema and troubleshooting workflow this part's §2 defers to).

Section G — Cost, Retention, and Governance

neshbooy

Part 17 — Workspace Cost Models and Retention Tiers

Why this part exists

[CONCEPT] Part 3 named the tier decision — Analytics tier or Data lake tier — as the fourth of four choices an engineer makes once, per data source, at ingestion configuration time, and then moved on because that part's scope was the ingestion pipeline, not the bill. DEH Part 25 §8 went a step further from the query-writing side: its SOC Management View callout points out that a scheduled rule's query shape and its source table's billing tier interact, because a table billed by the gigabyte scanned on every query turns a missing leading time filter into a recurring cost line, not a one-time inefficiency. Both parts pointed at this one and moved on. This is where the pointing stops.

This part treats cost and retention as one connected decision surface, not two separate conversations — a finance conversation about the monthly bill and an engineering conversation about how long data survives. They're the same decision because Microsoft Sentinel bills ingestion, retention, and (for one tier) query and processing as separate, addable meters, and which tier a table sits in determines which of those meters actually run. Get the tier wrong for a given source and you either pay full interactive-tier rates for data nobody queries, or you park a genuinely detection-relevant source somewhere that can never drive an analytics rule at all — the second failure is a coverage gap wearing a cost-optimization disguise, and it is exactly the kind of thing DEH Part 43 (Detection Debt) exists to name: a decision that looks like it only costs money now, and costs detection coverage later, quietly, with no error message pointing at it.

Two structural notes before the detail. First, this part does not re-teach KQL performance discipline — DEH Part 25 §8 already covers filtering on the indexed time column, projecting down before a join, and the mechanics of why an unfiltered scan is expensive; this part is about which tier a table lives in, not how to write a cheap query against it. Second, per this book's honesty constraint (`STYLE-GUIDE.md` §9), every dollar figure, discount percentage, and tier threshold below is something Microsoft can and does change on its own pricing cadence, with no lab tenant here to validate against. Treat the specific numbers as a snapshot dated to this chapter's `last_validated` date, and the framework around them — two billing surfaces, two tiers, a per-source decision — as the durable part.

1. Two billing surfaces, not one

[ENGINEERING] A Log Analytics workspace with Microsoft Sentinel enabled bills across at least two distinct dimensions, and conflating them is the single most common source of a cost surprise:

1. **Ingestion** — what it costs to get a gigabyte of data into a table at all, metered daily, per table, based on the data's uncompressed volume as analyzed at ingestion time.
2. **Retention and access** — what it costs to keep that data queryable over time, and (for one of the two tiers) what it costs to query, process, or search it at all, as a separate line from the storage itself.

Part 2 already established that once Sentinel is enabled on a workspace, every table in it — including tables Azure Monitor itself populates for unrelated operational telemetry — is billed under Sentinel

pricing, not plain Azure Monitor Logs pricing, whether or not a given table's contents have anything to do with security. This part picks up from there: within that Sentinel-priced workspace, the tier a table is assigned determines which of the two billing dimensions above actually apply, and at what rate.

PRODUCT VERSION NOTE (as of 2026-09-15) Microsoft Learn, "Plan costs and understand pricing and billing" for Microsoft Sentinel (learn.microsoft.com/azure/sentinel/billing, retrieved 2026-09-15), is the authoritative source for exactly how ingestion and retention are metered and priced, and it changes on Microsoft's own cadence — a new region rolling out different rates, a discount structure changing, or a tier's default retention window shifting are all things that have happened to this platform before and can happen again. This part states the shape of the billing model (which meters exist, what triggers them) as the durable content; check that page directly before building a budget or a business case on any specific number quoted here.

2. The Analytics tier

[ENGINEERING] The **Analytics tier** is the tier every earlier part in this book has implicitly assumed a table lives in, because it's the tier that supports everything the rest of the platform does: full KQL querying, analytics rules (Parts 4–5), workbooks (Part 8), hunting (Part 9), UEBA (Part 11), and entity mapping into the incident model (Part 6) all run against Analytics-tier data. Its retention and cost shape, per Microsoft Learn's "Log retention tiers in Microsoft Sentinel" documentation (learn.microsoft.com/azure/sentinel/log-plans, retrieved 2026-09-15 for this book), is:

- **Interactive retention**, meaning the data is immediately queryable with no separate search or restore step, for a default period that Microsoft's documentation states as 90 days, extensible per-table up to two years (Microsoft Learn, "Log retention tiers in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/log-plans, retrieved 2026-09-15).
- **No separate per-query charge**. Once a table's data is ingested and sitting in the Analytics tier, running a KQL query against it — however large the query's own scan, however many times an analyst re-runs it while triaging an incident — does not add its own line item the way it does for the Data lake tier's query mechanics (§3). This is the tier's central economic feature: cost is driven by what you ingest and how long you keep it, not by how hard you look at it afterward.
- **Full feature support**. Analytics rules, UEBA, entity mapping, and every other platform capability described earlier in this book assume an Analytics-tier source table. A table that isn't in the Analytics tier cannot back a scheduled or NRT analytics rule the way Parts 4–5 describe — this is the operational consequence, not just the cost consequence, of the tier choice, and it's the reason §6 below treats tier selection as a detection-design decision.

Extending retention beyond the default interactive window is a per-table setting, not a workspace-wide one — one table can carry a longer retention period than its neighbor in the same workspace, which matters for a source with a compliance-driven retention requirement that outlives the default the rest of the workspace runs on.

3. The Data lake tier

[ENGINEERING] The **Data lake tier** exists for a different job: holding high-volume, lower-per-record-value secondary security data — bulk network flow logs, verbose firewall traffic, or any source you want to retain for long-horizon hunting or compliance without paying Analytics-tier rates on every gigabyte, forever. Per the same Microsoft Learn retention-tiers documentation cited above, the Data lake tier's cost and access model differs from the Analytics tier's in a way that matters operationally, not just financially:

- **Ingestion, processing, storage, and query are billed as separate components**, rather than folded into one blended per-GB rate the way Analytics tier's ingestion-plus-default-retention pricing is. A table sitting quietly in the Data lake tier, untouched, costs one thing; running a large search or KQL job against months of that table's history costs something additional, on top of what storing it already cost.
- **It is not currently structured to drive analytics-rule alerting**. A source landed in the Data lake tier is not available to a scheduled or NRT rule's query the way an Analytics-tier table is — Part 3 already flagged this constraint at the ingestion-decision stage; this part is where it becomes a cost-versus-coverage tradeoff instead of a plumbing fact.
- **Access happens through a different query surface**. Microsoft's platform-layer documentation describes reaching Data lake tier data through mechanisms built for that tier specifically — KQL jobs and search-job-style asynchronous queries rather than the always-interactive query experience the Analytics tier supports. This is genuinely one of the fastest-moving surfaces in the current model; treat the exact set of supported access methods as something to re-verify against current Microsoft Learn documentation rather than as settled the way this chapter's tier-versus-tier framing is.

Blind Spot "Cheap to store" and "cheap to use" are not the same property, and the Data lake tier's pricing model separates them on purpose. A hunter who treats a Data lake tier table the way they'd treat an Analytics tier table — running the same kind of broad, exploratory, re-run-it-five-times-while-refining-the-filter query they'd run interactively against `SecurityEvent` — can generate a processing or query charge that a workspace owner never budgeted for, precisely because the Analytics tier's habit of "the query itself is free, only ingestion and retention cost anything" does not carry over. Anyone hunting against Data lake tier data needs to know that before they start, not after the job completes and the bill reflects it.

PRODUCT VERSION NOTE (as of 2026-09-15) *The two-tier model (Analytics tier and Data lake tier) described in §2–§3 is Microsoft's current framing, per the "Log retention tiers in Microsoft Sentinel" page cited above. This is a genuinely new-ish framing relative to a four-tier model (Analytics, Basic Logs, Auxiliary Logs, Archive) that circulated in Microsoft's own earlier documentation, training material, and certification content, and that four-tier language has not fully disappeared from the field — §4 below maps the two onto each other as best this book can, but a reader should treat that mapping as this book's own synthesis of the cited page, not as a claim Microsoft has published the mapping in exactly that form. Re-check the current retention-tiers page directly, especially if a vendor deck, a training course, or a colleague's notes still uses the older four-tier names.*

4. The tier model this replaced

[ENGINEERING] Anyone who learned Sentinel's cost model before the two-tier framing above became current is likely to still think in terms of four tiers: **Analytics**, **Basic Logs**, **Auxiliary Logs**, and **Archive**. That terminology hasn't vanished — DEH Part 25 §8's own SOC Management View callout, written from the query-performance side rather than the billing-architecture side, still refers to "Basic Logs and Auxiliary Logs tables" as the cheaper tiers some teams route high-volume, low-value sources into, because that callout is about the query-cost consequence of table choice and that consequence holds regardless of which generation of tier names a given workspace or a given reader's training used. Table 17.1 gives this book's best current understanding of how the two framings relate — read it as an orientation aid for a reader encountering both vocabularies, not as a Microsoft-published equivalence table.

Table 17.1 — Legacy four-tier terminology mapped to the current two-tier model. Supports a reader who learned Sentinel's cost model under the older naming and needs to translate it against this chapter's terms; not a claim that Microsoft has published this exact mapping, and not a claim that every property of a legacy tier carries over unchanged to its current-model counterpart.

Legacy tier name	Rough current-model equivalent	What to verify before relying on the mapping
Analytics	Analytics tier	Largely a direct carryover — full interactive querying, analytics-rule support, default retention window.
Basic Logs	Closest in spirit to Data lake tier's low-cost-storage role	Basic Logs' specific per-scan query billing model and the Data lake tier's own processing/query billing are not necessarily priced or structured identically — check current documentation rather than assuming a 1:1 cost match.
Auxiliary Logs	Data lake tier	Auxiliary Logs was positioned as a precursor concept for very-low-touch, high-volume data; the Data lake tier is its current-model successor per Microsoft's retention-tiers documentation, but confirm feature parity for any workload built around the older tier's specific behavior.
Archive	Long-term cold-storage retention, whose exact current placement (a Data lake tier property vs. a separate mechanism) this book could not confirm with confidence	Do not assume Archive's old search/restore workflow carries over unchanged — verify directly against the current retention-tiers page.

5. Pricing models: pay-as-you-go and Commitment tiers

[SOC MANAGEMENT] Independent of which retention tier a table sits in, a workspace as a whole chooses between two pricing structures for ingestion:

- **Pay-as-you-go** — every gigabyte ingested is billed at a published per-GB rate, with no upfront commitment and no minimum volume. This is the default, and it's the right choice for a workspace whose ingestion volume is small, unpredictable, or still being measured.
- **Commitment tiers** — a workspace commits to a fixed daily ingestion capacity in exchange for a discounted effective per-GB rate versus pay-as-you-go, billed whether or not the workspace actually uses the full committed capacity on a given day. Part 2 already named the 100 GB/day range as the rough point where a Commitment tier discount becomes worth evaluating, and flagged that the discount applies **per workspace** — splitting ingestion volume across several workspaces to keep each one "manageable" can quietly forfeit a discount that consolidating into fewer workspaces (or pooling volume through a dedicated cluster) would have captured.

Two mechanics worth naming explicitly because they change how a Commitment tier decision should be planned, not just priced:

- **Commitment tiers exist at multiple discrete capacity levels**, not as a continuously adjustable dial — a workspace picks a published tier level (Microsoft's documentation lists a defined ladder of GB/day levels above the entry threshold), and moving between levels is a deliberate change, not an automatic adjustment that tracks daily volume.
- **Changing tier level is constrained, not instantaneous, and the constraint is asymmetric.** Per Microsoft Learn, "Reduce costs for Microsoft Sentinel" (learn.microsoft.com/azure/sentinel/billing-reduce-costs, retrieved 2026-09-15): increasing the Commitment tier takes effect immediately but restarts a 31-day commitment period; moving back down to a lower Commitment tier or to pay-as-you-go is only allowed once that 31-day commitment period has finished. A workspace that increases its tier to absorb a short-lived

volume spike is committed to that higher rate for the full 31 days regardless of how quickly volume drops back down.

PRODUCT VERSION NOTE (as of 2026-09-15) *The specific Commitment tier capacity levels and their discount percentages relative to pay-as-you-go are published on Microsoft's Sentinel pricing page (azure.microsoft.com/pricing/details/microsoft-sentinel) and change on Azure's general pricing-update cadence — the same caution Part 14 already applies to Logic Apps per-action pricing applies here for the same reason. The 31-day commitment-period mechanic named above is documented on the "Reduce costs for Microsoft Sentinel" page cited above as of this writing, but is itself a policy Microsoft could change; this book does not quote a specific discount percentage or GB/day ladder above the published 100 GB/day entry point because either would likely be stale by the time it's read. Check the current Azure pricing calculator and Microsoft Sentinel cost-management documentation directly before sizing a Commitment tier decision, and re-check it again before assuming a cost model built even a few months earlier still holds.*

SOC Management View *A Commitment tier decision is a program-management commitment, not a technical setting — it's a multi-week-minimum financial obligation sized against a volume forecast, and forecasting ingestion volume badly in either direction has a real cost: underestimating means paying pay-as-you-go overage on top of the commitment; overestimating means paying for committed capacity the workspace never uses. Treat a Commitment tier evaluation as needing the same kind of volume-trend data Part 18's health-monitoring discipline already produces, not a one-time guess made at onboarding and never revisited.*

6. Free data sources

[ENGINEERING] Some data ingests into a Sentinel-enabled workspace at no charge at all, regardless of which tier the rest of the workspace's tables are billed under — not a cheaper tier, but zero ingestion cost. Microsoft's own documentation maintains a specific, named list of free data sources, and that list has grown and changed over the platform's history, so this section names it as a category with representative, high-confidence examples rather than reproducing a list this book cannot guarantee is current:

- The workspace's own **health monitoring data** — the `SentinelHealth` table Part 18 covers in depth — is documented as free, on the reasoning that a SOC shouldn't have to pay to find out its own platform is broken.
- Several **Microsoft security-product alert and incident feeds** — Microsoft Defender for Cloud alerts and Microsoft's own cross-product incident data among them — have historically been documented as free or free-with-conditions, on the logic that Microsoft wants friction-free

correlation of its own products' security signal into Sentinel rather than charging to bring in the alert stream that makes the unified incident queue (Part 15) work at all.

- **Azure Activity log** data has historically appeared on Microsoft's free-data-source list as well.

PRODUCT VERSION NOTE (as of 2026-09-15) *The authoritative, current free-data-sources list is the "Free data sources" section of Microsoft Learn, "Plan costs and understand pricing and billing" for Microsoft Sentinel ([Learn.microsoft.com/azure/sentinel/billing](https://learn.microsoft.com/azure/sentinel/billing), retrieved 2026-09-15), and is exactly the kind of list that gains and loses entries as Microsoft adjusts its own product bundling strategy — this book names three representative categories above with moderate confidence, not a complete or guaranteed-current enumeration. Before assuming any specific connector or table ingests free, check that page's current free-data-sources list directly; treat a table's free status as something to verify per source, not something to infer from this list or from what was true in an earlier version of this book.*

[SOC MANAGEMENT] The practical design implication: a table that ingests free is an attractive place to put context and enrichment data that a detection query joins against — the join itself costs nothing extra in ingestion terms, only in the query-time cost the source table it's joined from already carries. This is a reason (among several — see Part 10 on watchlists specifically) to prefer a free or already-flowing table over standing up a new paid ingestion path purely to hold reference data a rule needs to look up.

7. Tier as a detection-design decision, not just a finance one

[SOC MANAGEMENT] DEH Part 25 §8's SOC Management View callout makes the query-side half of this argument: a scheduled rule's query shape and its source table's billing tier interact, because an unfiltered scan against a per-query-billed table turns a query-writing habit into a recurring cost line. This part adds the other half, and it's the more consequential one: **which tier a source lives in determines whether an analytics rule can query it at all.** A table in the Data lake tier is not currently available to a scheduled or NRT rule the way an Analytics-tier table is (§3). That means every decision to park a source in the Data lake tier for cost reasons is simultaneously a decision that no automated analytics-rule alerting will ever fire against that source, directly, in its current form — full stop, regardless of how good the query would have been.

That tradeoff is sometimes exactly right: a bulk, low-signal, compliance-retention source that nobody was ever going to write a rule against anyway loses nothing by living in the Data lake tier. It's sometimes exactly wrong: a source with real detection value, routed to the Data lake tier purely because it's high-volume and someone was optimizing the ingestion bill without checking whether anything downstream needed to alert on it. The second case is a coverage gap that will not show up as a broken rule, a failed connector, or an error in [SentinelHealth](#) — there is no rule to fail, because no rule can exist against that table in the first place. It shows up only as an absence: an attack path nobody built a detection for, discovered the way most silent gaps are discovered, after the fact.

Engineering Reality A tier decision made once, at onboarding, for cost reasons, doesn't announce itself again later when the source's detection relevance changes. A source that looked purely like bulk compliance data at onboarding can become genuinely detection-relevant six months later — a new threat model, a new hunt finding, a new regulatory driver — and nothing in the platform prompts anyone to revisit the tier decision when that happens. The tier assignment just keeps quietly being what it was, unless someone deliberately re-checks it.

This is precisely the mechanism DEH Part 43 (Detection Debt) names in the abstract: a decision that looked cheap and reasonable at the time it was made, compounding, unflagged, into a gap that costs more to discover and fix later than it would have cost to get right at onboarding. Appendix A2's decision matrix exists as the companion reference for making this call deliberately, per source, rather than by default.

8. A workspace's cost shape, end to end

[CONCEPT] Figure 17.1 traces one workspace's billing surface from ingestion through the pricing-model choice, showing where each of this part's decisions — tier assignment, free-source exemption, and pay-as-you-go versus Commitment tier — actually sits in the model.

```
flowchart TD
    S[Data arriving via a connector/DCR<br/>see Part 3] --> FreeCheck{On the free<br/>data-source list?}
    FreeCheck -->|Yes| Free[No ingestion charge<br/>health/audit tables,<br/>select Microsoft security feeds]
    FreeCheck -->|No| Meter[Metered ingestion<br/>billed per GB]

    Meter --> Tier{Tier assignment<br/>decided at ingestion, per source}
    Free --> Tier

    Tier -->|Analytics tier| AT["Interactive retention<br/>90-day default, extensible<br/>No per-query charge<br/>Drives analytics rules, UEBA,<br/>workbooks, entity mapping"]
    Tier -->|Data lake tier| DL["Long-term low-cost storage<br/>Ingestion + processing + storage<br/>+ query billed separately<br/>KQL jobs / search jobs only —<br/>cannot drive a scheduled rule"]

    AT --> Price{Ingestion pricing model}
    DL --> Price

    Price -->|Pay-as-you-go| PAYG[Per-GB rate,<br/>no commitment]
    Price -->|Commitment tier| CT[Fixed daily capacity,<br/>discounted per-GB,<br/>per-workspace discount]
```

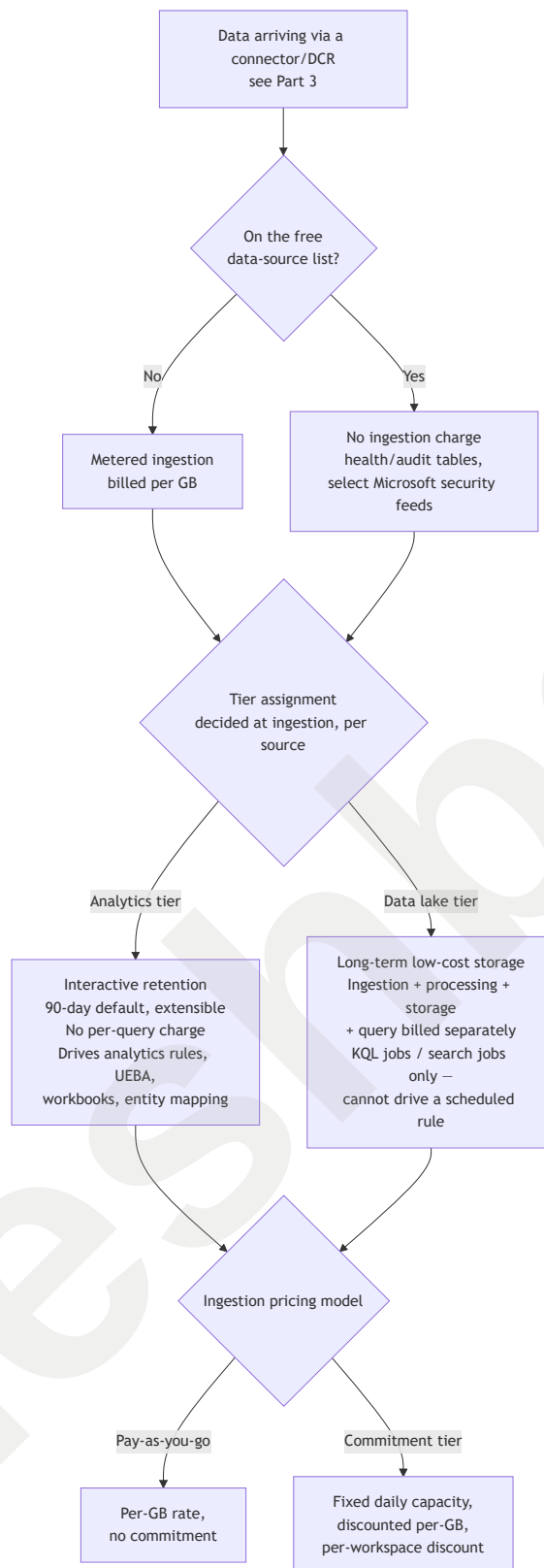


Figure 17.1 — A Sentinel workspace's cost shape, from ingestion to query. *CONCEPTUAL.*

Illustrates, at a structural level, how a free-data-source exemption, the Analytics-versus-Data-lake decision (§2–§3), and the pay-as-you-go-versus-Commitment-tier pricing choice (§5) combine to determine what a given piece of ingested data actually costs and what it can be used for afterward. This is this book's own structural sketch, built from the tier and billing documentation cited throughout this

part — not a reproduction of an official Microsoft architecture diagram, and not a capture of any live workspace, per this book's evidence-classification constraint (`STYLE-GUIDE.md` §9).

9. Keeping the decision from drifting

[SOC MANAGEMENT] Part 3 framed connector onboarding as four decisions made deliberately, per source, rather than accepted as a bundled default — connector family, transformation, destination table, and tier. This part's contribution to that discipline is the review side: a tier decision made correctly at onboarding can still go stale, because the two forces that would justify revisiting it — the source's actual detection relevance, and the workspace's actual ingestion volume relative to its pricing model — both change over time in ways nothing in the platform surfaces automatically.

Three practical anchors for keeping the review from being purely aspirational:

- **Tie a tier and pricing-model review to the same cadence Part 18's health-monitoring discipline already runs on.** A workspace that's already watching `SentinelHealth` for connector and rule failures is already looking at the platform regularly; adding "has any Data lake tier source become detection-relevant, and has ingestion volume moved enough to revisit the pricing model" to that same review is a marginal addition, not a new program.
- **Use Appendix A2's decision matrix as the documented record of why a given source landed where it did,** not just as a one-time onboarding checklist — a matrix that's never updated after the first pass is a snapshot of a decision, not a living record of one.
- **Treat a new Content Hub solution's bundled connector set (Part 3 §6) as a fresh instance of the same per-source tier question,** not an exemption from it — a bundle shipping several tables at once doesn't mean all of them belong in the same tier, and accepting a bundle's defaults without checking is exactly the "decided by not deciding" failure mode this part opened by naming.

What Would Change My Mind This part treats "review tier and pricing-model decisions on a recurring cadence, not just at onboarding" as necessary specifically because today's platform gives no automatic signal when a Data lake tier source becomes detection-relevant or a workspace's volume outgrows its pricing model. If Microsoft shipped a feature that let a Data lake tier table promote into analytics-rule eligibility without a full re-ingestion, or a workspace-level advisory that flagged "this table's query pattern suggests it belongs in a different tier," the specific recurring-review burden this section describes would shrink toward "verify the platform's own recommendation," a materially lighter operational ask than the deliberate, human-driven review this part currently recommends.

Cross-references

DEH Part 25 §8 (query-cost and billing-tier interaction — the query-writing half of this part's tier-as-detection-design argument) — DEH Part 43 (Detection Debt — the framework this part's §7 applies to a cost decision specifically) — Part 2 (Log Analytics Workspace Architecture and the Two Portals — Sentinel-wide billing and the Commitment tier threshold this part extends) — Part 3 (Data Ingestion:

Connectors, Data Collection Rules, and Table Tiers — where the tier decision is made, at the source) — Part 16 (Operating the Unified Security Operations Platform — portal-specific table-management surface differences) — Part 18 (Sentinel-Specific Tuning, Health Monitoring, and Troubleshooting — the [SentinelHealth](#) review cadence this part's §9 ties into) — Appendix A2 (Cost and Retention Decision Matrix — the companion reference for the per-source decision this part argues for).

neshboy

Section H — Tuning and Troubleshooting

neshbooy

Part 18 — Sentinel-Specific Tuning, Health Monitoring, and Troubleshooting

Why this part exists

[CONCEPT] DEH Part 25 §8 already names the failure mode this part exists to answer: Advanced Hunting and Sentinel table and field names are a Microsoft-controlled schema that changes over time, and a field renamed, moved, or retired produces a query that still compiles, still runs, and returns zero rows — with no ingest error anywhere to flag that anything broke. Zero rows is indistinguishable from "the technique genuinely isn't happening" without a separate mechanism confirming the rule still fires against a known-good event on some cadence. DEH Part 37 (Detection Testing) and DEH Part 39 (False Negative Engineering) teach why that confirmation has to be deliberate rather than assumed. What none of those three parts covers is what Sentinel specifically does, or doesn't do, to surface that failure on its own — whether the platform's own health telemetry would catch a silently broken rule, and what else can go wrong between a validated query and an incident an analyst sees that has nothing to do with the query's logic at all.

This part covers four things, in order: Sentinel's own classification of why a rule stops running (transient versus permanent failure, and what "permanent" actually triggers); the `SentinelHealth` and `SentinelAudit` tables as the queryable record of that classification, and the real limits on what they cover; ingestion delay as a failure class that affects a fixed-cadence Near-real-time (NRT) rule (fixed one-minute evaluation cadence, limited subset of scheduled-rule features) differently than it affects a Scheduled rule (the most configurable rule type — engineer-set query frequency and lookback window, full entity mapping and alert-grouping control); and incident-merge and correlation-engine behavior as a troubleshooting category distinct from anything a query's correctness can fix. It closes by tying all four back to DEH Part 25 §8's own warning: Sentinel's health telemetry — real, free, and genuinely useful — does not by itself solve the schema-drift problem DEH names. A canary-query discipline still has to sit on top of it, and this part states why.

1. Two failure classes: transient and permanent

[SENTINEL ENGINEER] Microsoft's own troubleshooting documentation for scheduled analytics rules sorts every rule-execution failure into one of two classes before deciding what to do about it, and the distinction is the single most useful thing to know before diagnosing a rule that has stopped producing incidents.

A **transient failure** is one Microsoft's platform expects to resolve on its own: a query that times out, a temporary connectivity break between a data source and the workspace (or between the workspace and Sentinel itself), or — by design — any new or unrecognized failure type, which Sentinel defaults to treating as transient. A rule experiencing a transient failure retries at increasing intervals for a while, then falls back to its next normal scheduled run. A rule is never disabled for a transient failure alone, no matter how many times it recurs, as long as each occurrence keeps getting classified as transient.

A **permanent failure** is one Microsoft's platform judges cannot self-resolve without a human changing something: the target workspace or table was deleted, Sentinel was removed from the workspace, a function the query calls was modified or removed, permissions on one of the query's data sources changed, or a data source was deleted outright. A separate permanent-failure category covers a query that consumes excessive compute relative to what Sentinel allows for a scheduled run — an engineering problem with the query itself, not an external dependency breaking. When a rule accumulates a predetermined number of consecutive permanent failures of the same type, Sentinel disables the rule, prepends the words "**AUTO DISABLED**" to the beginning of its name, and appends the failure reason to its description (Microsoft Learn, "Troubleshooting analytics rules in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/troubleshoot-analytics-rules, retrieved 2026-09-15). Microsoft doesn't publish the exact number of consecutive failures that crosses that line — only that the count is "predetermined" — so treat the threshold itself as unpublished rather than something to reverse-engineer and rely on.

PRODUCT VERSION NOTE (as of 2026-09-15) *The same troubleshooting page carries a banner steering readers toward Custom Detections — "now the best way to create new rules across Microsoft Sentinel SIEM Microsoft Defender XDR" [Microsoft Learn's own wording, not this book's] — as the unified rule-authoring surface (Part 5 owns this in full). The transient/permanent classification and **AUTO DISABLED** naming behavior described here are documented specifically for classic Scheduled and NRT analytics rules. Whether an equivalent auto-disable mechanism applies identically to a rule authored through Custom Detections is not something this book confirms — this book has no tenant to test it against. Re-check Microsoft Learn's current Custom Detections documentation before assuming this section's mechanics transfer unchanged.*

One permanent-failure case is worth naming on its own, because it lands squarely on the cross-tenant access pattern Part 2 already covers. When an analytics rule is created, Sentinel applies an access-permissions token to it so it can keep querying its target workspace even after the creator loses their own access — with one documented exception: a rule created to query a workspace in a different subscription or tenant (the scenario a Managed Security Service Provider, MSSP, distributed-workspace architecture produces, per Part 2 §2) carries the creating user's own credentials instead of an independent token. If that user later loses access to the target tenant — offboarded, role changed, guest access revoked — the rule fails with a health-monitoring message describing "insufficient access to resource," and gets auto-disabled through the same permanent-failure procedure as any other permission change (same source as above). Part 2 §6 already flags that Azure Lighthouse delegated access isn't supported for Sentinel data once viewed through the Microsoft Defender portal, and that Microsoft Entra B2B guest access is the documented alternative; this is the failure mode that fact produces when the substitution hasn't happened yet.

Engineering Reality The transient/permanent split matters operationally because it changes what "the rule looks broken" should make an engineer do first. A rule showing scattered, non-consecutive failures with varying reason text is very likely riding out transient noise on its own — the correct response is usually to wait and confirm via `SentinelHealth` (§2) that the failures aren't accumulating into a consecutive run of the same permanent-failure type. A rule that has actually gone `AUTO DISABLED` needs a person: the underlying condition (a deleted table, a revoked permission, a broken function) has to be fixed before re-enabling the rule does anything except set up the next disable cycle. Confusing the two — re-enabling an auto-disabled rule without touching the underlying cause, or panicking over a transient blip that was already self-healing — wastes exactly the kind of engineering time this section exists to save.

The table below separates the failure classes by what actually causes them and where the evidence of each shows up, as a first triage reference rather than an exhaustive list.

Failure class	Example causes	Sentinel's response	Where the evidence lands
Transient	Query timeout; temporary connectivity break between a data source and the workspace, or between the workspace and Sentinel; any new/unclassified failure type	Retries at increasing intervals, then resumes at the next scheduled run; rule stays enabled	<code>SentinelHealth</code> , <code>Status</code> == "Failure", with a transient-type reason string
Permanent — configuration change	Target workspace or table deleted; Sentinel removed from the workspace; a function the query calls was modified or removed; a data source's permissions changed or the source was deleted	After a predetermined run of consecutive same-type failures: rule disabled, <code>AUTO DISABLED</code> prepended to the rule's name, reason appended to its description	Rule list sorted by name; <code>SentinelHealth</code> <code>Reason</code> field; the rule's own description text
Permanent — resource drain	An improperly built query consumes excessive compute for a scheduled run	Same three-step auto-disable response, triggered by cost/performance rather than a hard dependency break	Same as above; requires rewriting the query before re-enabling does anything durable
Permanent — cross-tenant token loss	A rule queries a workspace in another subscription/tenant (the MSSP case above); the creating user later loses access to that tenant	"Insufficient access to resource" health message, then auto-disable under the same procedure	<code>SentinelHealth</code> ; Part 2 §2 and §6's cross-tenant/MSSP RBAC coverage for the broader access model this sits inside

2. `SentinelHealth` and `SentinelAudit`: the free, queryable audit trail

[ENGINEERING] Everything in §1 is diagnosable after the fact because Sentinel writes its own health telemetry into two ordinary Log Analytics tables — `SentinelHealth` and `SentinelAudit` — once an engineer turns the feature on. Neither exists by default; both are opt-in, enabled from Microsoft Sentinel's Settings page under **Auditing and health monitoring**, either as one "Enable" toggle covering every supported resource type, or through the underlying diagnostic settings for finer-grained

control (Microsoft Learn, "Turn on auditing and health monitoring in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/enable-monitoring, retrieved 2026-09-15). Once enabled, each table is created at the first eligible success or failure event — there's no backfill from before the feature was turned on.

The two tables split cleanly by what they answer. `SentinelHealth` answers "did this resource run, and did it succeed" — one row per execution event. `SentinelAudit` answers "did someone change this resource's configuration" — who, from where, and what the setting looked like before and after. That split matters for cost too: `SentinelHealth` is not billable, so health monitoring adds no ingestion cost regardless of scale; `SentinelAudit` is billable like any other ingested table, and its coverage is narrower — auditing today covers only analytics rules, not automation rules, playbooks, or connectors (same source as above).

Microsoft's own documentation is worth quoting rather than paraphrasing: build queries against these tables using the prebuilt `_SentinelHealth()` and `_SentinelAudit()` functions instead of querying the raw tables directly, because those functions "ensure the maintenance of your queries' backward compatibility in the event of changes being made to the schema of the tables themselves" (Microsoft Learn, "Auditing and health monitoring in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/health-audit, retrieved 2026-09-15). That's Microsoft's own health layer hedging against exactly the schema-drift risk DEH Part 25 §8 names for ordinary detection queries — a concrete sign the risk this part opened with isn't a hypothetical this book invented.

Resource type	What <code>SentinelHealth</code> / <code>SentinelAudit</code> records	Coverage caveat
Analytics rules (Scheduled, NRT)	Per-run success/failure, event count, whether the alert threshold was crossed; separately, <code>SentinelAudit</code> records create/update/delete changes to the rule itself	Full coverage once the feature is enabled — the only resource type with both health and audit coverage
Automation rules	Per-run status (<code>Success</code> , <code>Partial success</code> , <code>Failure</code>) and the list of playbooks the run invoked	Tells you the automation rule fired and what it called, not what happened inside a called playbook
Playbooks	Success/failure of a playbook triggered manually from the portal or via API	Does not, by itself, record a playbook invoked automatically by an automation rule's own action — that shows up on the automation-rule side instead
Data connectors	Data-fetch status changes and an hourly aggregated failure summary, per connector, per workspace	Supported for a named subset only — Amazon Web Services, Dynamics 365, Office 365, Microsoft Defender for Endpoint, Threat Intelligence–TAXII, Threat Intelligence Platforms, and any connector built on the Codeless Connector Framework; most other connectors have no <code>SentinelHealth</code> row at all and need the Data collection health monitoring workbook's <code>Heartbeat</code> -based view instead

[SENTINEL ENGINEER] The query behind §1's auto-disabled-rule check is a direct payoff of this table existing — the fastest way to answer "which of my rules are currently disabled, and why" without manually sorting the Active rules list.

```
// Adapted directly from Microsoft Learn, "Monitor the health and audit the
// integrity of your Microsoft Sentinel analytics rules"
// (learn.microsoft.com/azure/sentinel/monitor-analytics-rule-integrity),
// retrieved 2026-09-15. This book has no tenant to run the query against –
// treat the field and literal-string match below as documented, not
// independently verified, and re-check the source page if it returns
// nothing on a real workspace.
_SentinelHealth()
| where SentinelResourceType == "Analytics Rule"
| where Reason == "The analytics rule is disabled and was not executed."
```

A second query from the same source answers a related question — how failures break down by reason across every rule, the starting point for deciding whether a spike in transient failures is isolated or systemic:

```
// Adapted directly from the same Microsoft Learn page cited above.
_SentinelHealth()
| where SentinelResourceType == "Analytics Rule"
| summarize Occurrence = count(), Unique_rule = dcount(SentinelResourceId) by Status, Reason
```

Microsoft ships three purpose-built Content hub workbooks that wrap this data in a dashboard: **Analytics Health & Audit** (rule success/failure trends and audit activity by rule and caller), **Automation health** (automation-rule and playbook run status, correlated with Logic Apps diagnostics), and **Data collection health monitoring** (connector ingestion volume, latency, and anomaly detection via `series_decompose_anomalies()`). None substitutes for the underlying table — Part 8's framing of a workbook as "not a quality program" applies here too — but each is a reasonable first stop before writing a query from scratch.

flowchart TD

```

subgraph Resources["Monitored platform resources"]
    AR["Analytics rules<br/>(Scheduled + NRT)"]
    AUTORULE["Automation rules"]
    PB["Playbooks<br/>(portal/API-triggered runs only)"]
    DC["Data connectors<br/>(supported subset only)"]
end

TOGGLE["Settings -> Auditing and health monitoring -> Enable"]
AR --> TOGGLE
AUTORULE --> TOGGLE
PB --> TOGGLE
DC --> TOGGLE

TOGGLE --> SH["SentinelHealth<br/>not billable"]
TOGGLE --> SA["SentinelAudit<br/>billable; analytics rules only"]

LADIAG["Logic Apps diagnostics<br/>(enabled per playbook)"] --> AD["AzureDiagnostics"]

SH --> Q1["_SentinelHealth() queries"]
SA --> Q2["_SentinelAudit() queries"]
SH -. joined on RunId .-> AD

Q1 --> WB1["Analytics Health & Audit workbook"]
Q1 --> WB2["Automation health workbook"]
Q1 --> WB3["Data collection health<br/>monitoring workbook"]
Q2 --> WB1
Q1 --> ALERT["Azure Monitor alert rule<br/>(notification / automated action)"]

```

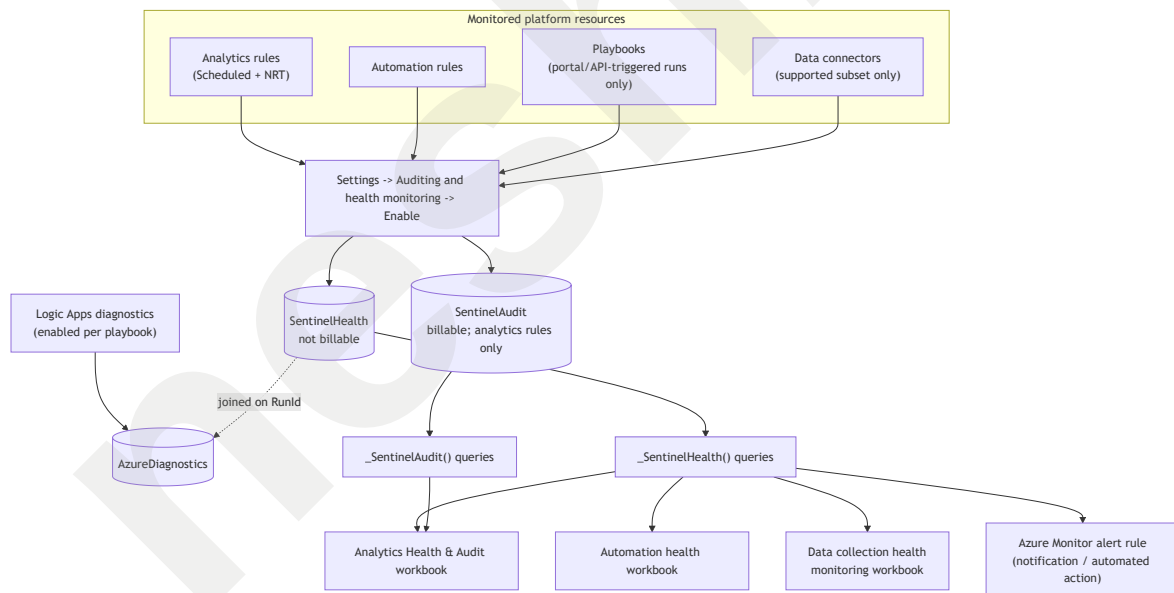


Figure 18.1 — Health and audit data flow, from platform resource to queryable table to dashboard. CONCEPTUAL. Illustrates how four resource types feed the `SentinelHealth`/`SentinelAudit` opt-in toggle, how the resulting tables are queried through the recommended `_SentinelHealth()`/`_SentinelAudit()` functions, and how that data surfaces in Microsoft's own built-in workbooks and (optionally) an Azure Monitor alert rule. Built from this part's own reading of the

Microsoft Learn pages cited throughout §2 — not a capture of any Sentinel UI screen, and not a reproduction of an official Microsoft architecture diagram.

3. Ingestion delay: the gap between generated and queryable

[ENGINEERING] A scheduled rule's lookback and run frequency being independent settings — the fact DEH Part 25 §3's Engineering Reality box covers, and this book's Part 1 leans on directly — has a concrete platform-level cause: an event's `TimeGenerated` timestamp reflects when it happened at the source, not when it became queryable in the workspace, and the gap between the two is ingestion delay. Microsoft's own worked example matches the failure this book has already described in the abstract: a rule running every five minutes with a five-minute lookback, against a source with a two-minute ingestion delay, misses an event outright — it lands after the run that should have caught it, and by the next run the time filter has already aged it out of the window (Microsoft Learn, "Handle Ingestion Delay in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/ingestion-delay, retrieved 2026-09-15). The rule doesn't error and shows no failure in `SentinelHealth`. It runs successfully, finds nothing, and produces no alert for an event that genuinely happened inside the window its settings would seem to cover.

Microsoft's documented fix is the one DEH already frames generically: widen the lookback beyond the known or estimated delay for the slowest source the query depends on. Widening alone introduces duplication — once lookback windows overlap, the same event can be evaluated, and alerted on, by more than one run. Microsoft's answer is to anchor the filter to `ingestion_time()` instead of the lookback window alone, so each event is attributed to exactly one run regardless of how wide the lookback grows:

```
// Adapted directly from Microsoft Learn, "Handle Ingestion Delay in
// Microsoft Sentinel" (learn.microsoft.com/azure/sentinel/ingestion-delay),
// retrieved 2026-09-15. Not run against a live tenant by this book –
// treat as documented, and re-verify ingestion_time() behavior against
// the current source table before relying on it in production.
let ingestion_delay = 2min;
let rule_look_back = 5min;
CommonSecurityLog
| where TimeGenerated >= ago(ingestion_delay + rule_look_back)
| where ingestion_time() > ago(rule_look_back)
```

False Positive Trap Widening a rule's lookback to compensate for ingestion delay, without also anchoring to `ingestion_time()`, is a platform-behavior false positive in exactly the sense this book's *STYLE-GUIDE* flags as distinct from a legitimate user tripping a threshold: the same real event, correctly matched twice by two overlapping lookback windows, produces two alerts for one occurrence. An analyst working the incident queue who doesn't know the rule was recently widened has no way to tell "this is a duplicate of yesterday's alert on the same event" from "this is a second, independent occurrence" without manually correlating timestamps — and a rule that starts double-alerting right after a well-intentioned lookback fix reads, to everyone downstream of the engineer who made the change, exactly like a rule that broke.

[SENTINEL ENGINEER] NRT rules inherit the same ingestion-delay mechanism but have no lookback knob to widen, because their evaluation window is tied to a fixed one-minute cadence rather than an engineer-set setting. Microsoft's health-monitoring documentation describes the retry behavior this produces: if an NRT rule's run fails, the system reconsiders that same failed window on the next run one minute later, continuing for up to sixty consecutive failures — one hour — before the pattern counts as sustained rather than a blip (Microsoft Learn, "Monitor the health and audit the integrity of your Microsoft Sentinel analytics rules," learn.microsoft.com/azure/sentinel/monitor-analytics-rule-integrity, retrieved 2026-09-15). That window covers outright run failures, not ordinary ingestion lag — but a source whose typical delay routinely exceeds roughly a minute puts an NRT rule in a structurally worse position than a scheduled rule facing the same delay: a scheduled rule's lookback can simply be set wider, while an NRT rule has no equivalent lever, and the only real fixes are a tighter-latency source or accepting systematically stale data on every run. Scheduled rules get an analogous but distinct retry behavior for ordinary query failures — up to five additional attempts on the exact same window before Sentinel treats it as fully skipped — a different mechanism aimed at run failures rather than lateness, but with the same lesson: a rule's own retry logic can mask a delay problem before it becomes visible as a real gap.

PRODUCT VERSION NOTE (as of 2026-09-15) Scheduled rules currently support a query interval ("Run query every") and lookback period ("Lookup data from the last") each ranging from five minutes to fourteen days, lookback required to be at least as long as the interval, with a default lookback of five minutes for a new rule (Microsoft Learn, "Create scheduled analytics rules in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/create-analytics-rules, and "Handle Ingestion Delay in Microsoft Sentinel," both retrieved 2026-09-15). These are exactly the kind of published numeric limits Microsoft can revise without a retirement-style announcement — re-check the current wizard's stated ranges directly if a rule's tuning depends on being close to either boundary. The retry-count figures earlier in this section (five additional attempts for a Scheduled rule, sixty consecutive failures for an NRT rule) are the same class of Microsoft Learn-published number and carry the same caveat, even though this book states them without repeating the hedge inline every time.

4. Incident-merge and correlation-engine surprises: a troubleshooting category of their own

[ANALYST] Everything covered so far explains why a rule might not run, or might run against incomplete data. This section covers a different kind of "wrong": the rule ran, the query was correct, the threshold was crossed, an alert was generated — and the incident an analyst sees still doesn't match what the rule's own configuration would predict. That gap is a troubleshooting category distinct from a broken query, because no amount of rewriting the KQL fixes it; the mismatch lives one layer up, in how alerts become incidents.

The first mechanism is a rule's own alert-grouping setting, which controls whether multiple alerts from one rule land in one incident or several. Microsoft documents a hard ceiling here that's easy to miss until it's hit: up to 150 alerts can be grouped into a single incident. If a run generates more matches than that under a grouping configuration, Sentinel doesn't drop the excess or force one oversized incident — it creates a second incident carrying the same incident details as the original, and routes everything past the 150th alert into that new incident instead (Microsoft Learn, "Create scheduled analytics rules in Microsoft Sentinel," learn.microsoft.com/azure/sentinel/create-analytics-rules, retrieved 2026-09-15). An engineer who configured "group all alerts into a single incident" and then sees two identical-looking incidents on a high-volume day hasn't found a bug — they've found the documented cap.

```
flowchart LR
    RULE["Scheduled rule fires<br/>200 matching alerts<br/>in one grouping window"] --> G["Alert grouping:<br/>'group all alerts into one incident'"]
    G --> INCA["Incident A<br/>(first 150 alerts)"]
    G --> INCB["151st alert onward | Incident B<br/>(same incident details as A,<br/>remaining 50 alerts)"]
```

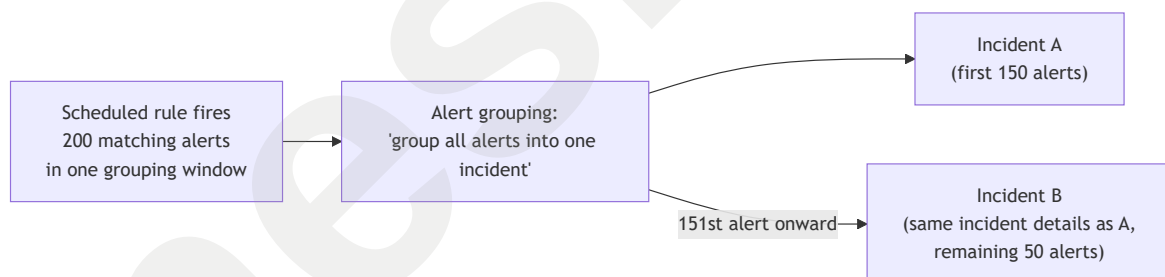


Figure 18.2 — The 150-alert grouping cap forking one rule's output into two incidents.

CONCEPTUAL. Illustrates the documented behavior when a single rule run's alert count exceeds Sentinel's per-incident grouping cap — not a capture of a real incident queue, and not a claim that 200 and 150 are anything but illustrative round numbers for this sketch.

The second mechanism is specific to a workspace onboarded to the Microsoft Defender portal, and it's a different kind of surprise because it isn't a cap — it's a documented loss of authority. Microsoft states this directly: once onboarded, a rule's alert-grouping settings "take effect only at the moment that the incident is created," because the Defender portal's own correlation engine owns alert correlation from that point on, and that engine "also might make decisions about alert correlation that don't take these settings into account" — with the consequence, in Microsoft's own words, that "the way alerts are

grouped into incidents might often be different than you would expect based on these settings" (same source as above). A related, narrower difference: "Re-open closed matching incidents" — letting a closed incident reopen for a new matching alert rather than spawning a separate one — isn't available in the Defender portal. Part 13 already treats correlation-engine behavior as a first-class operational hazard; this section's point is narrower and diagnostic — when an incident's shape doesn't match a rule's configuration, ask which portal created it before assuming the rule is broken.

Azure portal (retiring 2027-03-31)	Defender portal
A rule's alert-grouping settings directly and persistently determine how its alerts group into incidents	Alert-grouping settings apply only as an initial instruction at incident-creation time; the Defender XDR correlation engine can group differently afterward
"Re-open closed matching incidents" is available as a rule setting	This option is not available

PRODUCT VERSION NOTE (as of 2026-09-15) *The 150-alert grouping cap, the Defender portal's alert-grouping settings applying only as an initial instruction at incident-creation time, and the unavailability of "Re-open closed matching incidents" in the Defender portal are all documented on Microsoft Learn, "Create scheduled analytics rules in Microsoft Sentinel" (learn.microsoft.com/azure/sentinel/create-analytics-rules), retrieved 2026-09-15. All three are exactly the kind of numeric limit and portal-parity behavior Microsoft can revise without a retirement-style announcement, and the second two sit directly on top of the Azure-portal retirement this book's Part 2 already treats as its own single most consequential version note — re-check the Incident settings section of the current rule wizard before assuming any of the three still hold on a live workspace.*

Blind Spot `SentinelHealth`'s analytics-rule health record answers "did this run succeed, and how many alerts did it generate" — it does not record which incident, or how many incidents, those alerts ended up grouped into, and it has no visibility at all into a Defender-portal correlation decision that overrides a rule's own grouping configuration. A rule can show `Status == "Success"` with a healthy alert count in `SentinelHealth` while the incident queue tells a completely different story about how those alerts landed. Diagnosing an incident-shape surprise means looking at the incident and investigation graph directly (Part 6) or the Defender-portal-specific behavior Part 16 catalogs — not re-running the health query from §2 and expecting it to explain something it was never built to answer.

[SENTINEL ENGINEER] The diagnostic table below separates "the rule is unhealthy" symptoms from "the rule is healthy but the incident-formation layer did something unexpected" symptoms — the split this section exists to draw.

Symptom	Likely layer	Where to check
Rule shows Success in health telemetry, but no new incident appears	Alert threshold wasn't crossed, or incident creation is disabled on the rule	SentinelHealth reason text; the rule's own Incident settings tab
One rule produces more separate incidents than its grouping setting implies	The 150-alert grouping cap (§4), or a grouping mode narrower than expected	Count alerts per incident; re-check the rule's Alert grouping configuration
Incident title, severity, or provider field doesn't match the rule's own alert details	Defender-portal correlation engine treating grouping settings as an initial instruction only	Part 13 (automation-rule behavior differences) and Part 16 (portal parity gaps)
Rule is missing from Active rules, or its name now starts with AUTO DISABLED	Permanent failure and consecutive-failure auto-disable (§1)	§1's failure-class table; the rule's own description field
Alert fires later than expected, or misses a real event entirely	Ingestion delay exceeding the rule's lookback, or (for NRT) its fixed cadence (§3)	§3; the Workspace Usage Report's end-to-end latency view

5. Canary queries: the operational answer to schema drift

[THREAT HUNTER] Return to the failure mode this part opened with. DEH Part 25 §8 names it precisely: a table or field renamed, moved, or retired produces a query that keeps compiling and running, returns zero rows, and generates no ingest error — indistinguishable, from the outside, from "the technique genuinely isn't occurring." §2 already showed why Sentinel's own health telemetry doesn't rescue an engineer from this: Microsoft's documented rule-status descriptions list "Rule executed successfully, but didn't reach the threshold required to generate an alert" as a **Success** state, not a **Failure** (Microsoft Learn, "Monitor the health and audit the integrity of your Microsoft Sentinel analytics rules," retrieved 2026-09-15). A rule silently returning zero rows because a field it depends on was quietly renamed logs that same **Success** status as a rule that's genuinely finding nothing suspicious — from **SentinelHealth**'s point of view the two are identical. Nothing covered so far distinguishes "correctly found zero because the world is quiet" from "incorrectly found zero because the schema moved."

Detection Test DEH Part 25 §7's worked LSASS-memory-access detection anchors what a canary looks like as a platform object. The base query depends on specific columns from **DeviceProcessEvents** and **DeviceFileEvents** — exactly what a schema change could rename or relocate without warning. A canary for that detection isn't a copy of the detection logic; it's a simpler, separately scheduled query against the same base tables and load-bearing columns, checking only that they still exist and return non-null values for ordinary, high-volume activity every host generates constantly. If the canary itself returns zero rows, that's a fast signal the production rule's silence is a schema problem, not a quiet day — caught on a cadence measured in hours, not discovered weeks later.

[ENGINEERING] A canary belongs in the platform as its own scheduled rule, workbook query, or Azure Monitor alert rule reading the same base table, with a deliberately low bar: it should fire whenever a

source that reliably produces events over any short window goes quiet at the field level the production detection depends on, not whenever anything resembling an attack technique occurs. That's the general discipline DEH Part 37 and DEH Part 39 already teach — a detection's silence needs its own explicit test, because "no alerts" is not, on its own, evidence that nothing happened. This part's contribution is narrower: the canary has to be built and scheduled as an ordinary platform object, because nothing covered here — not `SentinelHealth`, not the auto-disable mechanism, not the retry logic — was built to catch a schema-drift-induced silent failure, and none of them will start catching it without a human adding a second query whose entire job is noticing the silence.

Engineering Reality *Everything in Parts 1 through 17 of this book exists to turn a correct query into durable production infrastructure. This part's real argument is that "correct" and "durably operating in production" are not the same claim, and the gap is exactly the four things covered here: a permanent failure the platform will eventually disable the rule for, an ingestion delay it will never flag as an error, a correlation-engine decision it documents but doesn't warn on, and a schema change its own health telemetry can't distinguish from a quiet day. None are query-logic bugs; all four still leave a SOC believing it's covered by a detection that, in practice, is not — and a canary-query habit catches all four with one question asked on a schedule: did a known-good signal still produce a known-good result today.*

What Would Change My Mind *This part treats a manually built, separately scheduled canary query as a necessary compensating control, on the grounds that nothing in Sentinel's current health-monitoring layer distinguishes "found nothing because nothing happened" from "found nothing because the schema moved." If Microsoft shipped a native coverage-staleness signal — something in `SentinelHealth`, or in whatever health surface a Custom Detections-based rule eventually gets, that flags a rule whose result count has dropped to zero against its own historical baseline, distinct from a rule that's merely running successfully — that would shift the canary from a necessary compensating control to a redundant one for any rule already covered by that native signal, and the honest update to this part would say so rather than keep recommending manual canaries out of habit once the platform closes the gap itself.*

6. A recurring health-review discipline, not a one-time setup step

[SOC MANAGEMENT] Every mechanism in this part shares one operational property worth stating plainly to whoever owns the platform-operations budget: turning on health monitoring costs nothing in ingestion (`SentinelHealth` isn't billable), and every check this part describes is a query or a workbook view, not new infrastructure to provision. The cost that actually matters is analyst and engineer time spent looking — which is exactly the kind of recurring, unglamorous review work that's easy to schedule once and then quietly stop doing once nothing looks broken for a few weeks. The table below is a minimum cadence, not an exhaustive program.

Cadence	Check	Query or surface
Once, at onboarding	Confirm auditing and health monitoring is actually turned on for the workspace	Settings → Auditing and health monitoring
Weekly	Scan for rules whose names now start with <code>AUTO DISABLED</code>	Active rules list sorted by name, or the §2 <code>_SentinelHealth()</code> disabled-rule query
Weekly	Review failure reasons across all rules, not just the disabled ones	§2's <code>Status / Reason</code> summary query
Monthly	Re-check whether ingestion delay for any source has drifted since it was last measured	Workspace Usage Report's end-to-end latency view
Monthly	Confirm each production rule's paired canary query is still firing as expected	§5's canary pattern
Per Defender-portal onboarding event	Re-verify alert-grouping and incident-shape behavior for every rule with a non-default grouping setting	Part 16's portal-parity coverage; §4's diagnostic table

SOC Management View *The recurring cost of this review is genuinely small relative to the cost of discovering a silently disabled or silently schema-drifted rule only when an incident it should have caught surfaces some other way — during an audit, a tabletop exercise, or, worse, an actual intrusion. Framing this as a standing weekly/monthly cadence owned by a named role, rather than a best-effort habit any engineer does "when there's time," is the practical difference between this part's mechanisms actually protecting a SOC and this part's mechanisms existing in documentation that nobody consults until something has already gone wrong.*

[CONCEPT] This part closes the loop this book opened in Part 1: a validated query is the starting line, not the finish line, and the six layers between a query and an incident an analyst can act on each have their own way of quietly failing without producing an error. `SentinelHealth` and `SentinelAudit` make most of those failures queryable, for free, once turned on — but they were built to answer "did this resource run and succeed," not "is this rule's silence meaningful," and the second question is the one a canary-query habit exists to answer instead. A SOC that treats Part 18's checklist as a recurring discipline, not a one-time setup task, is the practical difference between a platform that looks operated and one that actually is.

Cross-references: DEH Part 25 §3 (lookback-vs-frequency Engineering Reality, extended here to ingestion delay) and §8 (the Advanced Hunting schema-drift warning this part's canary discipline directly answers); DEH Part 37 (Detection Testing — the general discipline behind §5's canary pattern); DEH Part 39 (False Negative Engineering — why a detection's silence needs its own explicit test); Part 1 §3 (the six-layer query-to-incident pipeline this part closes); Part 2 §2 and §6 (workspace RBAC, cross-tenant/MSSP access patterns, and the Lighthouse/Entra B2B distinction behind §1's cross-tenant token case); Part 5 (Custom Detections as the newer unified rule-authoring surface flagged in §1's PRODUCT VERSION NOTE); Part 6 (entity mapping and the investigation graph, for diagnosing an incident-shape surprise directly); Part 8 (workbooks as a visualization layer, not a validated quality signal, echoed in §2);

Part 13 (automation-rule behavior differences on Defender-portal onboarding, extended here to alert grouping); Part 16 (Defender-portal migration and parity gaps, the full treatment behind §4's portal-comparison table).

neshbooy